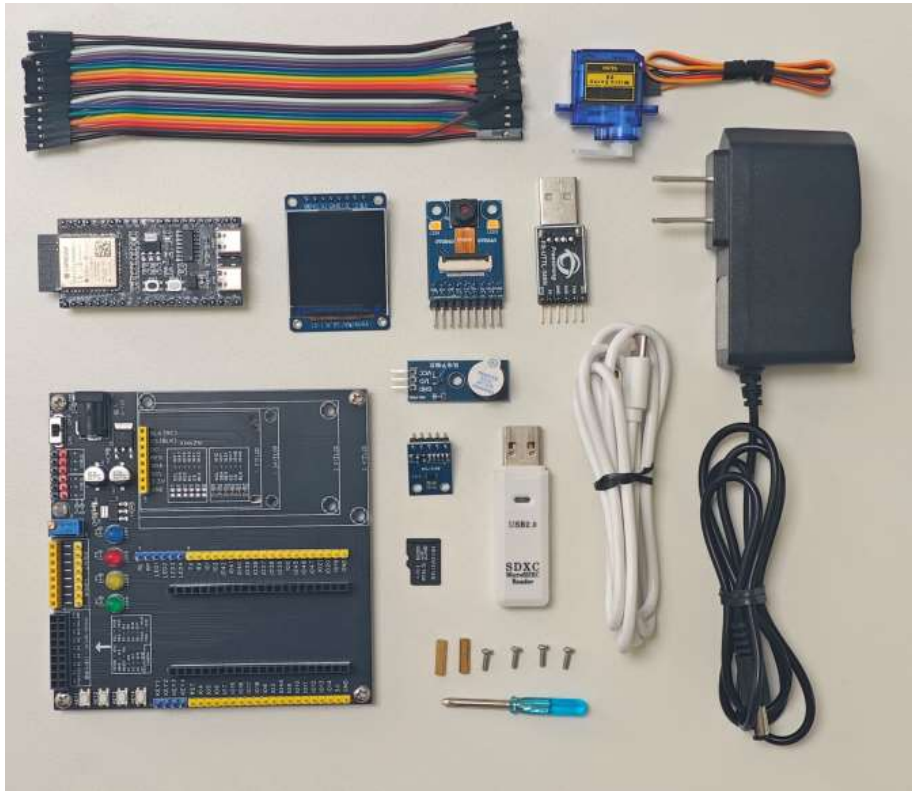
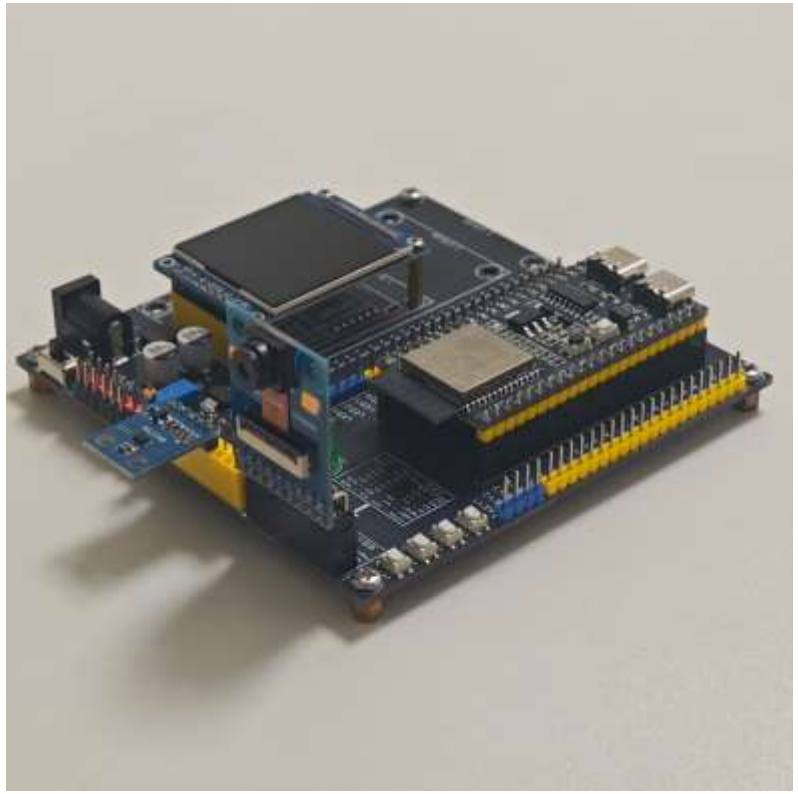
课程简介

艾谷科技
Aigo

- 开发框架： ESP-IDF（C语言）
- 外设实验： 24例
- 实验现象 – 讲述原理(内部架构框图) – 手写代码实现现象
- 硬件： ESP32-S3核心板+底板+模块

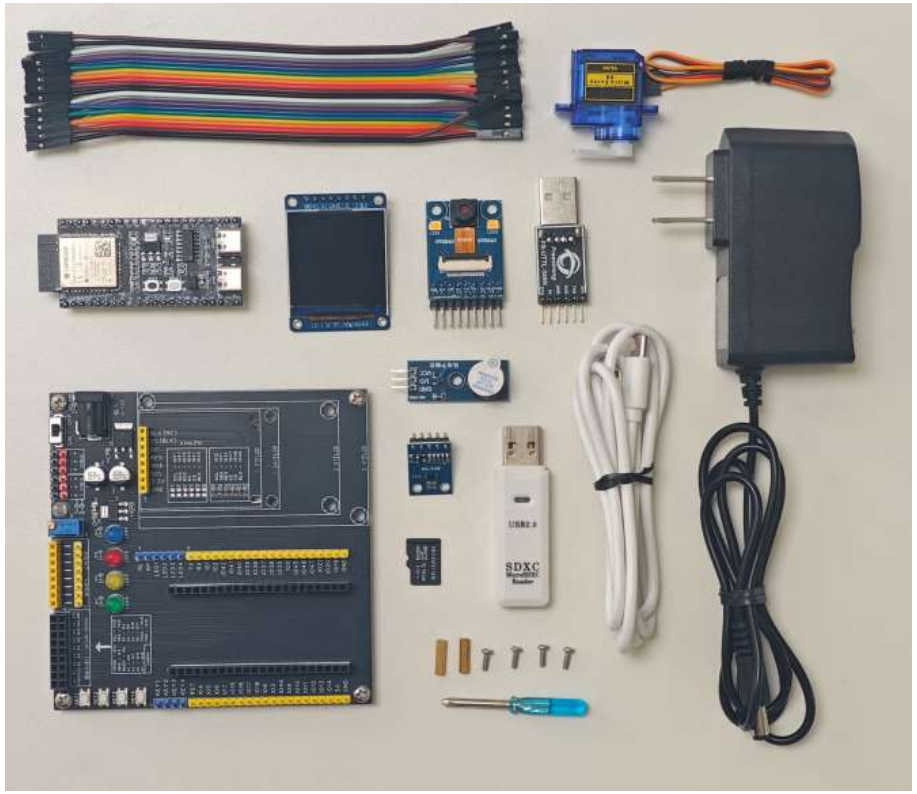




硬件设备

艾谷科技
Aigo

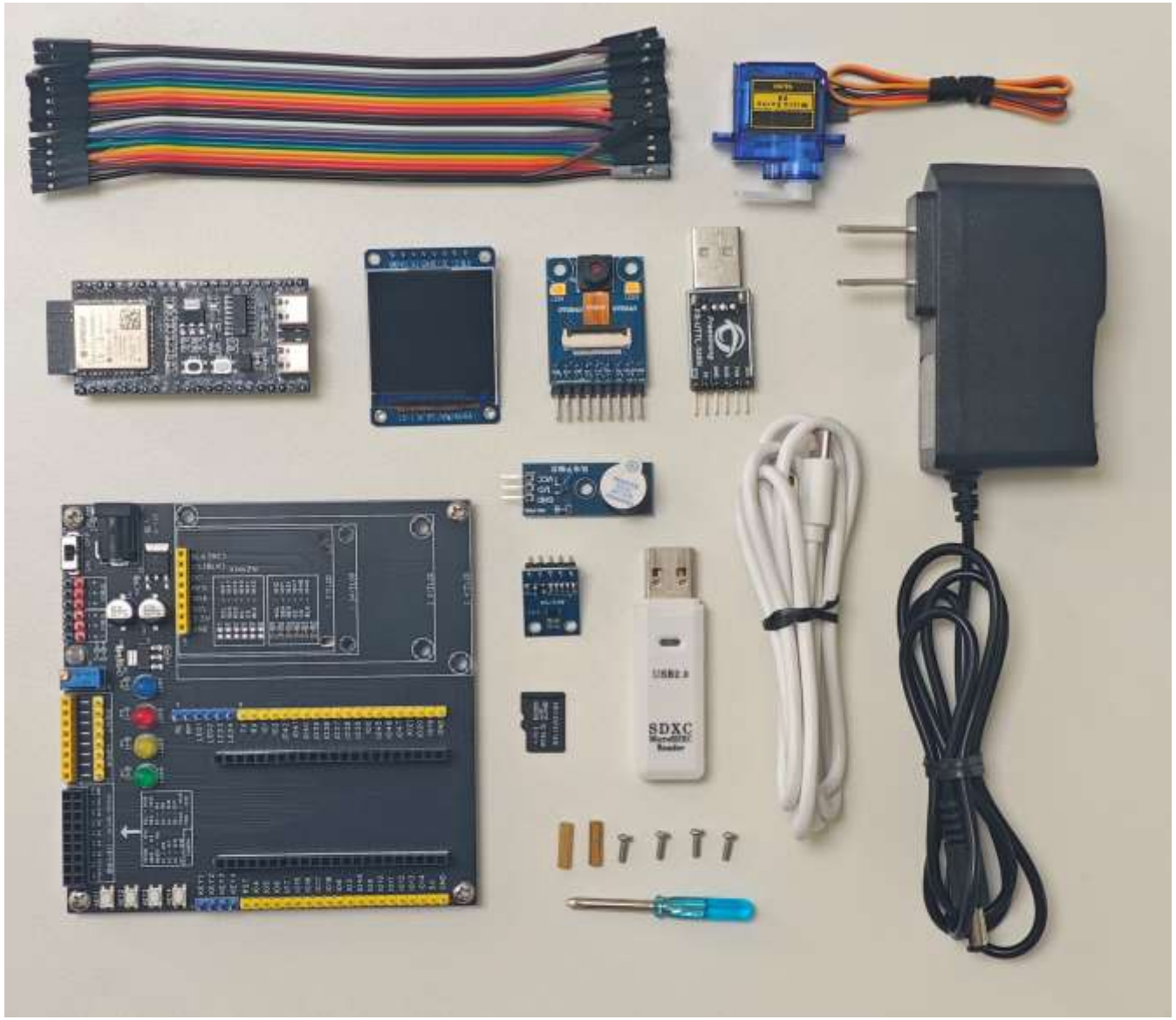
- ESP32S3开发套件
- Windows电脑





套件介绍

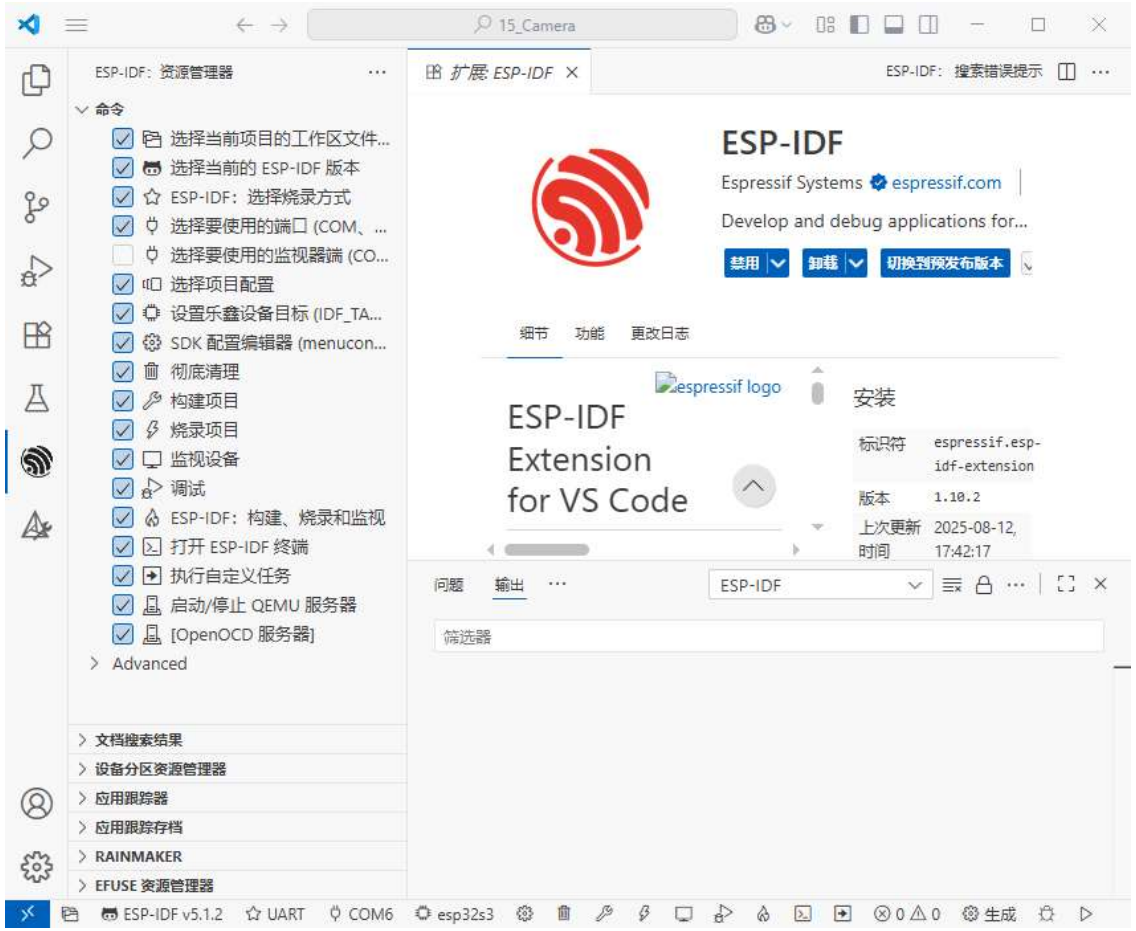
艾谷科技
Aigo



软件设备

艾谷科技
Aigo

VSCode + ESP-IDF插件



ESP32S3简介

艾谷科技
Aigo

- ESP32S3是乐鑫公司开发的MCU，芯片集成了Xtensa LX7双核处理器，带wifi、蓝牙
- ESP32S3常应用在嵌入式领域，如物联网、机器人、无线通信、智能车、娱乐电子产品等
- ESP32S3功能强大、性能优异、片上资源丰富、功耗低，是一款优秀的嵌入式微控制器



芯片、模组、开发板的关系

艾谷科技
Aigo

芯片



厂家：乐鑫

模组



厂家：乐鑫、第三方

开发板



厂家：第三方

模组内部

艾谷科技
Aigo



模组的优势

艾谷科技
Aigo

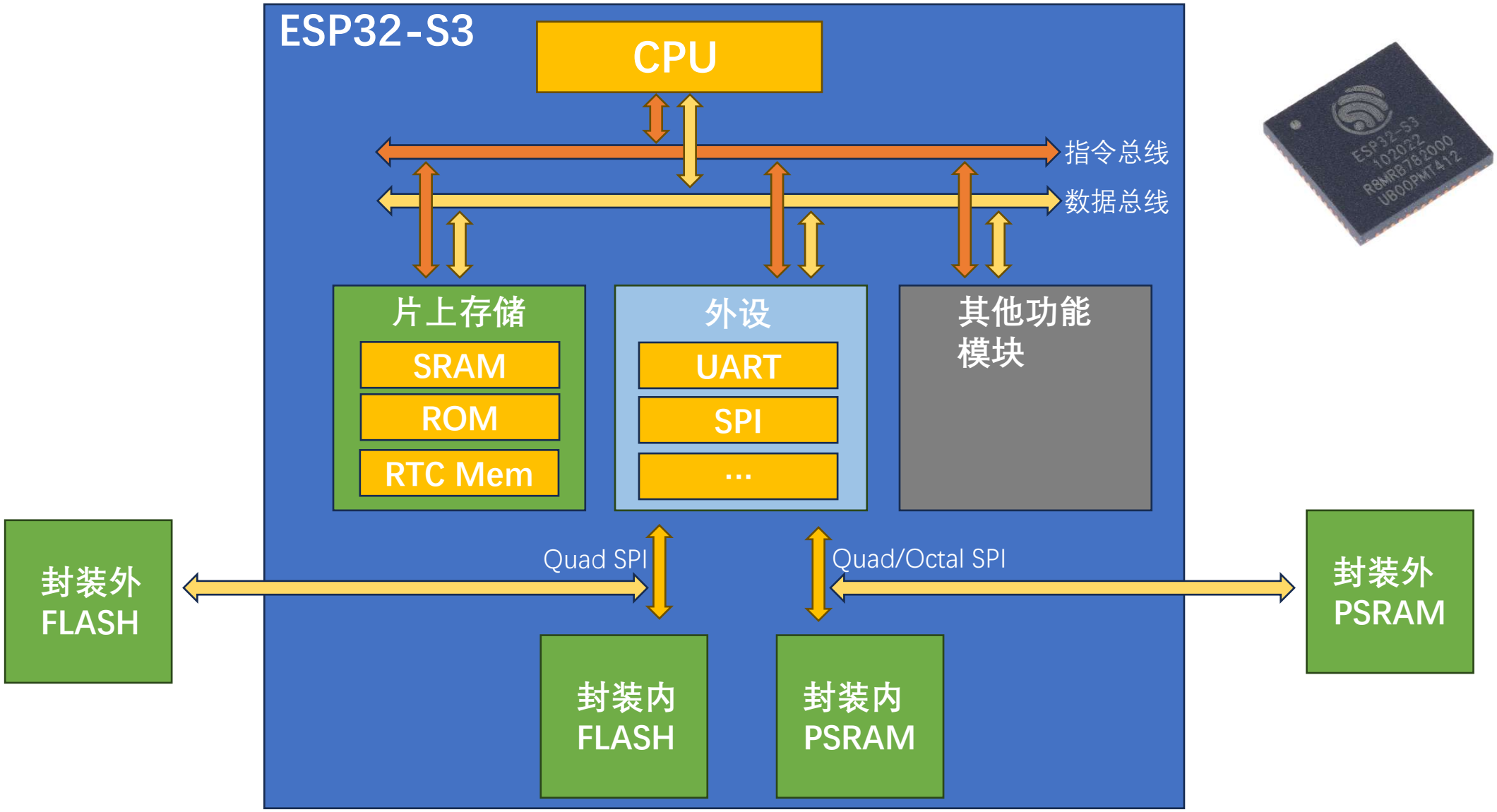




- QFN-56封装引脚，需要良好的焊接水平，否则容易短路
- 射频电路需要专门设计，设计不好WIFI、蓝牙的信号强度
- EMC问题，某些行业(医疗器械、铁路等)需要电磁兼容性认证，要求电路板有良好的抗干扰能力和较低的对外辐射，这就需要电路板信号进行分析，在电路板设计时做专门的处理。

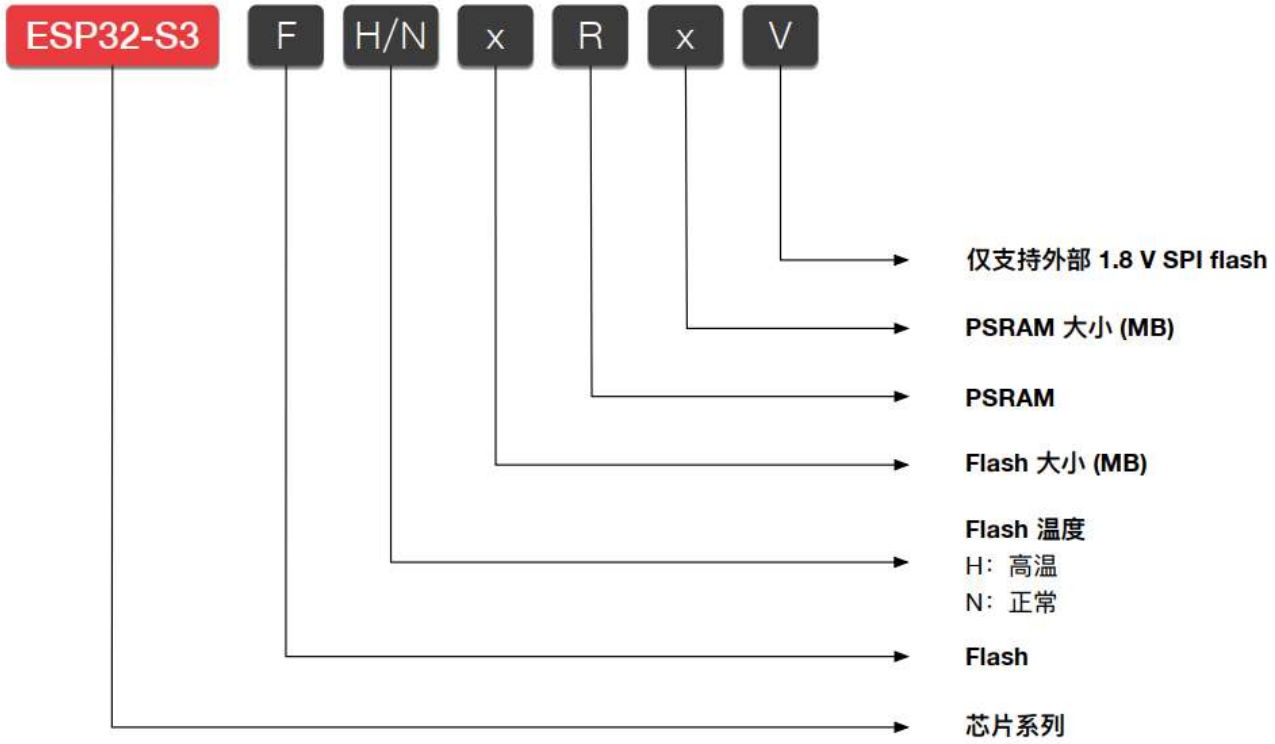
ESP32-S3内部结构

艾谷科技
Aigo



ESP32-S3型号

艾谷科技
Aigo





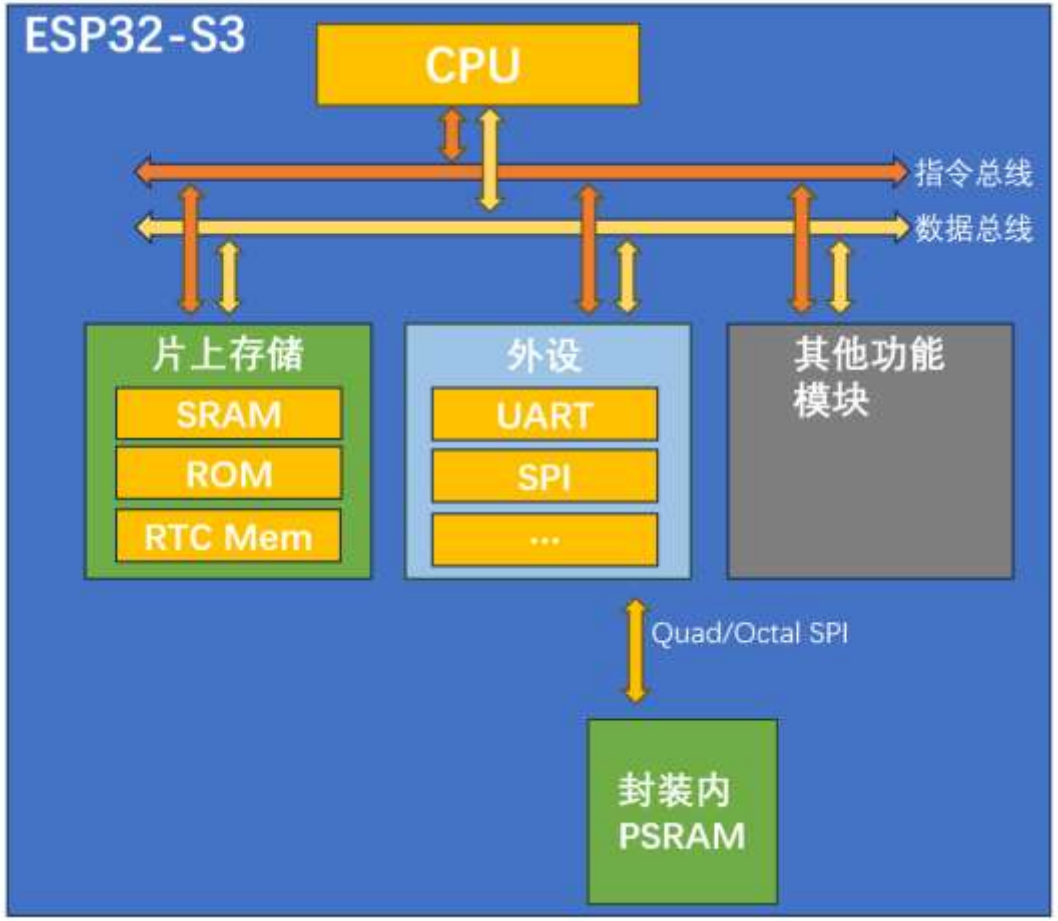
订购代码[1]	封装内 Flash	封装内 PSRAM	环境温度[2](°C)	VDD_SPI 电压[3]
ESP32-S3	—	—	–40 ~ 105	3.3 V/1.8 V
ESP32-S3FN8	8 MB (Quad SPI)[4]	—	–40 ~ 85	3.3 V
ESP32-S3R2	—	2 MB (Quad SPI)	–40 ~ 85	3.3 V
ESP32-S3R8	—	8 MB (Octal SPI)	–40 ~ 65	3.3 V
ESP32-S3R8V	—	8 MB (Octal SPI)	–40 ~ 65	1.8 V
ESP32-S3FH4R2	4 MB (Quad SPI)	2 MB (Quad SPI)	–40 ~ 85	3.3 V

ESP32-S3R8

艾谷科技
Aigo



- 系列：ESP32-S3系列
- 型号：ESP32-S3R8
- 内核：Xtensa LX7 32位双核
- 主频：240MHz
- RAM：512K（SRAM）+ 8M(PSRAM)
- ROM：384K（ROM）（不可编程）
- 供电：标准3.3V
- 封装：QFN-56



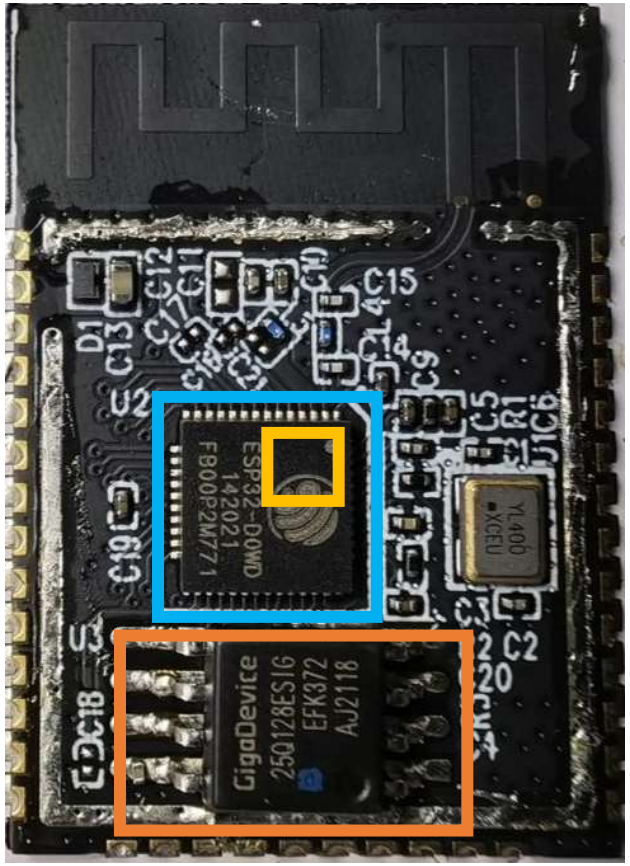
模组架构

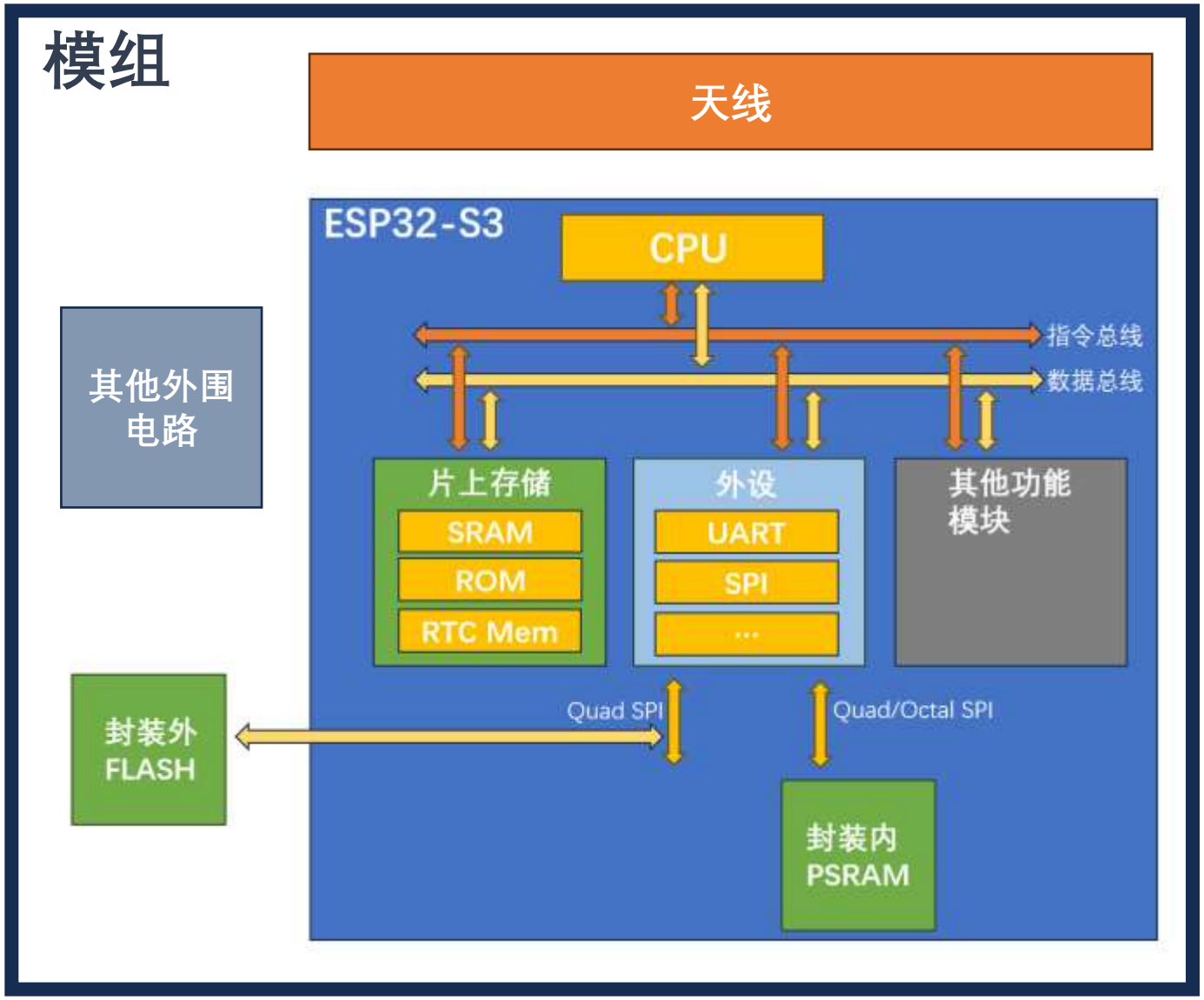
艾谷科技
Aigo

①ESP32-S3R8

②PSRAM

③FLASH





模组系列

艾谷科技
Aigo

ESP32-S3-WROOM-1

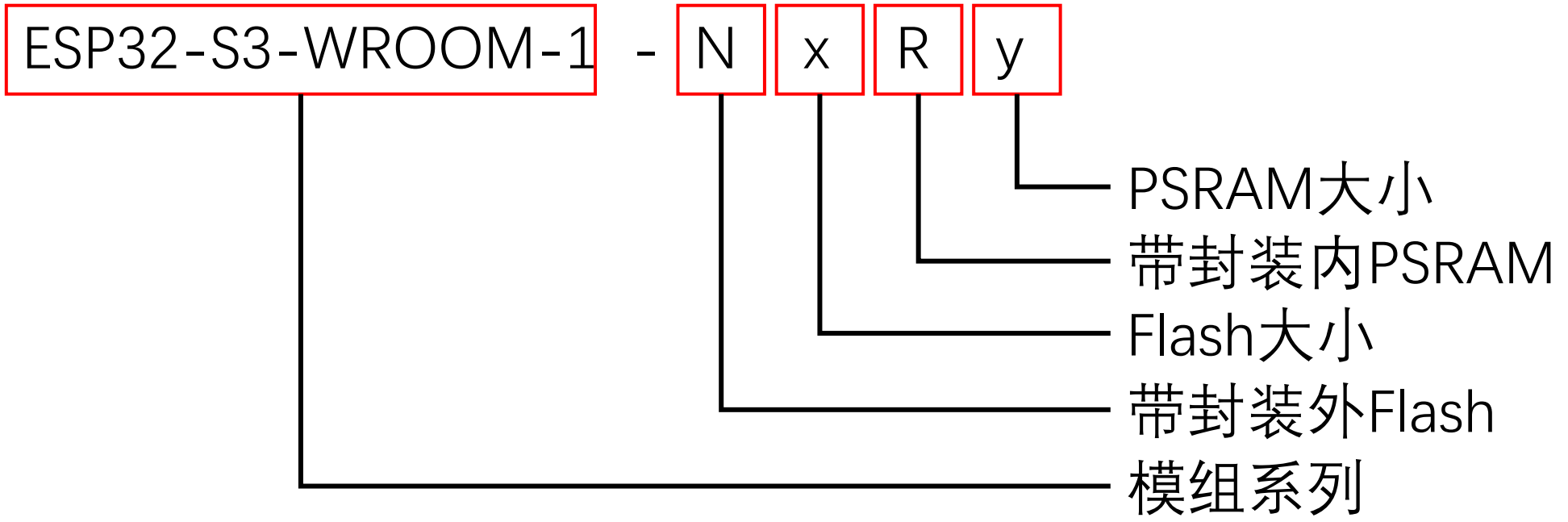


ESP32-S3-WROOM-1U



模组命名规则

艾谷科技
Aigo



订购代码[2]	Flash[3, 4]	PSRAM[4]	环境温度[5] (°C)	模组尺寸[6] (mm)
ESP32-S3-WROOM-1-N4	4 MB (Quad SPI)	-	–40 ~ 85	18.0 × 25.5 × 3.1
ESP32-S3-WROOM-1-N8	8 MB (Quad SPI)	-	–40 ~ 85	
ESP32-S3-WROOM-1-N16	16 MB (Quad SPI)	-	–40 ~ 85	
ESP32-S3-WROOM-1-H4	4 MB (Quad SPI)	-	–40 ~ 105	
ESP32-S3-WROOM-1-N4R2	4 MB (Quad SPI)	2 MB (Quad SPI)	–40 ~ 85	
ESP32-S3-WROOM-1-N8R2	8 MB (Quad SPI)	2 MB (Quad SPI)	–40 ~ 85	
ESP32-S3-WROOM-1-N16R2	16 MB (Quad SPI)	2 MB (Quad SPI)	–40 ~ 85	
ESP32-S3-WROOM-1-N4R8	4 MB (Quad SPI)	8 MB (Octal SPI)	–40 ~ 65	
ESP32-S3-WROOM-1-N8R8	8 MB (Quad SPI)	8 MB (Octal SPI)	–40 ~ 65	
ESP32-S3-WROOM-1-N16R8	16 MB (Quad SPI)	8 MB (Octal SPI)	–40 ~ 65	
ESP32-S3-WROOM-1-N16R16VA[7]	16 MB (Quad SPI)	16 MB (Octal SPI)	–40 ~ 65	

外设简介

艾谷科技
Aigo

- 外设是一个相对的概念，对于一个芯片来说，外设是指内核以外的某些部分。对于一个开发板来说，外设是指主控芯片以外的部分





ESP32-S3N16R8模组引脚

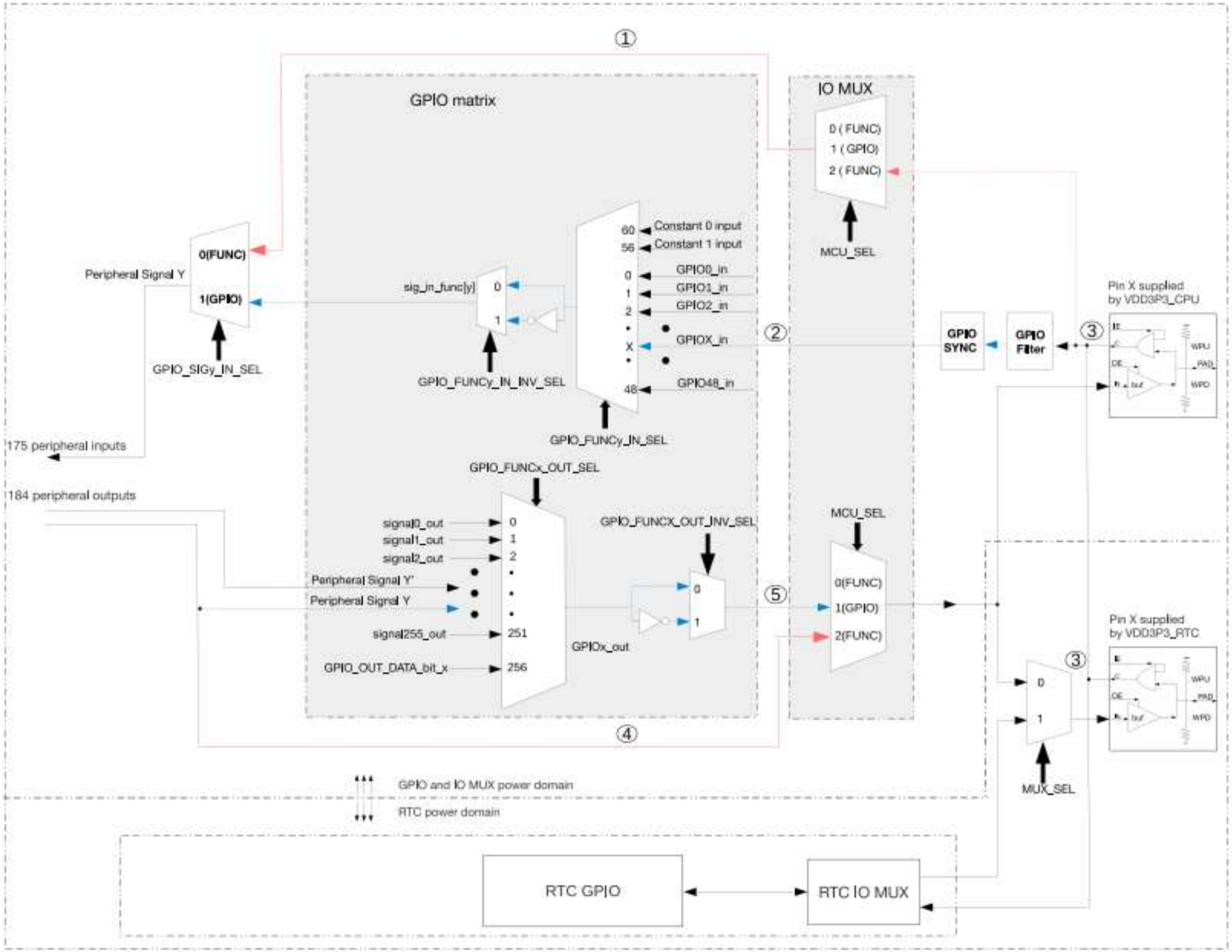
艾谷科技
Aigo

序号	引脚名称	引脚类型	模拟功能	上电默认功能	IO MUX直连的外设信号	其他功能	备注
1	GND	P	\	\	\		
2	3V3	P	\	\	\		
3	EN	I	\	\	\		高电平：芯片使能 低电平：芯片关闭 注意：EN引脚不能悬空
4	IO4	I/O/T	ADC1_CH3	GPIO4		TOUCH4、RTC_GPIO4	
5	IO5	I/O/T	ADC1_CH4	GPIO5	\	TOUCH5、RTC_GPIO5	
6	IO6	I/O/T	ADC1_CH5	GPIO6	\	TOUCH6、RTC_GPIO6	
7	IO7	I/O/T	ADC1_CH6	GPIO7	\	TOUCH7、RTC_GPIO7	
8	IO15	I/O/T	ADC2_CH4	GPIO15	U0RTS	RTC_GPIO15、XTAL_32K_P	
9	IO16	I/O/T	ADC2_CH5	GPIO16	U0CTS	RTC_GPIO16、XTAL_32K_N	
10	IO17	I/O/T	ADC2_CH6	GPIO17	U1TXD	RTC_GPIO17	
11	IO18	I/O/T	ADC2_CH7	GPIO18	U1RXD、CLK_OUT3	RTC_GPIO18	
12	IO8	I/O/T	ADC1_CH7	GPIO8	SUBSPICS1	TOUCH8、RTC_GPIO8	
13	IO19	I/O/T	ADC2_CH8	USB_D-	U1RTS、CLK_OUT2	RTC_GPIO19、GPIO19	
14	IO20	I/O/T	ADC2_CH9	USB_D+	U1CTS、CLK_OUT1	RTC_GPIO20、GPIO20	
15	IO3	I/O/T	ADC1_CH2	GPIO3	\	TOUCH3、RTC_GPIO3	
16	IO46	I/O/T	\	GPIO46	\	\	
17	IO9	I/O/T	ADC1_CH8	GPIO9	FSPIHD、SUBSPIHD	TOUCH9、RTC_GPIO9	
18	IO10	I/O/T	ADC1_CH9	GPIO10	FSPICS0、FSPIIO4、SUBSPICS0	TOUCH10、RTC_GPIO10	
19	IO11	I/O/T	ADC2_CH0	GPIO11	FSPID、FSPIIO5、SUBSPID	TOUCH11、RTC_GPIO11	
20	IO12	I/O/T	ADC2_CH1	GPIO12	FSPICLK、FSPIIO6、SUBSPICLK	TOUCH12、RTC_GPIO12	
21	IO13	I/O/T	ADC2_CH2	GPIO13	FSPIQ、FSPIIO7、SUBSPIQ	TOUCH13、RTC_GPIO13	
22	IO14	I/O/T	ADC2_CH3	GPIO14	FSPIWP、FSPIDQS、SUBSPIWP	TOUCH14、RTC_GPIO14	
23	IO21	I/O/T	\	GPIO21	\	RTC_GPIO21	
24	IO47	I/O/T	\	GPIO47	SPICLK_P_DIFF、SUBSPICLK_P_DIFF	\	
25	IO48	I/O/T	\	GPIO48	SPICLK_N_DIFF、SUBSPICLK_N_DIFF	\	
26	IO45	I/O/T	\	GPIO45	\	\	
27	IO0	I/O/T	\	GPIO0	\	RTC_GPIO0	
28	IO35	I/O/T	\	GPIO35	SPIIO6、FSPID、SUBSPID	\	
29	IO36	I/O/T	\	GPIO36	SPIIO7、FSPICLK、SUBSPICLK	\	
30	IO37	I/O/T	\	GPIO37	SPIDQS、FSPIQ、SUBSPIQ	\	
31	IO38	I/O/T	\	GPIO38	FSPIWP、SUBSPIWP	\	
32	IO39	I/O/T	\	MTCK	CLK_OUT3、SUBSPICS1	GPIO39	
33	IO40	I/O/T	\	MTDO	CLK_OUT2	GPIO40	
34	IO41	I/O/T	\	MTDI	CLK_OUT1	GPIO41	
35	IO42	I/O/T	\	MTMS	\	GPIO42	
36	RXD0	I/O/T	\	U0RXD	U0RXD、CLK_OUT2	GPIO44	
37	TXD0	I/O/T	\	U0TXD	U0TXD、CLK_OUT1	GPIO43	
38	IO2	I/O/T	ADC1_CH1	GPIO2	\	TOUCH2、RTC_GPIO2	
39	IO1	I/O/T	ADC1_CH0	GPIO1	\	TOUCH1、RTC_GPIO1	
40	GND	P	\	\	\	\	接地
41	EPAD	P	\	\	\	\	接地

	红色为电源引脚，不可用于其他功能
	黄色为Strapping引脚，表示在芯片上电过程中，需要固定到特定的电平，控制芯片的启动模式，上电后，可正常使用
	蓝色为特殊功能引脚，EN引脚为复位引脚，无法用于其他用途，IO19、IO20为USB-OTG引脚，调试下载程序，不建议用于其他用途
	橙色为内部PSRAM通信引脚，在开启PSRAM后，该引脚不可用于其他用途，不开启PSRAM时，该引脚可正常使用
	绿色引脚为正常IO引脚，可正常使用

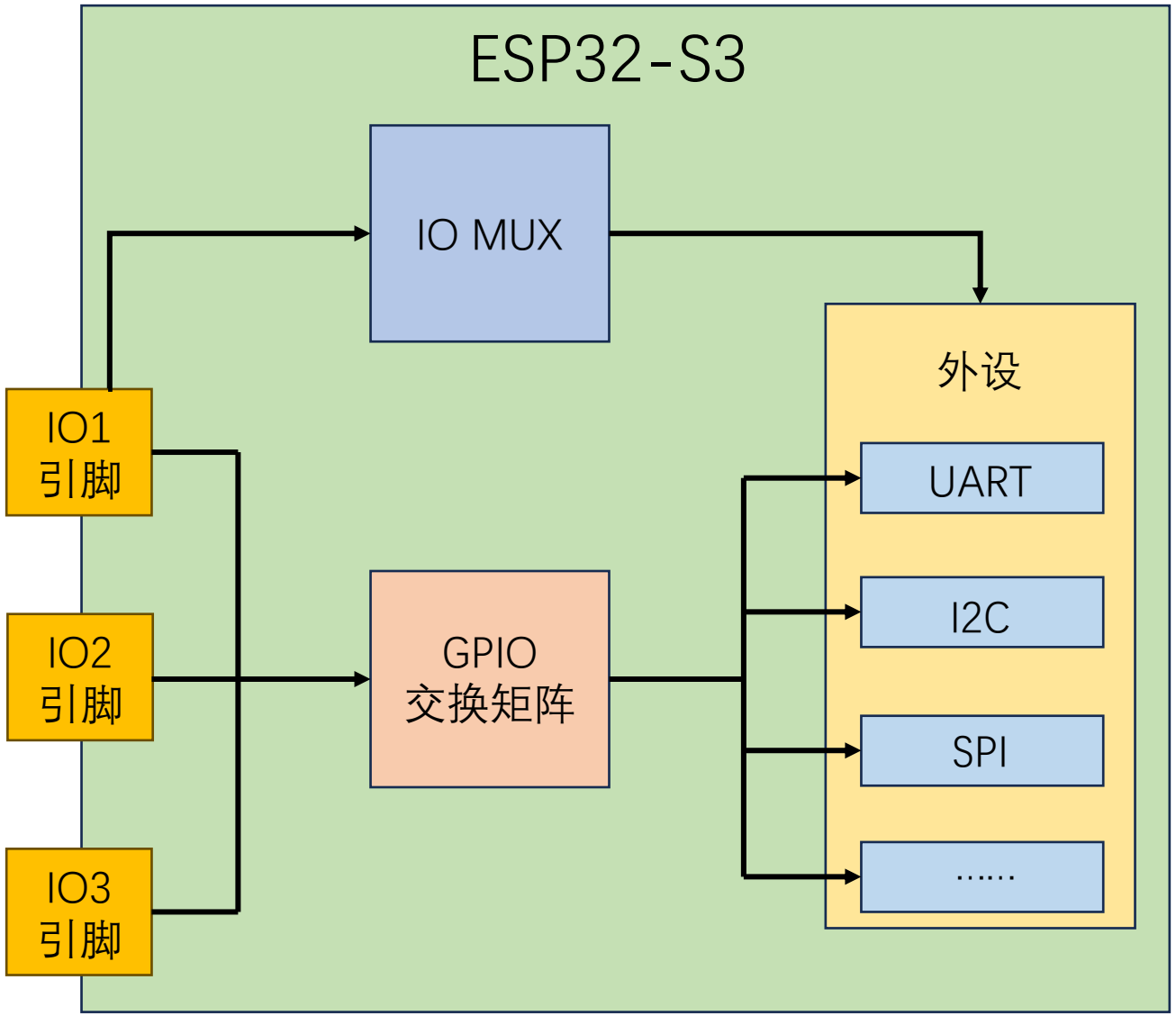
管脚功能	信号	描述
U...TXD	发送数据 (Transmit)	UART0/1 接口
U...RXD	接收数据 (Receive)	
U...RTS	请求发送 (Request to send)	
U...CTS	允许发送 (Clear to send)	
MTCK	测试时钟 (Test clock)	用于调试功能的 JTAG 接口
MTDO	测试数据输出 (Test Data Out)	
MTDI	测试数据输入 (Test Data In)	
MTMS	测试模式选择 (Test Mode Select)	
SPIQ	数据输出 (Data out)	SPIO/1 接口，由 VDD_SPI 供电，通过 SPI 总线连接封装内或封装外 flash/PSRAM，支持单线、双线、四线 SPI 模式。详见章节 2.6 芯片与 Flash/PSRAM 的管脚对应关系
SPID	数据输入 (Data in)	
SPIHD	暂停 (Hold)	
SPIWP	写保护 (Write protect)	
SPICLK	时钟 (Clock)	
SPICS...	片选 (Chip select)	
SPIIO...	数据 (Data)	八线 SPI 模式下 SPIO/1 接口的高 4 位数据线接口及 DQS 接口，由 VDD_SPI 或 VDD3P3_CPU 供电
SPIDQS	数据选通/数据掩码 (Data strobe/data mask)	
SPICLK_N_DIFF	负极性时钟信号 (Negative clock signal)	SPI 总线差分时钟的负极/正极端
SPICLK_P_DIFF	正极性时钟信号 (Positive clock signal)	
SUBSPIQ	数据输出 (Data out)	SPIO/1 接口，由 VDD3P3_RTC 或 VDD3V3_CPU 供电，通过 SUBSPI 总线连接封装内或封装外 flash/PSRAM，支持单线、双线、四线 SPI 模式
SUBSPID	数据输入 (Data in)	
SUBSPIHD	暂停 (Hold)	
SUBSPIWP	写保护 (Write protect)	
SUBSPICLK	时钟 (Clock)	
SUBSPICS...	片选 (Chip select)	
SUBSPICLK_N_DIFF	负极性时钟信号 (Negative clock signal)	SUBSPI 总线差分时钟的负极/正极端
SUBSPICLK_P_DIFF	正极性时钟信号 (Positive clock signal)	
FSPIQ	数据输出 (Data out)	用于快速 SPI 传输的 SPI2 主接口，支持单线、双线、四线 SPI 模式
FSPID	数据输入 (Data in)	
FSPIHD	暂停 (Hold)	
FSPIWP	写保护 (Write protect)	
FSPICLK	时钟 (Clock)	
FSPICS0	片选 (Chip select)	
FSPIIO...	数据 (Data)	八线 SPI 模式下 SPI2 接口的高 4 位数据线接口及 DQS 接口
FSPIDQS	数据选通/数据掩码 (Data strobe/data mask)	
CLK_OUT...	时钟输出 (Clock output)	芯片内部组件生成的输出时钟信号

GPIO交换矩阵


①
GPIO matrix
IO MUX
0 (FUNC)
1 (GPIO)
2 (FUNC)
MCU_SEL
Peripheral Signal Y
0(FUNC)
1(GPIO)
GPIO_SIGy_IN_SEL
sig_in_func[y]
GPIO_FUNCy_IN_INV_SEL
Constant 0 input
Constant 1 input
GPIO0_in
GPIO1_in
GPIO2_in
GPIOX_in
GPIO48_in
GPIO_FUNCy_IN_SEL
175 peripheral inputs
184 peripheral outputs
GPIO_FUNCx_OUT_SEL
GPIO_FUNCX_OUT_INV_SEL
signal0_out
signal1_out
signal2_out
Peripheral Signal Y'
Peripheral Signal Y
signal255_out
GPIO_OUT_DATA_bit_x
GPIOx_out
②
③
④
⑤
GPIO SYNC
GPIO Filter
Pin X supplied by VDD3P3_CPU
Pin X supplied by VDD3P3_RTC
MUX_SEL
GPIO and IO MUX power domain
RTC power domain
RTC GPIO
RTC IO MUX

GPIO交换矩阵

艾谷科技
Aigo



前置小知识

高电平和低电平

日常工程讨论时：

高电平 – 供电电压

低电平 – 0V

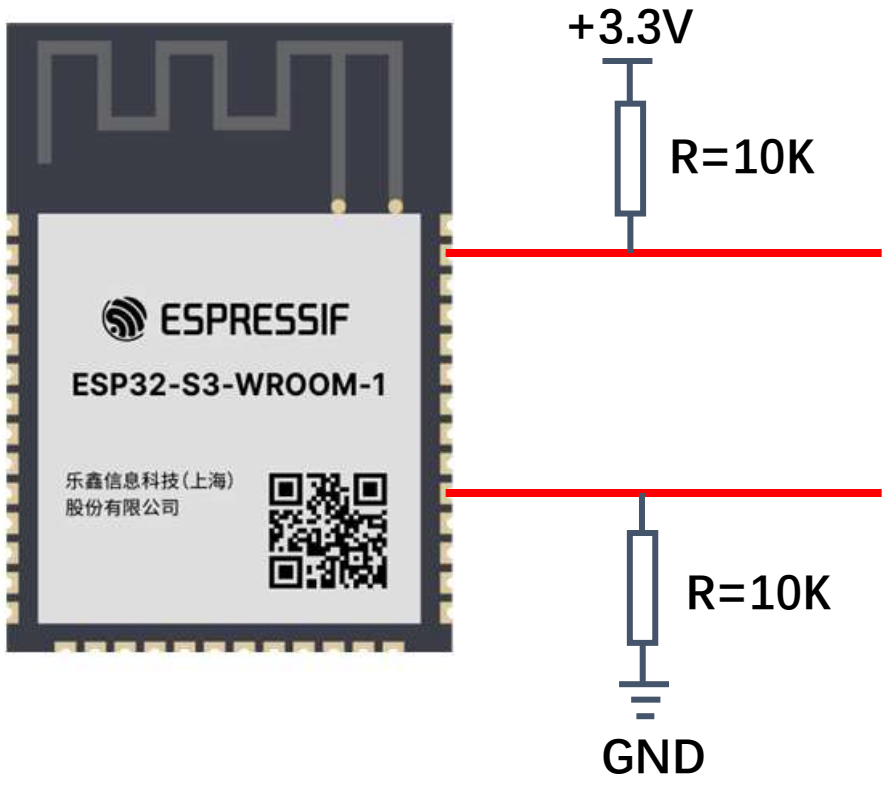
比如：

STM32/ESP32等芯片，使用3.3V电压供电，此时我们说的高电平即为3.3V。

51系列单片机等芯片，使用5V电压供电，此时我们说的高电平即为5V。



IO口的上拉和下拉



前置小知识

艾谷科技
Aigo

USB转串口(UART)通信

芯片通常带串口通信，而电脑都是USB接口，通过USB转串口芯片，可以将USB信号和串口信号相互转化，实现电脑和芯片之间的通信。

如：CH340,CP2102等。







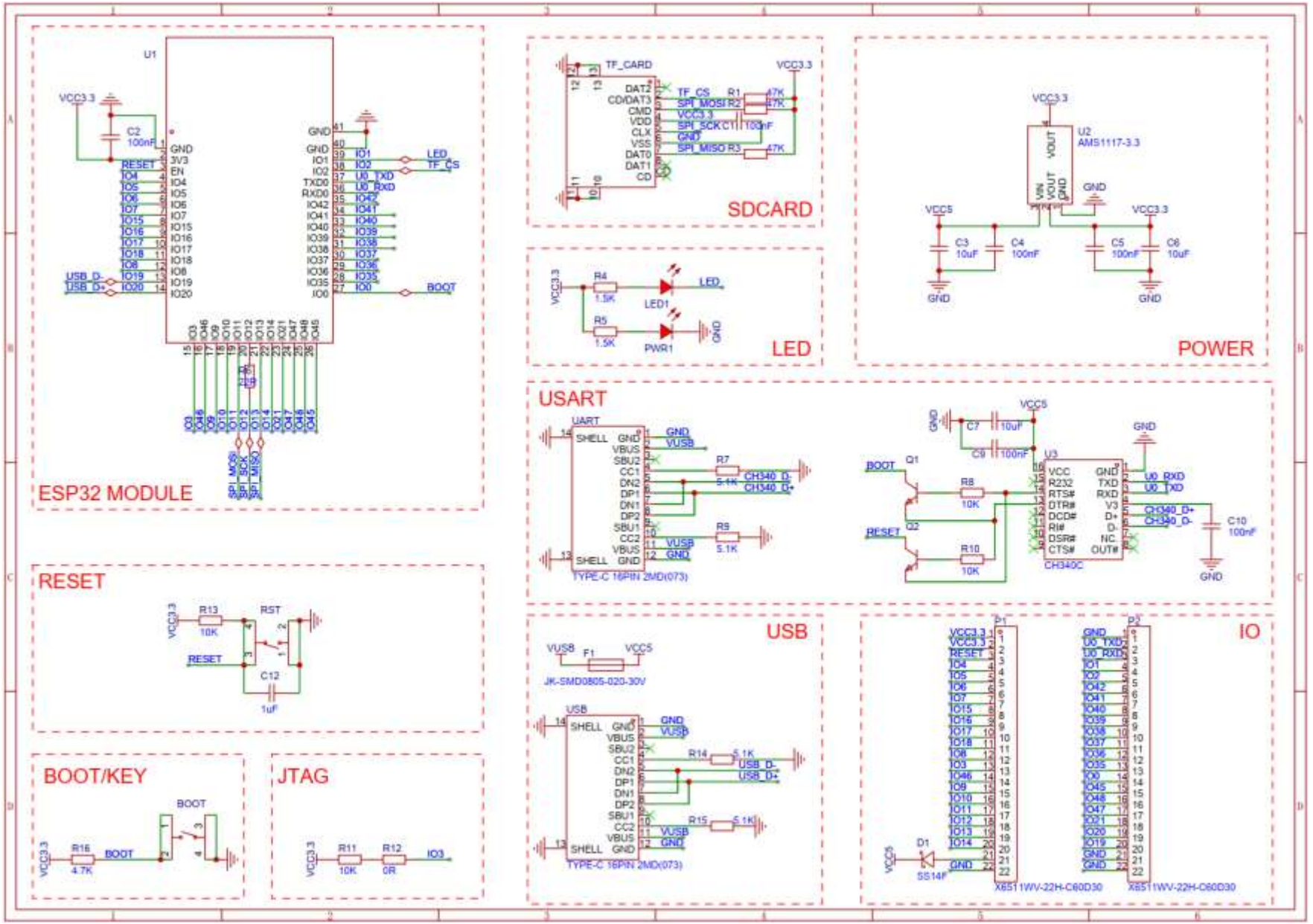






ESP32-S3开发板

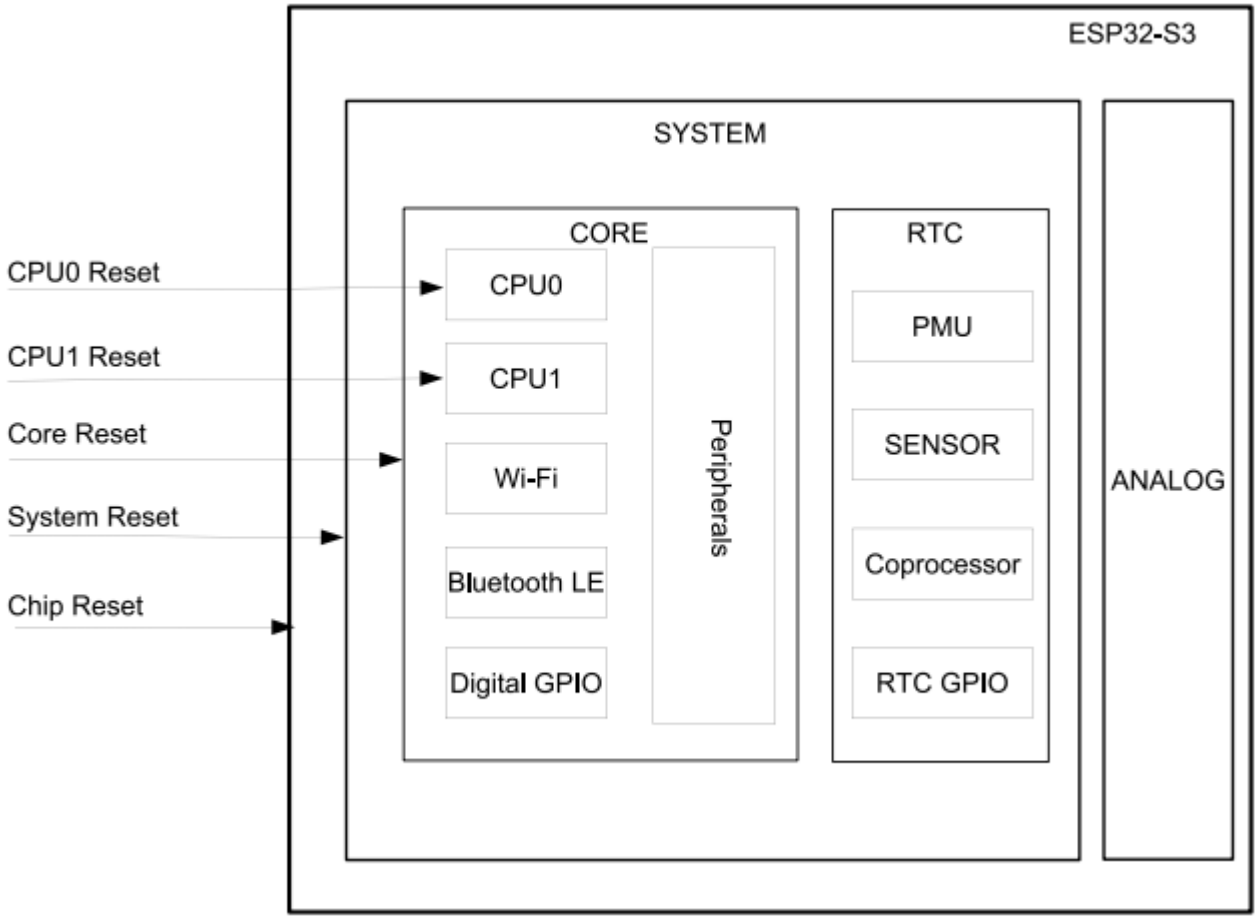
艾谷科技
Aigo



ESP32-S3复位

艾谷科技
Aigo

4种级别复位：CPU复位、内核复位、系统复位、芯片复位



Strapping引脚

Strapping引脚是特殊的引脚，芯片在上电启动时，会先检测Strapping引脚的电平状态，依据其电平状态，进行不同的配置操作。

Strapping 管脚	默认配置	值
GPIO0	弱上拉	1
GPIO3	浮空	-
GPIO45	弱下拉	0
GPIO46	弱下拉	0

芯片模式配置

艾谷科技
Aigo

GPIO0和GPIO46共同决定模组上电时的启动模式

启动模式	GPIO0	GPIO46
SPI Boot	1	任意值
Download Boot	0	0

SPI Boot模式下，芯片正常读取程序并启动
Download Boot模式下，用户可以通过USB或UART0将编译好的二进制文件下载到Flash中。

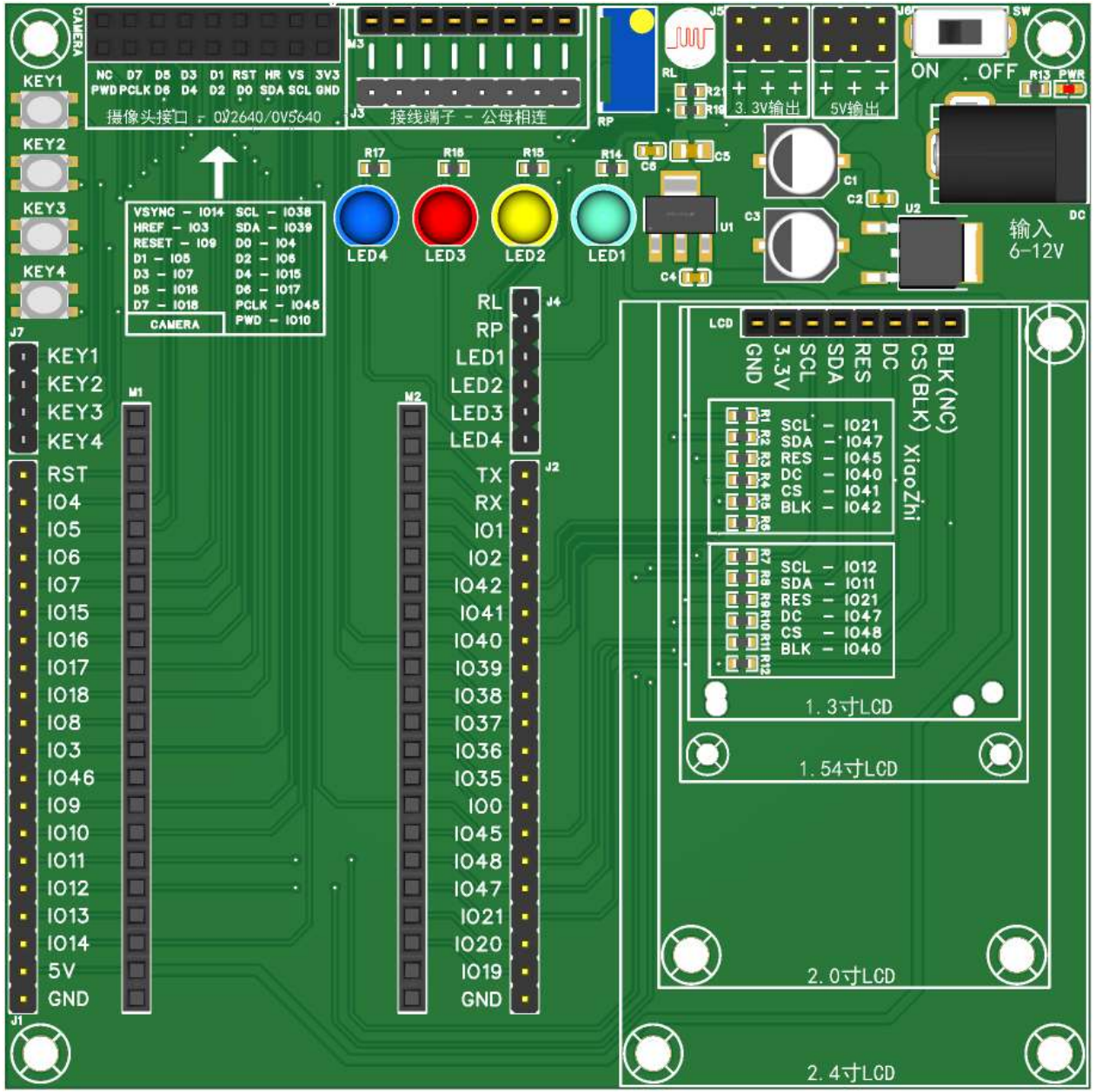
JTAG信号源控制

JTAG接口：是一种国际标准测试协议，主要用于芯片内部测试。
ESP32-S3内部集成JTAG接口，但是需要对JTAG接口的信号源（输入信号）进行选择。
JATG信号源：
正常情况下 – USB串口/内部JTAG控制器
异常情况 – 外部JTAG调试器

JTAG 信号源	EFUSE_DIS_PAD_JTAG	EFUSE_DIS_USB_JTAG	EFUSE_STRAP_JTAG_SEL	GPIO3
USB 串口/JTAG 控制器	0	0	0	忽略
	0	0	1	1
	1	0	忽略	忽略
JTAG 管脚 [2]	0	0	1	0
	0	1	忽略	忽略
JTAG 关闭	1	1	忽略	忽略

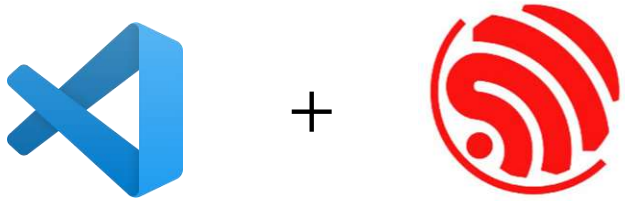
ESP32-S3底板

艾谷科技
Aigo



开发环境搭建

艾谷科技
Aigo



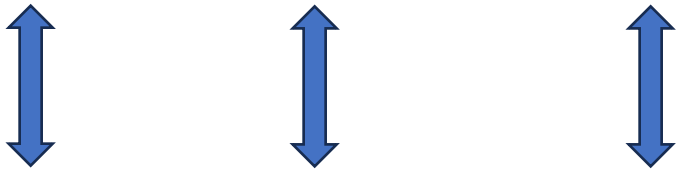
VSCode　　ESP-IDF插件

1.安装ESP-IDF底层配置
2.定义编译程序时所需要的环境变量(IDF_PATH 、IDF_TOOLS_PATH)
3.安装VSCode
4.安装必要的VSCODE插件（CC++插件/CHINESE插件）
5.安装ESP-IDF插件
6.添加ESP-IDF底层配置
7.安装CH340驱动

环境变量

艾谷科技
Aigo

变量类型　变量名称　=　变量值



系统变量:　环境变量　ESP_PATH　=　C:\Espressif\frameworks\esp-idf-v5.1.2



C语言变量:　int　ESP_PATH　=　10

新建工程

1.安装硬件开发平台
2.新建工程：每500ms打印一次HELLO WORLD
3.配置芯片：
    Serial Flasher config – 配置封装外Flash
    ESP PSRAM – 配置封装内PSRAM
    ESP System Setting – ESP32系统配置(主频)
    FreeRTOS – 配置时钟节拍

GPIO外设简介

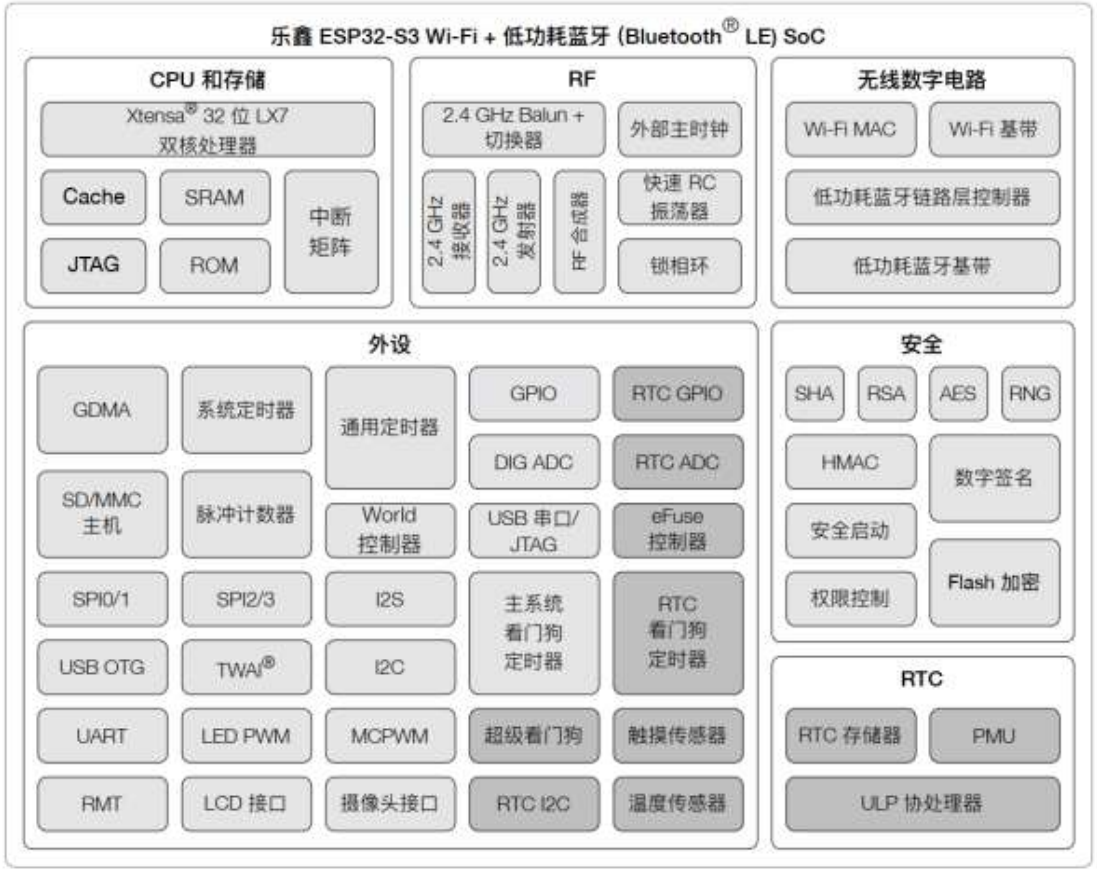
艾谷科技
Aigo

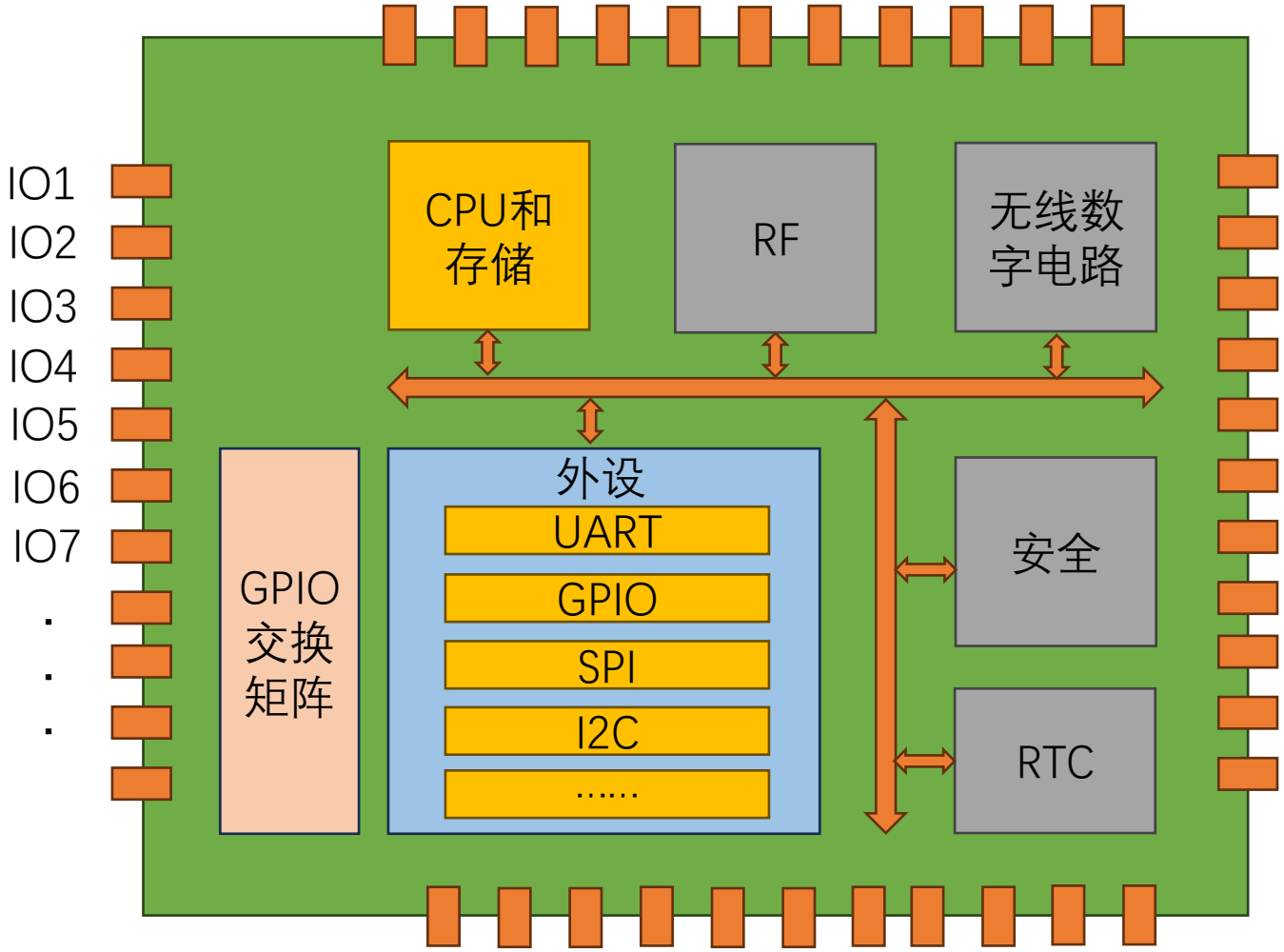
- GPIO（General Purpose Input Output）通用输入输出口
- 可配置成6种输入输出模式
- 引脚电平：0V、3.3V
- 输出模式下可控制端口输出高低电平，用以驱动LED、控制蜂鸣器、模拟通信协议时序等
- 输入模式下可读取端口的高低电平或电压，用于读取按键输入、外接模块电平信号输入、模拟通信协议接收数据等

IO和GPIO

艾谷科技
Aigo







模式配置

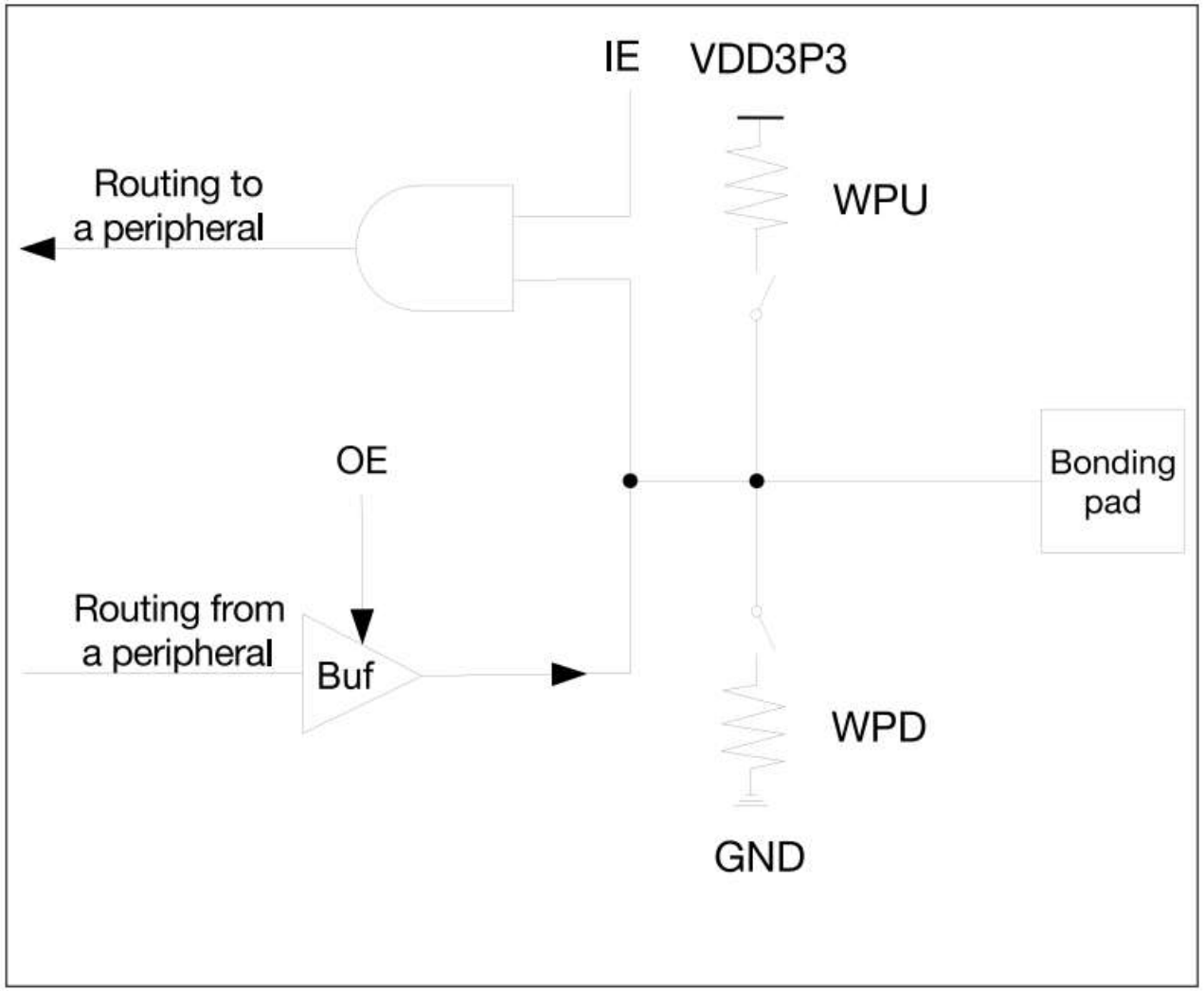
艾谷科技
Aigo

- 可以配置ESP32S3的IO为以下6种模式：

模式名称	特征
关闭输入输出	关闭GPIO的输入输出功能
输入模式	可读取引脚电平
输出模式	可输出引脚电平，高电平：3.3V，低电平：0V
开漏输出	可输出引脚电平，高电平：高阻态，低电平：0V
开漏输入输出	可输出和读取引脚电平，高电平：高阻态，低电平：0V
输入输出	可输出引脚电平和读取引脚电平，高电平：3.3V，低电平：0V

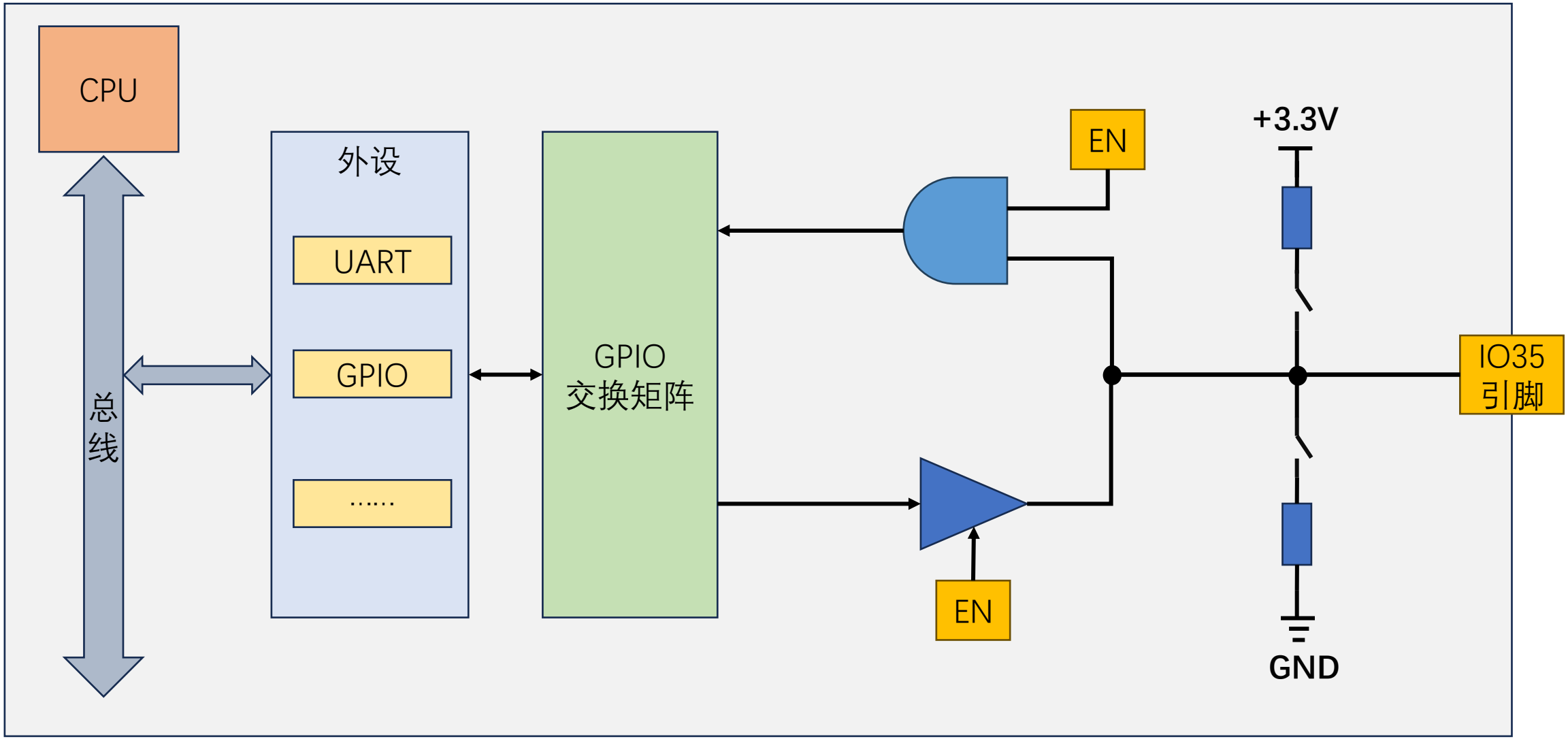
IO引脚内部架构图

艾谷科技
Aigo

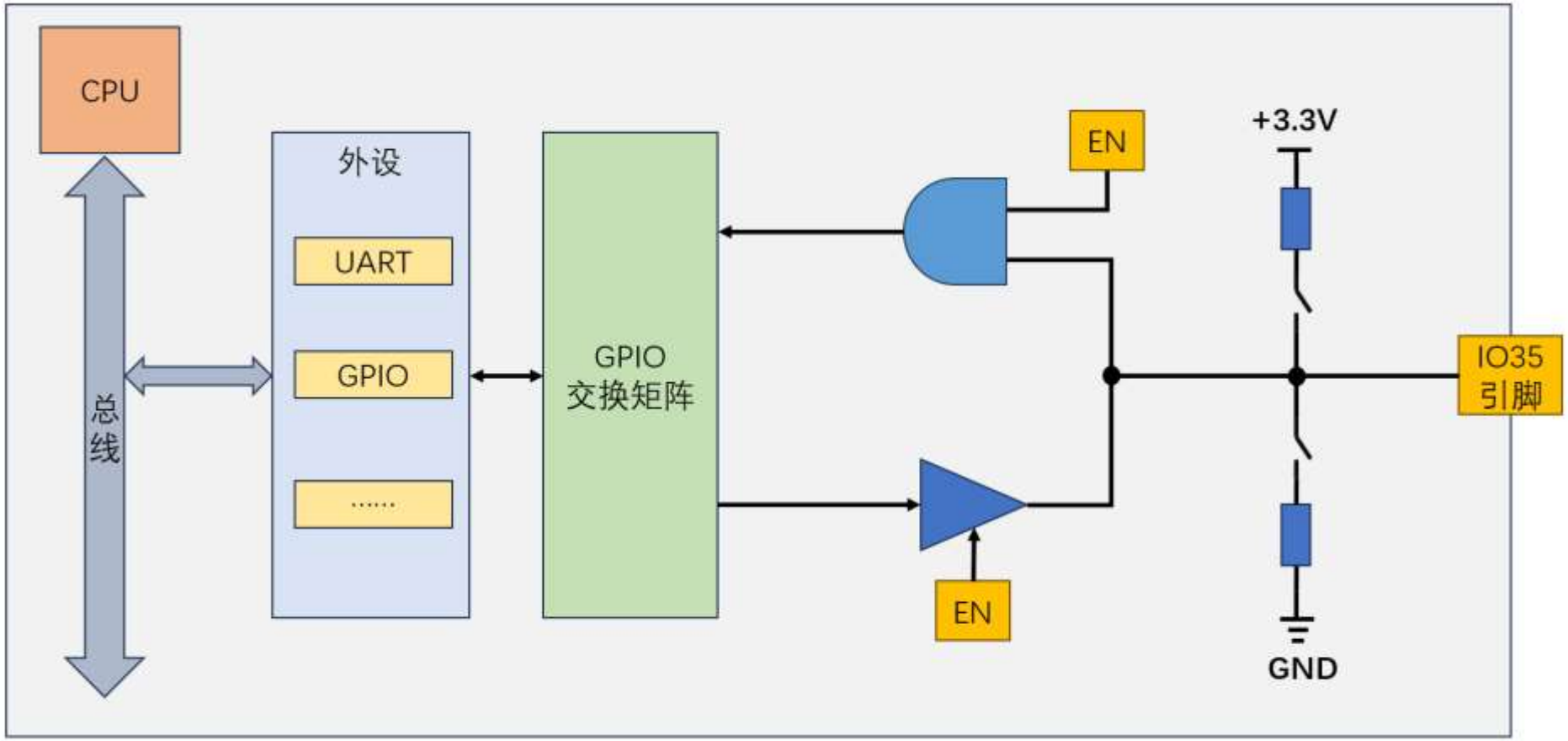


GPIO配置流程

艾谷科技
Aigo



代码控制流程

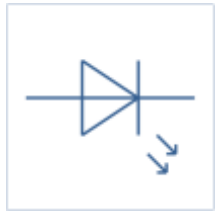


- 使用gpio_config()函数来配置GPIO
- 使用gpio_set_level()函数来控制IO口输出高电平/低电平
- 使用gpio_get_level()函数来读取当前IO口的电平状态

LED简介

艾谷科技
Aigo

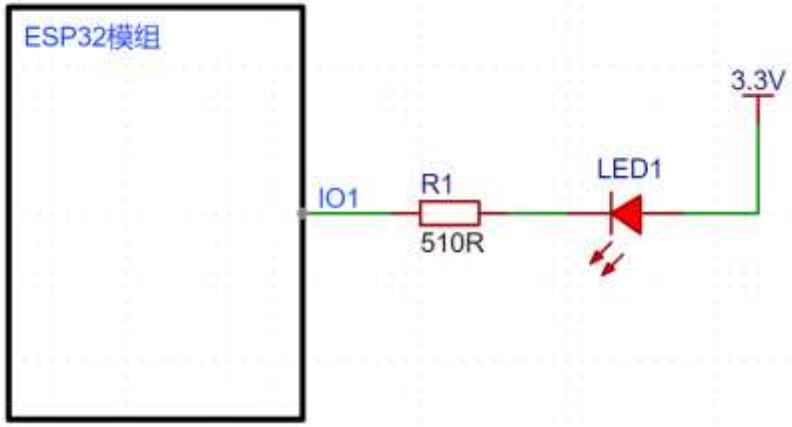




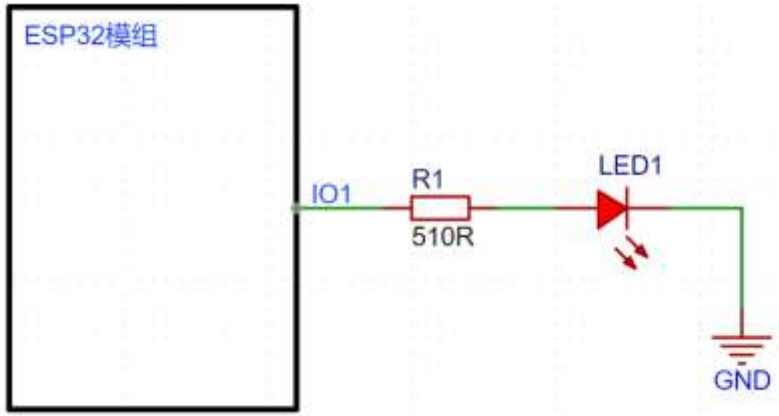
- LED：发光二极管，正向通电点亮，反向通电不亮

LED硬件电路

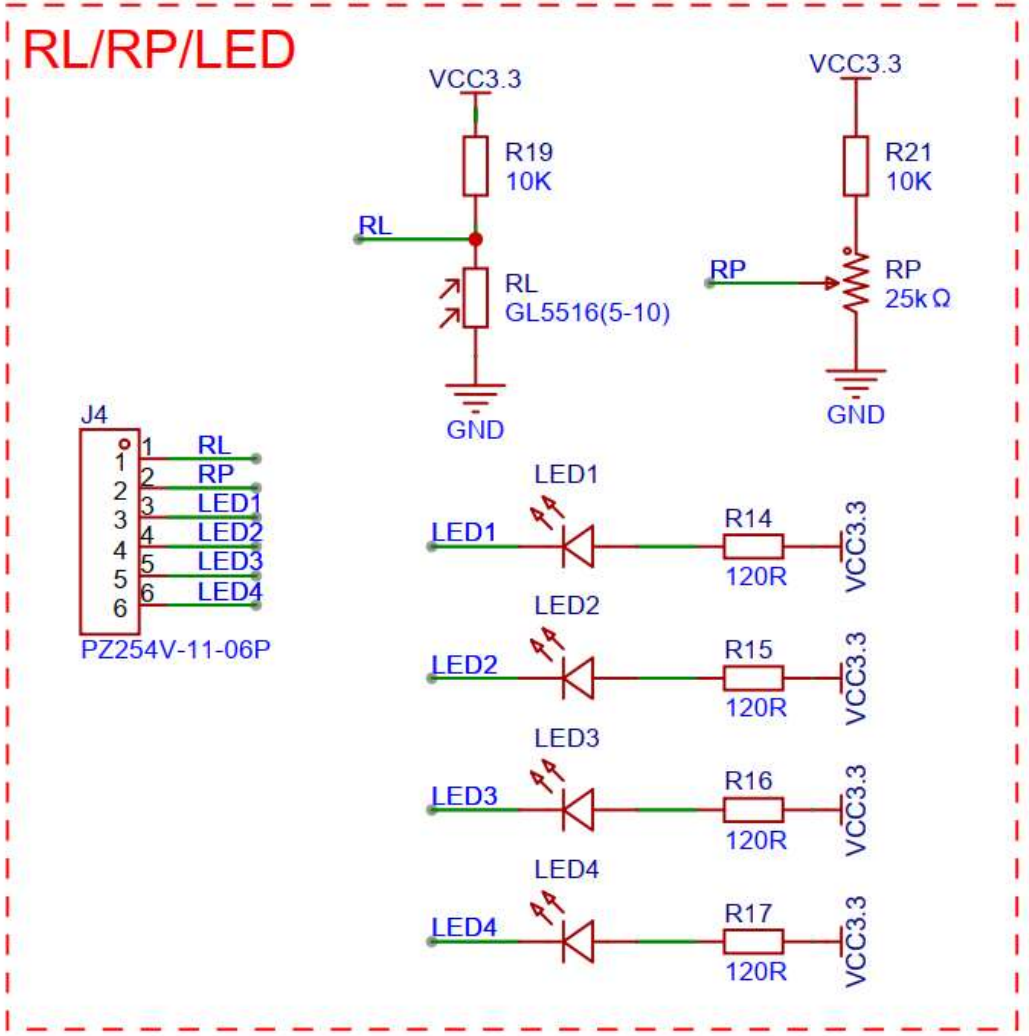
艾谷科技
Aigo



低电平：点亮
高电平：熄灭



高电平：点亮
低电平：熄灭



C语言数据类型

艾谷科技
Aigo

关键字	位数	表示范围	stdint关键字
char	8	-128 ~ 127	int8_t
unsigned char	8	0 ~ 255	uint8_t
short	16	-32768 ~ 32767	int16_t
unsigned short	16	0 ~ 65535	uint16_t
int	32	-2147483648 ~ 2147483647	int32_t
unsigned int	32	0 ~ 4294967295	uint32_t
long long	64	-(2^64)/2 ~ (2^64)/2-1	int64_t
unsigned long long	64	0 ~ (2^64)-1	uint64_t
float	32	-3.4e38 ~ 3.4e38	
double	64	-1.7e308 ~ 1.7e308	

结构体初始化

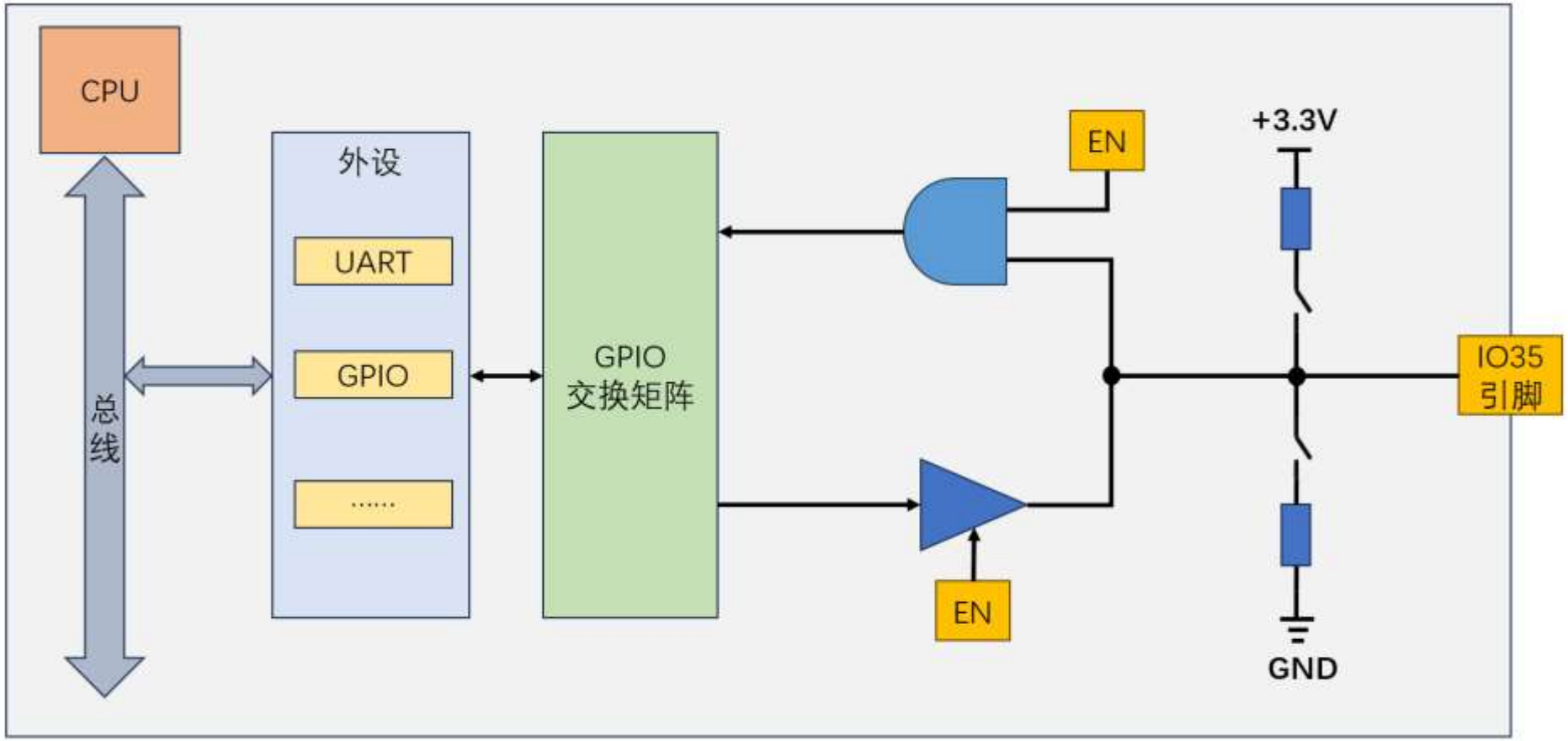
艾谷科技
Aigo

```c
typedef struct {
    uint64_t pin_bit_mask;
    gpio_mode_t mode;
    gpio_pullup_t pull_up_en;
    gpio_pulldown_t pull_down_en;
    gpio_int_type_t intr_type;
} gpio_config_t;
```

```c
gpio_config_t gpio_structure = {
    .intr_type    = GPIO_INTR_DISABLE,
    .mode         = GPIO_MODE_OUTPUT,
    .pin_bit_mask = 1ull << GPIO_NUM_35,
    .pull_down_en = GPIO_PULLDOWN_DISABLE,
    .pull_up_en   = GPIO_PULLUP_ENABLE,
};
```

```c
gpio_config_t gpio_structure = {0};
gpio_structure.intr_type    = GPIO_INTR_DISABLE;
gpio_structure.mode         = GPIO_MODE_OUTPUT;
gpio_structure.pin_bit_mask = 1ull << GPIO_NUM_35;
gpio_structure.pull_down_en = GPIO_PULLDOWN_DISABLE;
gpio_structure.pull_up_en   = GPIO_PULLUP_ENABLE;
```

代码控制流程

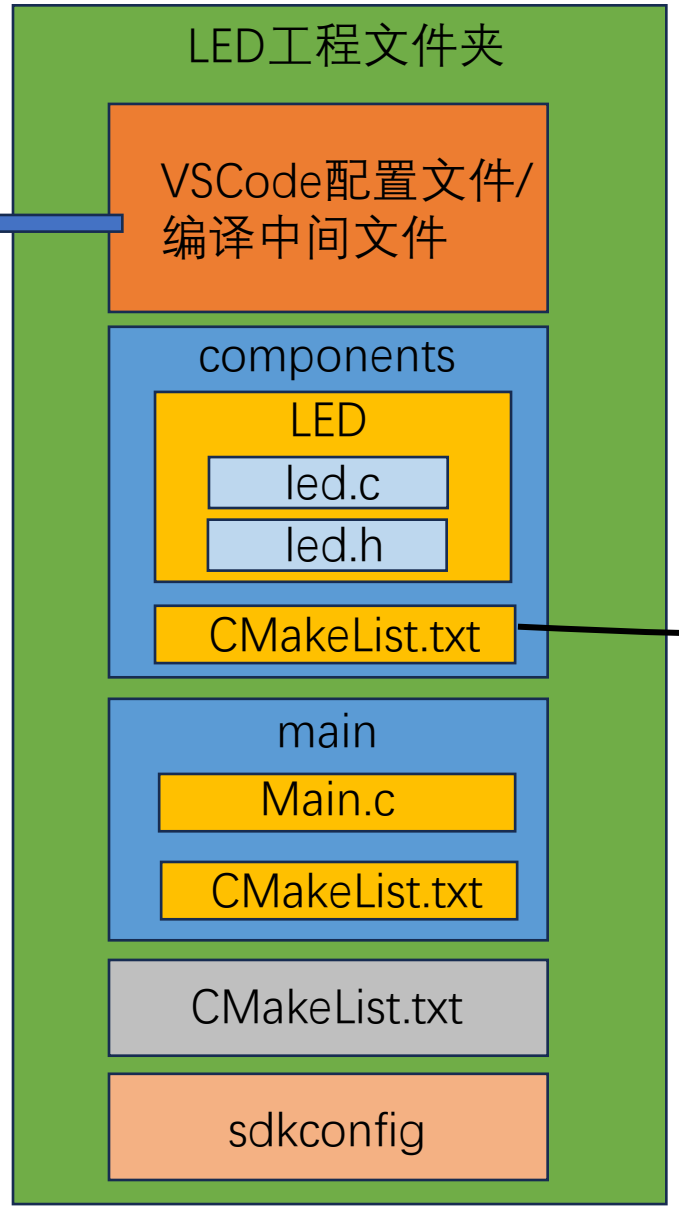


- 使用gpio_config()函数来配置GPIO
- 使用gpio_set_level()函数来控制IO口输出高电平/低电平
- 使用gpio_get_level()函数来读取当前IO口的电平状态

工程架构

艾谷科技
Aigo

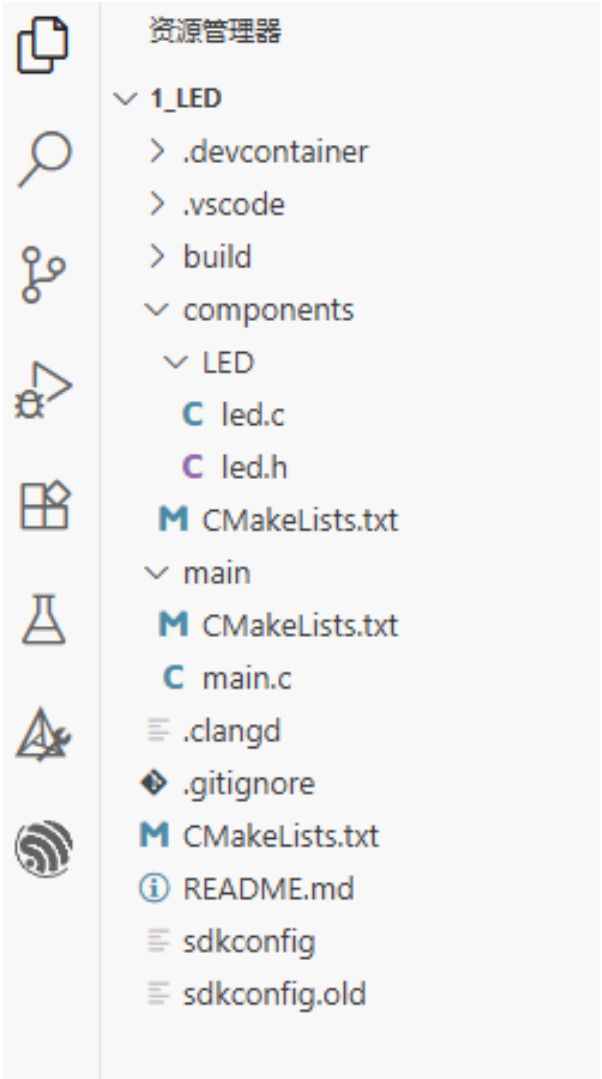
.devcontainer
.vscode
Build
.clangd
.gitignore
README.md



源文件：LED文件夹

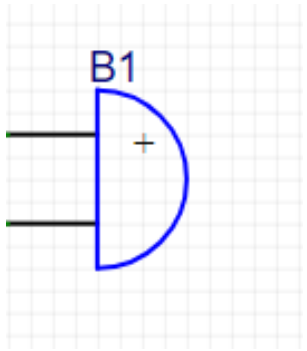
头文件：LED文件夹

依赖官方文件：driver



蜂鸣器简介





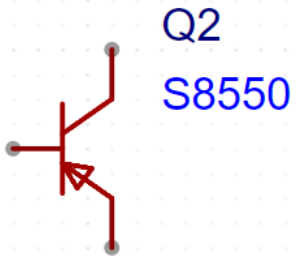


- 有源蜂鸣器：内部自带振荡源，将正负极接上直流电压即可持续发声，频率固定
- 无源蜂鸣器：内部不带振荡源，需要控制器提供振荡脉冲才可发声，调整提供振荡脉冲的频率，可发出不同频率的声音

蜂鸣器硬件电路

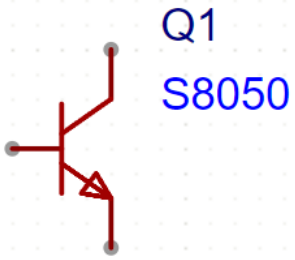
艾谷科技
Aigo

PNP型

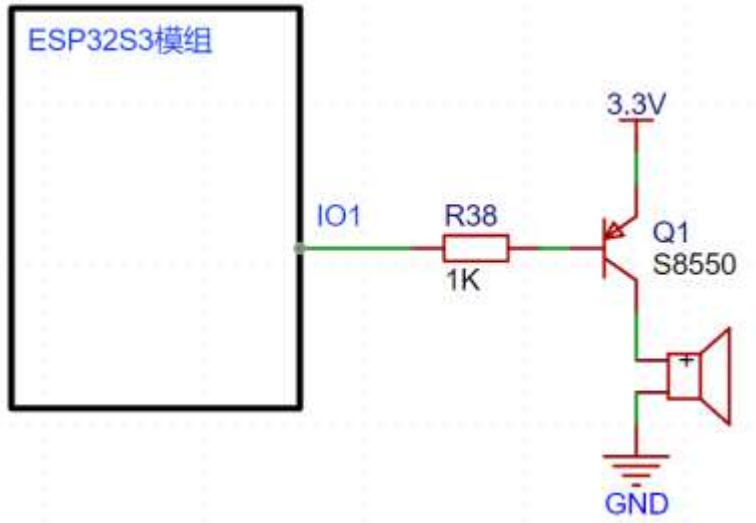


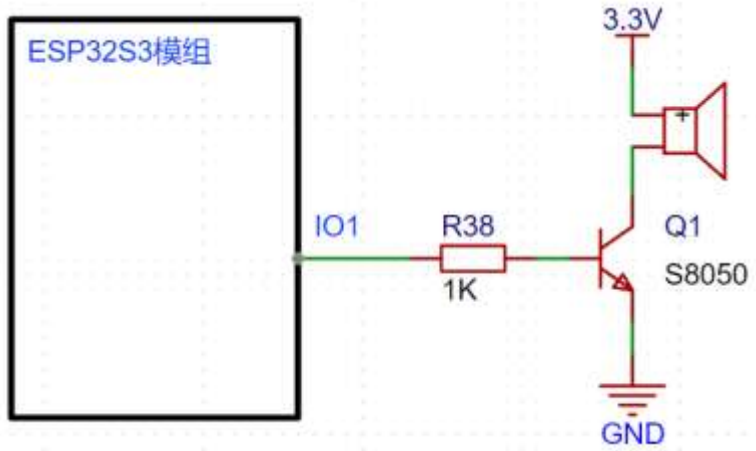
低电平：导通
高电平：截至

NPN型



高电平：导通
低电平：截至





按键简介

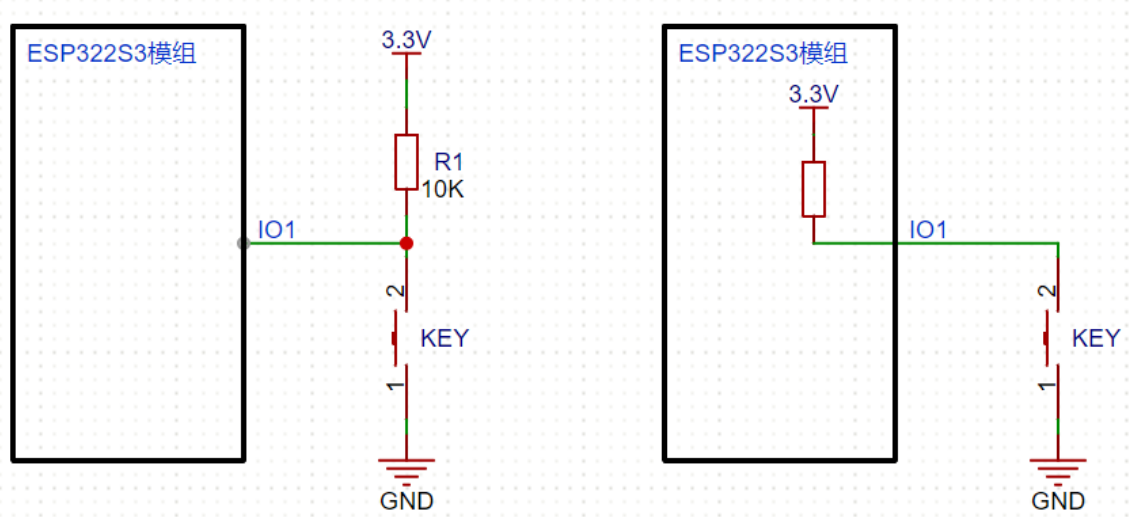
- 按键：常见的输入设备，按下导通，松手断开



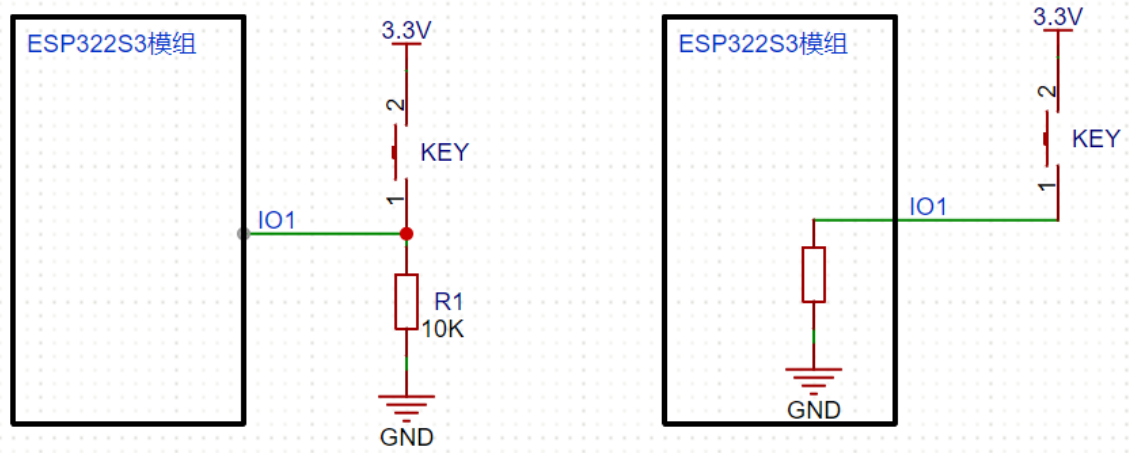
按键硬件电路

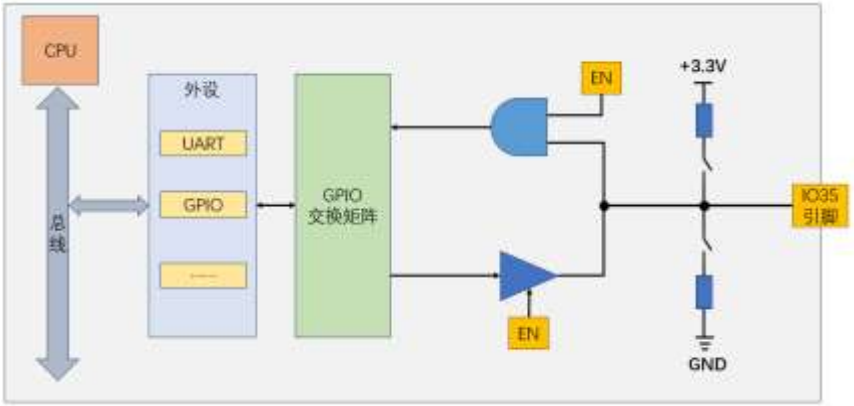
艾谷科技
Aigo

没按：高电平
按下：低电平



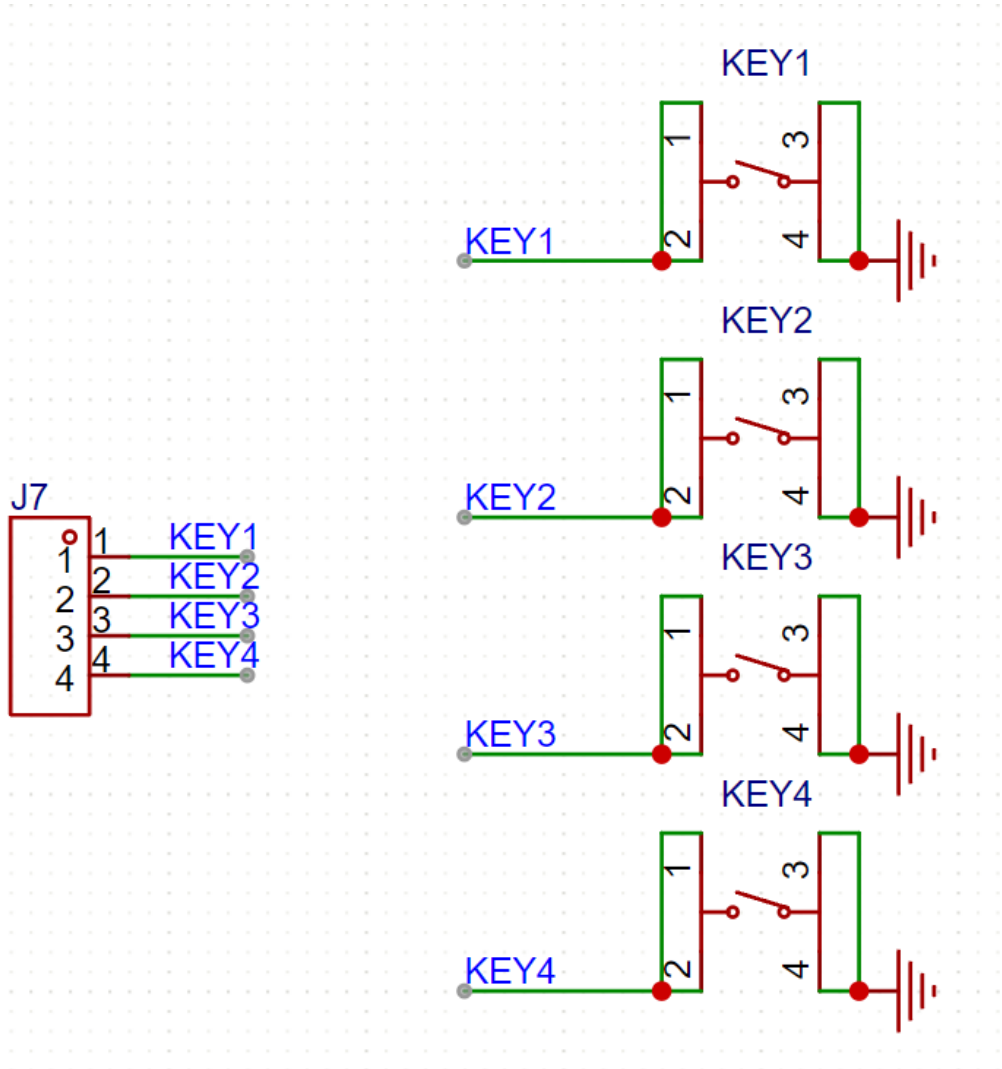
没按：低电平
按下：高电平





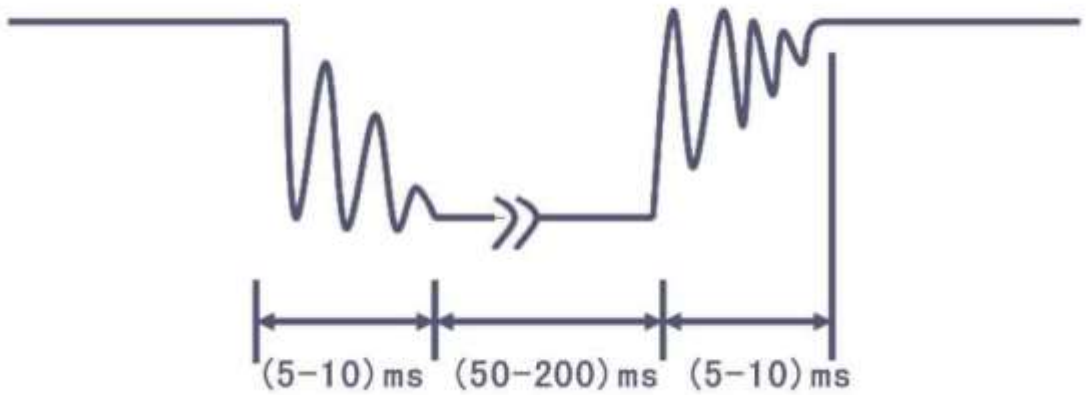
底板按键电路图

艾谷科技
Aigo



按键抖动

按键抖动：由于按键内部使用的是机械式弹簧片来进行通断的，所以在按下和松手的瞬间会伴随有一连串的抖动



Freertos多任务实验

艾谷科技
Aigo

项目要求:

1. LED1以100ms的频率闪烁（LED1连接IO38引脚）
2. 按键1控制LED2的亮灭（按键1连接IO9引脚、LED2连接IO39引脚）
3. 程序运行不能有任何卡顿。

实现方式

ESPIDF的开发框架，自带FreeRTOS库，可以直接使用FreeRTOS的多任务功能，实现多个while循环的“并行”运行。

```
while (1)
{
    key_num = key_scan();
    if(key_num == 1)
    {
        gpio_toggle(GPIO_NUM_39);
    }
}
```

```
while (1)
{
    gpio_toggle(GPIO_NUM_38);
    vTaskDelay(100);
}
```

FreeRTOS的作用

使用FreeRTOS的多任务调度功能，实现了裸机程序不容易实现的功能，优化了代码。

而ESPIDF框架虽然集成了FreeRTOS库，但它和ESP32S3芯片本身并没有绑定关系。

初学阶段代码功能单一，逻辑并不复杂，使用裸机写代码完全能够完成。当学会ESP32S3的所有功能之后，开始做大型项目，随着项目功能越来越复杂，代码越来越多之后，裸机无法完成，开始考虑使用FreeRTOS来优化我们的代码，让代码更加流畅。

中断系统

- 中断：在主程序运行过程中，出现了特定的中断触发条件（中断源），使得CPU暂停当前正在运行的程序，转而去处理中断程序，处理完成后又返回原来被暂停的位置继续运行

- 中断优先级：当有多个中断源同时申请中断时，优先响应更加紧急的中断源

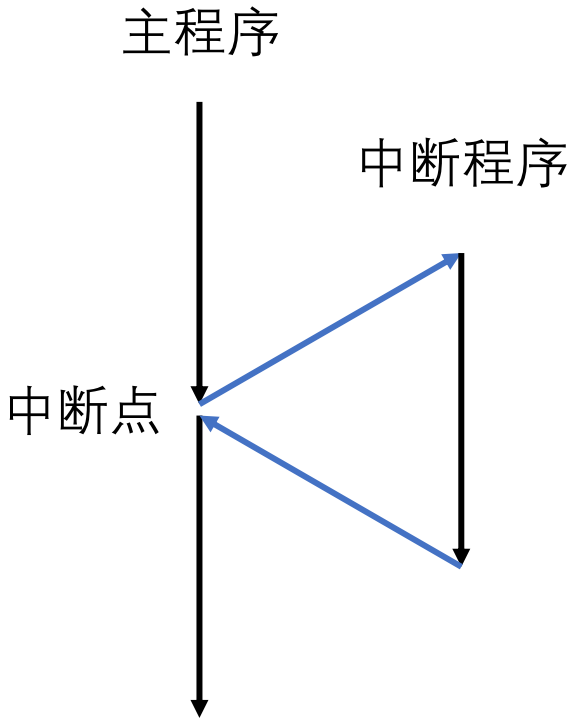
- 中断嵌套：当一个中断程序正在运行时，又有新的更高优先级的中断源申请中断，CPU再次暂停当前中断程序，转而去处理新的中断程序，处理完成后依次进行返回

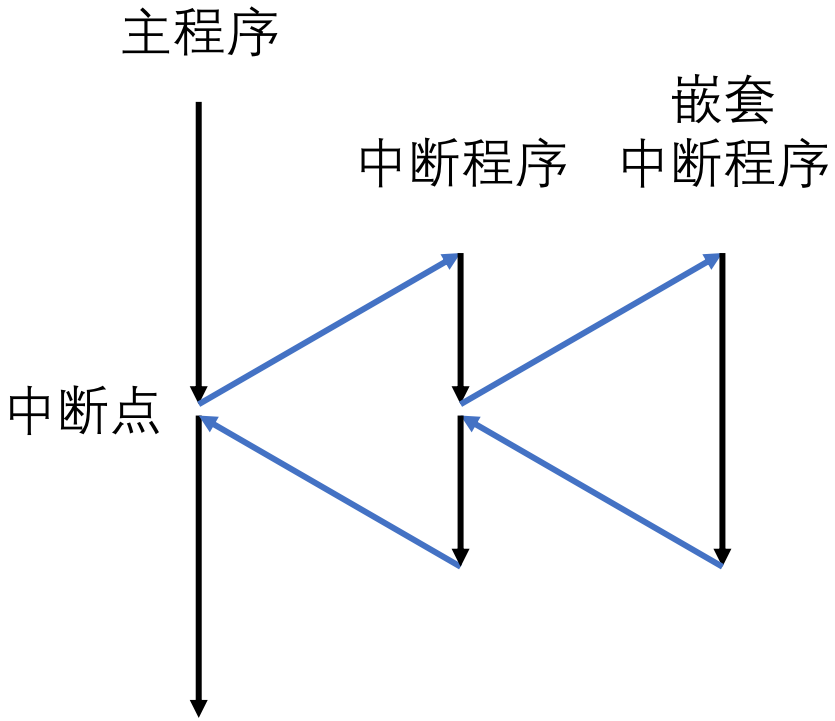
中断执行流程

艾谷科技
Aigo

```c
void app_main(void)
{
    while (1)
    {
        /* 主程序 */
    }
}

void isr_handler(void *param)
{
    /* 中断程序 */
}
```





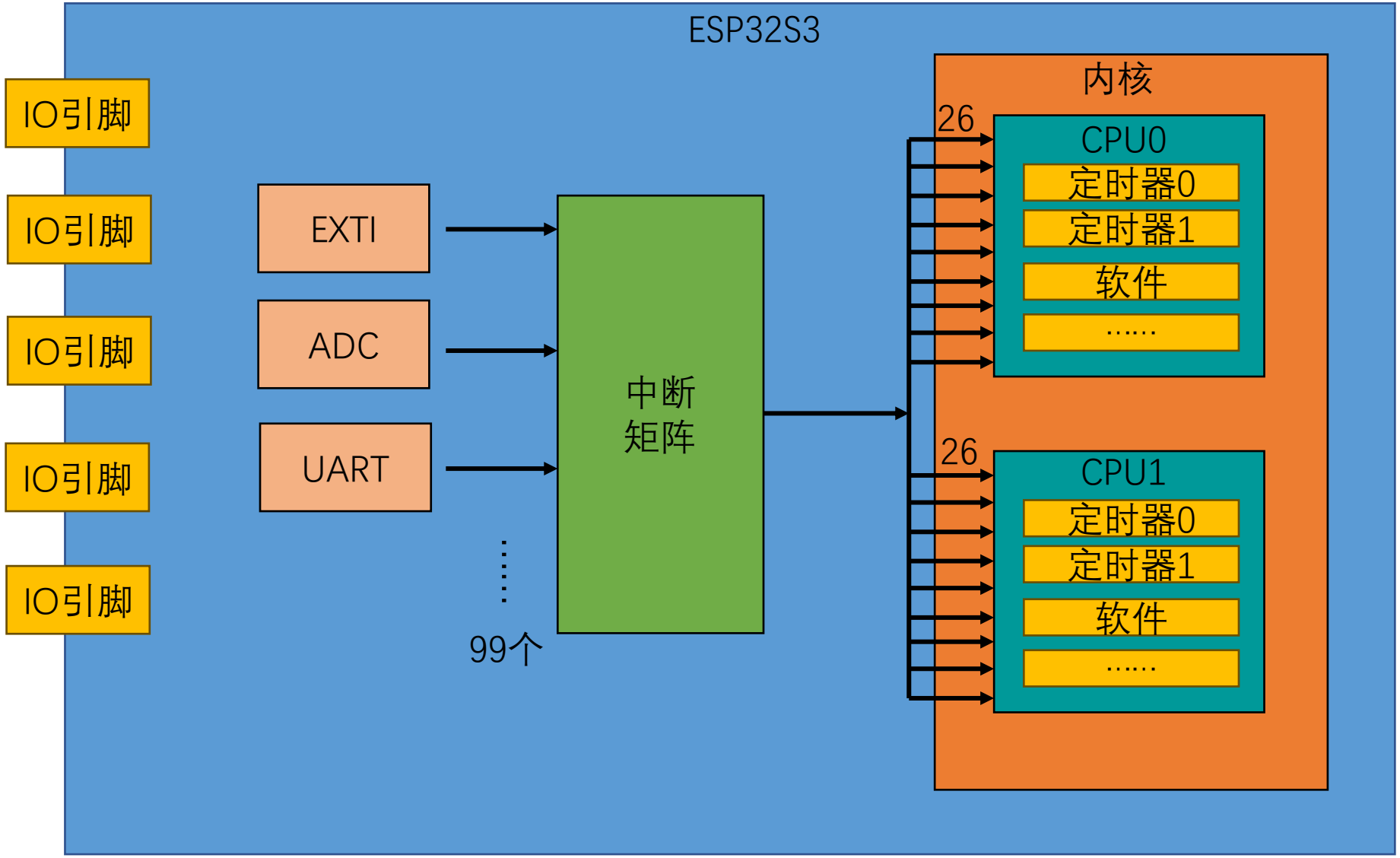
ESP32S3中断

- 99个外部中断源，包含EXTI、ADC、UART、SPI等多个外设
- 而每个CPU只有32个中断通道，因此使用“中断矩阵”管理中断源和CPU的连接关系。

No.	中断源
0	MAC_INTR
1	MAC_NMI
2	PWR_INTR
3	BB_INT
4	BT_MAC_INT
5	BT_BB_INT
6	BT_BB_NMI
7	RWBT_IRQ
8	RWBLE_IRQ
9	RWBT_NMI
10	RWBLE_NMI
11	I2C_MST_INT
12	保留
13	保留
14	UHCI0_INTR
15	保留
16	GPIO_INTERRUPT_CPU
17	GPIO_INTERRUPT_CPU_NMI
18	保留
19	保留
20	SPI_INTR_1
21	SPI_INTR_2
22	SPI_INTR_3
23	保留
24	LCD_CAM_INT
25	I2S0_INT
26	I2S1_INT
27	UART_INTR
28	UART1_INTR
29	UART2_INTR
30	SDIO_HOST_INTERRUPT
31	PWM0_INTR
32	PWM1_INTR
33	保留
34	保留

No.	中断源
35	LEDC_INT
36	EFUSE_INT
37	TWAI_INT
38	USB_INTR
39	RTC_CORE_INTR
40	RMT_INTR
41	PCNT_INTR
42	I2C_EXT0_INTR
43	I2C_EXT1_INTR
44	保留
45	保留
46	保留
47	保留
48	保留
49	保留
50	TG_T0_INT
51	TG_T1_INT
52	TG_WDT_INT
53	TG1_T0_INT
54	TG1_T1_INT
55	TG1_WDT_INT
56	CACHE_IA_INT
57	SYSTIMER_TARGET0_INT
58	SYSTIMER_TARGET1_INT
59	SYSTIMER_TARGET2_INT
60	SPI_MEM_REJECT_INTR
61	DCACHE_PRELOAD_INT
62	ICACHE_PRELOAD_INT
63	DCACHE_SYNC_INT
64	ICACHE_SYNC_INT
65	APB_ADC_INT
66	DMA_IN_CH0_INT
67	DMA_IN_CH1_INT
68	DMA_IN_CH2_INT
69	DMA_IN_CH3_INT
70	DMA_IN_CH4_INT
71	DMA_OUT_CH0_INT

中断系统


ESP32S3
IO引脚
IO引脚
IO引脚
IO引脚
IO引脚
EXTI
ADC
UART
99个
中断
矩阵
内核
26
CPU0
定时器0
定时器1
软件
……
26
CPU1
定时器0
定时器1
软件
……

EXTI外部中断简介

艾谷科技
Aigo

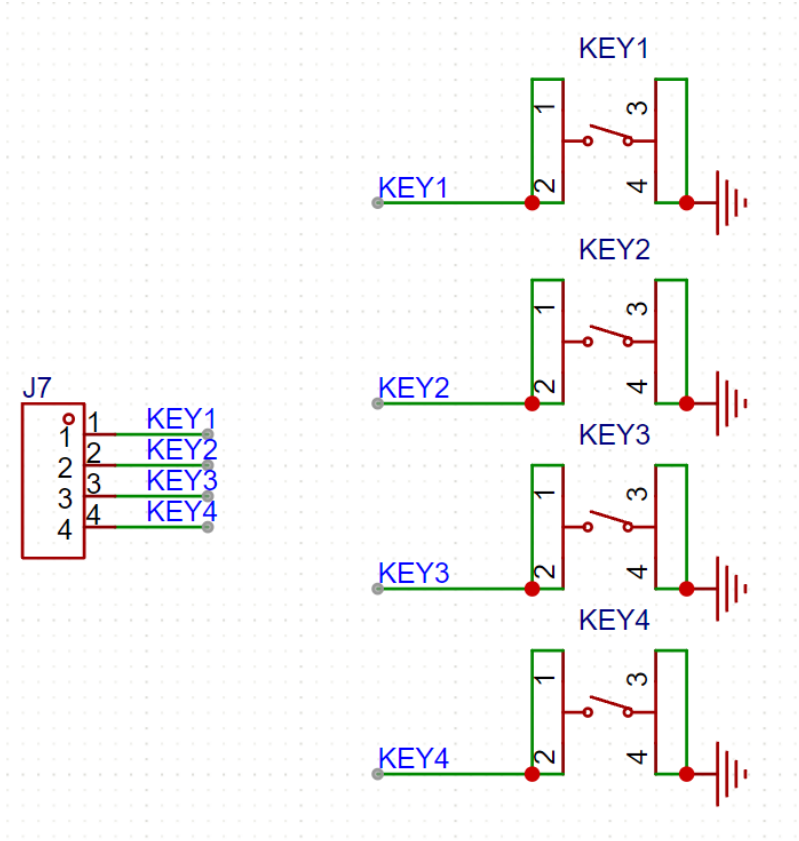
- EXTI（Extern Interrupt）外部中断
- EXTI可以监测指定IO口的电平信号，当其指定的IO口产生电平变化时，EXTI将立即向中断矩阵发出中断申请，经过中断矩阵裁决后即可中断CPU主程序，使CPU执行EXTI对应的中断程序
- 支持的触发方式：上升沿/下降沿/双边沿/高电平/低电平
- 支持的IO口：所有可正常配置的IO口

```
{
    uint8_t key_num = 0 ;
    while(1)
    {
        key_num = key_scan();
         if(key_num == 1)
         {
             gpio_toggle(GPIO_NUM_39);
         }
         vTaskDelay(5);
    }
}
```

触发方式

艾谷科技
Aigo

- 使用按键，进行下降沿触发



EXTI外设配置过程

艾谷科技
Aigo

- 使用 “gpio_config” 函数配置GPIO
- 使用 “gpio_install_isr_service” 函数注册中断
- 使用 “gpio_isr_handler_add” 函数将IO口与中断服务函数匹配
- 使用 “gpio_intr_enable” 函数开启中断

调试方式

- 串口调试：通过串口通信，将调试信息发送到电脑端，电脑使用串口监视器显示调试信息（printf、ESP_LOGx）

- 显示屏调试：直接将显示屏连接到开发板，将调试信息打印在显示屏上

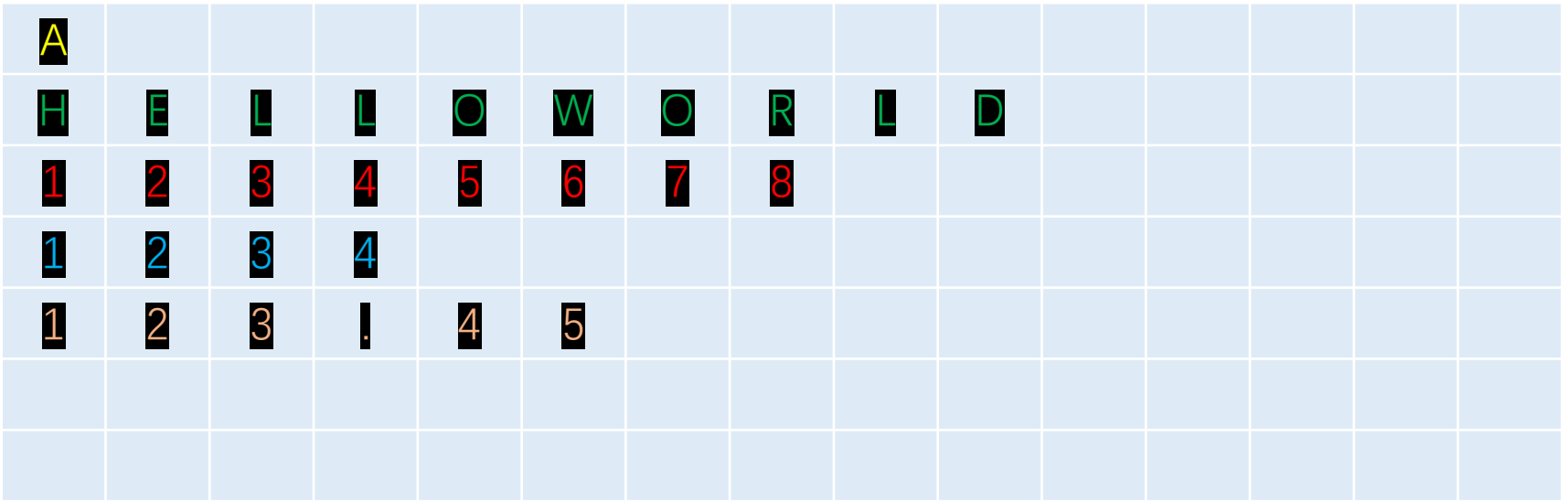
显示方式

艾谷科技
Aigo



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1															
2															
3															
4															
5															
6															
7															

LCD驱动函数



函数	功能
lcd_init ()	初始化
lcd_clear(BLACK)	清屏
lcd_show_char(1,1,'A',YELLOW,BLACK)	显示一个字符
lcd_show_string(2,1,"HELLOWORLD",GREEN,BLACK)	显示字符串
lcd_show_num(3,1,12345678,8,RED,BLACK)	显示十进制数字
lcd_show_hexnum(4,1,0x1234,4,BLUE,BLACK)	显示十六进制数字
lcd_show_float(5,1,123.45,5,BROWN,BLACK)	显示浮点数

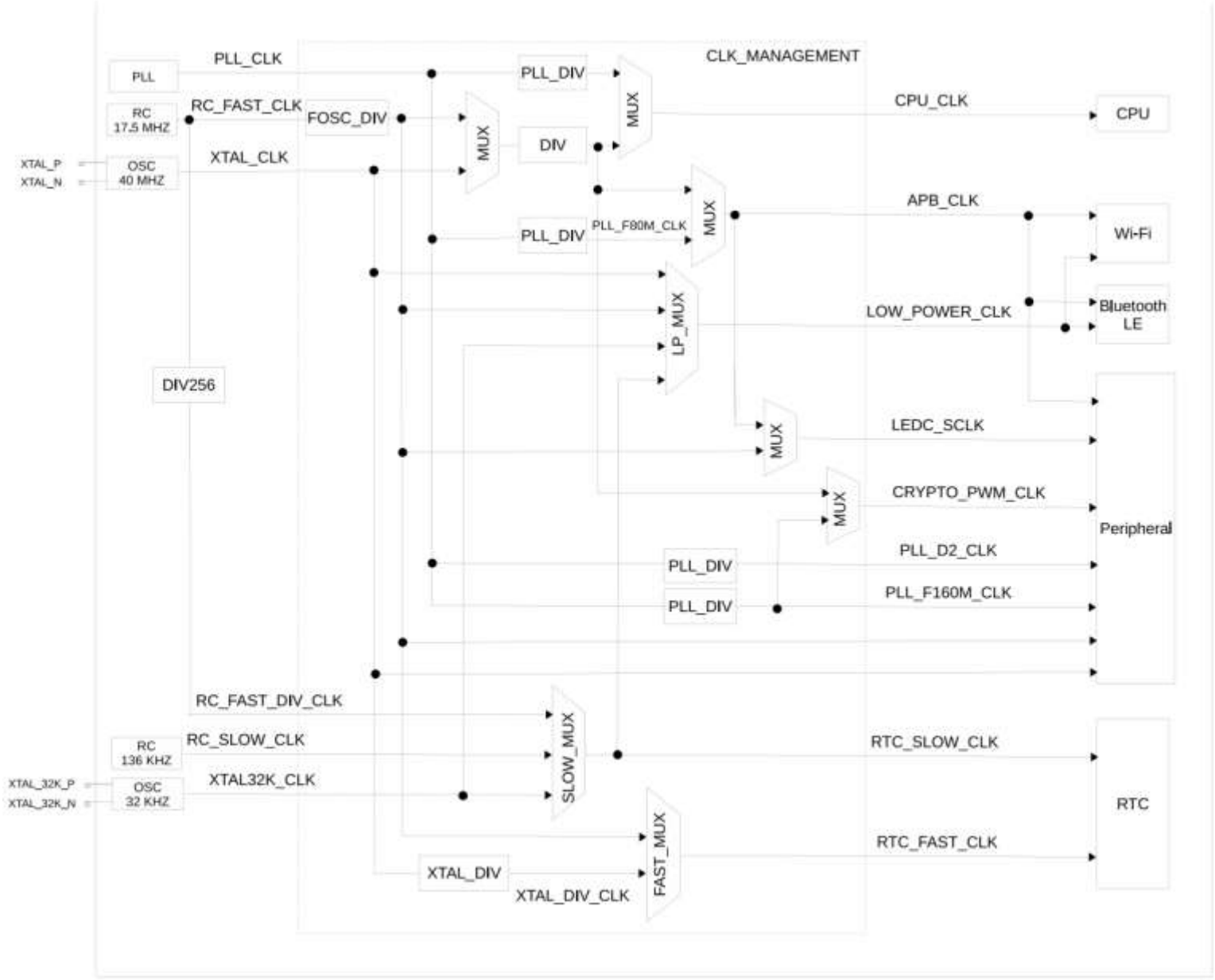
ESP32-S3时钟系统

艾谷科技
Aigo

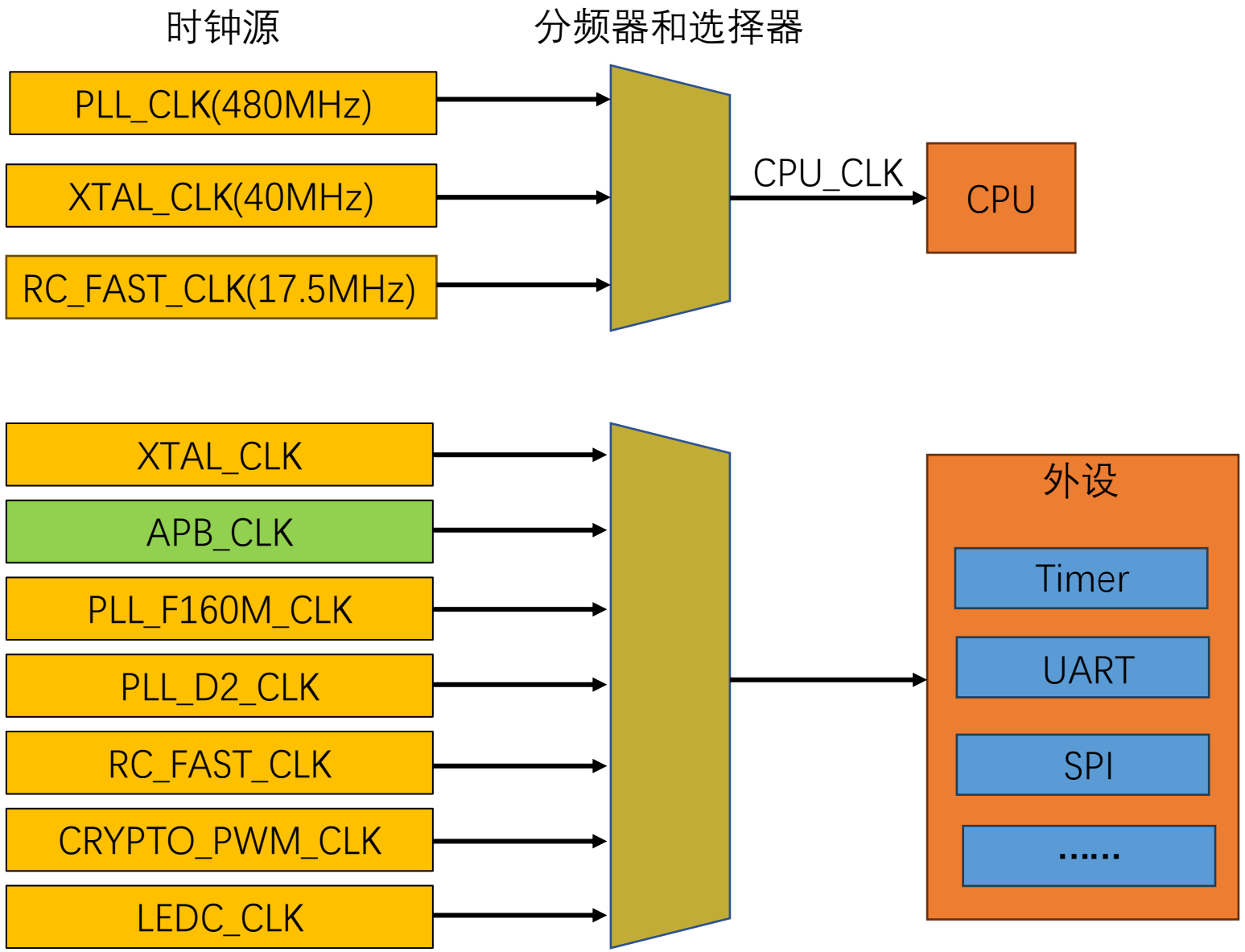
- ESP32-S3的时钟主要用来给CPU、外设或其他功能电路提供脉冲，在芯片内部，所有功能模块的工作均按照时钟节拍进行。相当于芯片的“心脏”。
- ESP32-S3的时钟主要来源于振荡器、RC震荡电路和PLL时钟生成电路。产生的时钟，经过各种分频器、选择器处理之后，使得大部分功能模块可以根据不同功耗和性能需求来获取对应频率的工作时钟。

ESP32-S3时钟系统

艾谷科技
Aigo



CPU和外设时钟


时钟源
分频器和选择器
PLL_CLK(480MHz)
XTAL_CLK(40MHz)
RC_FAST_CLK(17.5MHz)
CPU_CLK
CPU
XTAL_CLK
APB_CLK
PLL_F160M_CLK
PLL_D2_CLK
RC_FAST_CLK
CRYPTO_PWM_CLK
LEDC_CLK
外设
Timer
UART
SPI
.....

外设时钟选择

艾谷科技
Aigo

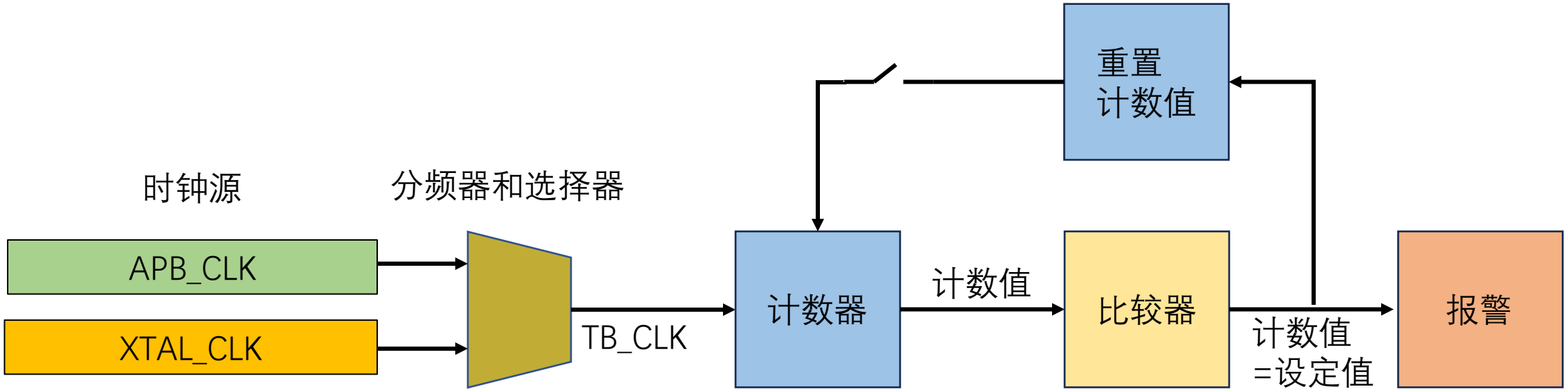
外设	XTAL_CLK	APB_CLK	PLL_F160M_CLK	PLL_D2_CLK	RC_FAST_CLK	CRYPTO_PWM_CLK	LEDC_CLK
TIMG	Y	Y					
I2S	Y		Y	Y			
UHCI		Y					
UART	Y	Y			Y		
RMT	Y	Y			Y		
PWM						Y	
I2C	Y				Y		
SPI	Y	Y					
PCNT		Y					
eFuse Controller	Y	Y					
SARADC		Y		Y			
USB		Y					
CRYPTO						Y	
TWAI Controller		Y					
SDIO HOST	Y		Y				
LEDC	Y	Y			Y		Y
LCD_CAM	Y		Y	Y			
SYS_TIMER	Y	Y					

通用定时器简介

- ESP32-S3包含两组通用定时器，即定时器组0和定时器组1
- 每个定时器组都有两个通用定时器和一个主系统看门狗定时器

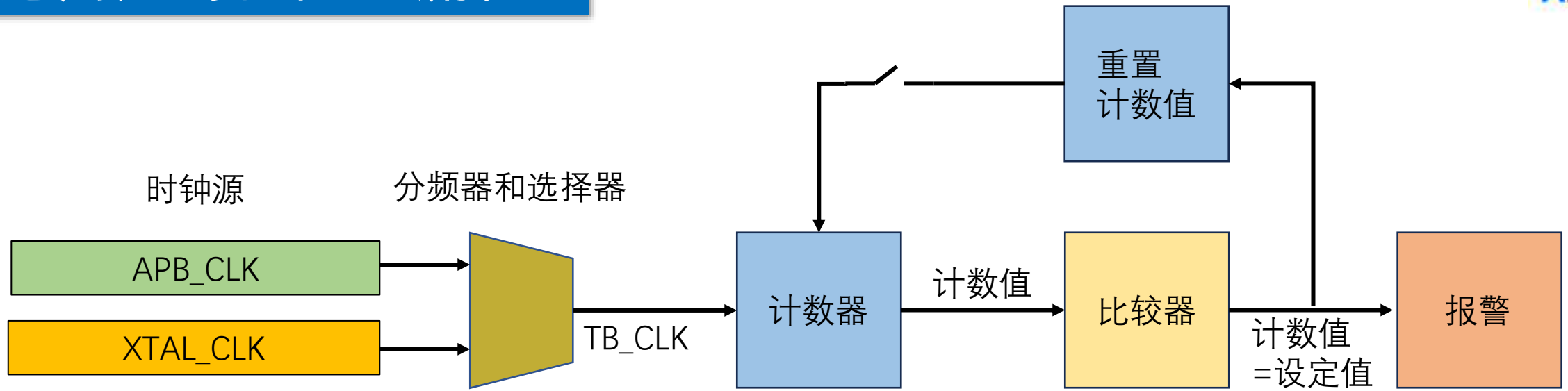


通用定时器工作流程


重置
计数值
时钟源
分频器和选择器
APB_CLK
XTAL_CLK
TB_CLK
计数器
计数值
比较器
计数值
=设定值
报警

通用定时器配置流程

艾谷科技
Aigo

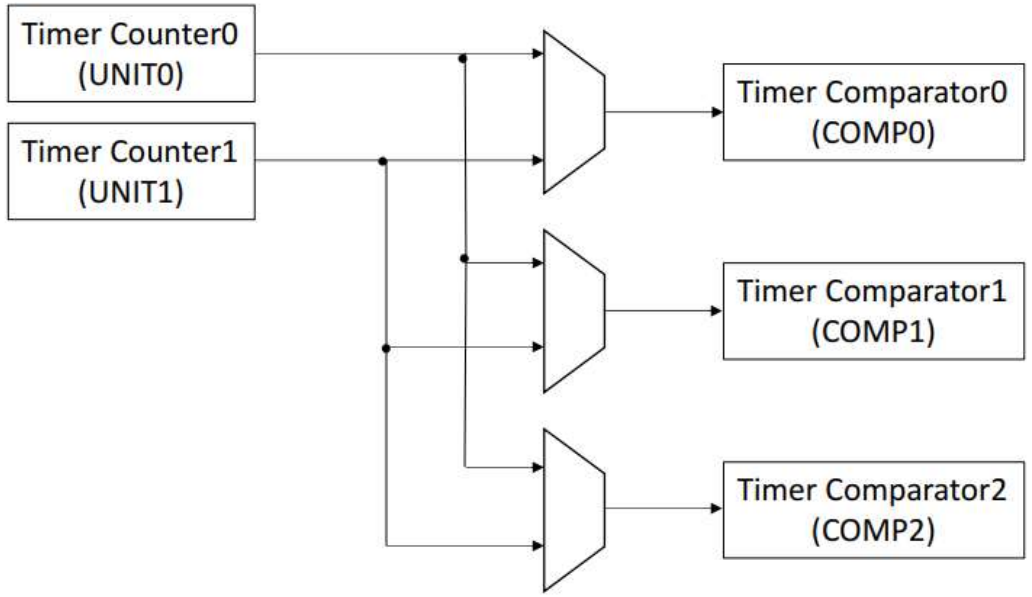


1.使用gptimer_handle_t定义定时器句柄（给定时器起名）
2.使用 “gptimer_new_timer” 函数配置时钟源和计数器
3.使用 “gptimer_set_alarm_action” 函数配置比较器的动作
4.使用 “gptimer_register_event_callbacks” 函数配置报警事件
5.使用 “gptimer_enable” 函数使能通用定时器
6.使用 “gptimer_start” 函数开启通用定时器

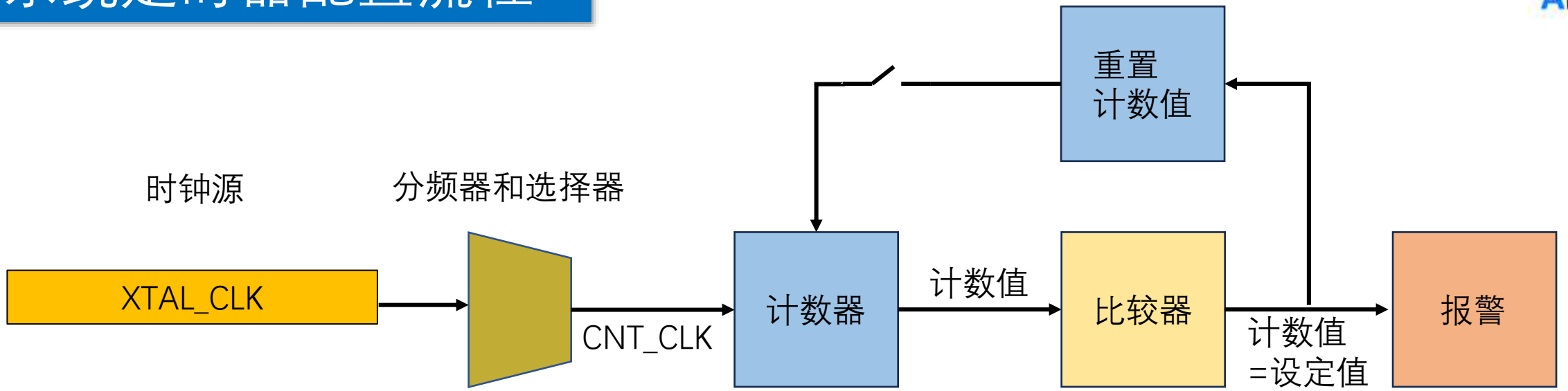
系统定时器

艾谷科技
Aigo

- ESP32-S3包含一组系统定时器可用于生成操作系统所需的滴答定时中断，也可以用作普通定时器使用。



系统定时器配置流程

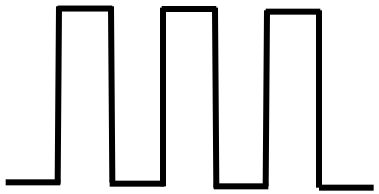


1.使用 esp_timer_handle_t定义定时器句柄
2.使用 “esp_timer_create” 函数创建一个定时器并配置报警事件
3.使用 “esp_timer_start_periodic” 函数配置比较器值，并开启定时器
“esp_timer_start_once”
注意：计数器的计数步长是固定的（1us）

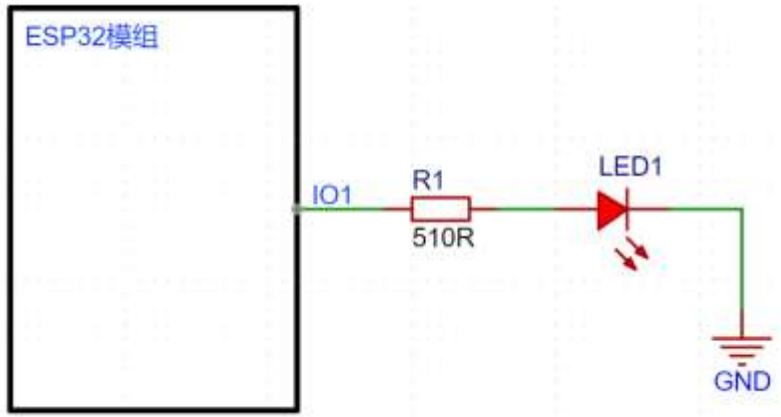
数字信号->电压值

艾谷科技
Aigo

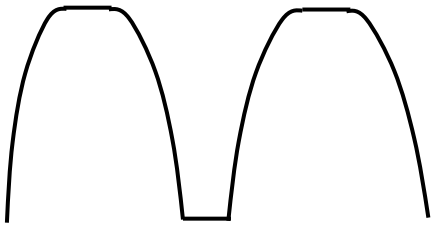
高电平：3.3V



低电平：0V



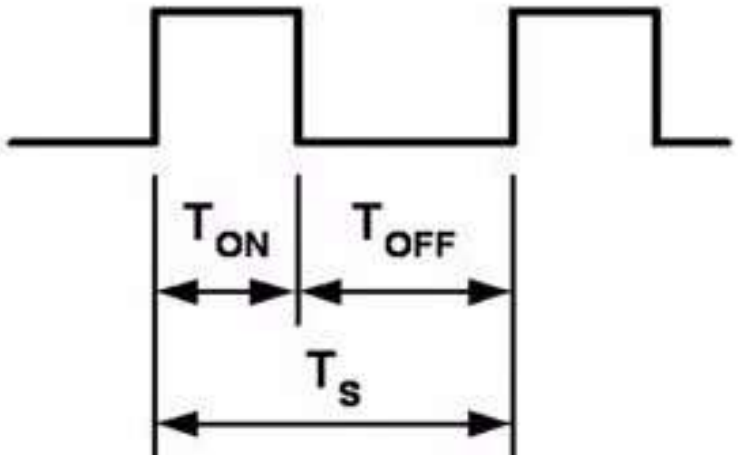
LED1+R1的电压值变化



PWM简介

艾谷科技
Aigo

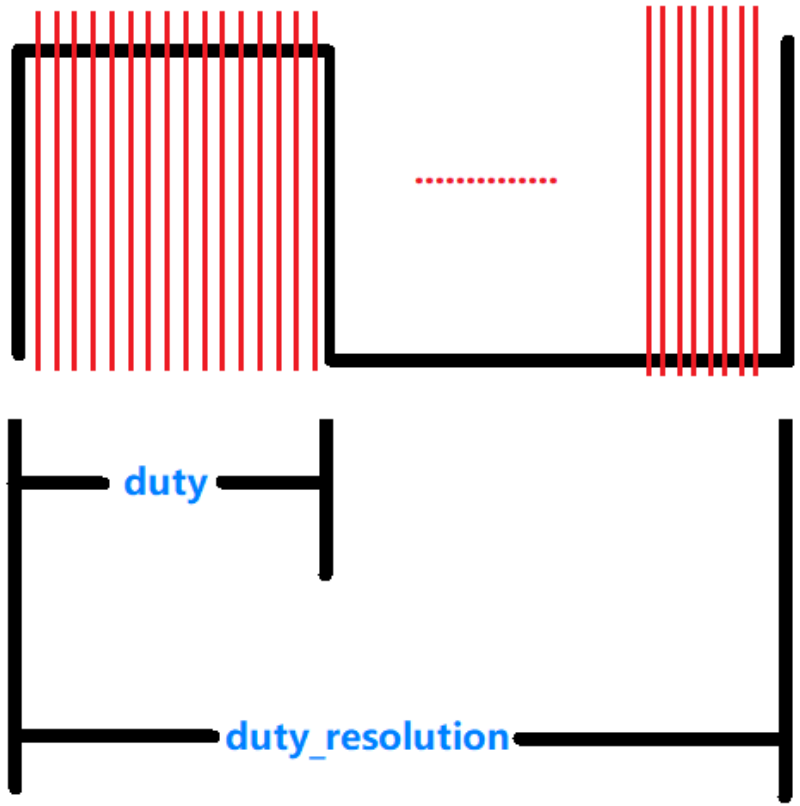
- PWM（Pulse Width Modulation）脉冲宽度调制
- 在具有惯性的系统中，可以通过对一系列脉冲的宽度进行调制，来等效地获得所需要的模拟参量，常应用于电机控速等领域
- PWM参数：
  频率 = 1 / T_S　　占空比 = T_{ON} / T_S　　分辨率 = 占空比变化步距

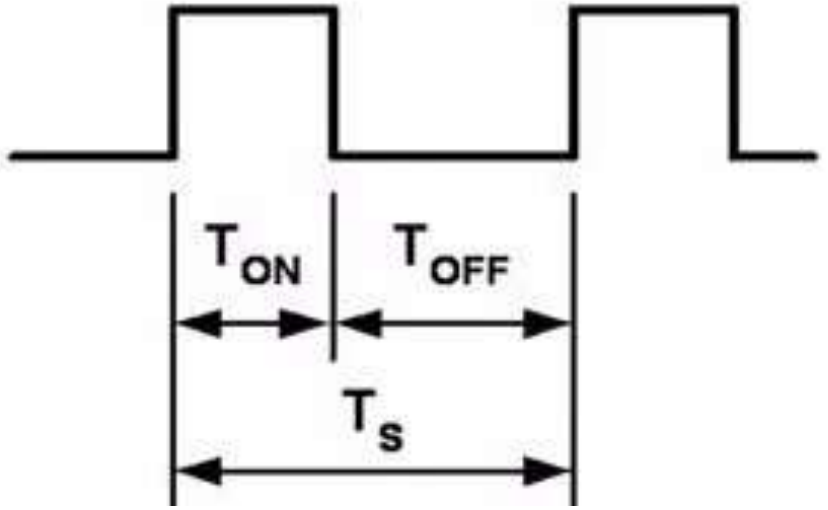



```
while (1)
{
    gpio_set_level(GPIO_NUM_38,1);
    vTaskDelay(5);
    gpio_set_level(GPIO_NUM_38,0);
    vTaskDelay(5);
}
```

占空比表示方式

- 实际应用中的占空比 = (高电平切分数量 / 分辨率) * 100%
  = (duty / duty_resolution) * 100%

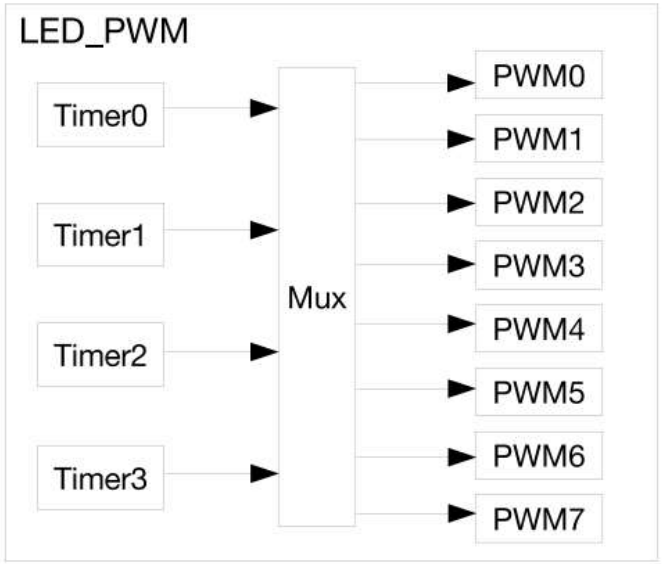




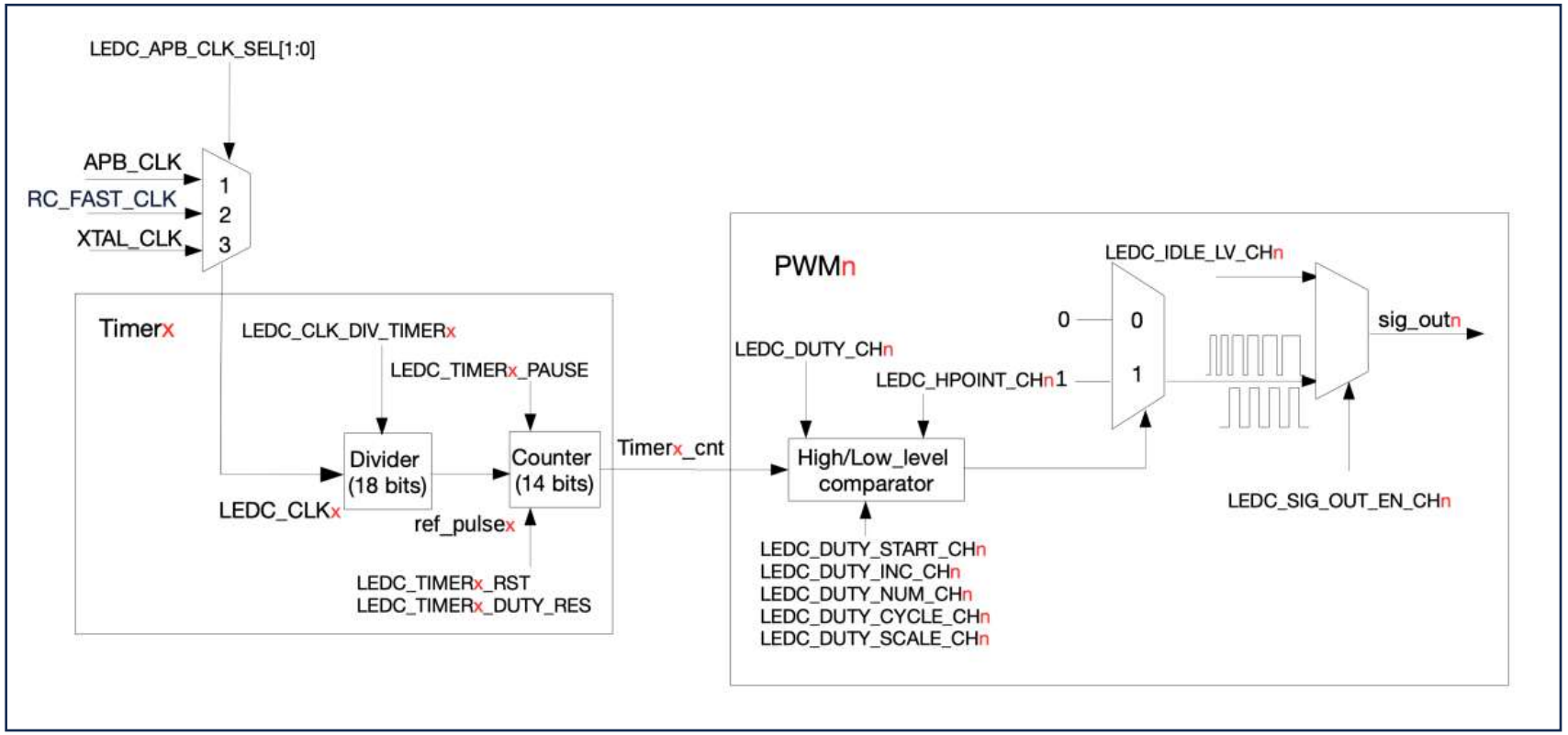
LED PWM控制器

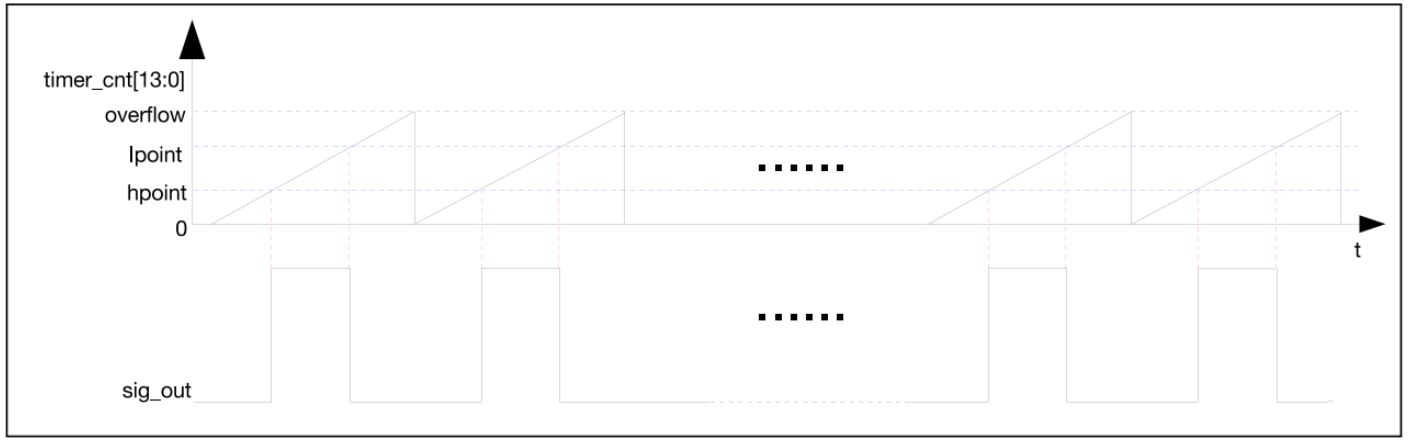
艾谷科技
Aigo

- ESP32S3中有两种PWM控制器：LED PWM 和 MC PWM
- LED PWM控制器用于生成控制LED的PWM，具有占空比自动渐变等专门功能。该外设也可生成PWM信号用作其他用途。
- ESP32S3具有八个独立的PWM生成器（即八个PWM输出通道）和四个独立定时器给PWM生成器提供时钟。
- 四个定时器可独立配置，但是共用一个时钟源。
- 每个PWM生成器会在四个定时器中选择一个，以该定时器为基准生成PWM信号。



PWM控制器架构



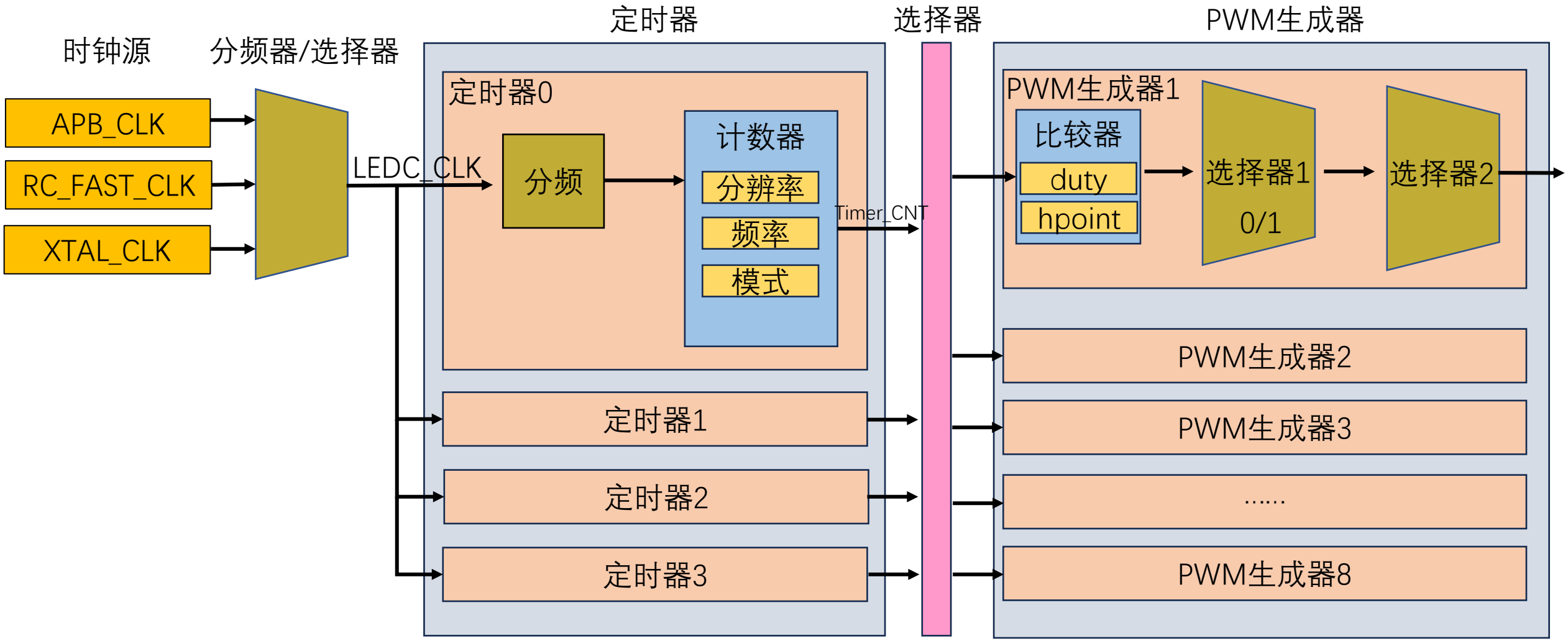


Timer_cnt < hpoint， 低电平
hpoint < Timer_cnt < lpoint ,高电平
Timer_cnt > lpoint, 低电平
Timer_cnt = overflow， Timer_cnt重置

Timer_cnt < lpoint , 高电平
Timer_cnt > lpoint , 低电平

PWM控制器配置流程

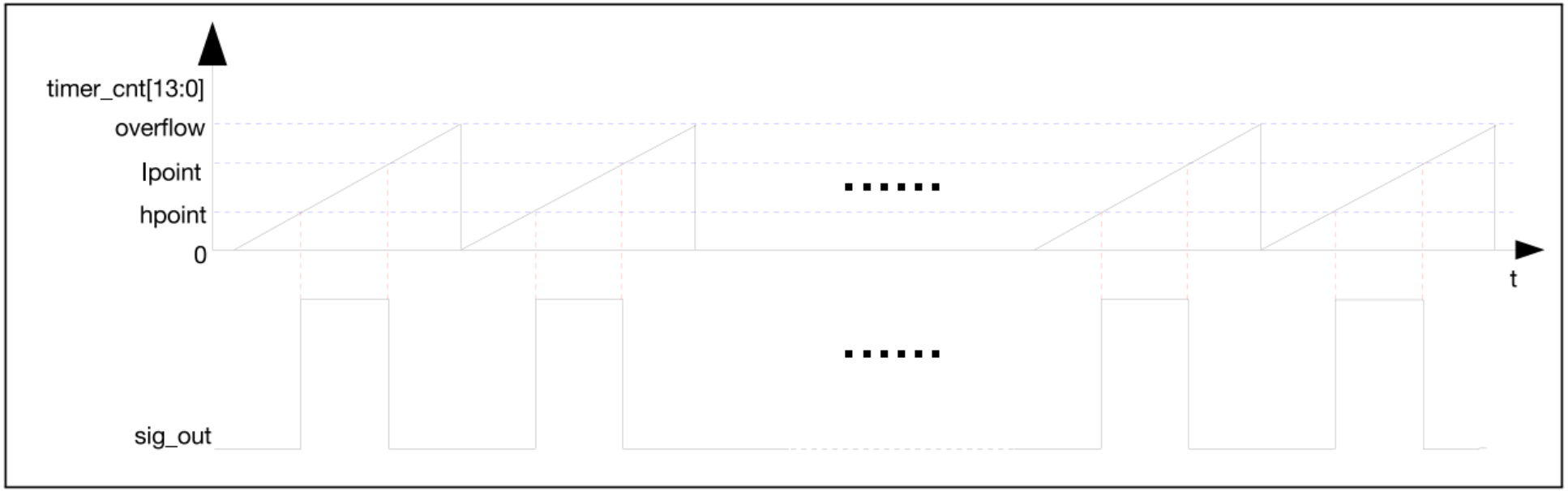
艾谷科技 Aigo



1. 使用 ledc_timer_config() 函数配置时钟源和定时器参数　2. 使用 ledc_channel_config() 函数配置PWM控制器参数
3. 使用 ledc_set_duty（）函数修改占空比，使用 ledc_update_duty() 函数更新占空比

脉冲生成过程

艾谷科技
Aigo



Timer_cnt < hpoint，低电平
hpoint < Timer_cnt < lpoint ,高电平
Timer_cnt > lpoint, 低电平
Timer_cnt = overflow，Timer_cnt重置

Timer_cnt < lpoint , 高电平
Timer_cnt > lpoint , 低电平

频率和分辨率配置

表 35.2-1. 常用配置频率及精度

LEDC_CLK*x*	PWM 频率	最高精度（位）[1]	最低精度（位）[2]
APB_CLK (80 MHz)	1 kHz	14	7
APB_CLK (80 MHz)	5 kHz	13	4
APB_CLK (80 MHz)	10 kHz	12	3
XTAL_CLK (40 MHz)	1 kHz	14	6
XTAL_CLK (40 MHz)	4 kHz	13	4
RC_FAST_CLK (17.5 MHz)	1 kHz	14	5
RC_FAST_CLK (17.5 MHz)	1.75 kHz	13	4

电机简介

艾谷科技
Aigo

- 电机分类：直流有刷电机、直流无刷电机、步进电机等





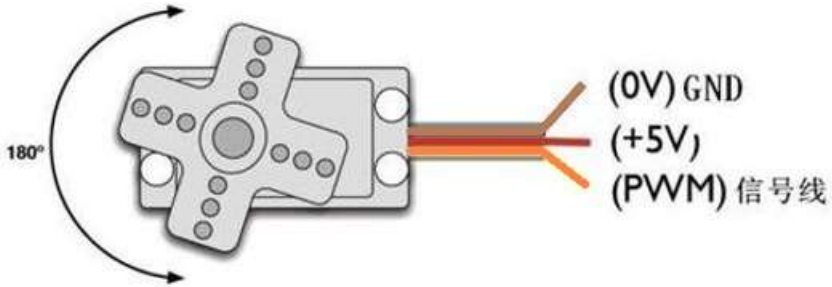


舵机简介

艾谷科技
Aigo

- 舵机是一种根据输入PWM信号占空比来控制输出角度的电机
- 常用舵机分类：180°舵机(方向、角度)；360°舵机(方向、转速)
- 角度控制方式：PWM信号占空比控制



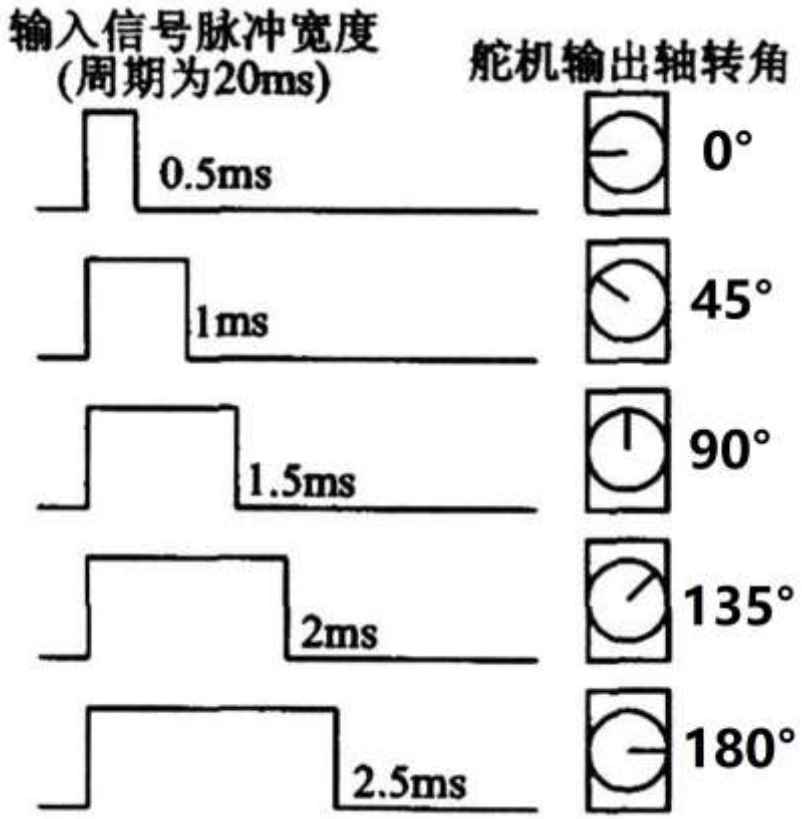




SG90舵机控制方式

艾谷科技
Aigo

- 输入PWM信号要求：周期为20ms(50Hz)，高电平宽度为0.5ms~2.5ms



占空比
0.025
0.05
0.075
0.1
0.125

duty和angle映射关系

角度	占空比
0	0.025
45	0.05
90	0.075
145	0.1
180	0.125

$$占空比 = \frac{duty}{1024}$$

$$y = kx + b$$

$$k = \frac{1}{1800}$$

$$b = 0.025$$

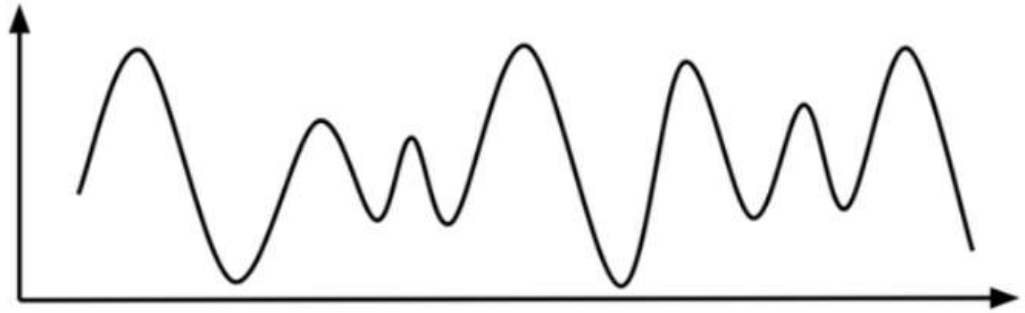
$$占空比 = \frac{1}{1800} * 角度 + 0.025$$

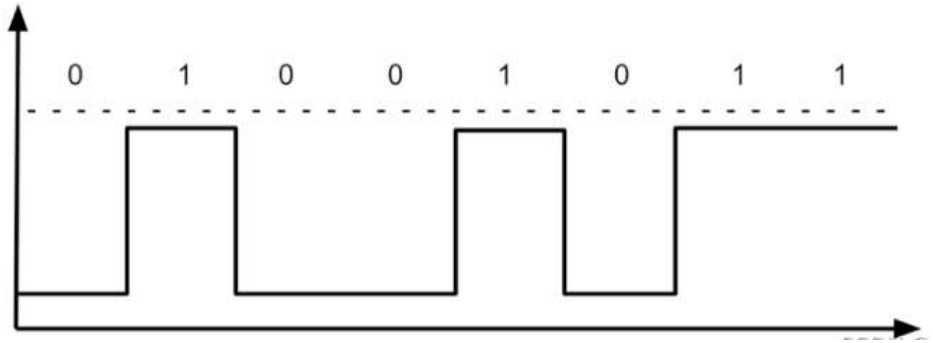
$$duty = \left(\frac{1}{1800} * 角度 + 0.025\right) * 1024$$

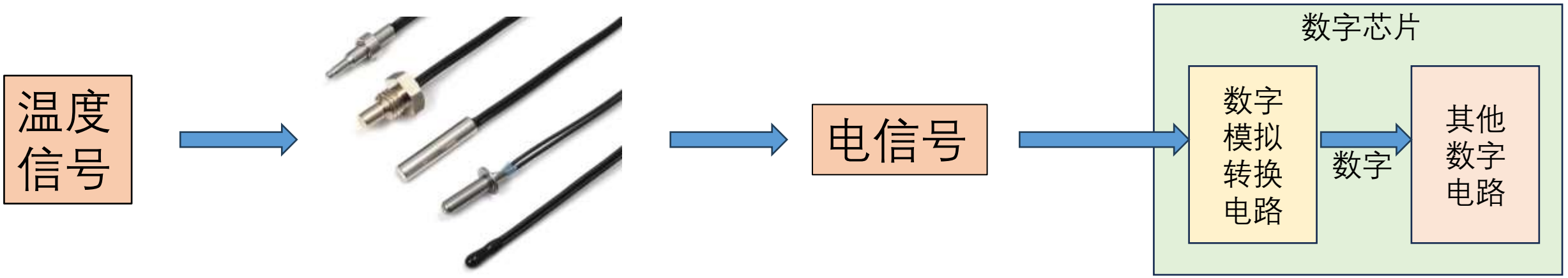
信号简介

艾谷科技
Aigo

- 信号分类：模拟信号、数字信号
- 模拟信号是一段连续的信号，可以是某一范围内的任意值
- 数字信号是一段离散的信号，只有0和1







ADC简介

- ADC（Analog-Digital Converter）模拟-数字转换器
- ADC可以将引脚上连续变化的模拟电压转换为内存中存储的数字变量，建立模拟电路到数字电路的桥梁
- 2个12位逐次逼近型ADC
- 输入电压范围：0~3.3V，转换结果范围：0~4095

- ESP32S3 ADC资源：SAR ADC1、SAR ADC2，共20路输入通道

说明：

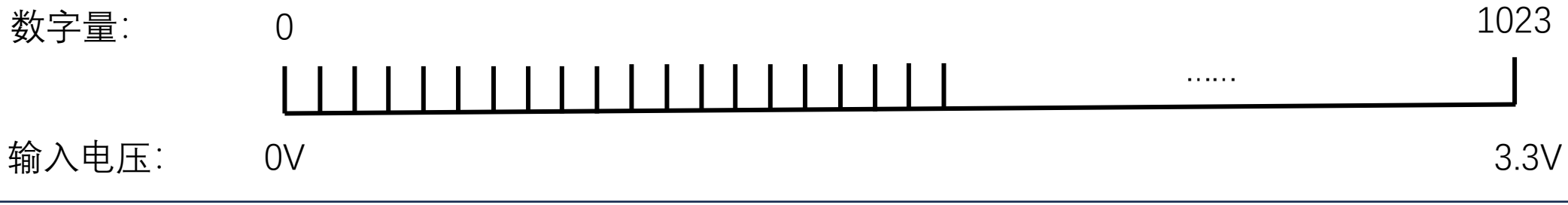
ESP32-S3 的 DIG ADC2 控制器无法正常工作，该章节中已删除其相关信息，详见 ESP32-S3 系列芯片勘误表。

ADC分辨率

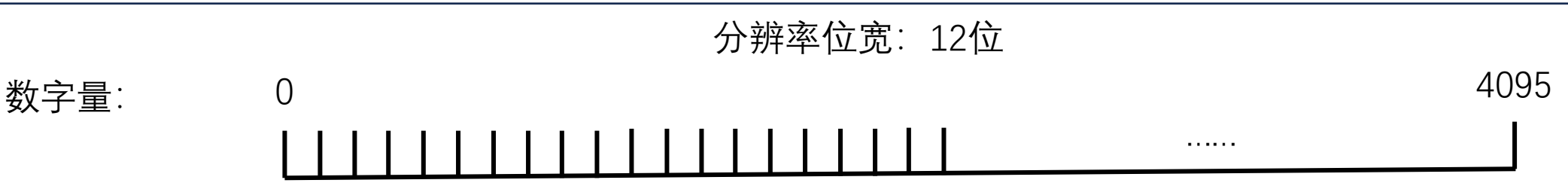
艾谷科技
Aigo

- ADC分辨率：输入电压值的分割数量

分辨率位宽：10位



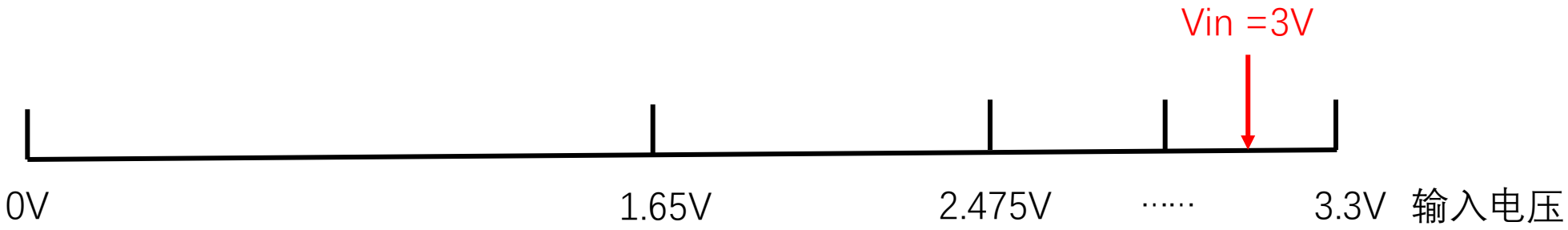
分辨率位宽：12位

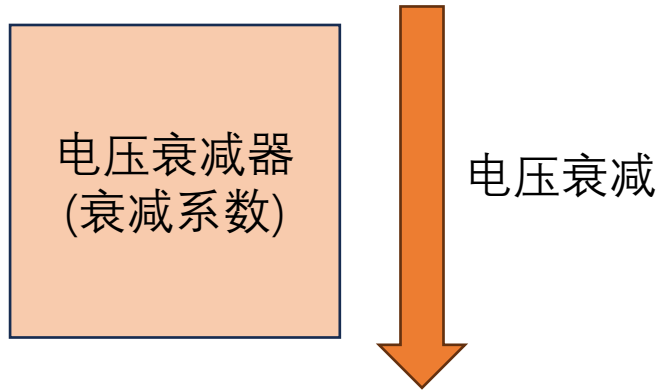


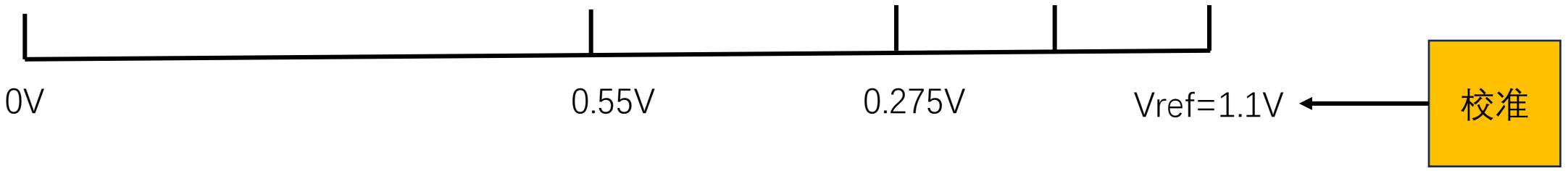
$$输入电压值 = \frac{数字量}{2^{12}-1} \times 3.3V$$

电压输入与电压衰减

艾谷科技
Aigo







SAR ADC简介

A/D 转换过程通常为 4 步：采样、保持、量化和编码，如下图所示。

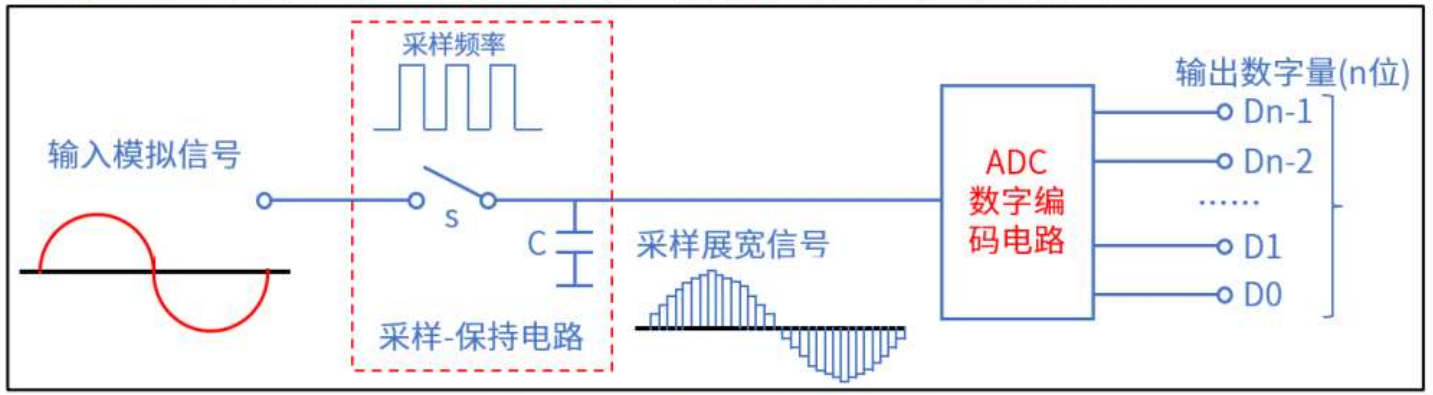


图 24.1.1.1 A/D 转换过程图

采样：把时间连续变化的信号变换为时间离散的信号。

保持：保持采样信号，使有充分时间转换为数字信号。

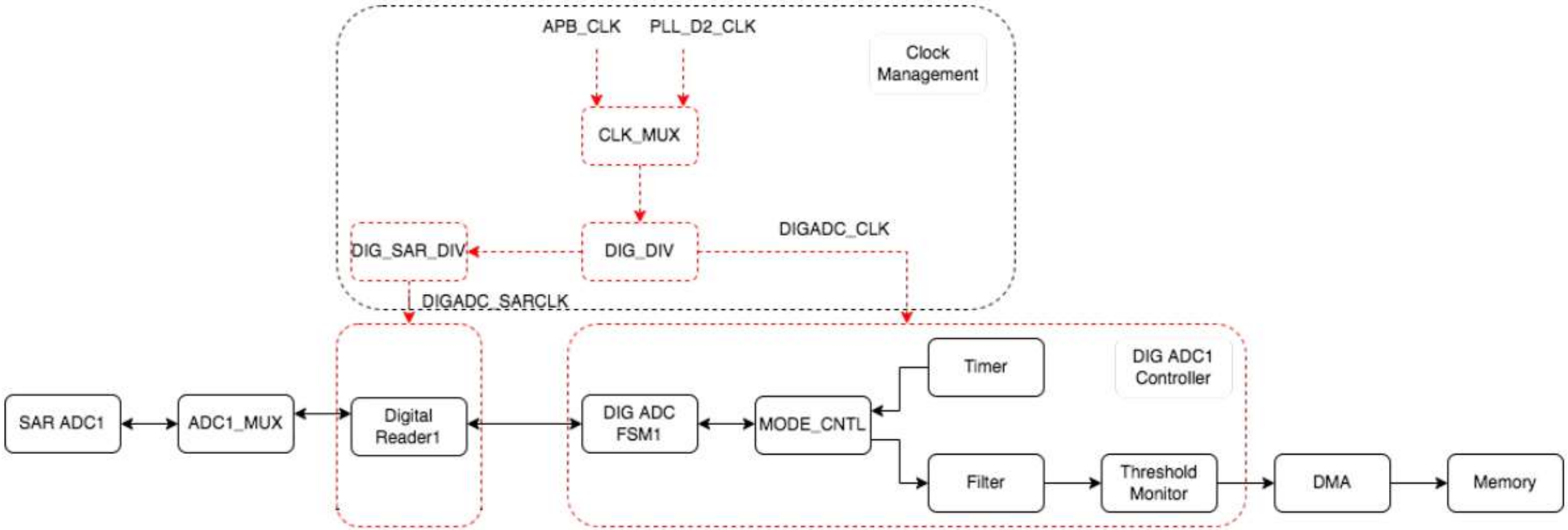
量化：把采样保持电路的输出信号用单位量化电压的整数倍表示。

编码：把量化的结果用二进制代码表示。

采样和保持通常是在采样-保持电路中完成，量化和编码通常在 ADC 数字编码电路中完成。

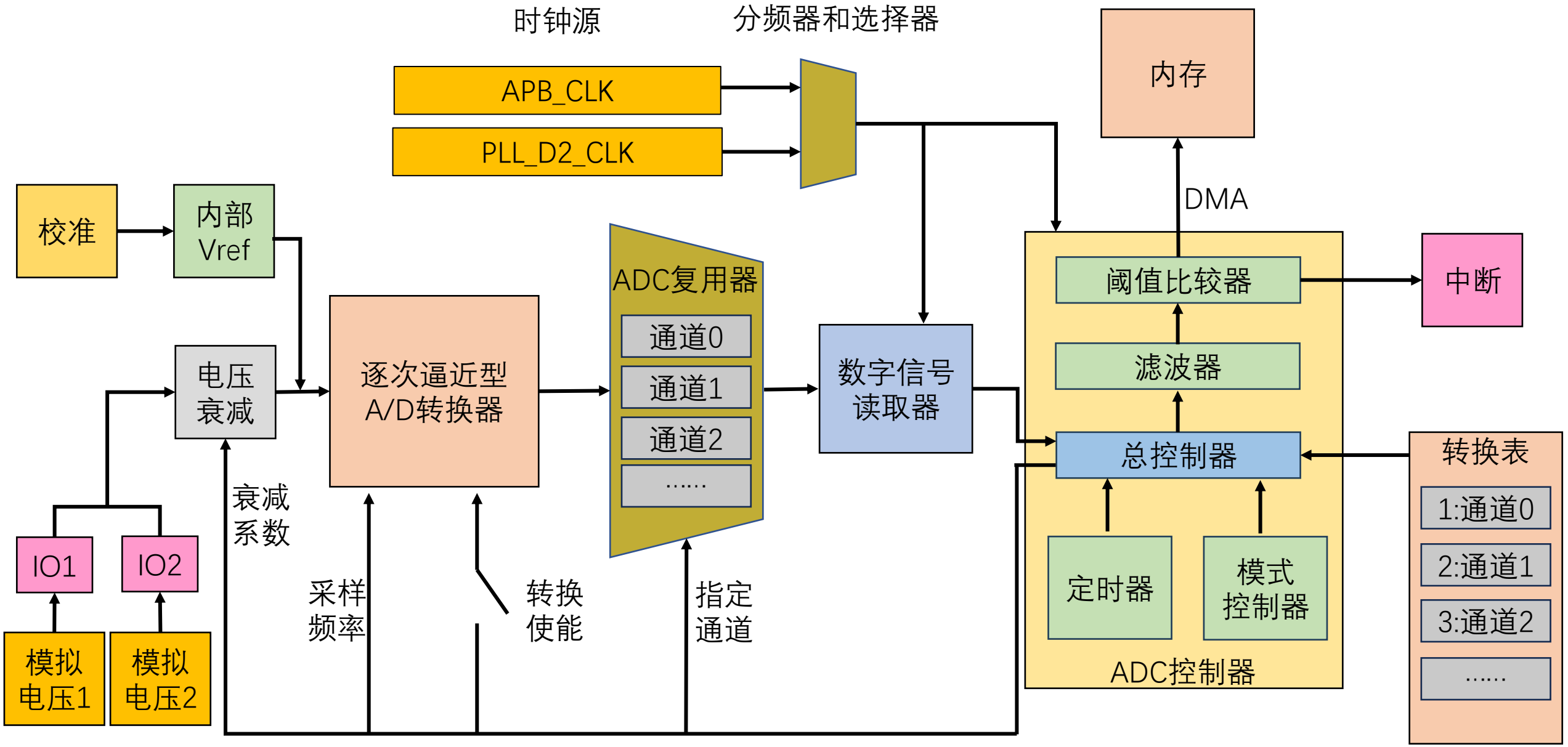
ADC内部框图

艾谷科技
Aigo



ADC工作流程

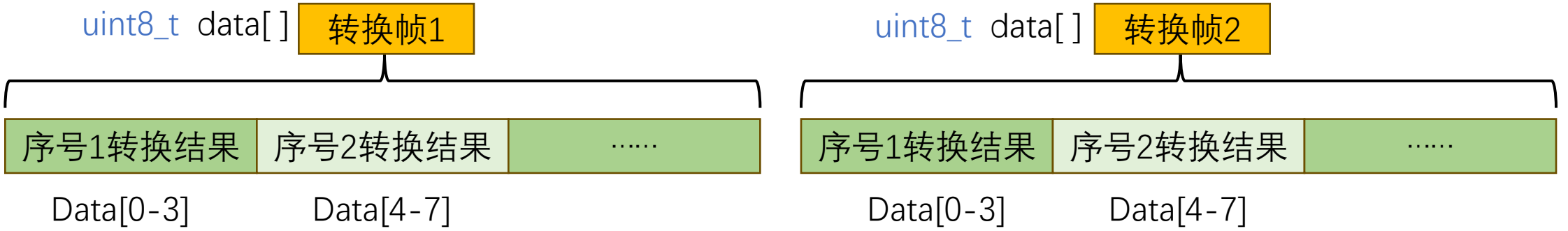
艾谷科技 Aigo

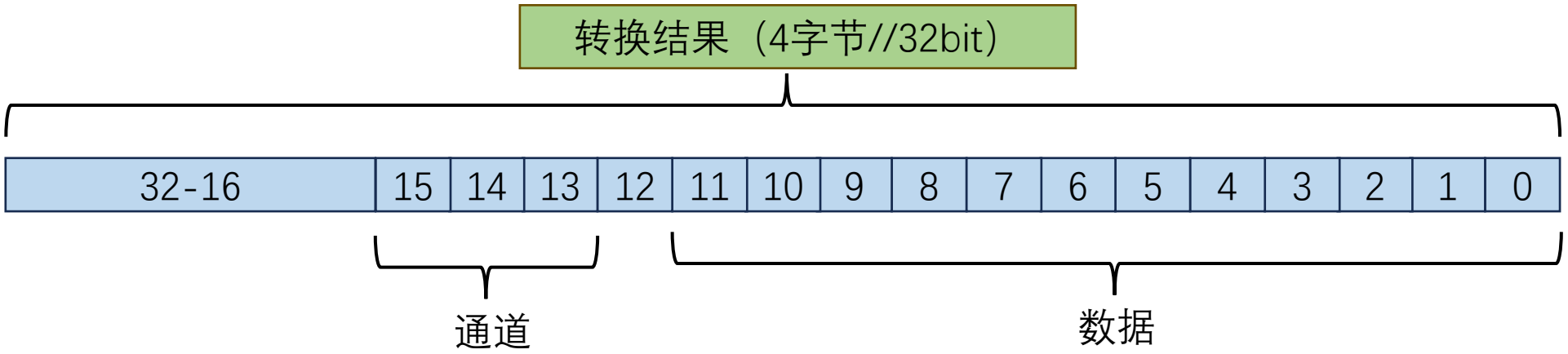


内存中的数据结构

艾谷科技
Aigo

- 一次转换完成，形成一个转换帧
- 一个转换帧内部包含多个转换结果
- 一个转换结果由4个字节（32个比特位）组成



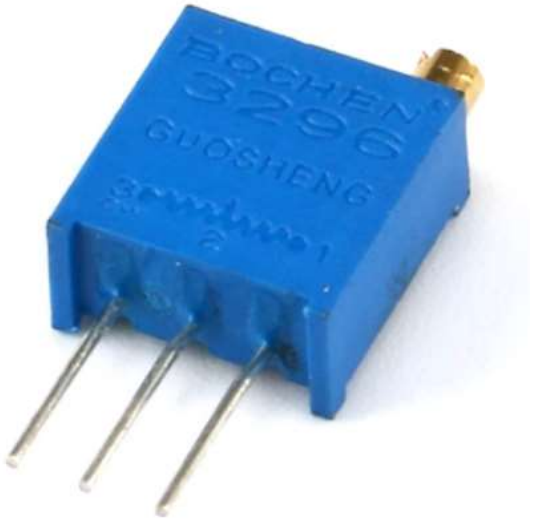


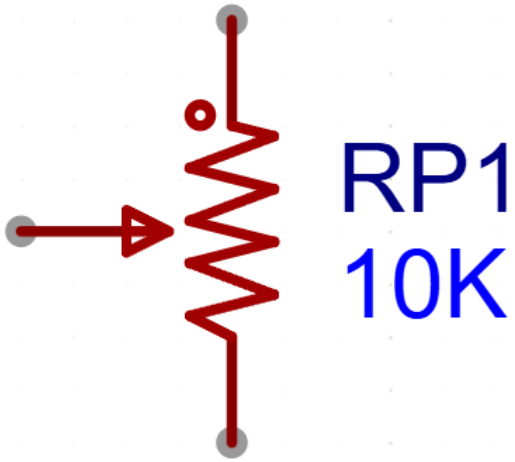
ADC连续转换配置流程

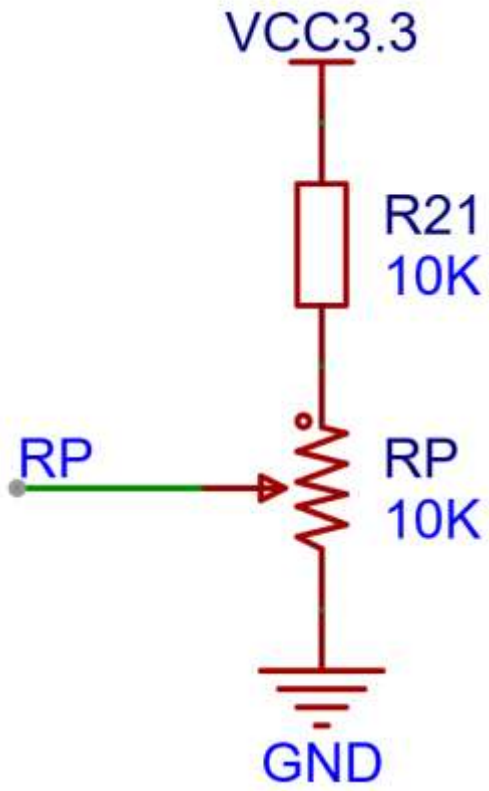
艾谷科技
Aigo

- 使用 “adc_continuous_new_handle” 函数配置转换结果存储单元大小、转换帧存储单元大小
- 使用 “adc_digi_pattern_config_t” 结构体配置转换表
- 使用 “adc_continuous_config” 函数配置ADC总控制器需要下发和执行的参数
- 使用 “adc_continuous_register_event_callbacks” 函数注册回调函数，用于数据读取
- 使用 “adc_continuous_start” 函数开启连续转换

可调电阻





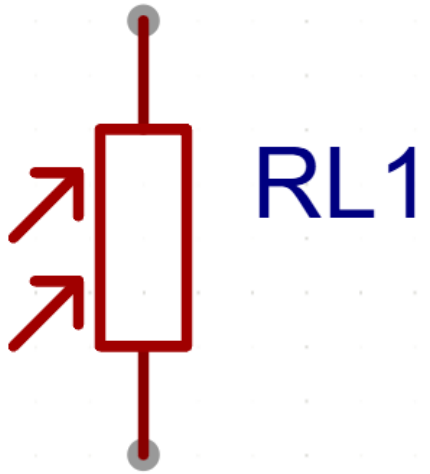


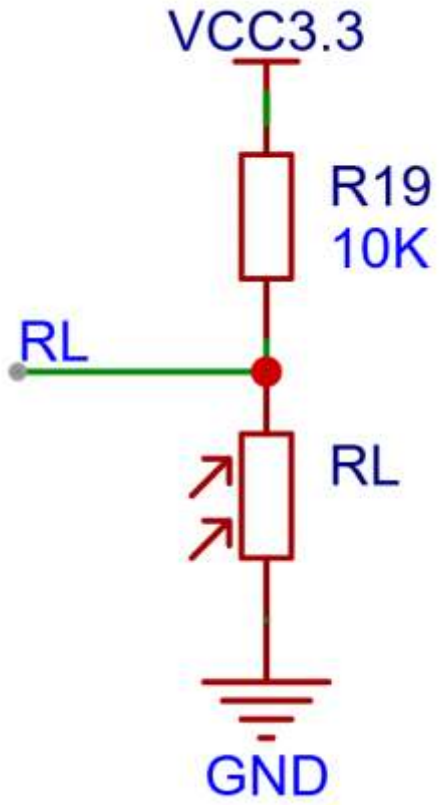
光敏电阻

艾谷科技
Aigo

- 光照强度越强，阻值越小
- 光照强度越弱，阻值越大

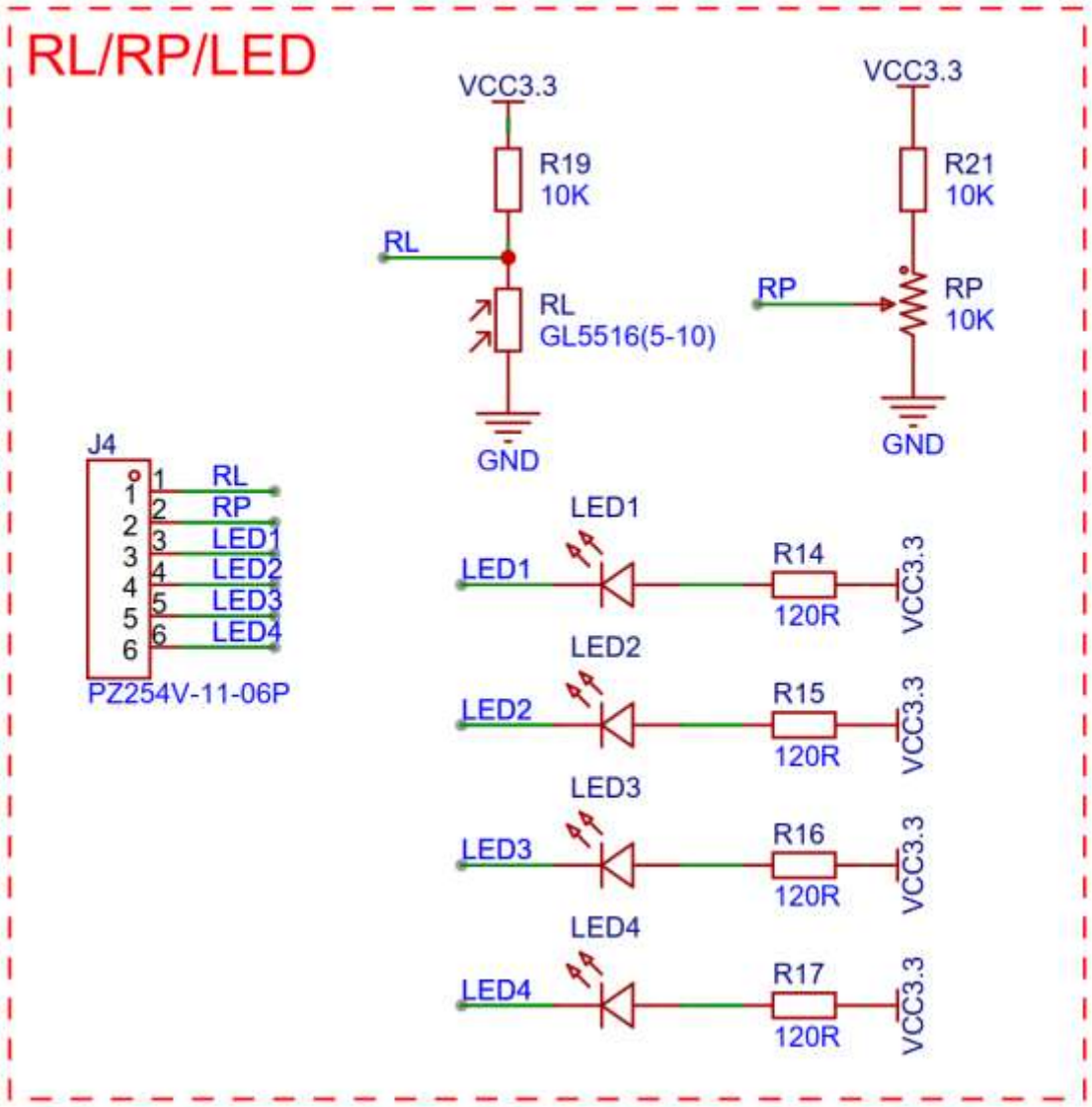






底板原理图

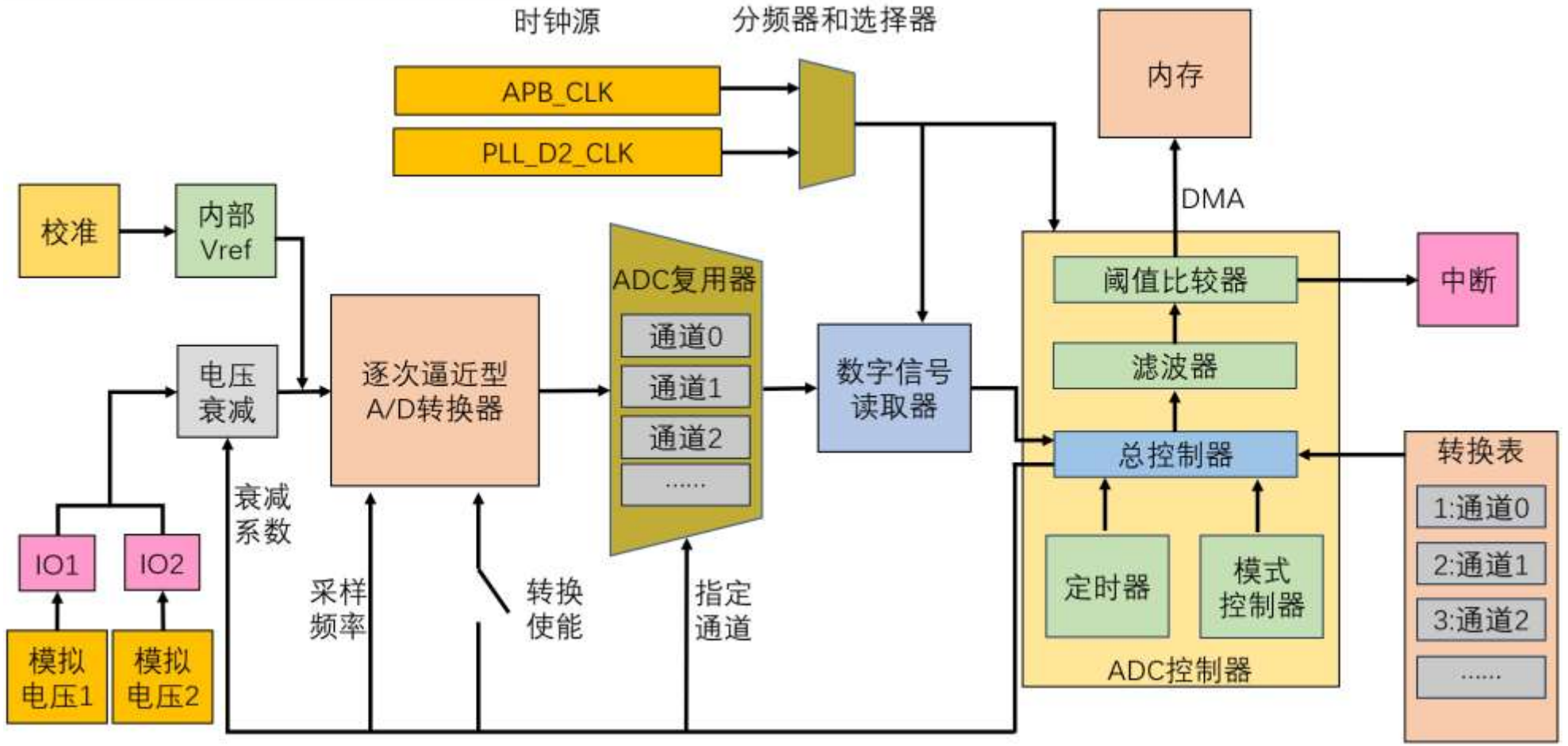
艾谷科技
Aigo



衰减系数

输入电压范围	衰减系数
0-3.3V	ADC_ATTEN_DB_11
0-2.2V	ADC_ATTEN_DB_6
0-1.5V	ADC_ATTEN_DB_2_5
0-1.1V	ADC_ATTEN_DB_0

ADC单次转换配置流程



- 使用 “adc_oneshot_new_unit” 函数配置时钟源、ADC参数
- 使用 “adc_oneshot_config_channel” 函数配置总控制器要下发和执行的参数
- 使用 “adc_oneshot_read” 函数开启一次转换并获取转换数据

单次转换和连续转换

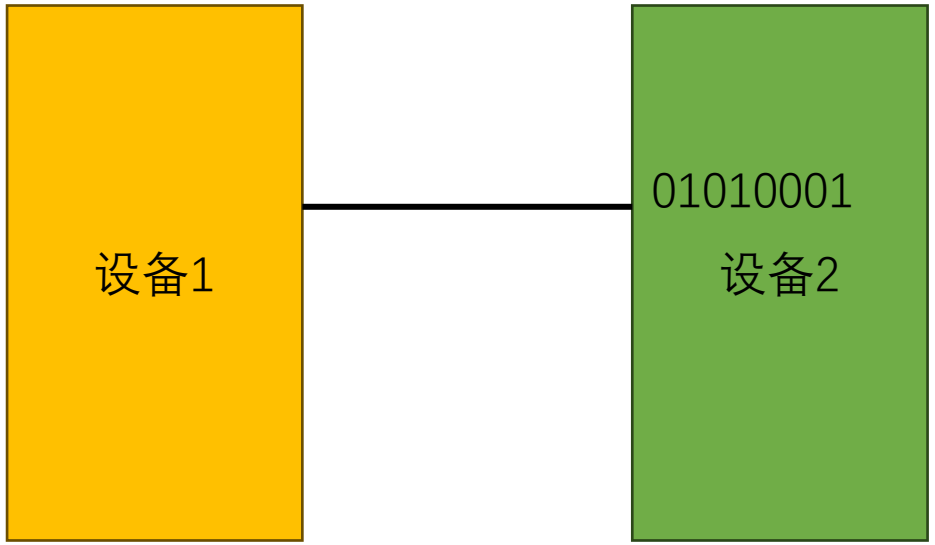
艾谷科技
Aigo

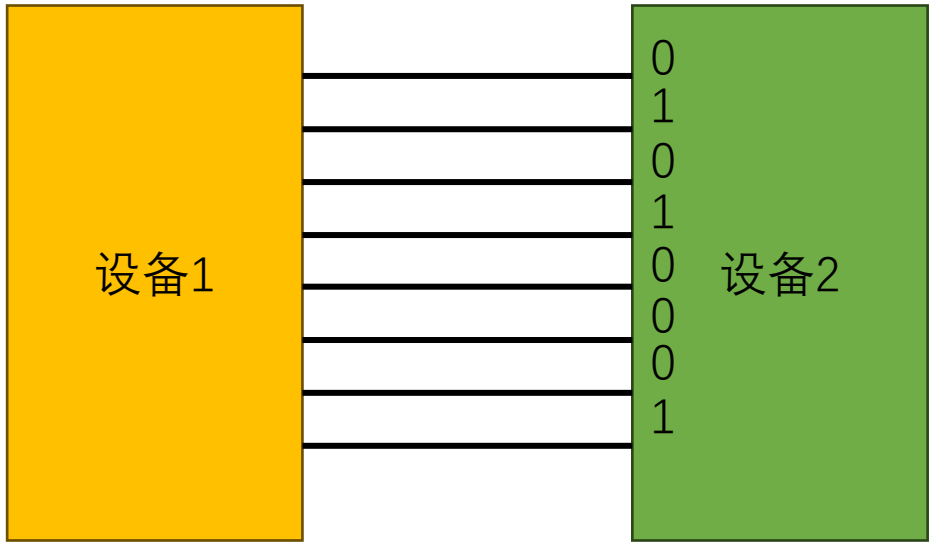
	ADC连续转换模式	ADC单次转换模式
配置	相对复杂	简单
运行	全自动执行	需软件操作
转换频率	适用于频率较高场合	适用于频率较低场合

通信分类

艾谷科技
Aigo

- 通信的目的：将一个设备的数据传送到另一个设备
- 通信协议：制定通信的规则，通信双方按照协议规则进行数据收发
- 通信分类：
  -依据传输线数量：串行通信（UART、I2C、SPI）、并行通信
  -依据传输方向：单工通信、半双工通信、全双工通信
  -依据同步方式：同步、异步





ESP32S3串行接口

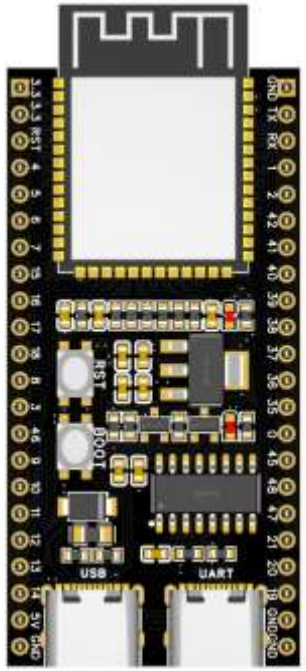
艾谷科技
Aigo

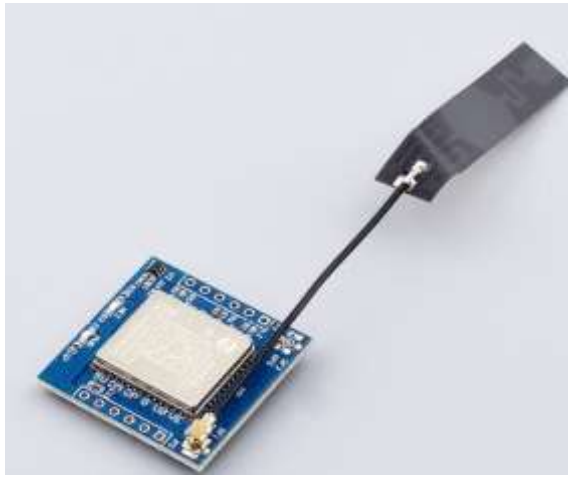
名称	引脚	双工	时钟	设备
UART	TX、RX	全双工	异步	一对一
I2C	SCL、SDA	半双工	同步	一对多
SPI	SCLK、MOSI、MISO、CS	全双工	同步	一对多

串口通信

- 串口是一种应用十分广泛的通讯接口，串口成本低、容易使用、通信线路简单，可实现两个设备的互相通信
- 芯片的串口可以使芯片与芯片、芯片与电脑、芯片与各式各样的模块互相通信，极大地扩展了芯片的应用范围



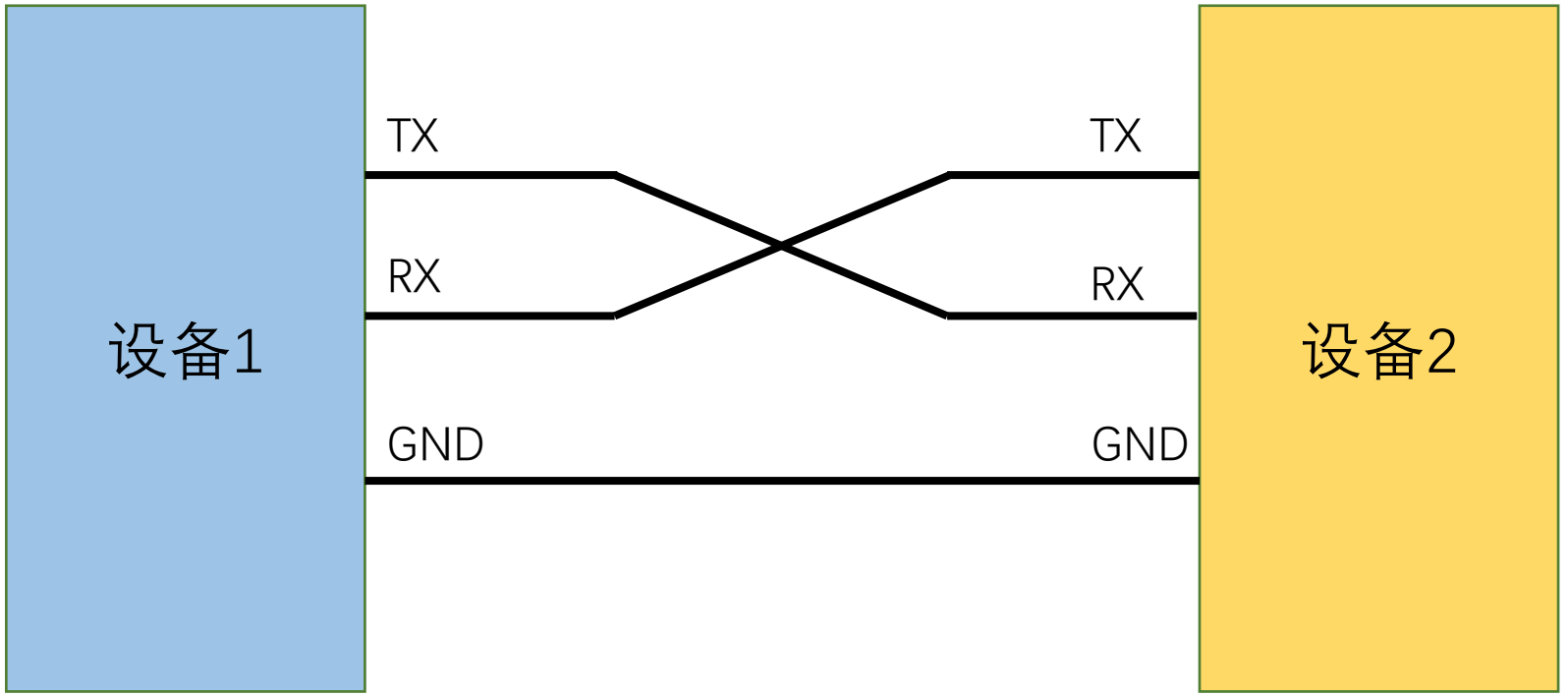




硬件电路

艾谷科技
Aigo

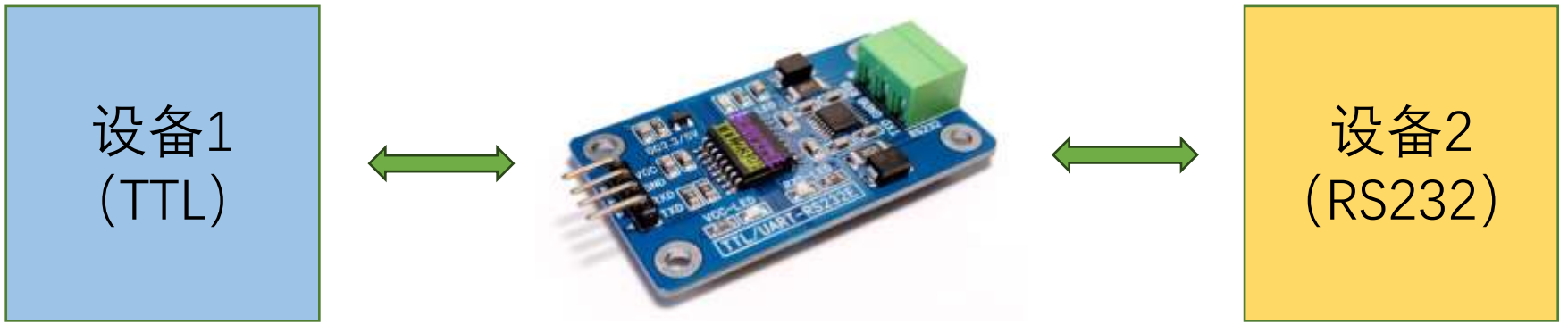
- 简单双向串口通信有两根通信线（发送TX和接收RX）
- TX与RX要交叉连接
- 当电平标准不一致时，需要加电平转换芯片



电平标准

- 电平标准是数据1和数据0的表达方式，是传输线缆中人为规定的电压与数据的对应关系，串口常用的电平标准有如下三种：

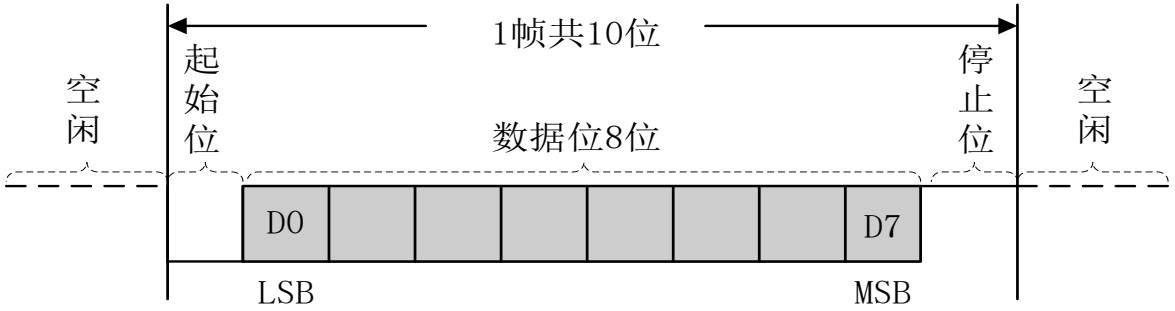
- TTL电平：+3.3V(电源电压)表示1，0V表示0
- RS232电平：-3~-15V表示1，+3~+15V表示0
- RS485电平：两线压差+2~+6V表示1，-2~-6V表示0（差分信号）

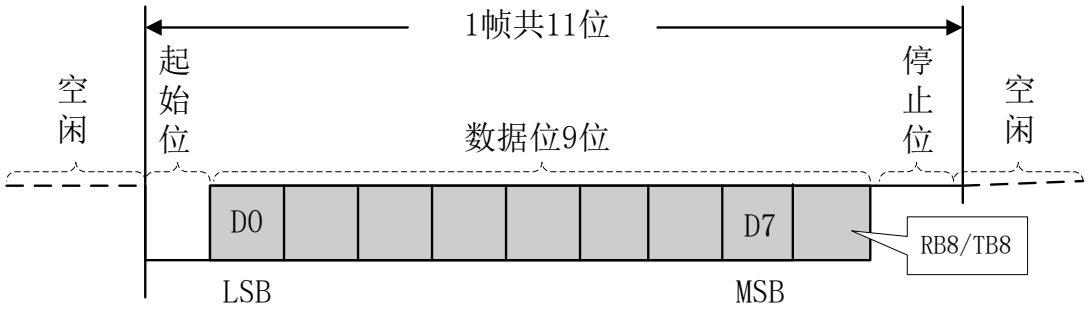


串口参数及时序

艾谷科技
Aigo

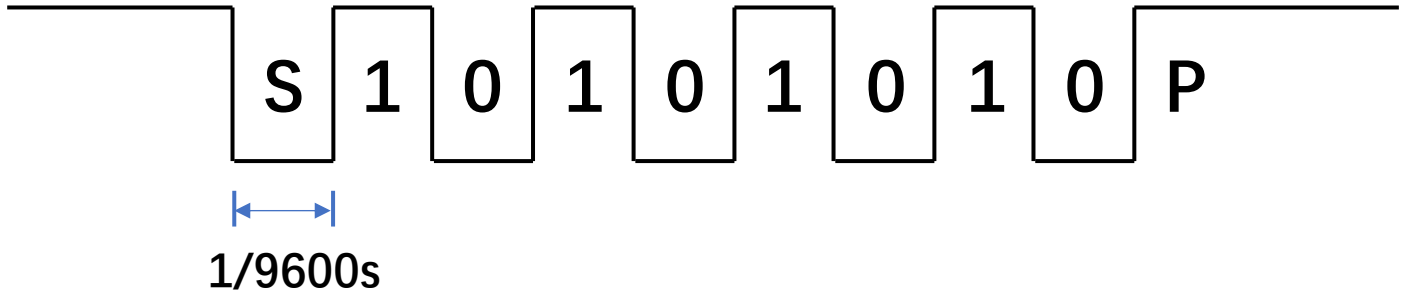
- 波特率：串口通信的速率
- 起始位：标志一个数据帧的开始，固定为低电平
- 停止位：用于数据帧间隔，固定为高电平
- 数据位：数据帧的有效载荷，1为高电平，0为低电平，低位先行
- 校验位：用于数据验证，根据数据位计算得来



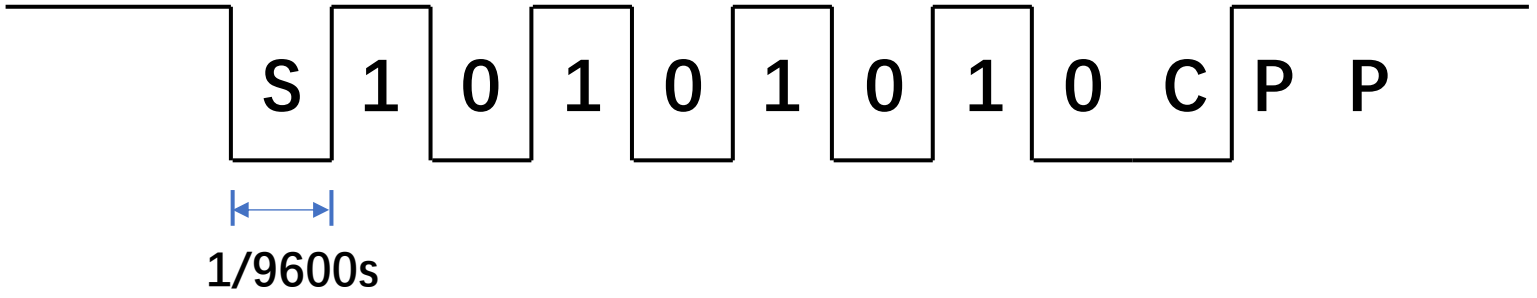


数据帧分析

发送数据：0x55（01010101）
波特率：9600
起始位：1位
停止位：1位
数据位：8位
校验位：无



发送数据：0x55（01010101）
波特率：9600
起始位：1位
停止位：2位
数据位：9位
校验位：偶校验



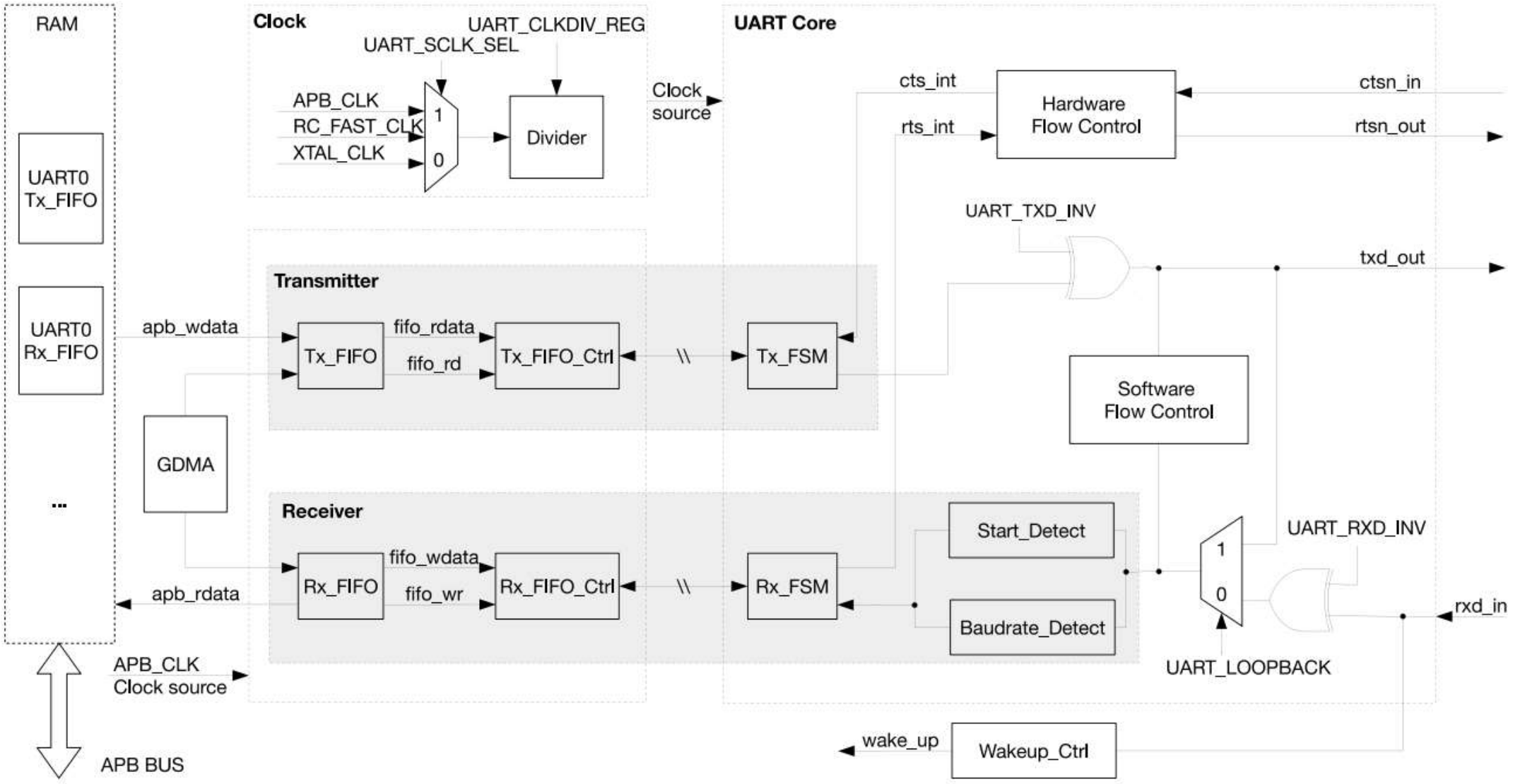
ESP32S3的UART

艾谷科技
Aigo

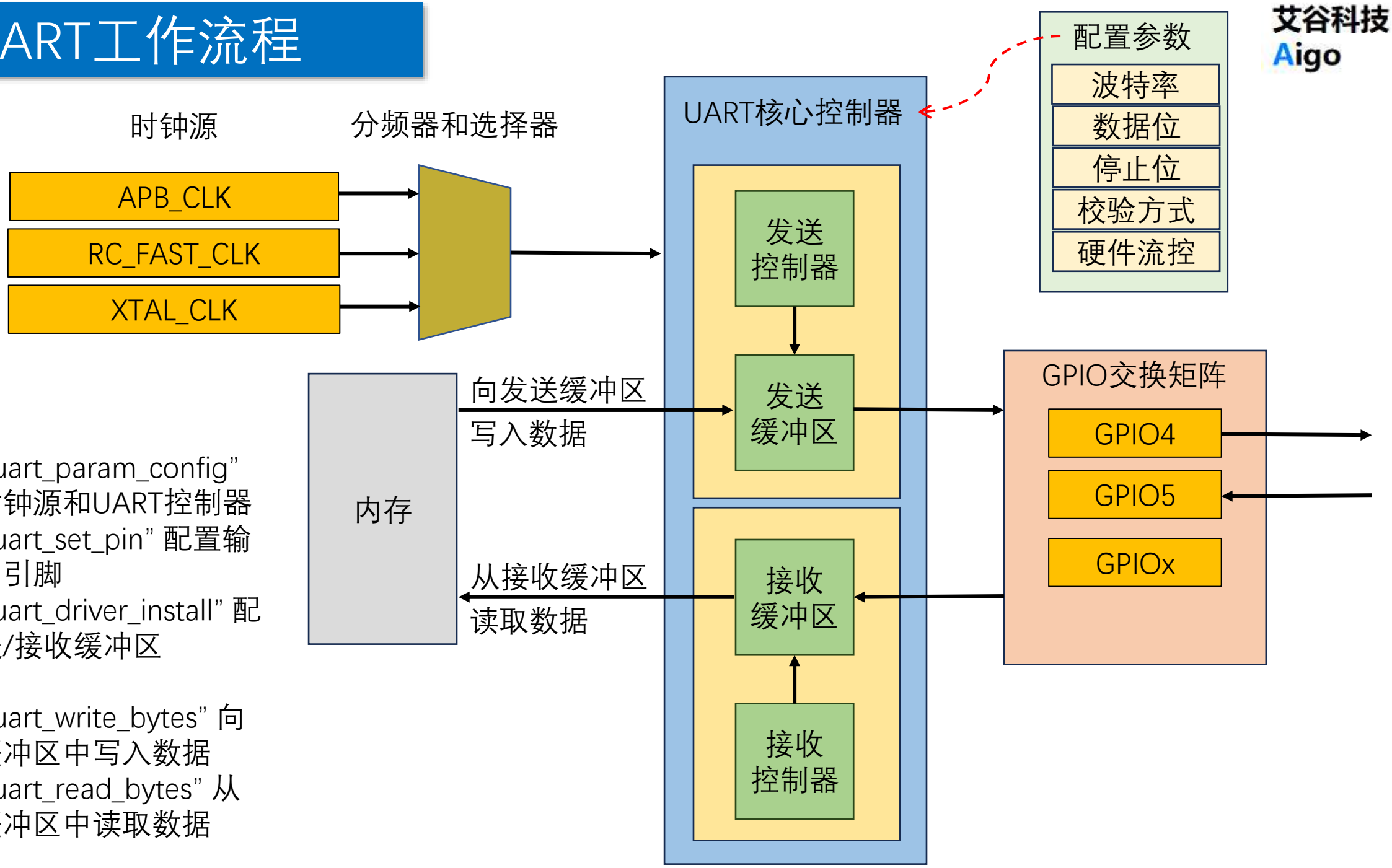
- UART（Universal Asynchronous Receiver/Transmitter）通用异步收发器
- UART是ESP32S3内部集成的硬件外设，可以将数据缓冲区中的一个字节数据自动生成数据帧时序，从TX引脚发送出去，也可自动接收RX引脚的数据帧时序，拼接为一个字节数据，存放在数据缓冲区里
- 串口通信速度最高达5Mbits/s
- 可配置数据位长度（5/6/7/8）、停止位长度（1/1.5/2/3）
- 可选校验位（无校验/奇校验/偶校验）

- ESP32S3 UART资源： UART0、 UART1、 UART2

UART内部架构图


RAM
UART0 Tx_FIFO
UART0 Rx_FIFO
...
GDMA
APB_CLK Clock source
APB BUS
Clock
UART_CLKDIV_REG
UART_SCLK_SEL
APB_CLK
RC_FAST_CLK
XTAL_CLK
1
0
Divider
Clock source
UART Core
cts_int
rts_int
Hardware Flow Control
ctsn_in
rtsn_out
UART_TXD_INV
txd_out
Transmitter
apb_wdata
Tx_FIFO
fifo_rdata
fifo_rd
Tx_FIFO_Ctrl
Tx_FSM
Software Flow Control
Receiver
apb_rdata
Rx_FIFO
fifo_wdata
fifo_wr
Rx_FIFO_Ctrl
Rx_FSM
Start_Detect
Baudrate_Detect
UART_RXD_INV
UART_LOOPBACK
rxd_in
wake_up
Wakeup_Ctrl

UART工作流程



- 使用 “uart_param_config” 配置时钟源和UART控制器
- 使用 “uart_set_pin” 配置输入输出引脚
- 使用 “uart_driver_install” 配置发送/接收缓冲区

- 使用 “uart_write_bytes” 向发送缓冲区中写入数据
- 使用 “uart_read_bytes” 从接收缓冲区中读取数据

数据格式

艾谷科技
Aigo

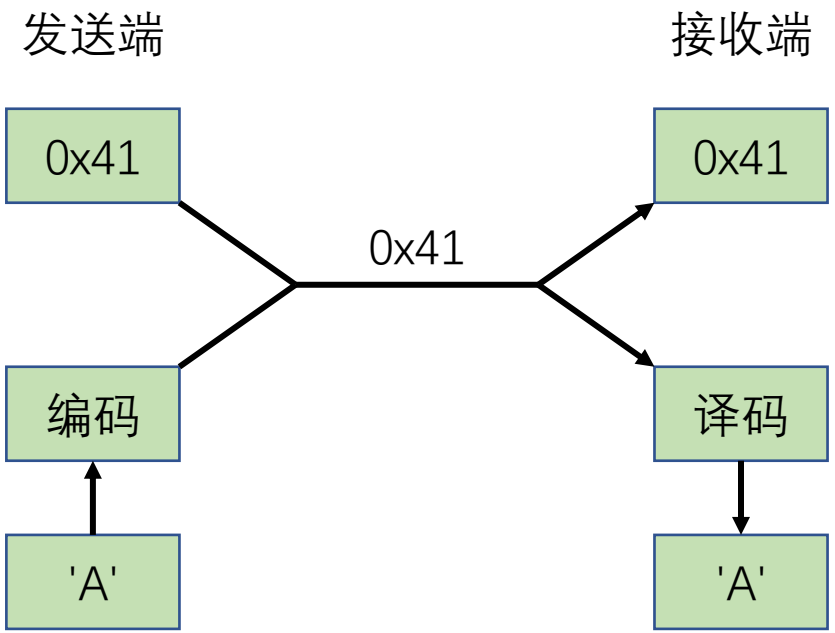
- HEX模式(十六进制模式)：以原始数据的形式显示
- 文本模式/字符模式：以原始数据编码后的形式显示

DEC	HEX	字符	解释
0	0x00	(NUL)	空字符
1	0x01	(SOH)	标题开始
2	0x02	(STX)	正文开始
3	0x03	(ETX)	正文结束
4	0x04	(EOT)	传输结束
5	0x05	(ENQ)	请求
6	0x06	(ACK)	收到通知
7	0x07	(BEL)	响铃 (\a)
8	0x08	(BS)	退格 (\b)
9	0x09	(HT)	水平制表符 (\t)
10	0x0A	(LF)	换行键 (\n)
11	0x0B	(VT)	垂直制表符 (\v)
12	0x0C	(FF)	换页键 (\f)
13	0x0D	(CR)	回车键 (\r)
14	0x0E	(SO)	不用切换
15	0x0F	(SI)	启用切换
16	0x10	(DLE)	数据链路转义
17	0x11	(DC1)	设备控制1
18	0x12	(DC2)	设备控制2
19	0x13	(DC3)	设备控制3
20	0x14	(DC4)	设备控制4
21	0x15	(NAK)	拒绝接收
22	0x16	(SYN)	同步空闲
23	0x17	(ETB)	结束传输块
24	0x18	(CAN)	取消
25	0x19	(EM)	媒介结束
26	0x1A	(SUB)	代替
27	0x1B	(ESC)	换码
28	0x1C	(FS)	文件分隔符
29	0x1D	(GS)	分组符
30	0x1E	(RS)	记录分隔符
31	0x1F	(US)	单元分隔符

DEC	HEX	字符	解释
32	0x20	(space)	空格
33	0x21	!	叹号
34	0x22	"	双引号
35	0x23	#	井号
36	0x24	$	美元符
37	0x25	%	百分号
38	0x26	&	与号
39	0x27	'	单引号
40	0x28	(	左括号
41	0x29	)	右括号
42	0x2A	*	星号
43	0x2B	+	加号
44	0x2C	,	逗号
45	0x2D	-	减号
46	0x2E	.	句号
47	0x2F	/	斜杠
48	0x30	0	字符0
49	0x31	1	字符1
50	0x32	2	字符2
51	0x33	3	字符3
52	0x34	4	字符4
53	0x35	5	字符5
54	0x36	6	字符6
55	0x37	7	字符7
56	0x38	8	字符8
57	0x39	9	字符9
58	0x3A	:	冒号
59	0x3B	;	分号
60	0x3C	<	小于
61	0x3D	=	等号
62	0x3E	>	大于
63	0x3F	?	问号

DEC	HEX	字符	解释
64	0x40	@	电子邮件符号
65	0x41	A	大写字母A
66	0x42	B	大写字母B
67	0x43	C	大写字母C
68	0x44	D	大写字母D
69	0x45	E	大写字母E
70	0x46	F	大写字母F
71	0x47	G	大写字母G
72	0x48	H	大写字母H
73	0x49	I	大写字母I
74	0x4A	J	大写字母J
75	0x4B	K	大写字母K
76	0x4C	L	大写字母L
77	0x4D	M	大写字母M
78	0x4E	N	大写字母N
79	0x4F	O	大写字母O
80	0x50	P	大写字母P
81	0x51	Q	大写字母Q
82	0x52	R	大写字母R
83	0x53	S	大写字母S
84	0x54	T	大写字母T
85	0x55	U	大写字母U
86	0x56	V	大写字母V
87	0x57	W	大写字母W
88	0x58	X	大写字母X
89	0x59	Y	大写字母Y
90	0x5A	Z	大写字母Z
91	0x5B	[	左方括号
92	0x5C	\	反斜杠
93	0x5D	]	右方括号
94	0x5E	^	次方号
95	0x5F	_	下划线

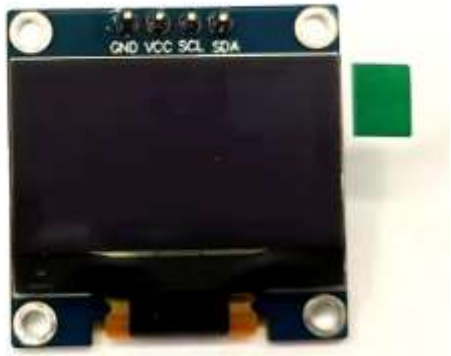
DEC	HEX	字符	解释
96	0x60	`	反单引号
97	0x61	a	小写字母a
98	0x62	b	小写字母b
99	0x63	c	小写字母c
100	0x64	d	小写字母d
101	0x65	e	小写字母e
102	0x66	f	小写字母f
103	0x67	g	小写字母g
104	0x68	h	小写字母h
105	0x69	i	小写字母i
106	0x6A	j	小写字母j
107	0x68	k	小写字母k
108	0x6C	l	小写字母l
109	0x6D	m	小写字母m
110	0x6E	n	小写字母n
111	0x6F	o	小写字母o
112	0x70	p	小写字母p
113	0x71	q	小写字母q
114	0x72	r	小写字母r
115	0x73	s	小写字母s
116	0x74	t	小写字母t
117	0x75	u	小写字母u
118	0x76	v	小写字母v
119	0x77	w	小写字母w
120	0x78	x	小写字母x
121	0x79	y	小写字母y
122	0x7A	z	小写字母z
123	0x78	{	左花括号
124	0x7C	|	竖线
125	0x7D	}	右花括号
126	0x7E	~	波浪号
127	0x7F	(DEL)	删除

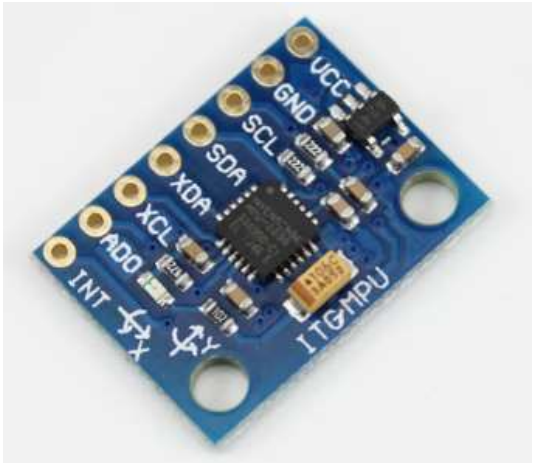


I2C通信简介

艾谷科技
Aigo

- I2C/IIC（Inter IC Bus）是由Philips公司开发的一种通用数据总线
- 两根通信线：SCL（Serial Clock）、SDA（Serial Data）
- 同步，半双工
- 带数据应答
- 总线挂载多设备(一主多从，多主多从)



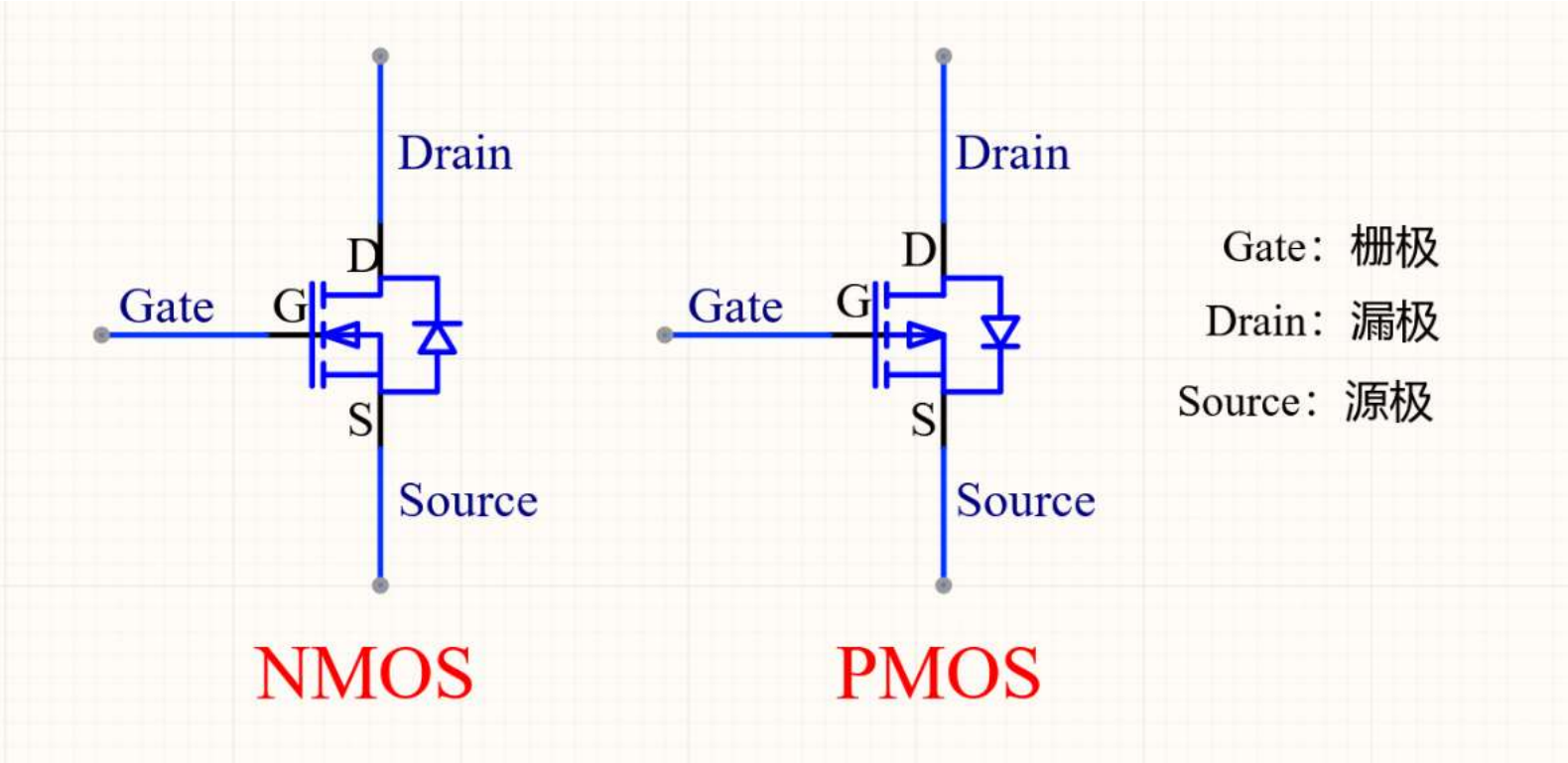




前置小知识

艾谷科技
Aigo

MOSFET晶体管

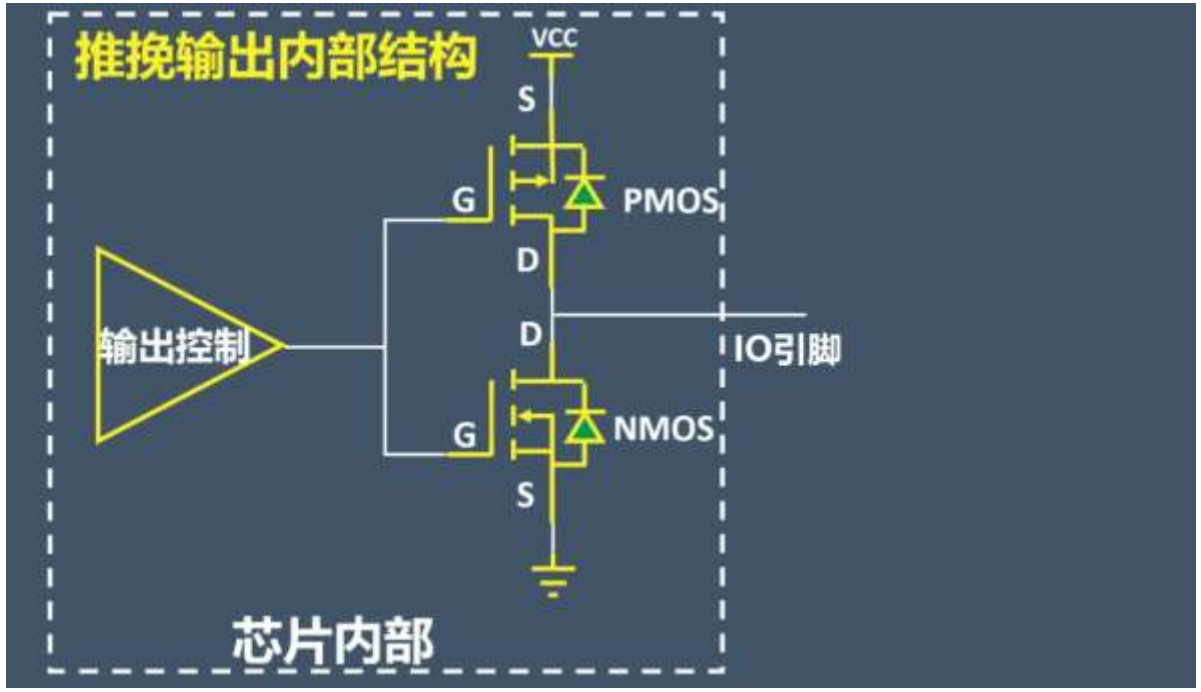


$V_G - V_S > V_{th}$，DS导通　　$V_S - V_G > V_{th}$，SD导通

前置小知识

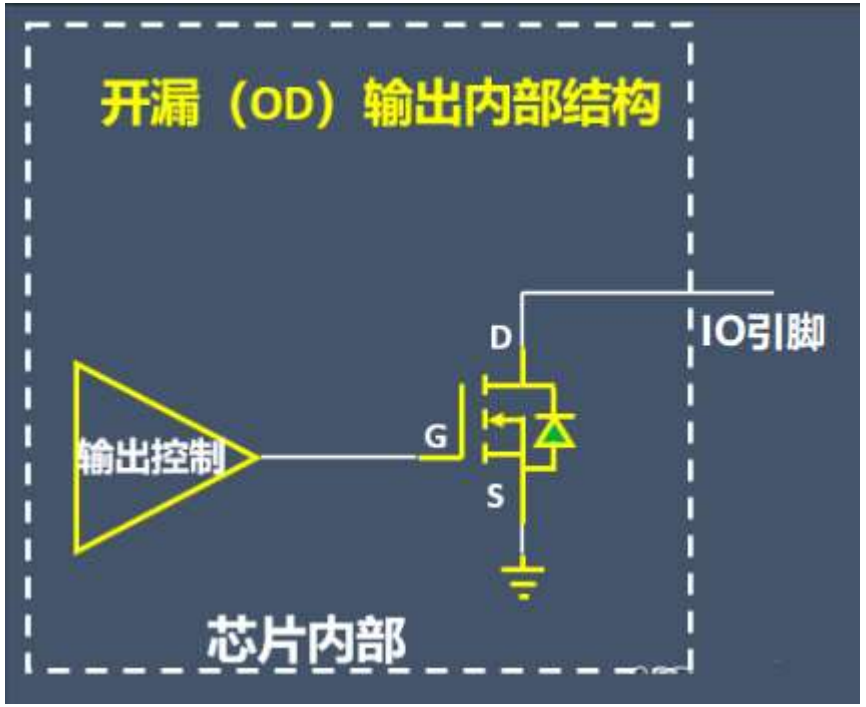
艾谷科技
Aigo

推挽输出



输出高电平，3.3V
输出低电平，0V

开漏输出

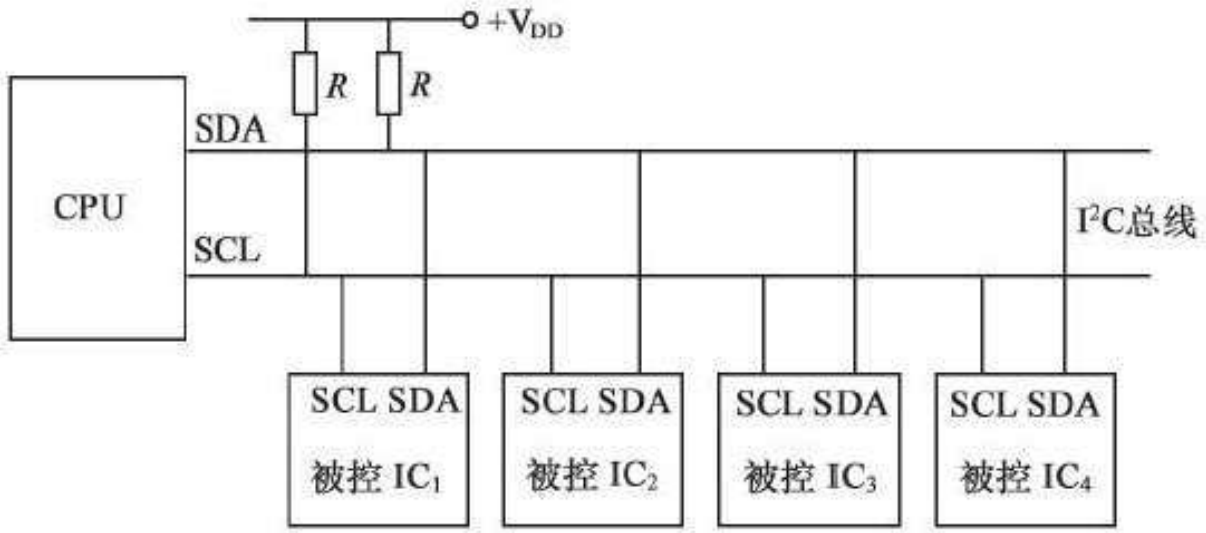


输出高电平，浮空电平
输出低电平，0V

硬件电路

艾谷科技
Aigo

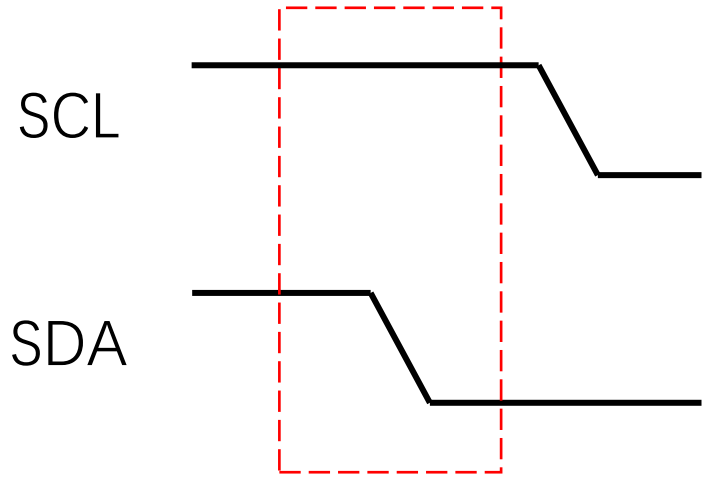
- 所有I2C设备的SCL连在一起，SDA连在一起
- 设备的SCL和SDA均要配置成开漏输出模式
- SCL和SDA各添加一个上拉电阻，阻值一般为4.7KΩ左右

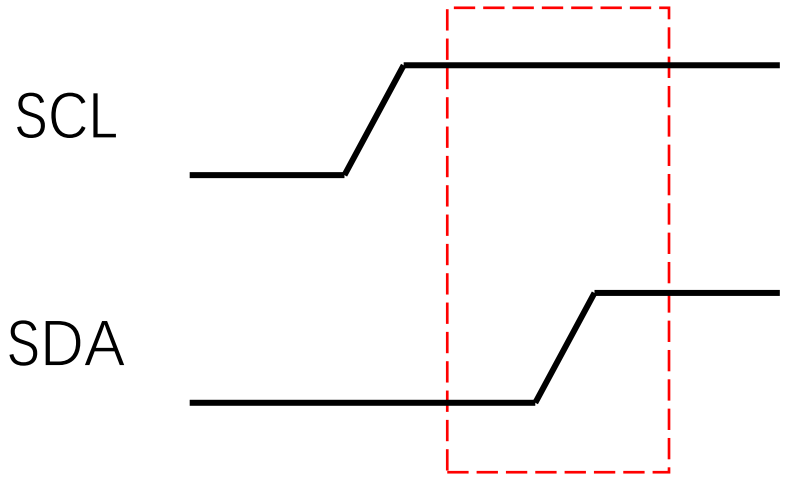


I2C时序基本单元

艾谷科技
Aigo

- 起始条件：SCL高电平期间，SDA从高电平切换到低电平
- 终止条件：SCL高电平期间，SDA从低电平切换到高电平

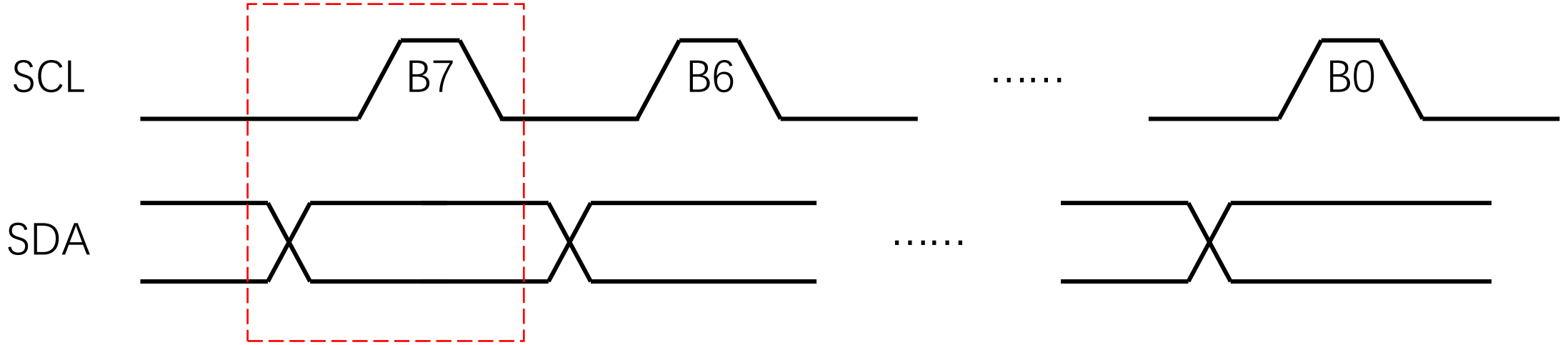




I2C时序基本单元

艾谷科技
Aigo

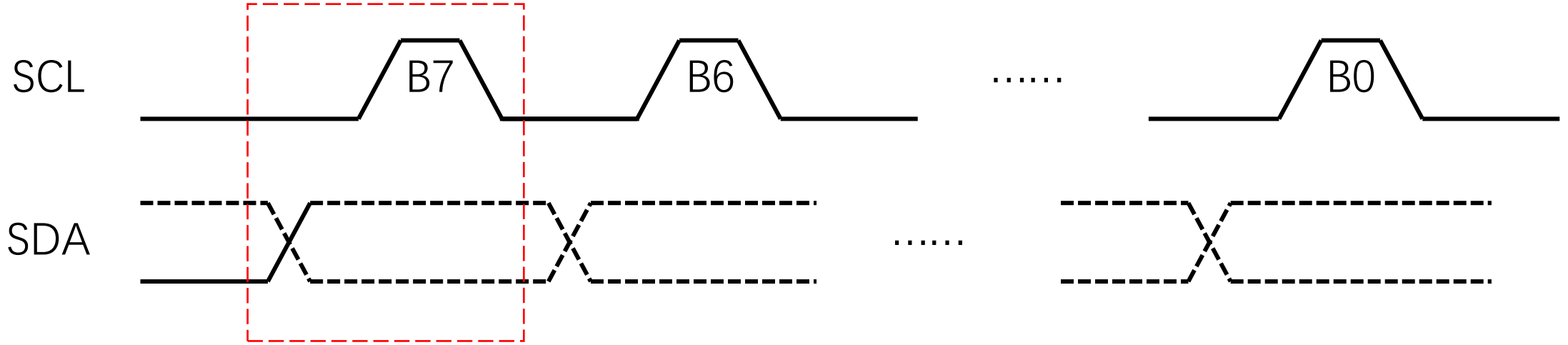
- 发送一个字节：SCL低电平期间，主机将数据位依次放到SDA线上（高位先行），然后释放SCL，从机将在SCL高电平期间读取数据位，依次循环上述过程8次，即可发送一个字节



I2C时序基本单元

艾谷科技
Aigo

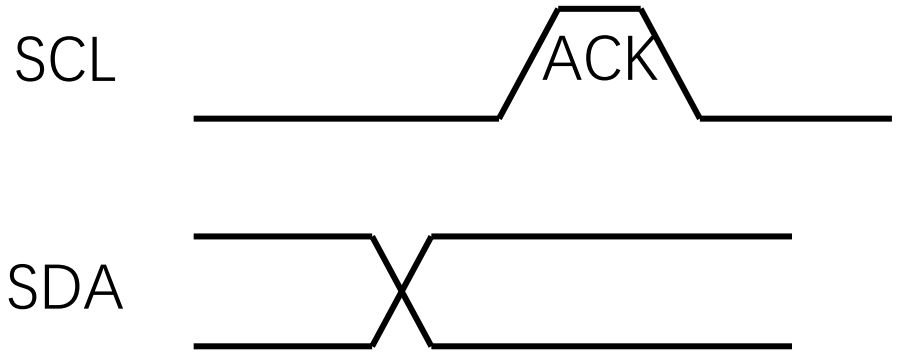
- 接收一个字节：SCL低电平期间，从机将数据位依次放到SDA线上（高位先行），然后释放SCL，主机将在SCL高电平期间读取数据位，依次循环上述过程8次，即可接收一个字节（主机在接收之前，需要释放SDA）

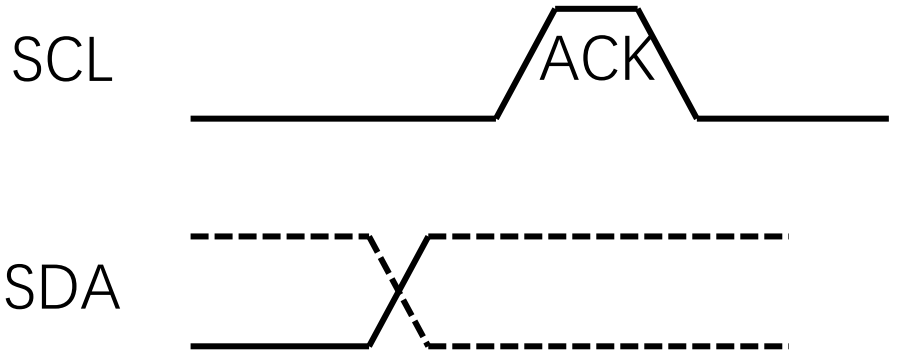


I2C时序基本单元

艾谷科技
Aigo

- 发送应答：主机在接收完一个字节之后，在下一个时钟发送一位数据，数据0表示应答，数据1表示非应答
- 接收应答：主机在发送完一个字节之后，在下一个时钟接收一位数据，判断从机是否应答，数据0表示应答，数据1表示非应答（主机在接收之前，需要释放SDA）

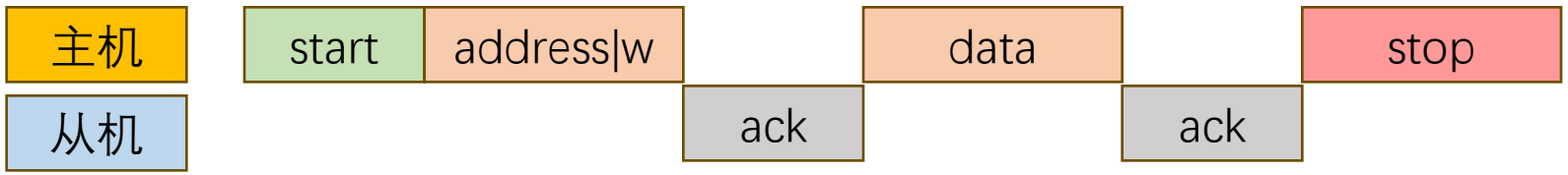




I2C时序

艾谷科技
Aigo

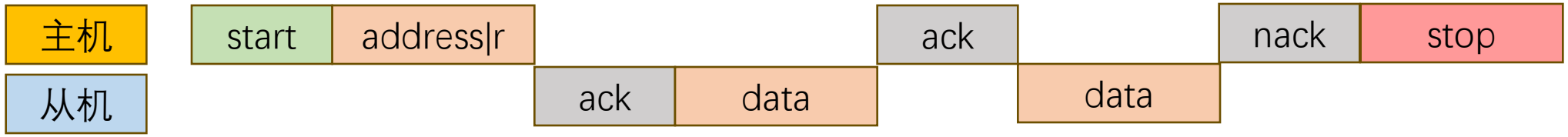
主机向从机发送数据



I2C时序

艾谷科技
Aigo

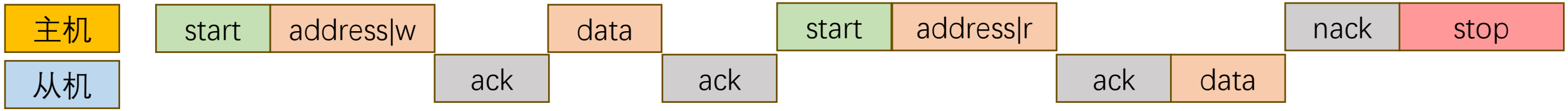
主机从从机读取数据



I2C时序

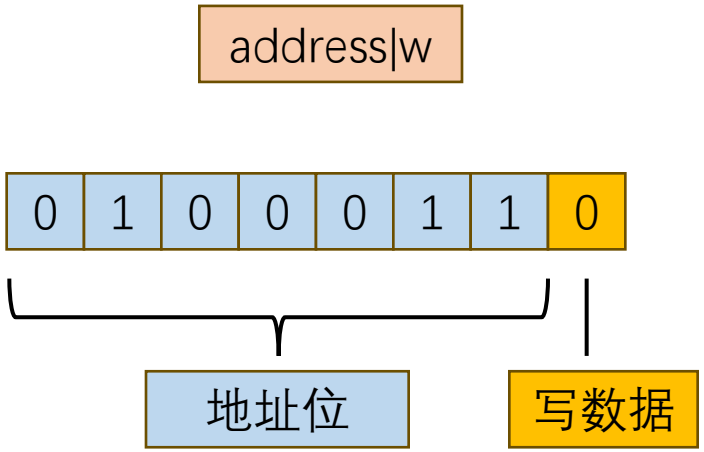
艾谷科技
Aigo

主机向从机发送数据并读取数据

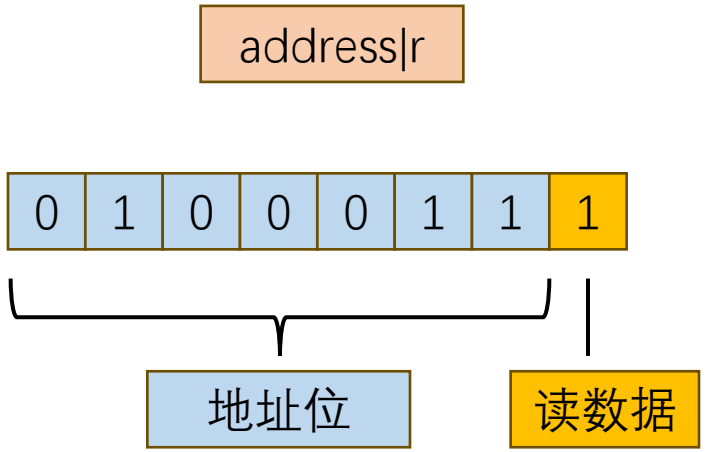


地址字节的构成

艾谷科技
Aigo



0x46



0x47

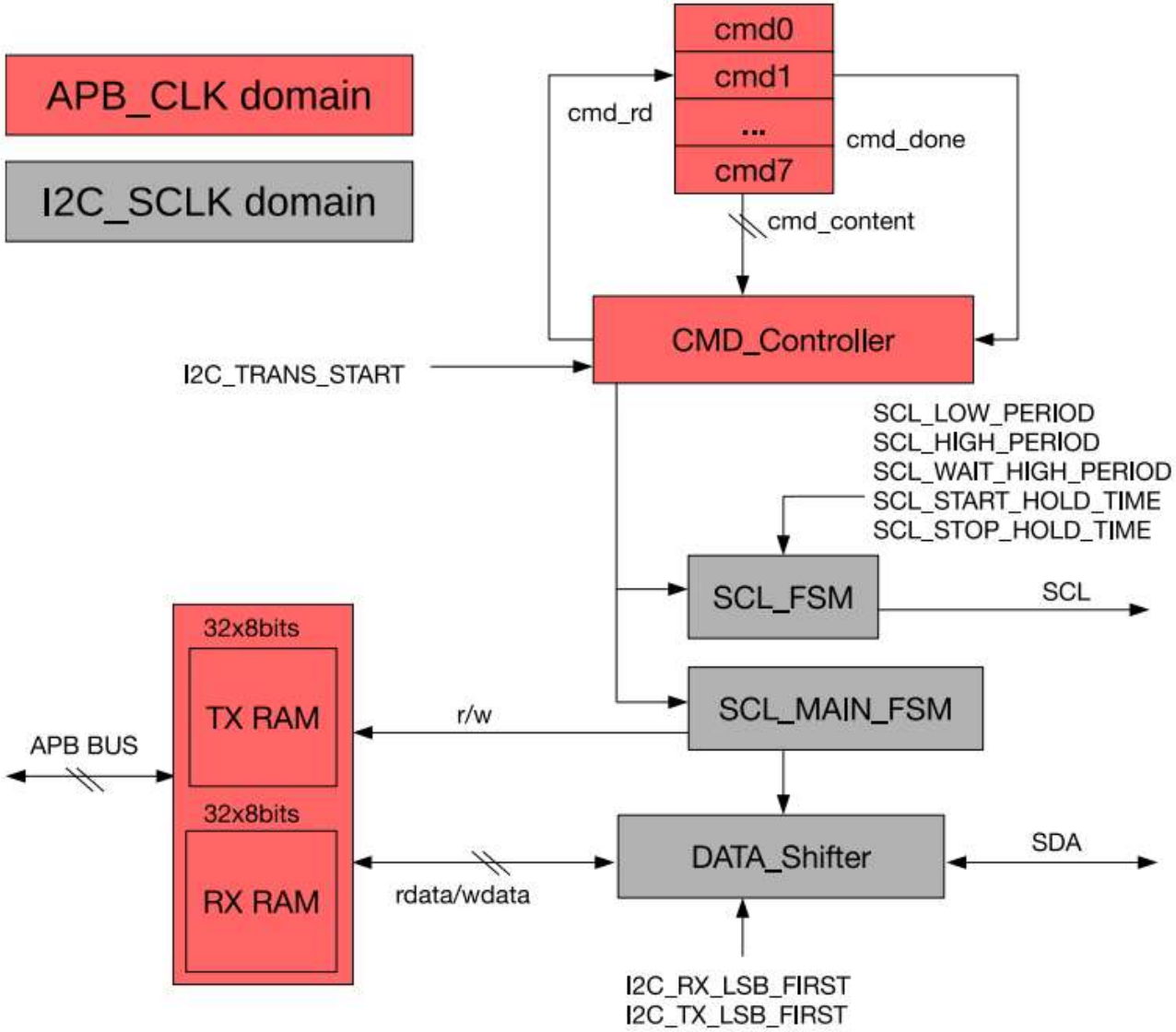
I2C外设简介

艾谷科技
Aigo

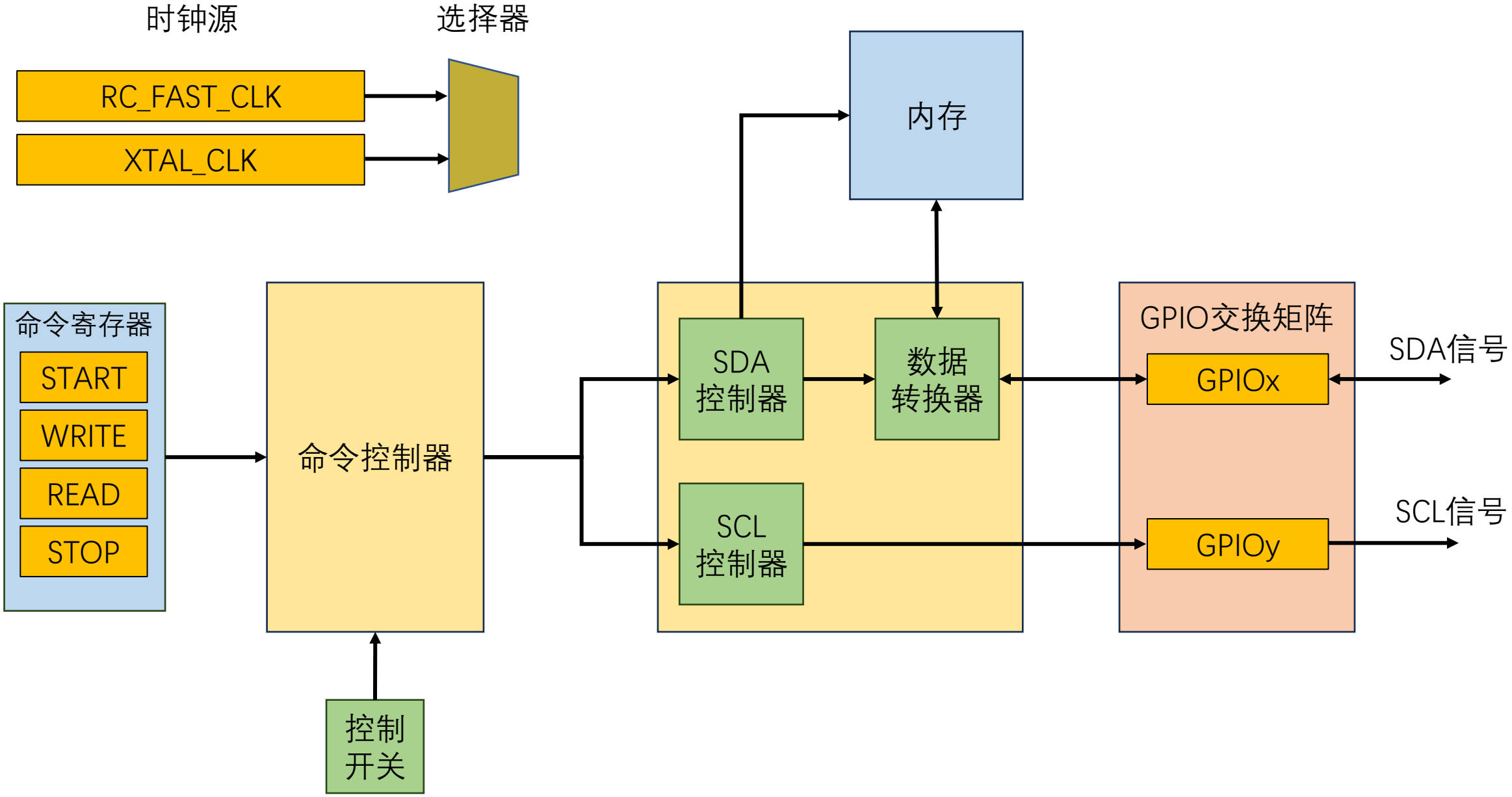
- ESP32-S3内部集成2个I2C控制器，负责处理I2C总线上的通信
- 支持主机模式、从机模式
- 支持7位/10位地址模式
- 支持不同的通讯速度，标准速度(100 kHz)，快速(400 kHz)

- ESP32-S3硬件I2C资源：I2C0、I2C1

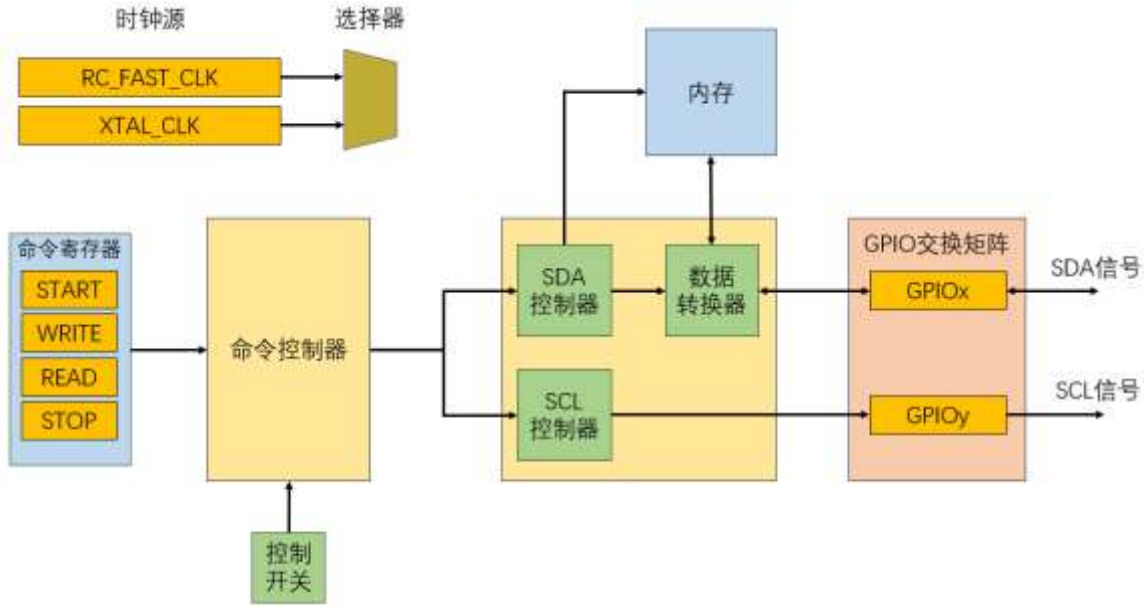
I2C框图


APB_CLK domain
I2C_SCLK domain
cmd0
cmd1
...
cmd7
cmd_rd
cmd_done
cmd_content
CMD_Controller
I2C_TRANS_START
SCL_LOW_PERIOD
SCL_HIGH_PERIOD
SCL_WAIT_HIGH_PERIOD
SCL_START_HOLD_TIME
SCL_STOP_HOLD_TIME
SCL_FSM
SCL
32x8bits
TX RAM
r/w
SCL_MAIN_FSM
APB BUS
32x8bits
RX RAM
rdata/wdata
DATA_Shifter
SDA
I2C_RX_LSB_FIRST
I2C_TX_LSB_FIRST

I2C工作流程


时钟源
选择器
RC_FAST_CLK
XTAL_CLK
内存
命令寄存器
START
WRITE
READ
STOP
命令控制器
SDA
控制器
数据
转换器
SCL
控制器
GPIO交换矩阵
GPIOx
GPIOy
SDA信号
SCL信号
控制
开关

I2C配置流程



- 使用 “i2c_param_config” 函数配置时钟源、GPIO交换矩阵引脚、I2C参数
- 使用 “i2c_driver_install” 向CPU注册I2C

- 使用 “i2c_cmd_link_create” 函数创建命令寄存器中的命令链(存放命令的数组)
- 使用 “i2c_master_start” 函数向命令链中存入I2C_START命令
- 使用 “i2c_master_write_byte” 函数向命令链中存入I2C_ WRITE命令
- 使用 “i2c_master_read_byte” 函数向命令链中存入I2C_ READ命令
- 使用 “i2c_master_stop” 函数向命令链中存入I2C_ STOP命令
- 使用 “i2c_master_cmd_begin” 函数开启命令控制器，开始IIC通信
- 使用 “i2c_cmd_link_delete” 函数删除命令链，释放内存空间

BH1750简介

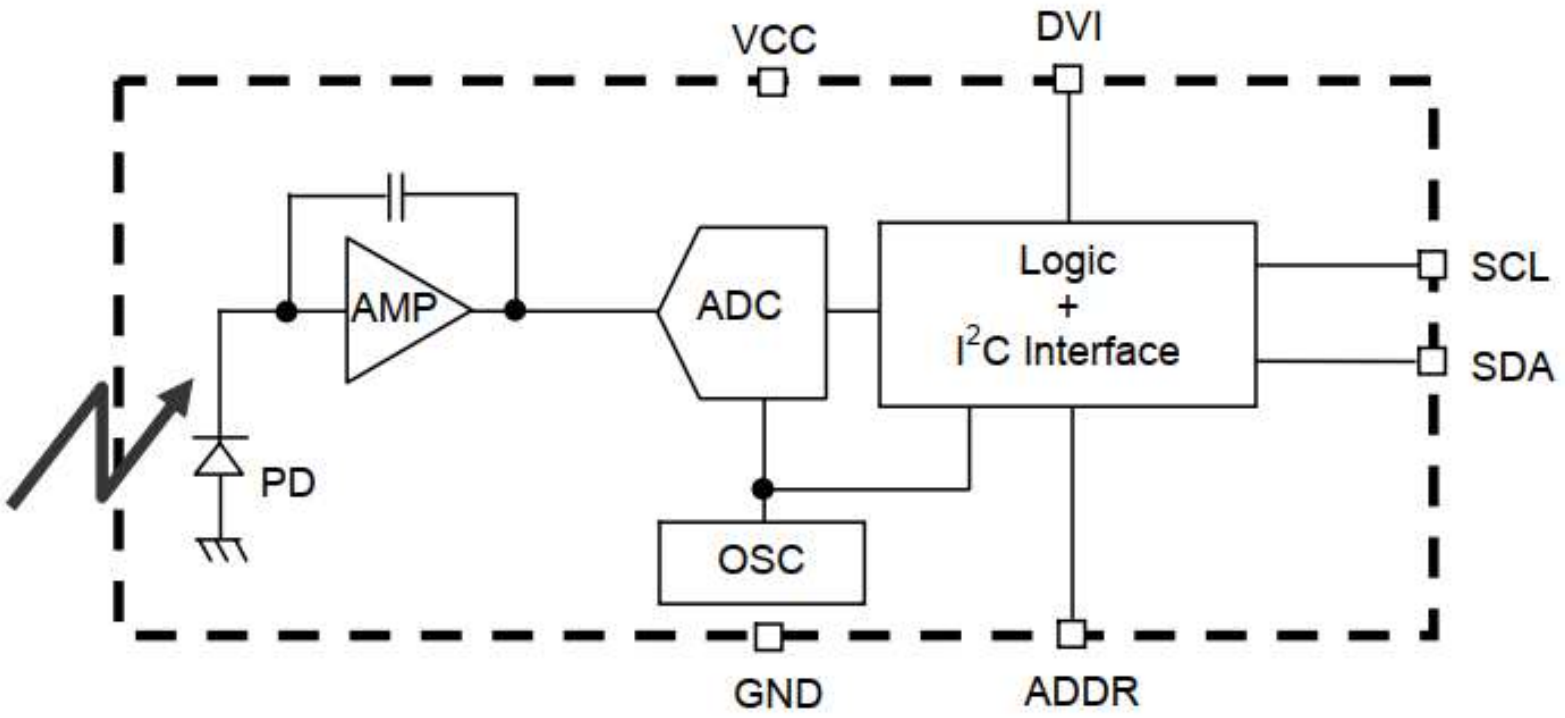
艾谷科技
Aigo

- BH1750FVI 是一款适用于 I2C 总线接口的数字环境光传感器 IC
- 适用于获取环境光数据，以调整 LCD 和键盘背光
- 检测光照强度分辨率：1 - 65535 勒克斯
- 支持标准I2C（100KHz）、和快速I2C（400KHz）



BH1750结构框图

艾谷科技
Aigo



环境光 → 电信号 → 数字信号 → I2C通信输出

IIC通信必要的参数

1.从机模块的工作流程：确定指令发送顺序

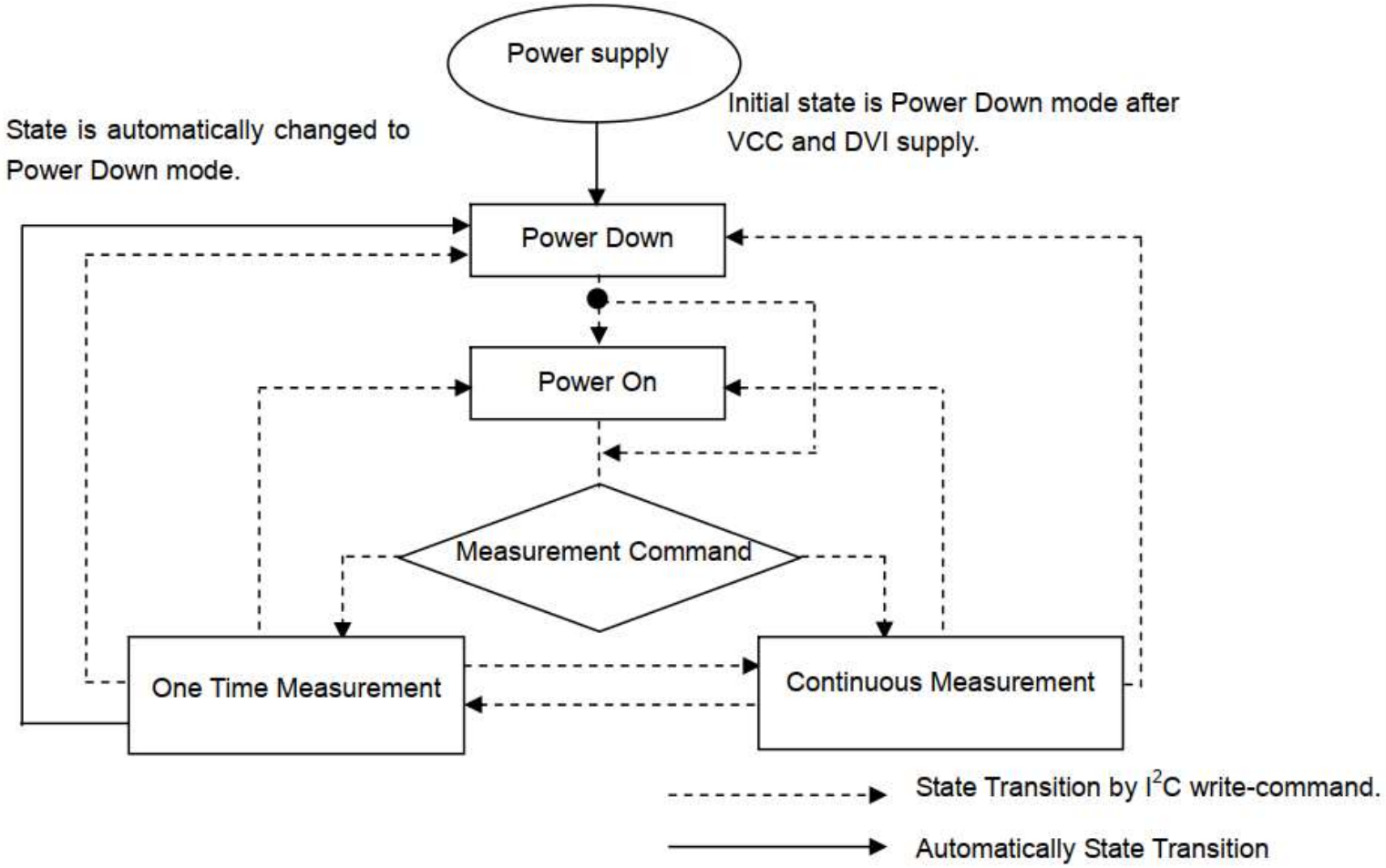
2.从机模块的指令集：确保能够正确操作从机

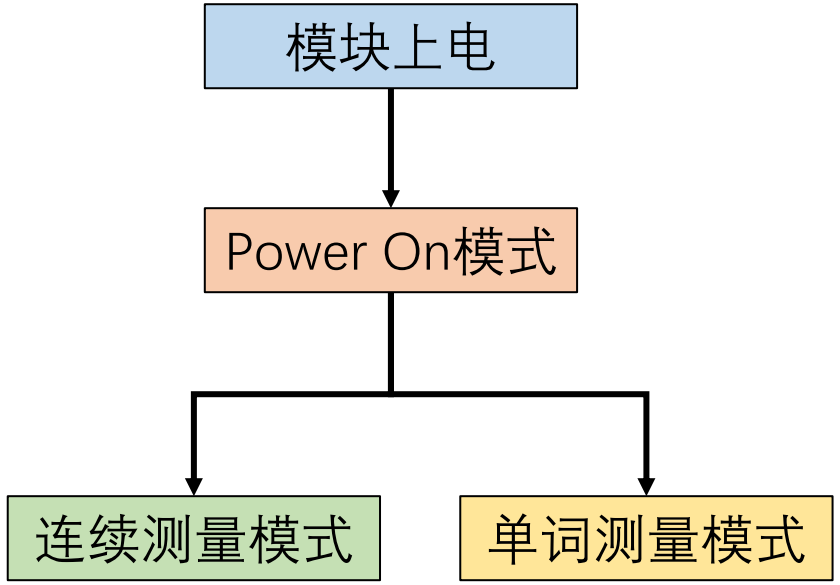
3.从机模块的I2C通信速度：确保IIC通信正常进行

4.从机模块的地址：确保主机能够正确呼唤从机

5.从机模块的数据帧结构：确保主机和从机能够正确解析数据

BH1750工作流程


Power supply
Initial state is Power Down mode after VCC and DVI supply.
State is automatically changed to Power Down mode.
Power Down
Power On
Measurement Command
One Time Measurement
Continuous Measurement
State Transition by I2C write-command.
Automatically State Transition


模块上电
Power On模式
连续测量模式
单词测量模式

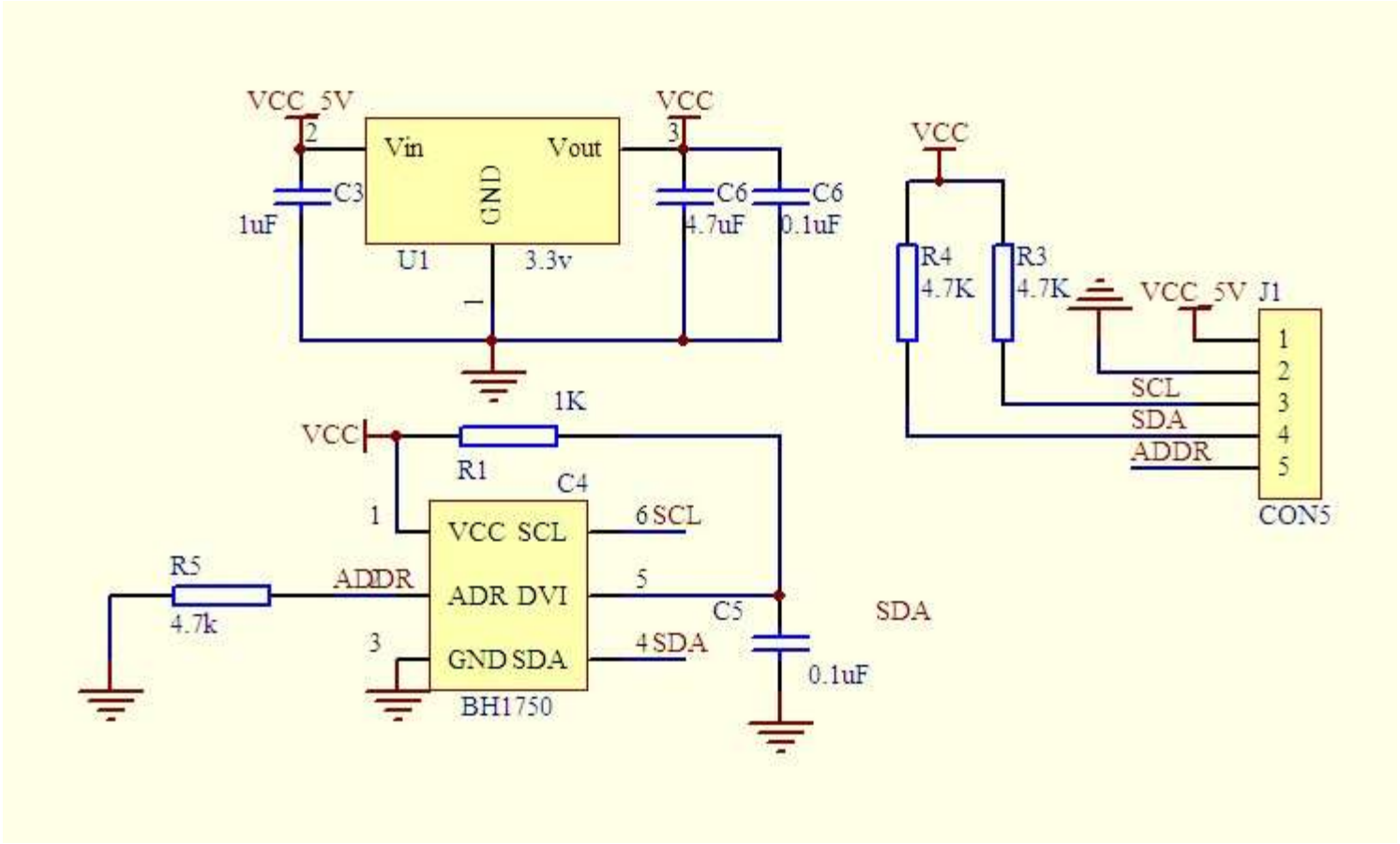
BH1750指令集

艾谷科技
Aigo

指令集	操作码	功能
掉电模式	0000_0000（0x00）	无活跃状态，低功耗
上电模式	0000_0001（0x01）	等待测量指令
复位指令	0000_0111（0x07）	复位数据寄存器，掉电模式无响应
连续高分辨率模式	0001_0000（0x10）	1lx分辨率，测量时间120ms
连续高分辨率模式2	0001_0001（0x11）	0.5lx分辨率，测量时间120ms
连续低分辨率模式	0001_0011（0x13）	4lx分辨率，测量时间16ms
单次高分辨率模式	0010_0000（0x20）	1lx分辨率，测量时间120ms
单次高分辨率模式2	0010_0001（0x21）	0.5lx分辨率，测量时间120ms
单次低分辨率模式	0010_0011（0x23）	4lx分辨率，测量时间16ms

BH1750模块原理图和地址

艾谷科技
Aigo



从机读地址:
01000111（0x47）
从机写地址:
01000110（0x46）

2) Slave Address
Slave Address is 2 types, it is determined by ADDR Terminal
ADDR = 'H' (ADDR ≧ 0.7VCC) → "1011100"
ADDR = 'L' (ADDR ≦ 0.3VCC) → "0100011"

BH1750数据帧结构

艾谷科技
Aigo

from Master to Slave

from Slave to Master

① Send "Continuously H-resolution mode " instruction

ST	0100011	0	Ack	00010000	Ack	SP

② Wait to complete 1st H-resolution mode measurement.(max. 180ms.)

③ Read measurement result.

ST	0100011	1	Ack	High Byte [15:8]	Ack

Low Byte [7:0]	$\overline{\text{Ack}}$	SP

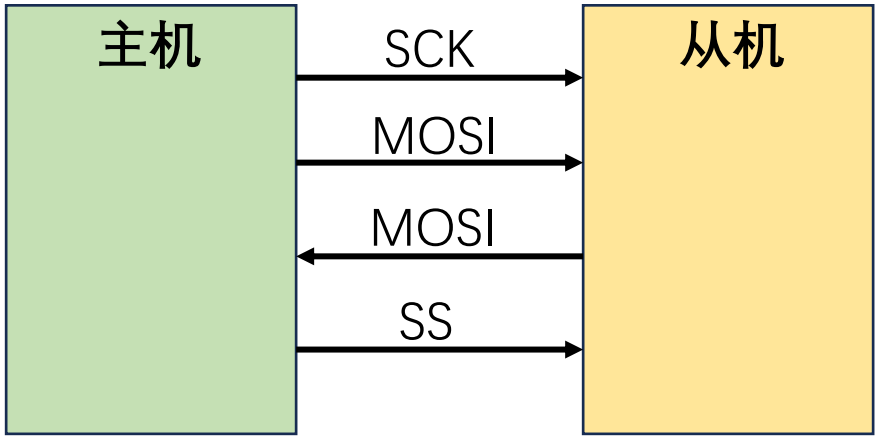
How to calculate when the data High Byte is "10000011" and Low Byte is "10010000"

$(2^{15} + 2^{9} + 2^{8} + 2^{7} + 2^{4}) / 1.2 \fallingdotseq 28067$ [lx]

SPI通信简介

艾谷科技
Aigo

- SPI（Serial Peripheral Interface）是由Motorola公司开发的一种通用数据总线
- 支持总线挂载多设备（一主多从）
- 四根通信线：SCK（Serial Clock）、MOSI（Master Output Slave Input）、MISO（Master Input Slave Output）、SS（Slave Select）
- 同步，全双工



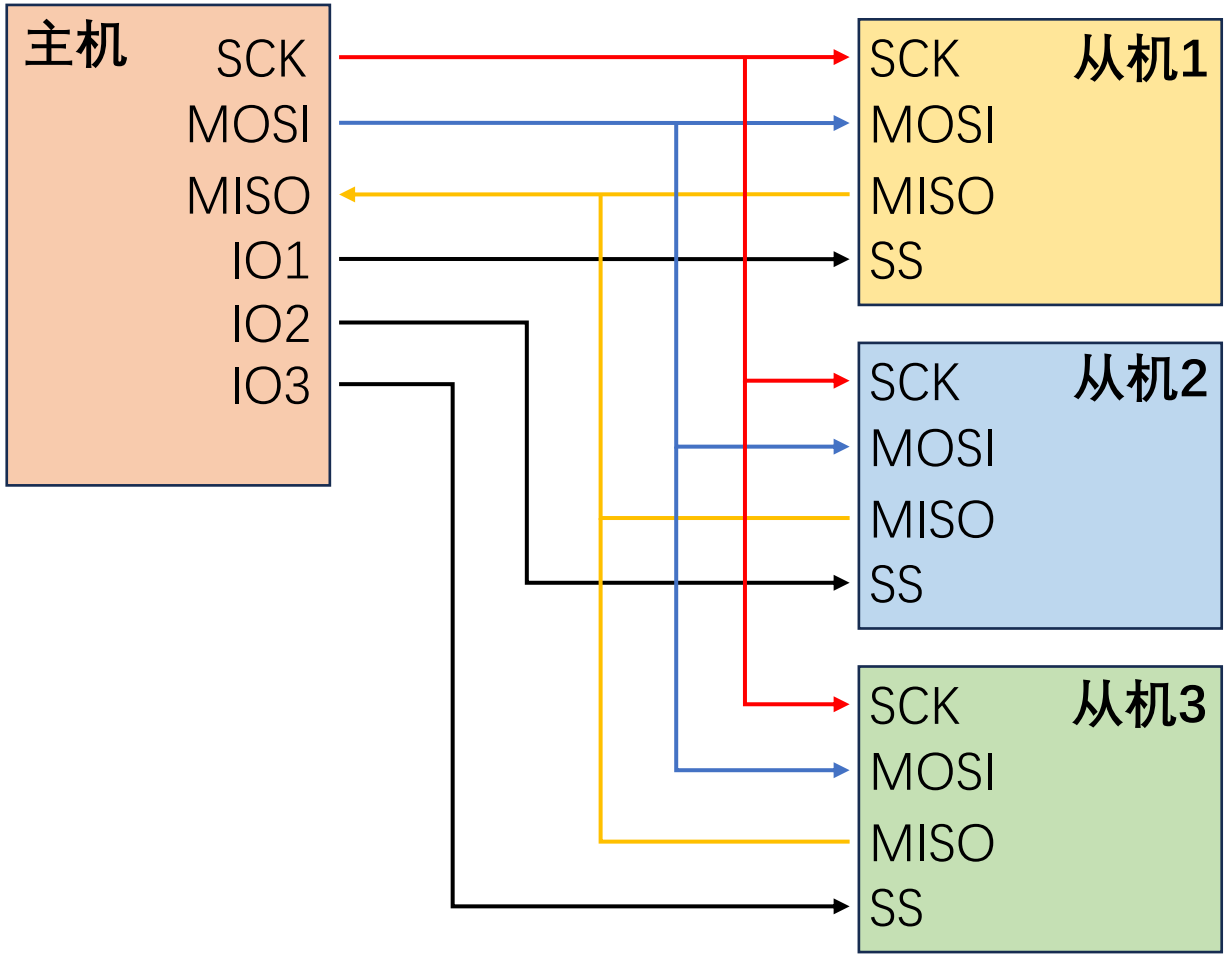




硬件电路

艾谷科技
Aigo

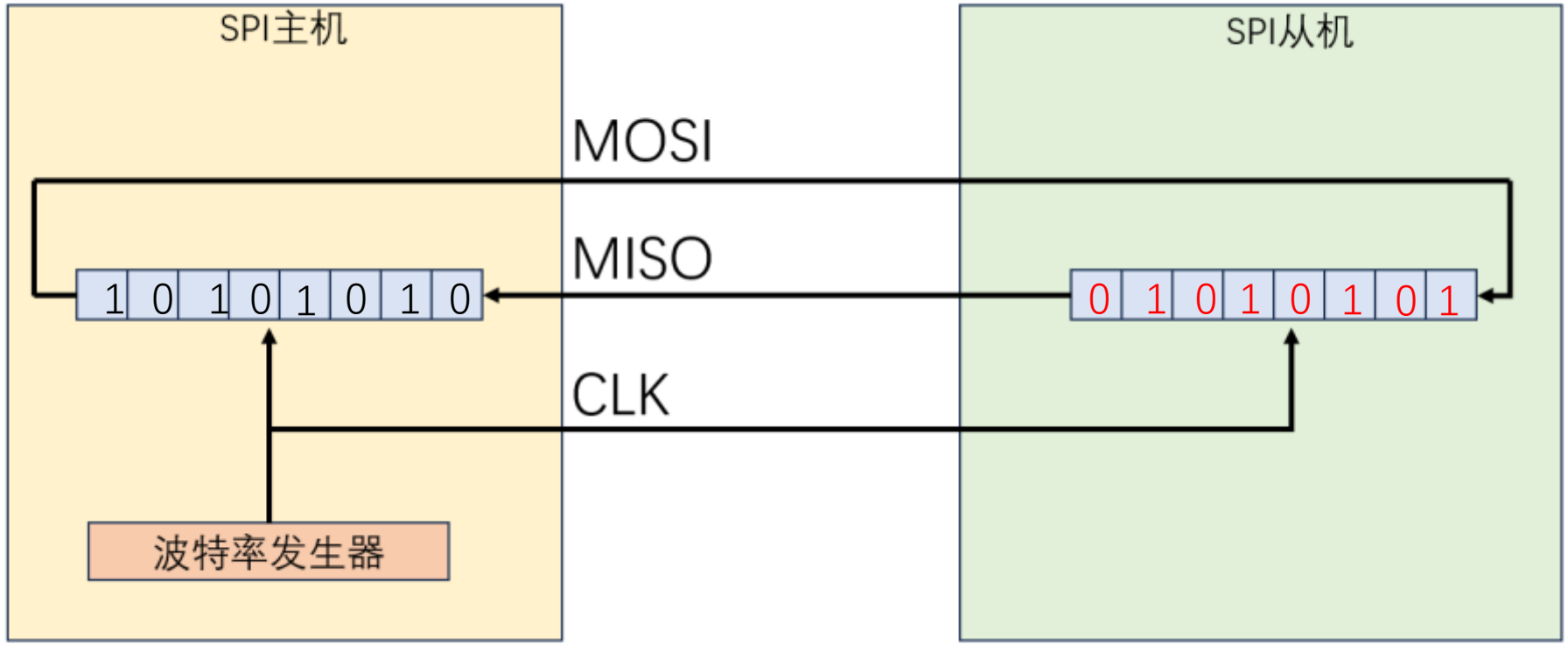
- 所有SPI设备的SCK、MOSI、MISO分别连在一起
- 主机另外引出多条SS控制线，分别接到各从机的SS引脚



通信示意图

艾谷科技
Aigo

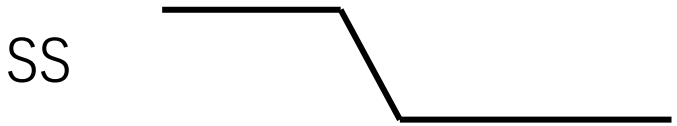
- SPI通信过程：数据交换



SPI时序基本单元

艾谷科技
Aigo

- 起始条件：SS从高电平切换到低电平
- 终止条件：SS从低电平切换到高电平





SPI模式真值表

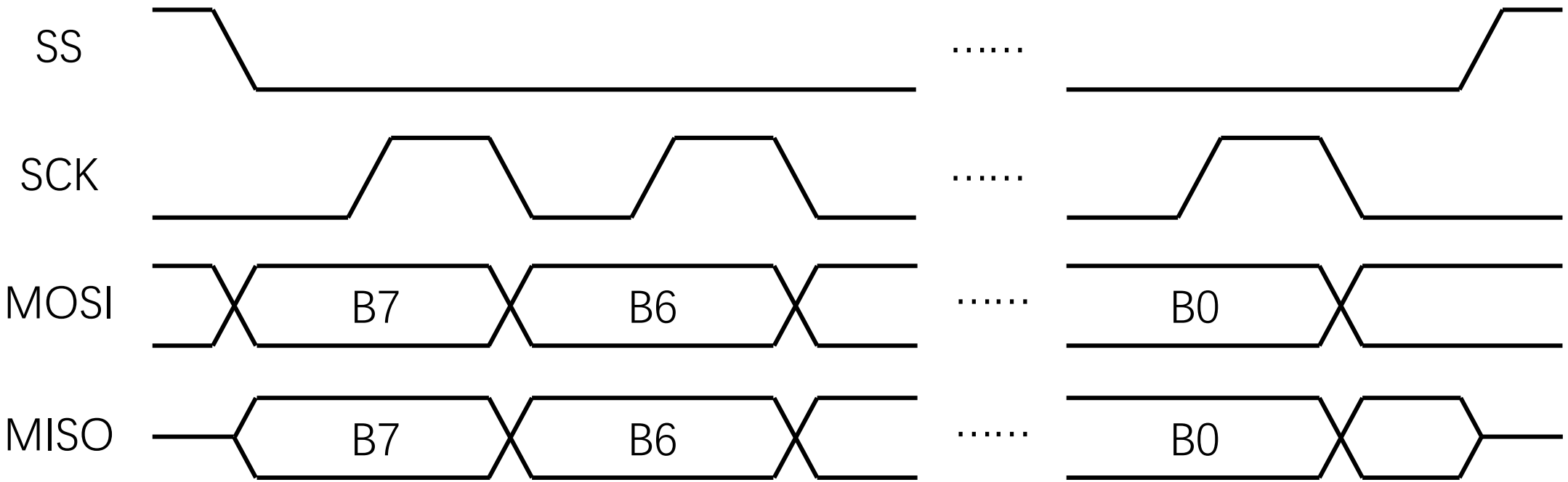
艾谷科技
Aigo

SPI工作模式	CPOL	CPHA	SCL空闲状态	数据移入	数据移出
0	0	0	低电平	奇数边沿	偶数边沿
1	0	1	低电平	偶数边沿	奇数边沿
2	1	0	高电平	奇数边沿	偶数边沿
3	1	1	高电平	偶数边沿	奇数边沿

SPI时序基本单元

艾谷科技
Aigo

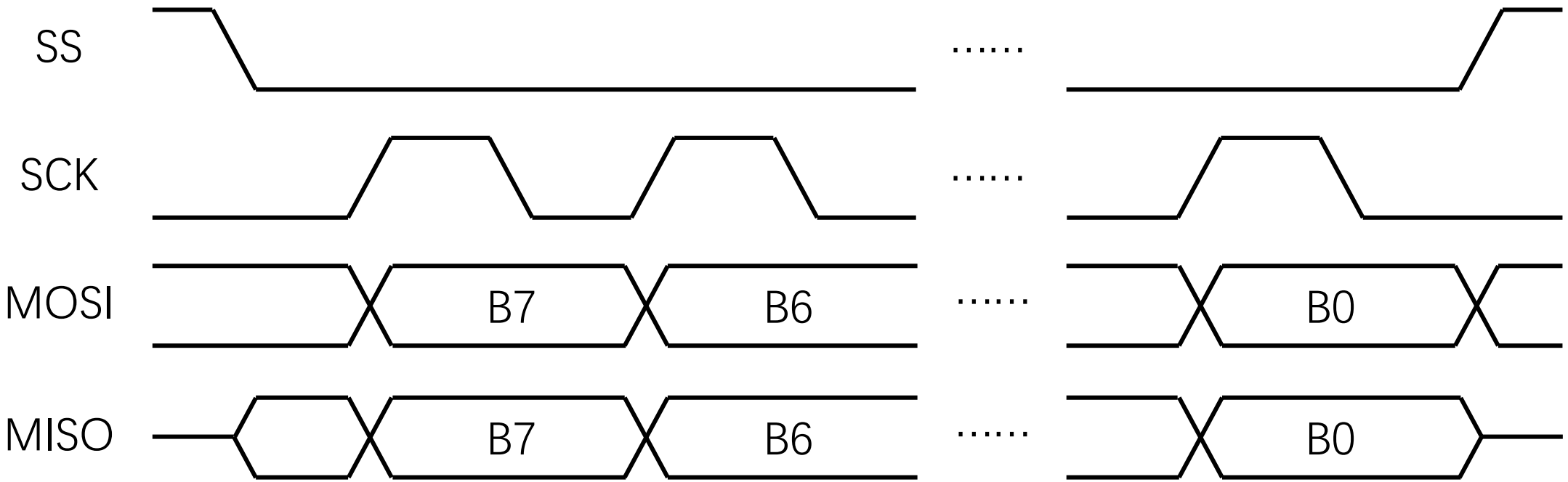
- 交换一个字节（模式0）
- CPOL=0：空闲状态时，SCK为低电平
- CPHA=0：SCK奇数边沿移入数据，偶数边沿移出数据



SPI时序基本单元

艾谷科技
Aigo

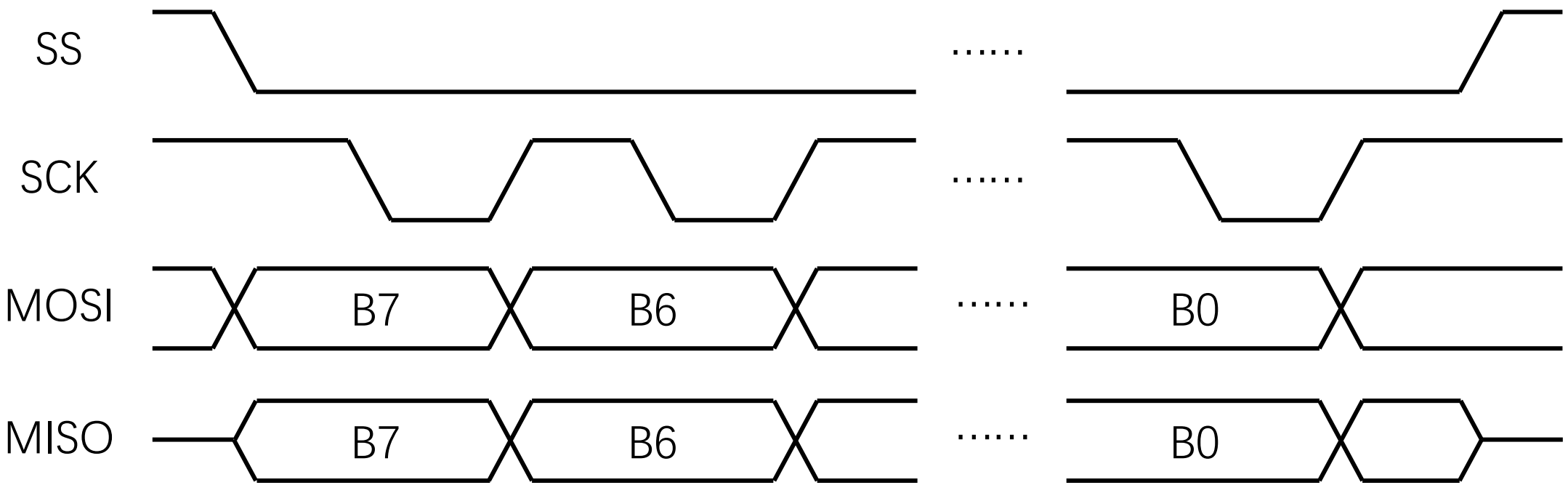
- 交换一个字节（模式1）
- CPOL=0：空闲状态时，SCK为低电平
- CPHA=1：SCK奇数边沿移出数据，偶数边沿移入数据



SPI时序基本单元

艾谷科技
Aigo

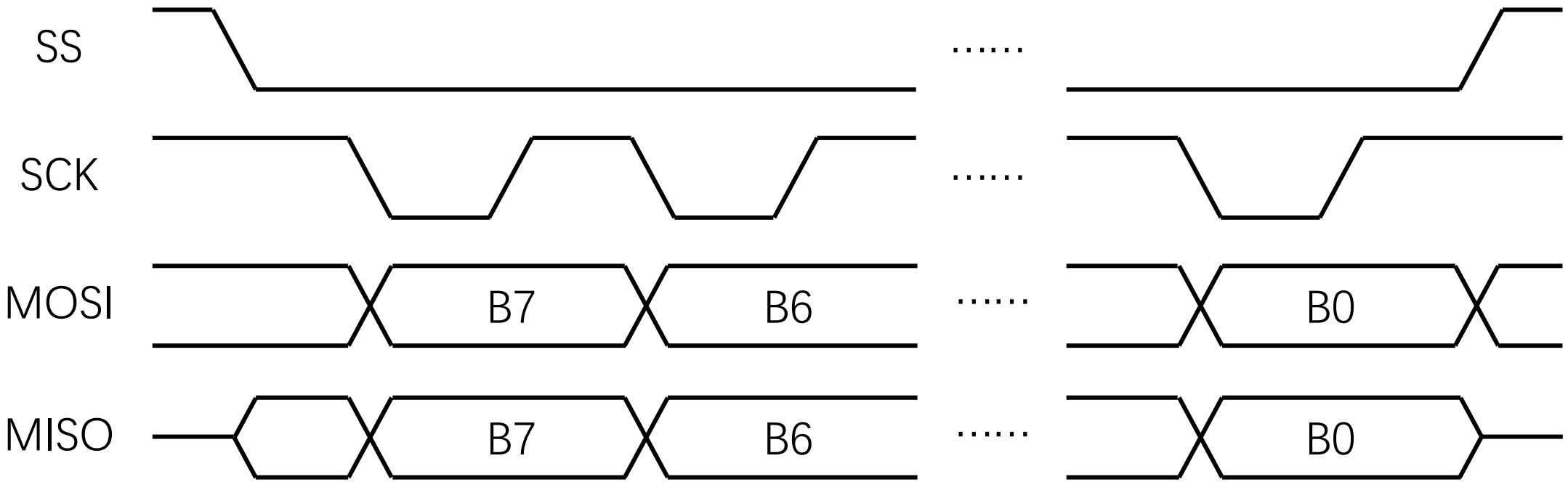
- 交换一个字节（模式2）
- CPOL=1：空闲状态时，SCK为高电平
- CPHA=0：SCK奇数边沿移入数据，偶数边沿移出数据



SPI时序基本单元

艾谷科技
Aigo

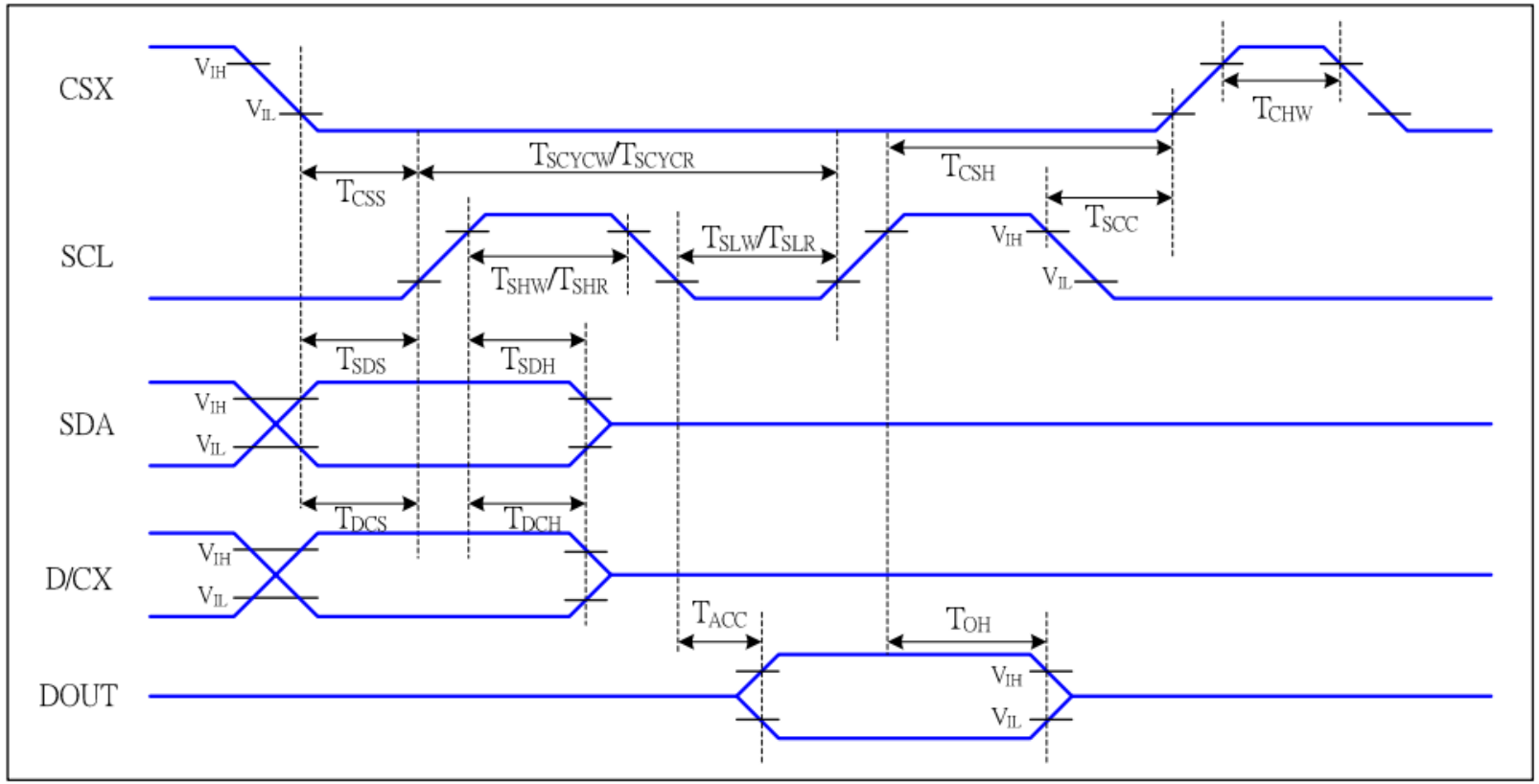
- 交换一个字节（模式3）
- CPOL=1：空闲状态时，SCK为高电平
- CPHA=1：SCK奇数边沿移出数据，偶数边沿移入数据



LCD字节交换模式

艾谷科技
Aigo

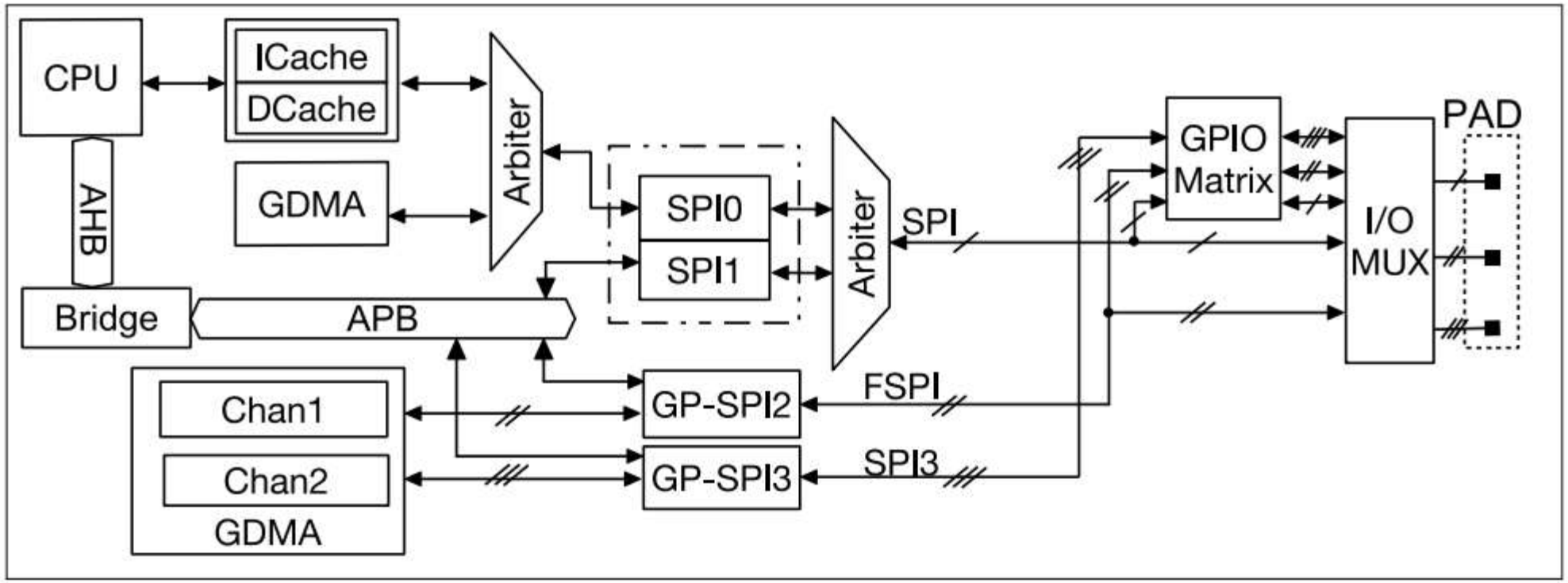
- 交换一个字节（模式0）
- CPOL=0：空闲状态时，SCK为低电平
- CPHA=0：SCK奇数边沿移入数据，偶数边沿移出数据



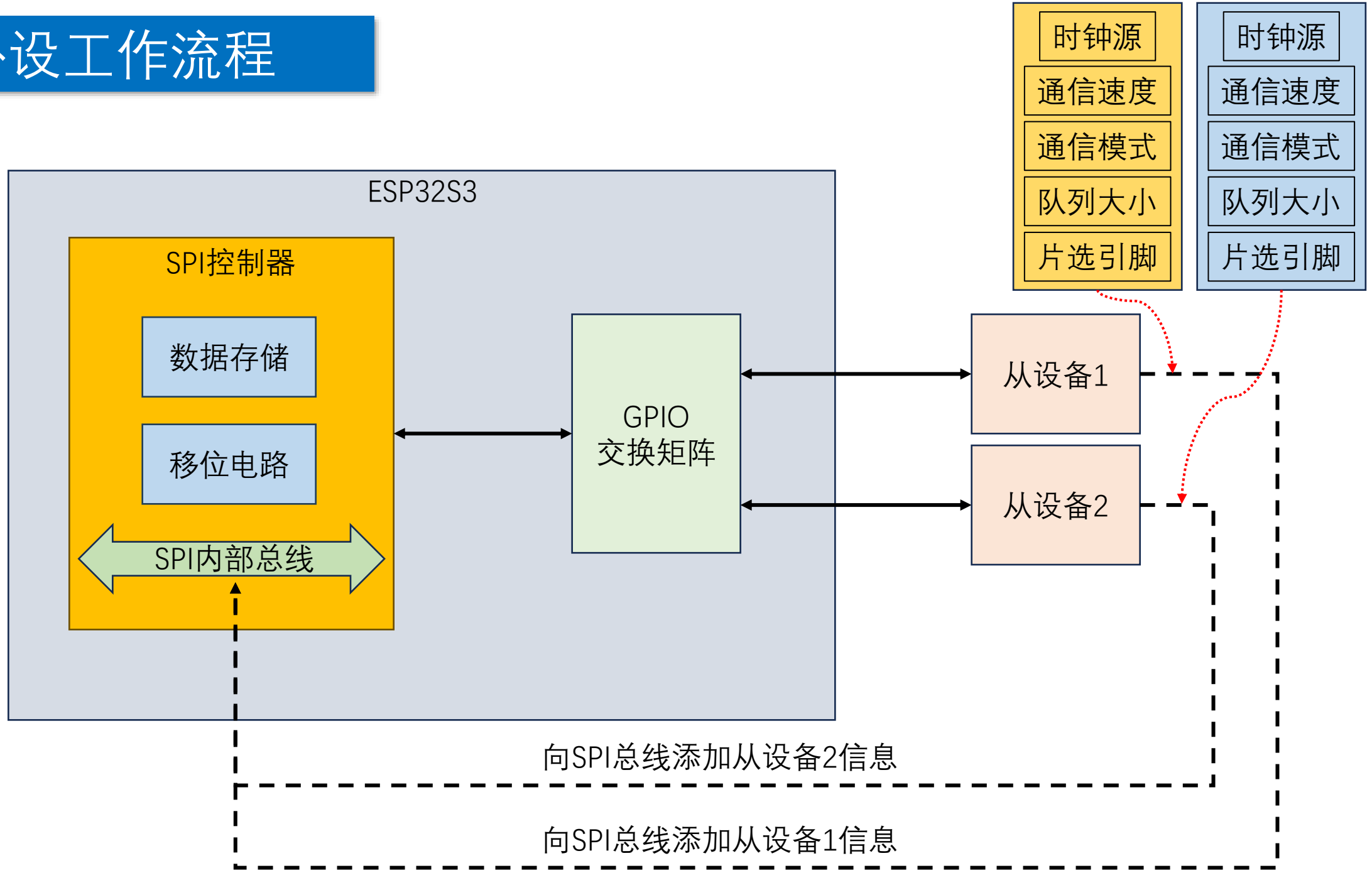
SPI外设

- ESP32-S3内部集成了4个SPI控制器：
  SPI0、SPI1、通用SPI2（GP-SPI2）、通用SPI3（GP-SPI3）
- GP-SPI2最多可挂载6个从设备，GP-SPI3最多可挂载3个从设备
- 支持主机模式和从机模式
- 主机模式最大时钟速度：80MHz
- 支持半双工通信和全双工通信
- 支持多种数据格式：
  1-bit SPI：1个时钟周期传输1bit
  2-bit Dual SPI：1个时钟周期传输2bit
  4-bit Quad SPI：1个时钟周期传输4bit
  8-bit Octal SPI：1个时钟周期传输8bit

SPI外设内部架构


CPU
ICache
DCache
AHB
GDMA
Arbiter
Bridge
APB
SPI0
SPI1
Arbiter
SPI
GPIO
Matrix
I/O
MUX
PAD
Chan1
Chan2
GDMA
GP-SPI2
GP-SPI3
FSPI
SPI3

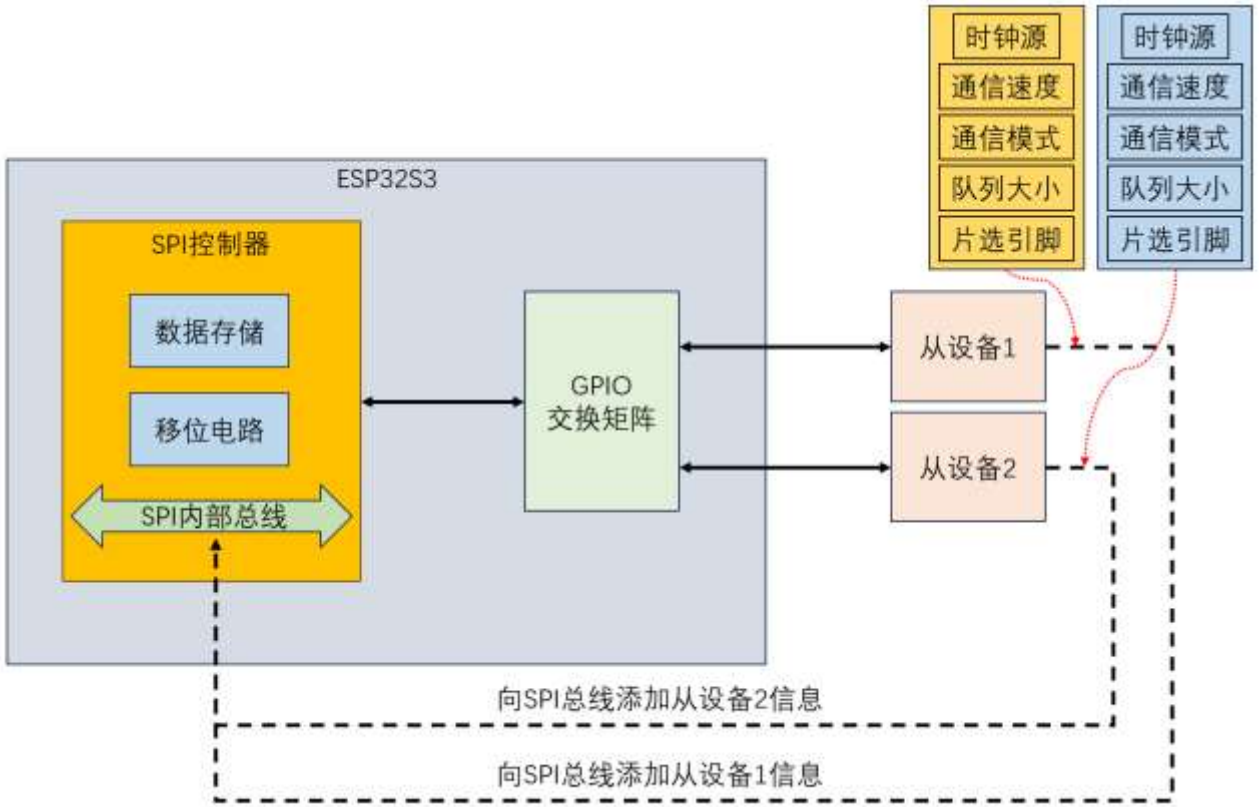
SPI外设工作流程



艾谷科技
Aigo

SPI外设代码配置流程

艾谷科技
Aigo



- 使用 “spi_bus_initialize” 函数配置SPI控制器参数和GPIO交换矩阵
- 使用 “spi_bus_add_device” 函数向SPI总线添加从设备信息
- 使用 “spi_device_polling_transmit” 函数进行数据交换

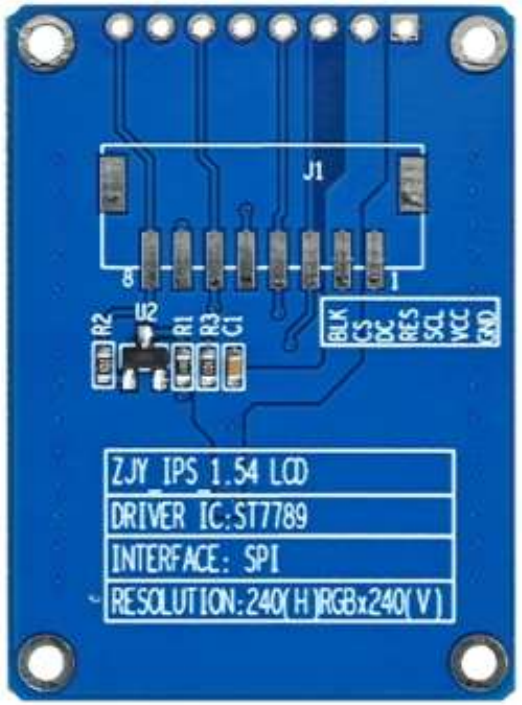
TFT-LCD简介

艾谷科技
Aigo

TFT-LCD，全称Thin Film Transistor Liquid Crystal Display，中文名：薄膜晶体管液晶显示器，也叫做真彩液晶显示器。

LCD模块组成：裸屏+底板+外围电路





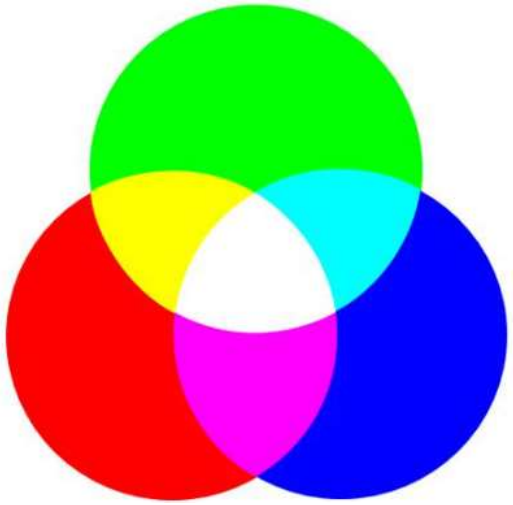


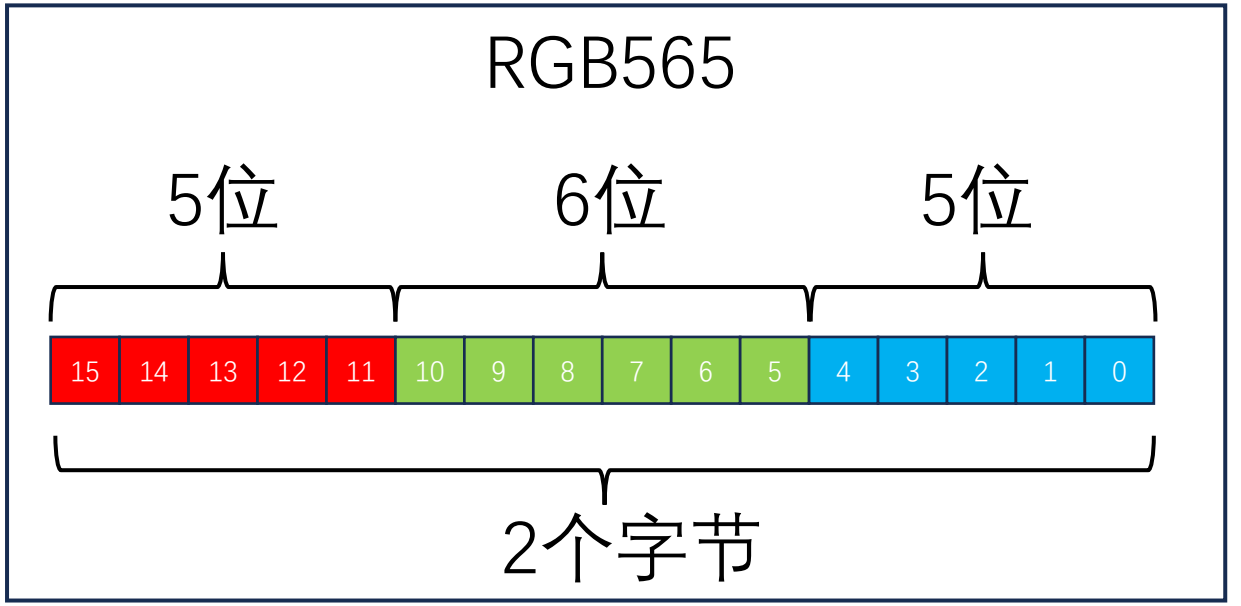
LCD参数与显示原理

艾谷科技
Aigo

- 尺寸：1.54寸
- 分辨率：240*240
- 色彩空间：颜色是一种物理量，在计算机中人们将物理世界中的颜色进行量化，利用二进制来表示不同的颜色，从而有了RGB的概念。RGB又分为RGB565、RGB666、RGB888







LCD屏幕引脚定义

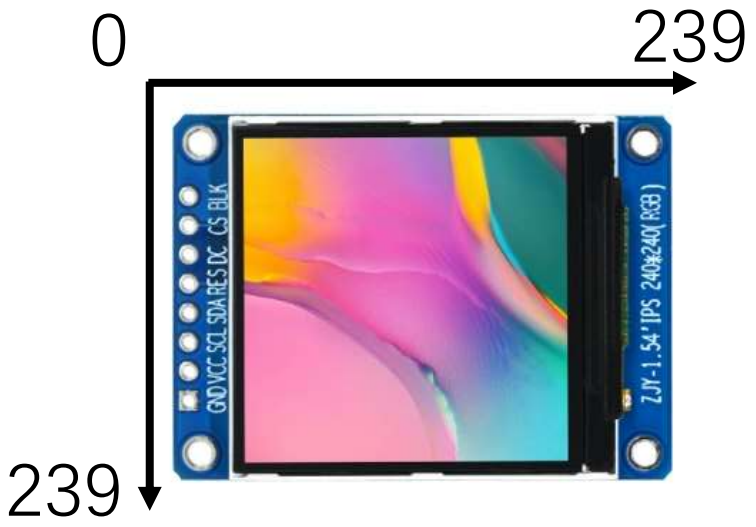
艾谷科技
Aigo



引脚序号	引脚名称	功能说明	连接
1	GND	电源地	
2	VCC	供电，3.3V	
3	SCL	SPI时钟引脚	IO12
4	SDA（MOSI）	SPI数据输入引脚	IO11
5	RES	复位引脚(常态高电平，低电平复位)	IO21
6	DC	LCD数据/命令选择引脚(0-命令 \ 1-数据)	IO47
7	CS	从机片选引脚	IO48
8	BLK	背光控制(默认打开背光、低电平关闭背光)	IO40
	MISO	主机输入从机输出引脚	IO13

LCD常用命令





命令	说明
0x2A	设置起始点列地址
0x2B	设置起始点行地址
0x2C	内存写命令(将指令和数据写入LCDRAM)
0x36	设置扫描方向(控制显示方向) 0x00\0xC0\0x70\0xA0

LCD初始化

LCD初始化：在使用LCD之前，需要发送特定的命令和数据，对LCD进行初始化配置。配置命令由数据手册中给出。

SD卡简介

艾谷科技
Aigo

SD卡是一种小型非易失性闪存存储设备，由松下、东芝、闪迪联合制定标准，是目前嵌入式系统外接大容量存储的最主流方案。

常用SD卡类型：SD卡、Micro SD卡(TF卡)

TF卡按照容量分类：SDSC、SDHC、SDXC

TF卡通信接口：SDIO接口、SPI接口





类型	全称	容量	文件系统
SDSC	Standard Capacity SD	≤2GB	FAT16/FAT32
SDHC	High Capacity SD	2~32GB	FAT32
SDXC	eXtended Capacity SD	32GB~2TB	exFAT

TF卡引脚定义

艾谷科技
Aigo

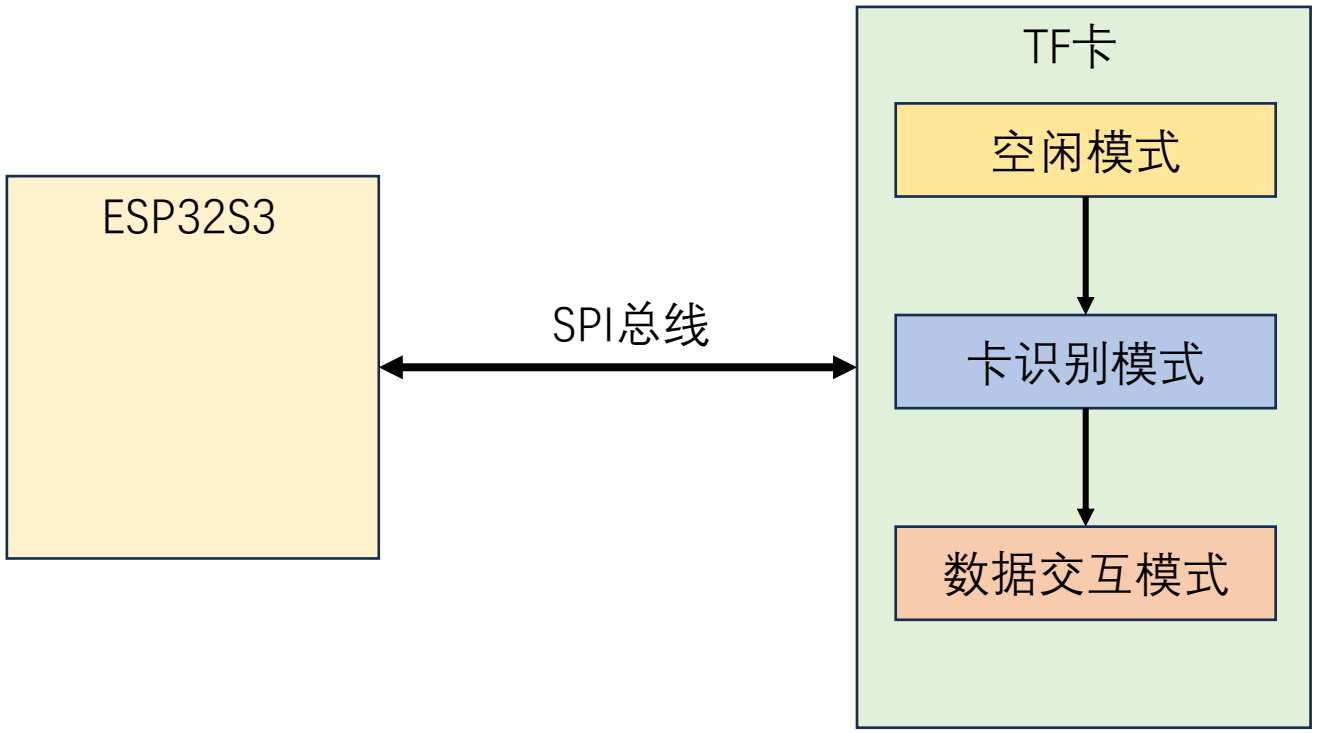


引脚编号	引脚名称	SDIO功能	SPI模式功能	与ESP32S3引脚连接
1	DAT2	数据线2		
2	CD/DAT3	数据线3	SS	IO2
3	CMD	命令/响应	MOSI引脚	IO11
4	VDD	供电(2.7~3.6V)	供电(2.7~3.6V)	
5	CLK	时钟信号	SCK引脚	IO12
6	VSS1	电源地	电源地	
7	DAT0	数据线0	MISO引脚	IO13
8	DAT1	数据线1		

TF卡工作模式

艾谷科技
Aigo

TF卡工作模式：空闲模式、卡识别模式、数据交互模式

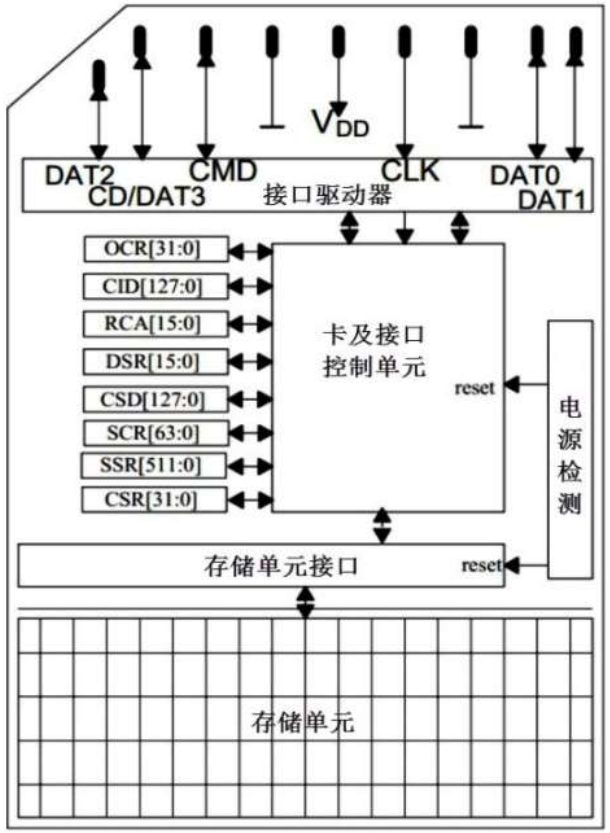


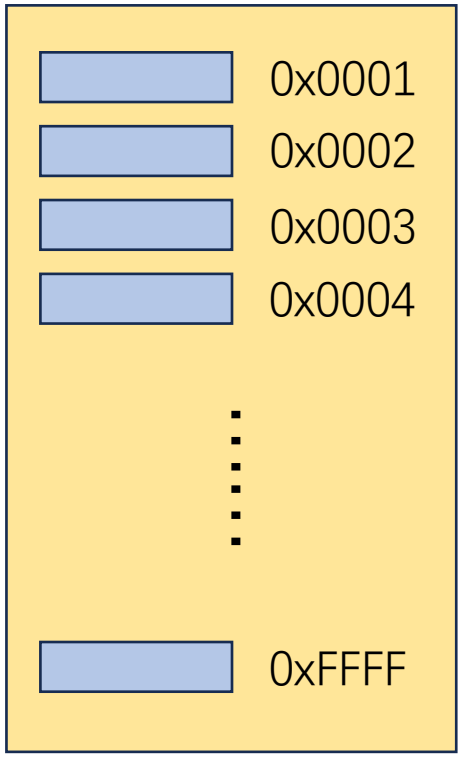
TF卡数据存储

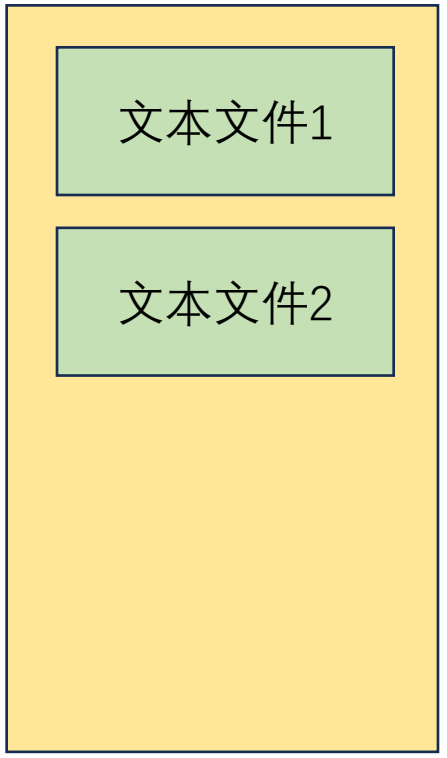
艾谷科技
Aigo

常见的数据交互方式:

- 直接向芯片的物理地址中，写入数据并存储(适用小容量存储设备)
- 以文件的形式，写入数据并存储(适用大容量存储设备)

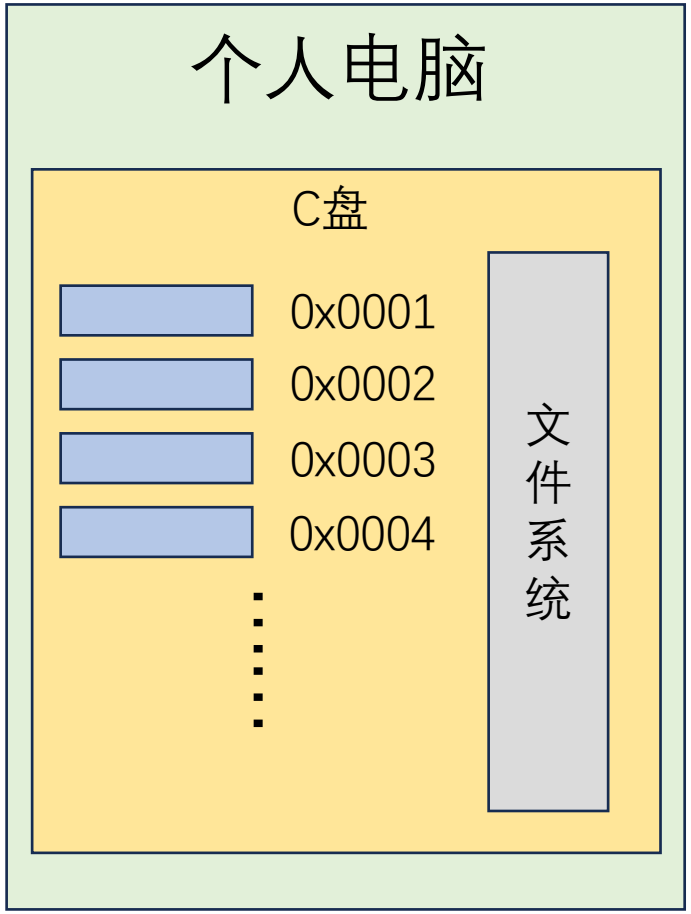


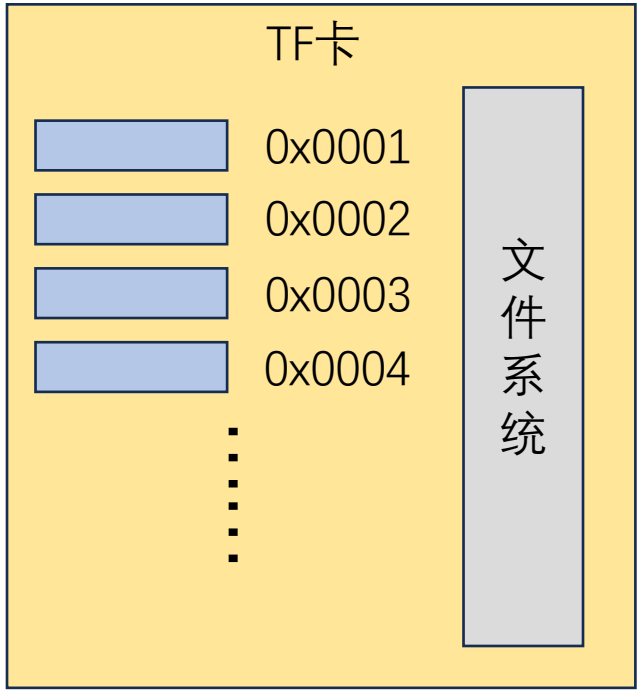




文件系统

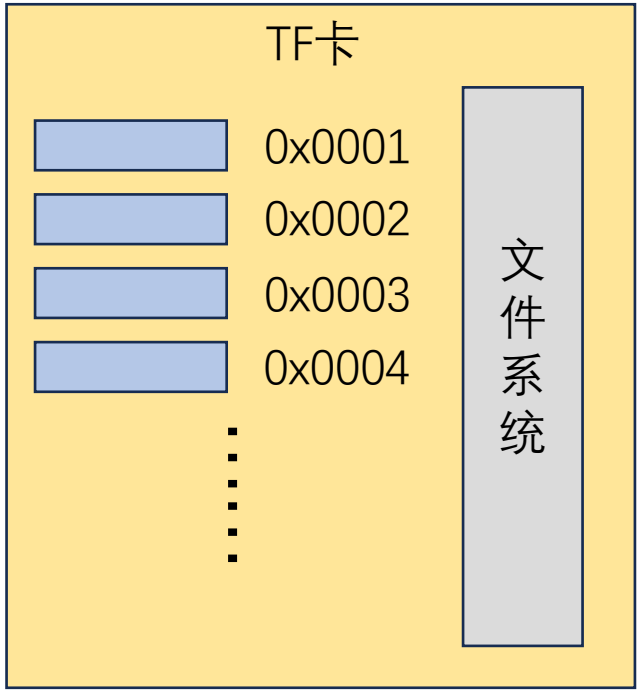
文件系统(File System,FS)，是一套规范规则 + 软件实现，用于管理存储设备上的数据存放、查找、修改、删除，让存储设备可以按文件 / 文件夹的形式使用。

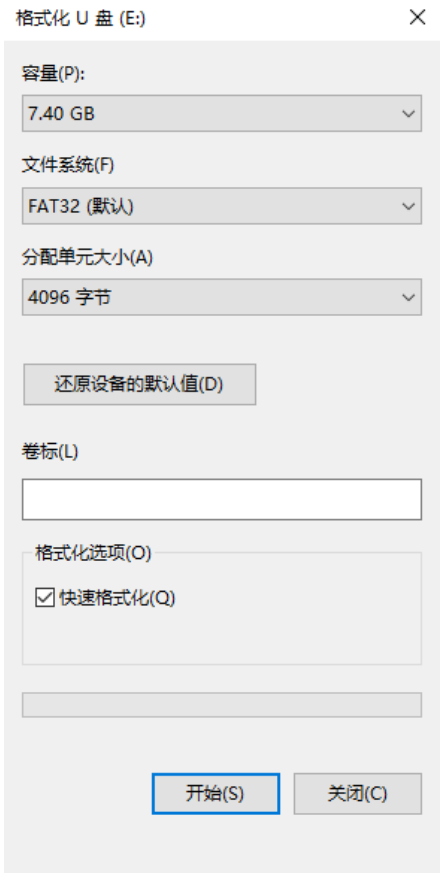




文件系统

文件系统(File System,FS)，是一套规范规则 + 软件实现，用于管理存储设备上的数据存放、查找、修改、删除，让存储设备可以按文件 / 文件夹的形式使用。





FATFS

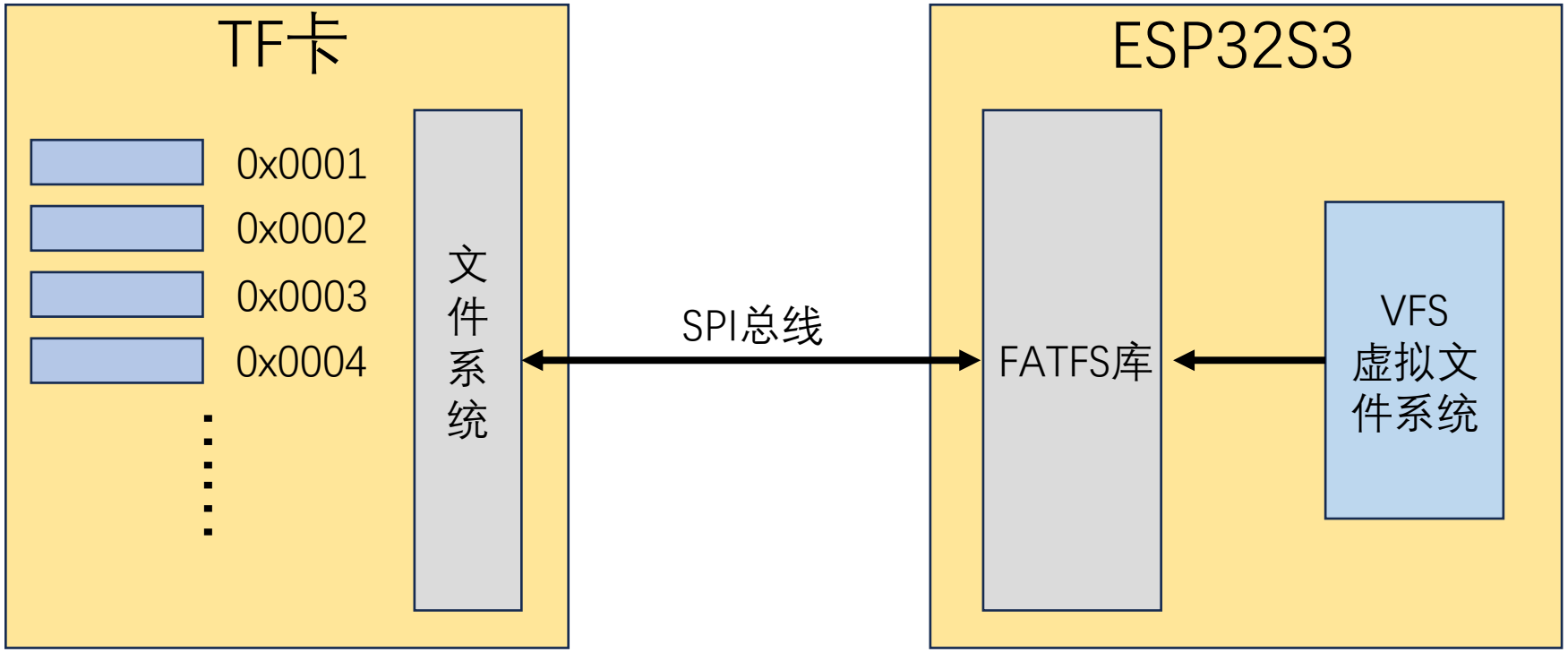
艾谷科技
Aigo

FATFS 是一个开源、轻量级、专为嵌入式单片机开发使用的 FAT 文件系统驱动库。本质：一套C语言代码

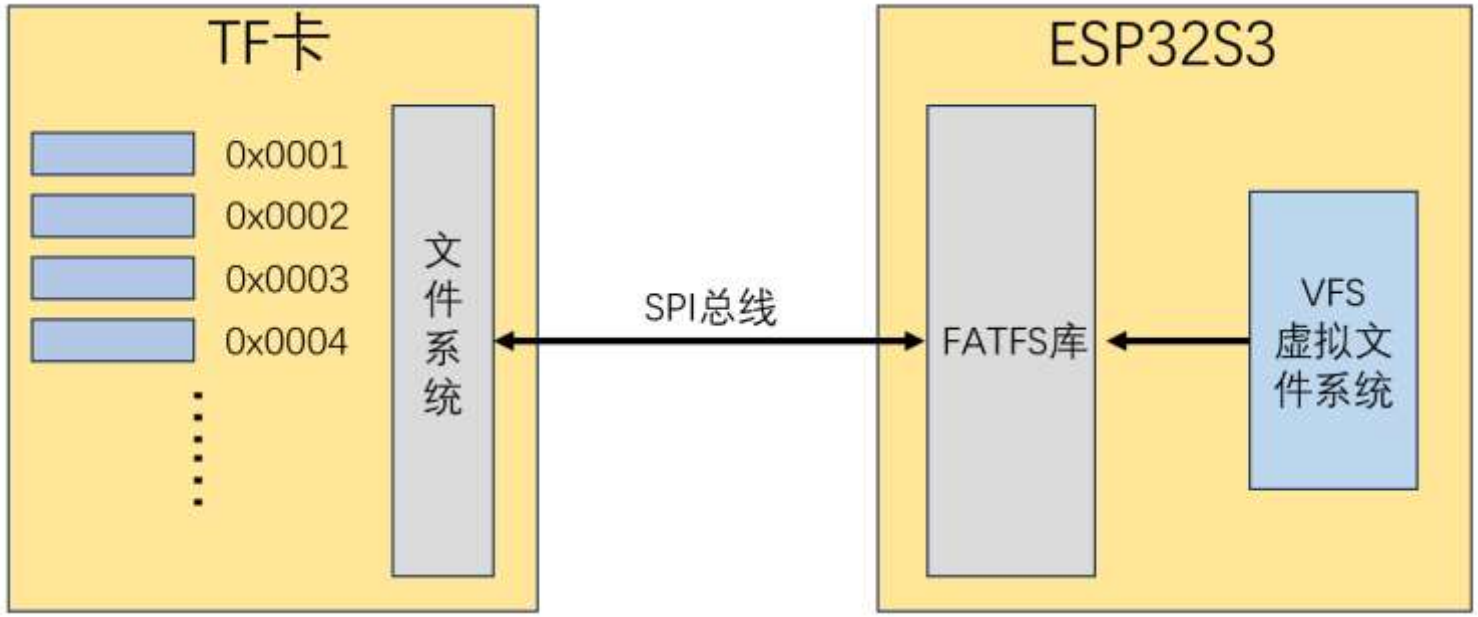
VFS(Virtual File System),虚拟文件系统,将C函数转换成FATFS函数。

FATFS库函数：f_open()、f_read()、f_write()、f_close()

C语言函数：fopen()、fread()、fwrite()、fclose()



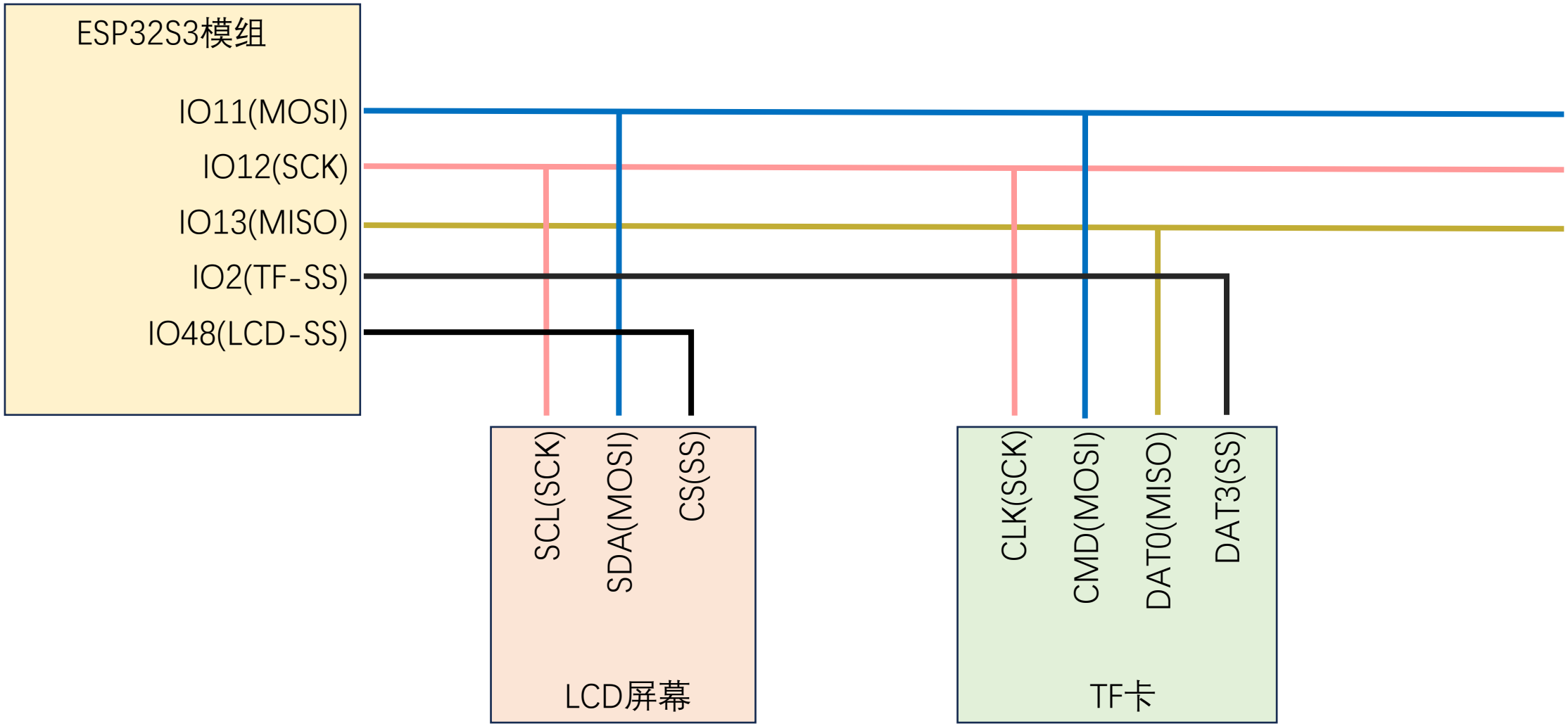
配置流程



- 格式化TF卡(文件系统类型：FAT32　分配单元大小：4096)
- 使用 “esp_vfs_fat_sdspi_mount” 函数向SPI总线添加从设备信息、配置SD卡的物理参数(电压、时钟速度等)、配置SD卡的SPI总线信息(使用哪一个SPI、SS引脚)、建立VFS与FATFS与SD卡的匹配关系

SPI总线连接图

艾谷科技
Aigo



RAM和FLASH简介

- RAM(Random Access Memory),中文名：随机存取存储器。读写速度极快、容量小、掉电数据丢失。(内存条)

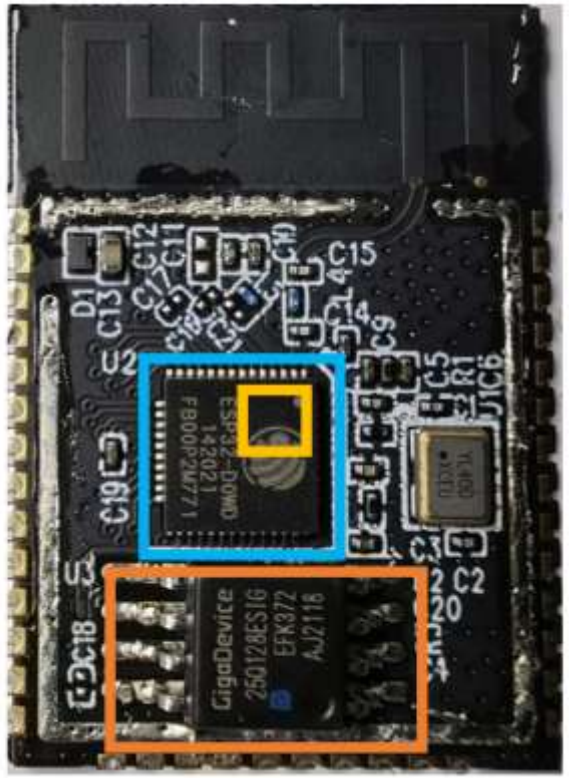
- FLASH(Flash Memory),中文名：闪存存储器。读写速度较RAM慢、容量大、掉电数据不丢失。(硬盘)

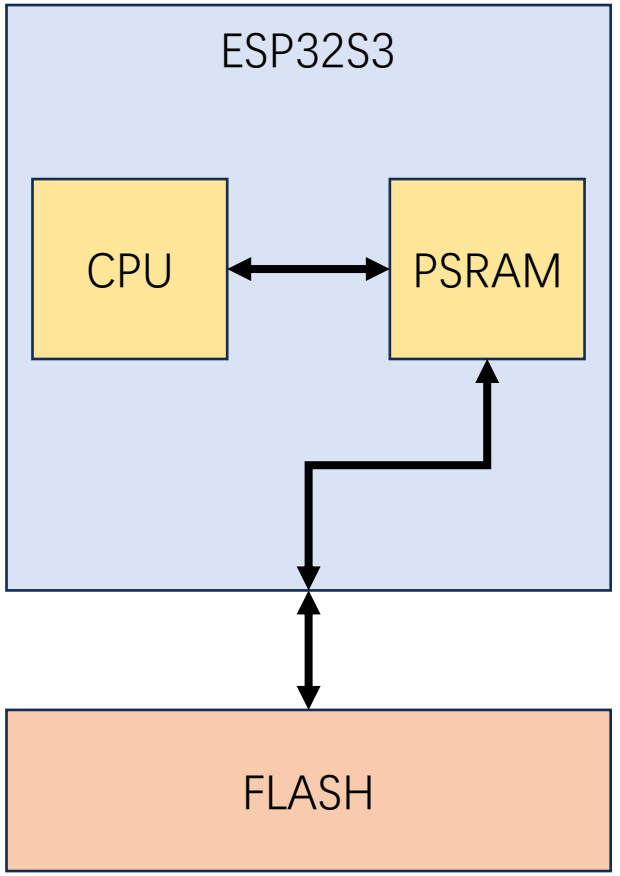
RAM和FLASH功能

艾谷科技
Aigo

- 核心板搭载乐鑫ESP32S3-WROOM-1-N16R8模组
- PSRAM：8MB
- FLASH：16MB

①ESP32-S3R8
②PSRAM
③FLASH

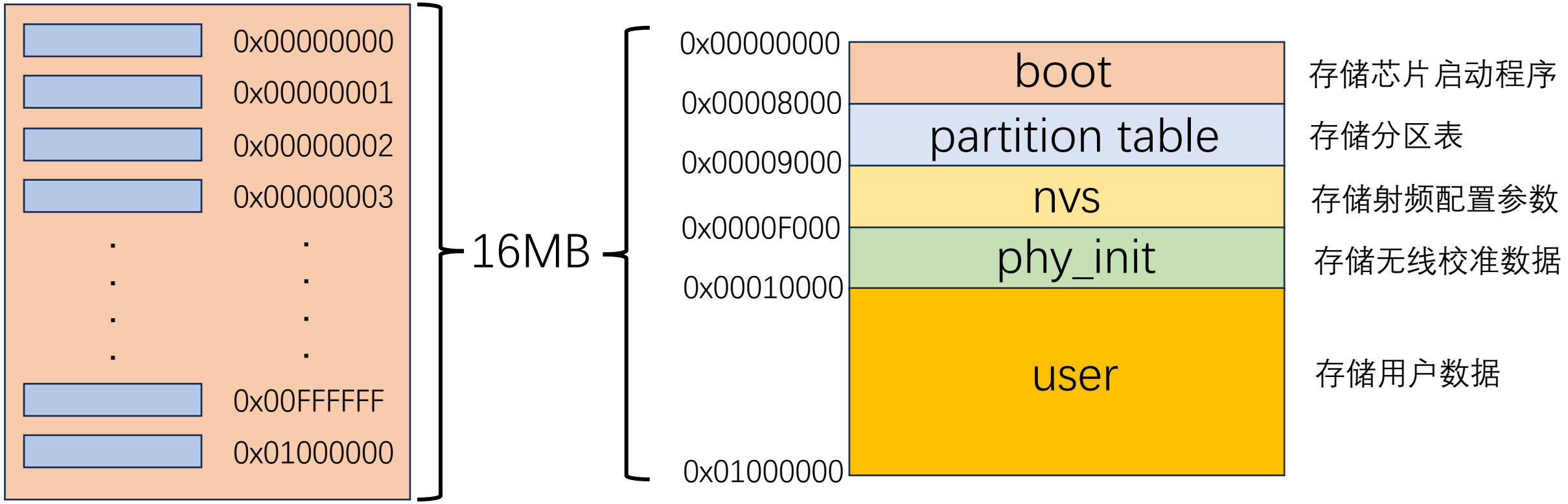




分区表简介

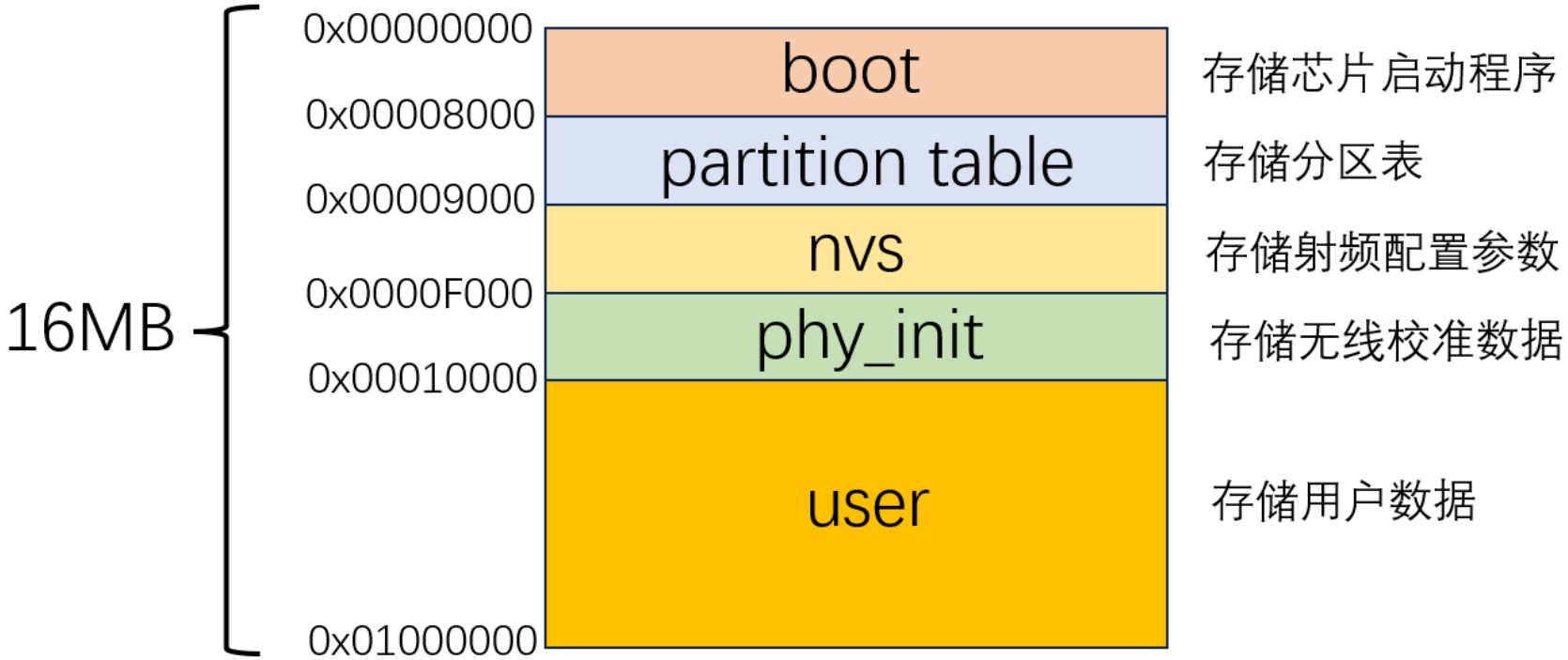
艾谷科技 Aigo

- 分区表：对FLASH进行分区的表格
- FLASH大小：16MB
- FLASH物理地址范围：16MB = 16*1024KB = 16*1024*1024字节
  = 16,777,216 = 0x01000000



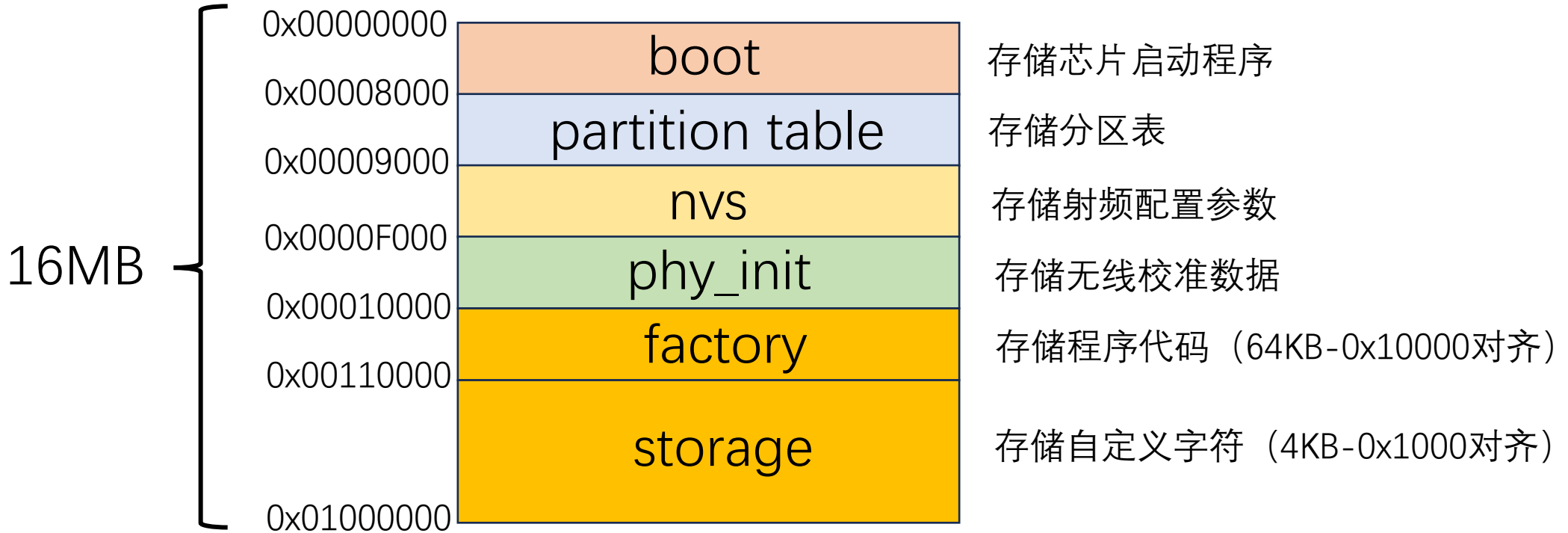
分区表格式


```
# ESP-IDF Partition Table
# Name,   Type, SubType, Offset,  Size,   Flags
nvs,      data, nvs,     0x9000,  0x6000,
phy_init, data, phy,     0xf000,  0x1000,
factory,  app,  factory, 0x10000, 1M,
```



自定义分区表

艾谷科技
Aigo



区域类型

艾谷科技
Aigo

```
# ESP-IDF Partition Table
# Name,   Type, SubType, Offset,  Size,   Flags
nvs,      data, nvs,     0x9000,  0x6000,
phy_init, data, phy,     0xf000,  0x1000,
factory,  app,  factory, 0x10000, 1M,
```

Type	SubType	功能
data	fat	FAT文件系统类型
	ota	OTA远程升级相关数据
	phy	WIFI/蓝牙射频校准参数
	nvs	WIFI/蓝牙配置信息
	nvs_key	NVS加密密钥
	spiffs	SPIFFS文件系统类型
	coredump	存储系统崩溃数据
app	factory	存储程序代码固件
	ota0~ota15	存储OTA远程升级固件
	test	存储测试程序固件

代码配置流程

- Sdkmenuconfig中将默认分区表改成自定义分区表
- 使用分区表编辑器，按照框图对FLASH进行分区

- 使用esp_vfs_spiffs_register()函数初始化文件系统参数
- 调用C语言文件操作函数操作文件(建立文件/打开文件/写数据/读数据)

ESP-IDF组件库简介

艾谷科技
Aigo

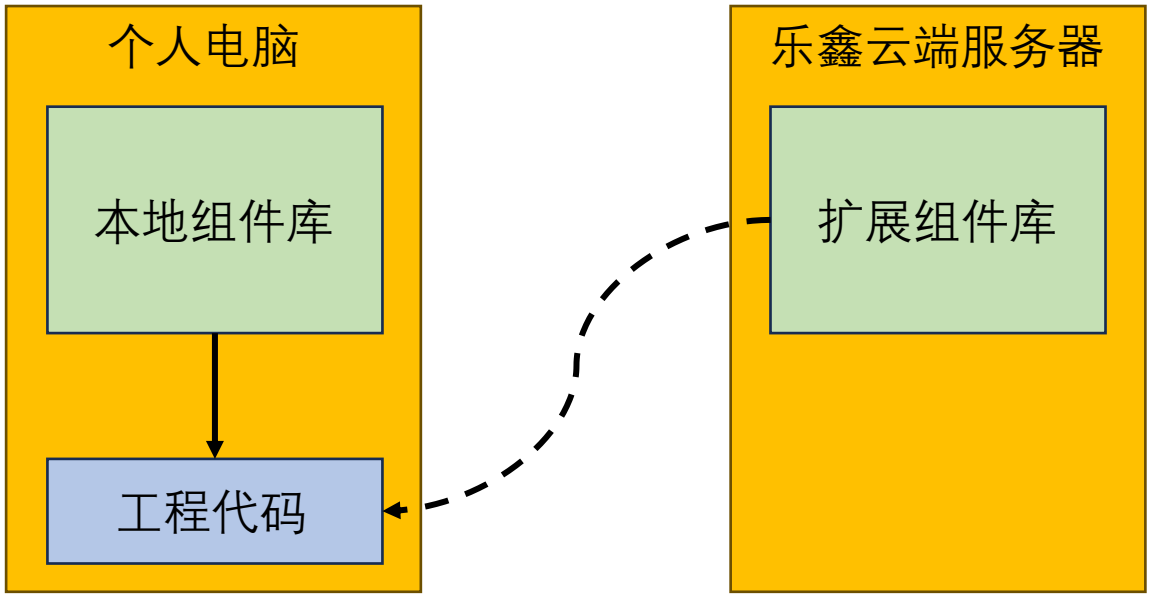
ESP-IDF组件库是乐鑫官方为ESP32全系列芯片打造的官方模块化核心库。所有组件代码经乐鑫严格测试，稳定性好。

ESP-IDF组件库包含：

- 本地组件库：在安装ESP-IDF已经安装在本地。

保存路径：安装文件夹/Espressif/frameworks/esp-idf-v5.1.2/components

- 扩展组件库：部署在乐鑫云端服务器



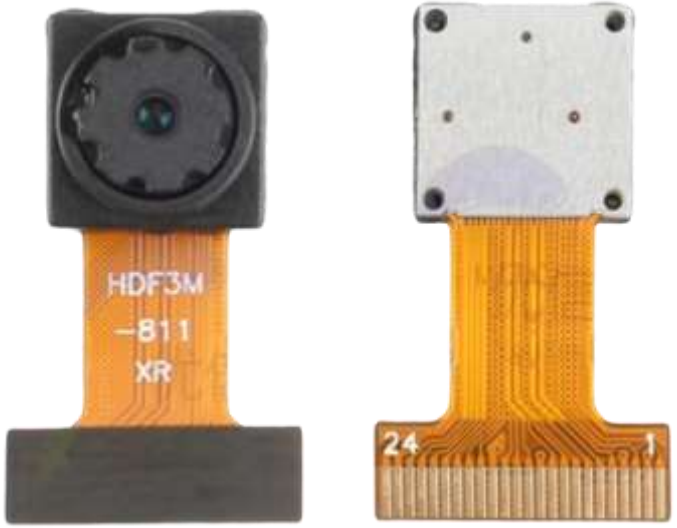
OV2640摄像头简介

艾谷科技
Aigo

OV2640是豪威科技(OmniVision)推出的一款低功耗、高集成度的200万像素的摄像头，专为嵌入式领域设计。

OV2640可以输出多种格式的图像，如：RGB565、RGB888等，也可以输出多种分辨率的图像，如：240*240，320*240等。

OV2640摄像头模块：镜头+底板+外围电路

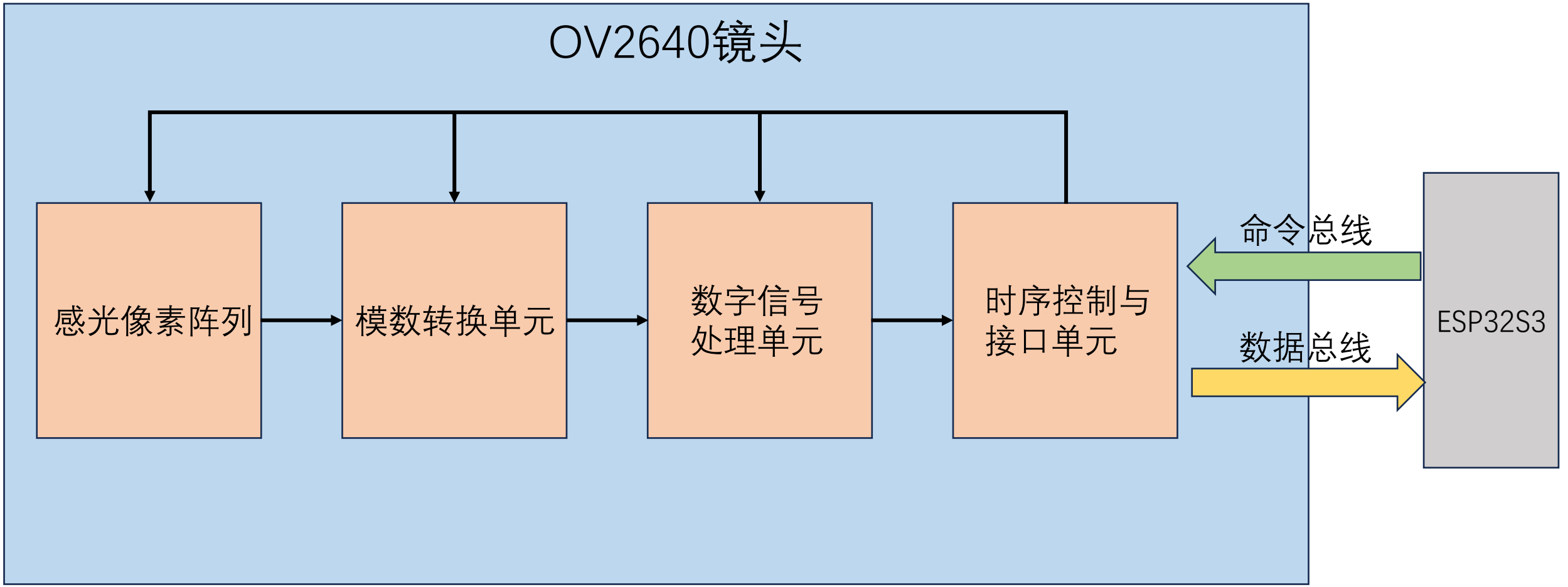






OV2640工作原理

艾谷科技
Aigo



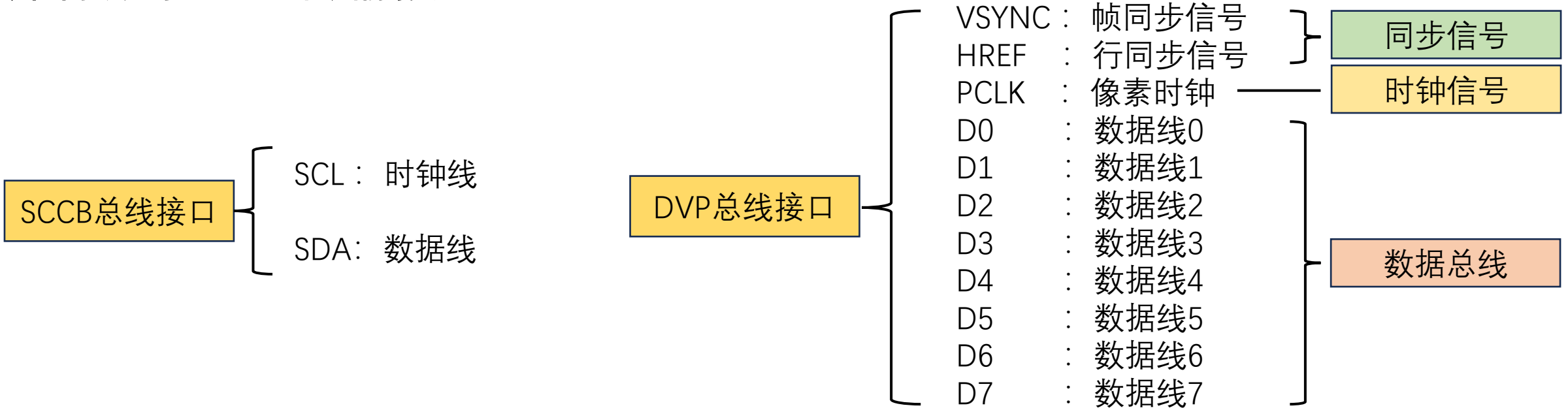
OV2640总线通信

艾谷科技 Aigo

OV2640带有两组总线接口，分别是命令接口-SCCB总线接口、数据接口-DVP并行数字视频总线接口。

SCCB是在I2C的基础上简化/定制而来，其核心规则完全一致。可以将ESP32S3的I2C外设直接应用于OV2640的SCCB总线接口。

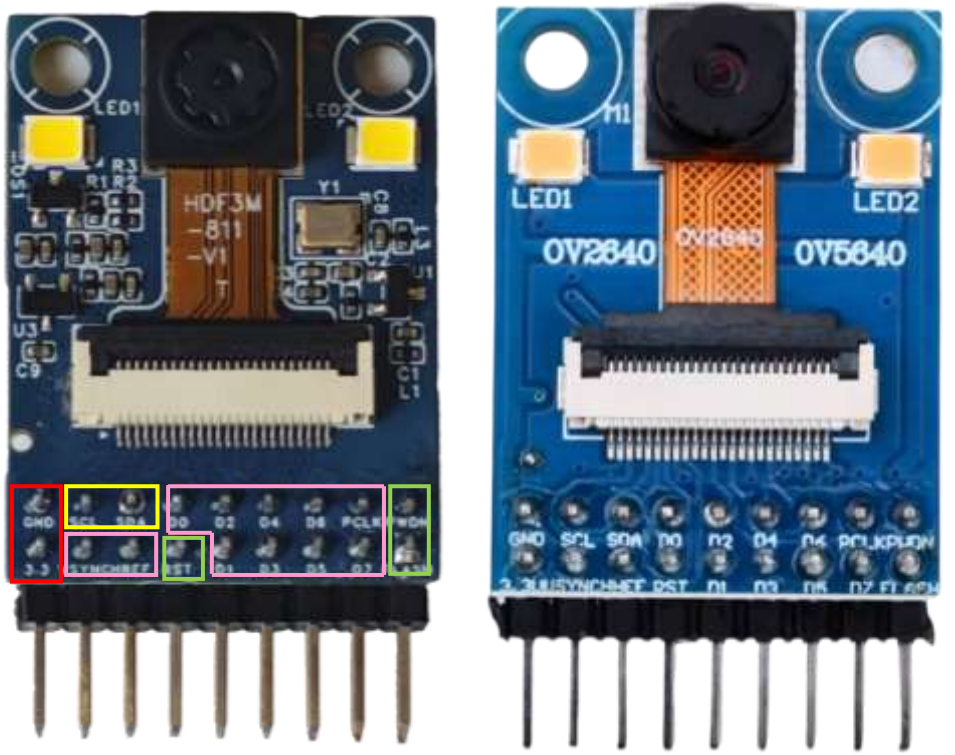
DVP总线接口是一种面向嵌入式中/低分辨率图像传感器设计的并行数字视频传输接口。



OV2640模块引脚定义

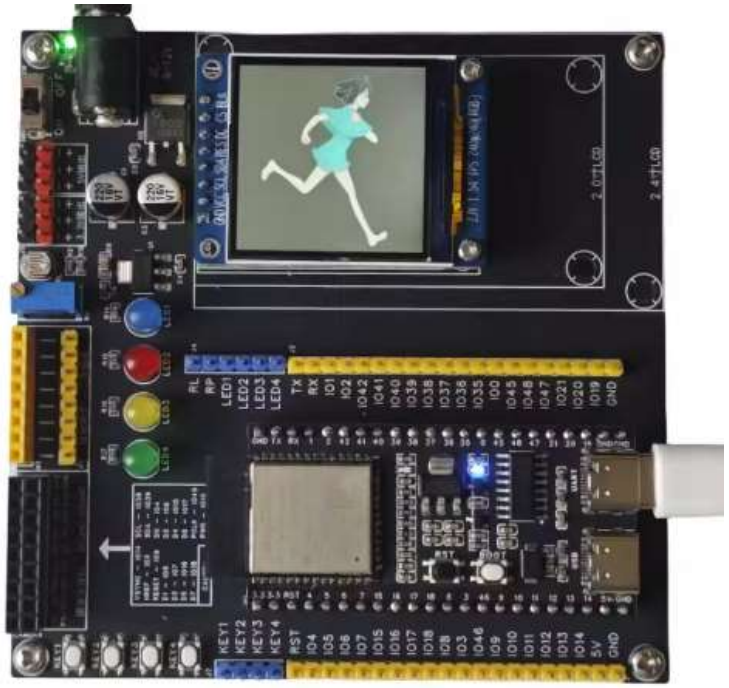
艾谷科技
Aigo

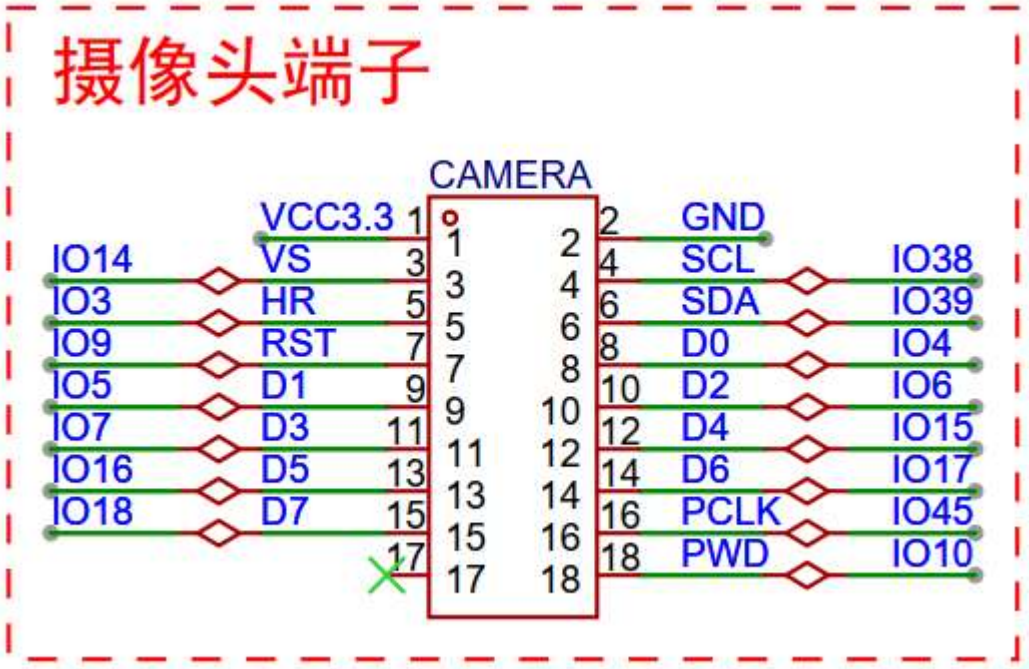
引脚名称	引脚定义
GND	电源地
3.3	电源正极
SCL	SCCB接口-时钟线
SDA	SCCB接口-数据线
VSYNC	DVP接口-帧同步信号
HREF	DVP接口-行同步信号
PCLK	DVP接口-时钟信号
D0~D7	DVP接口-数据信号
RST	复位引脚(低电平复位)
PWDN	掉电控制引脚(常态低电平、高电平低功耗模式)
FLASH	闪光灯控制



底板摄像头接口

艾谷科技
Aigo

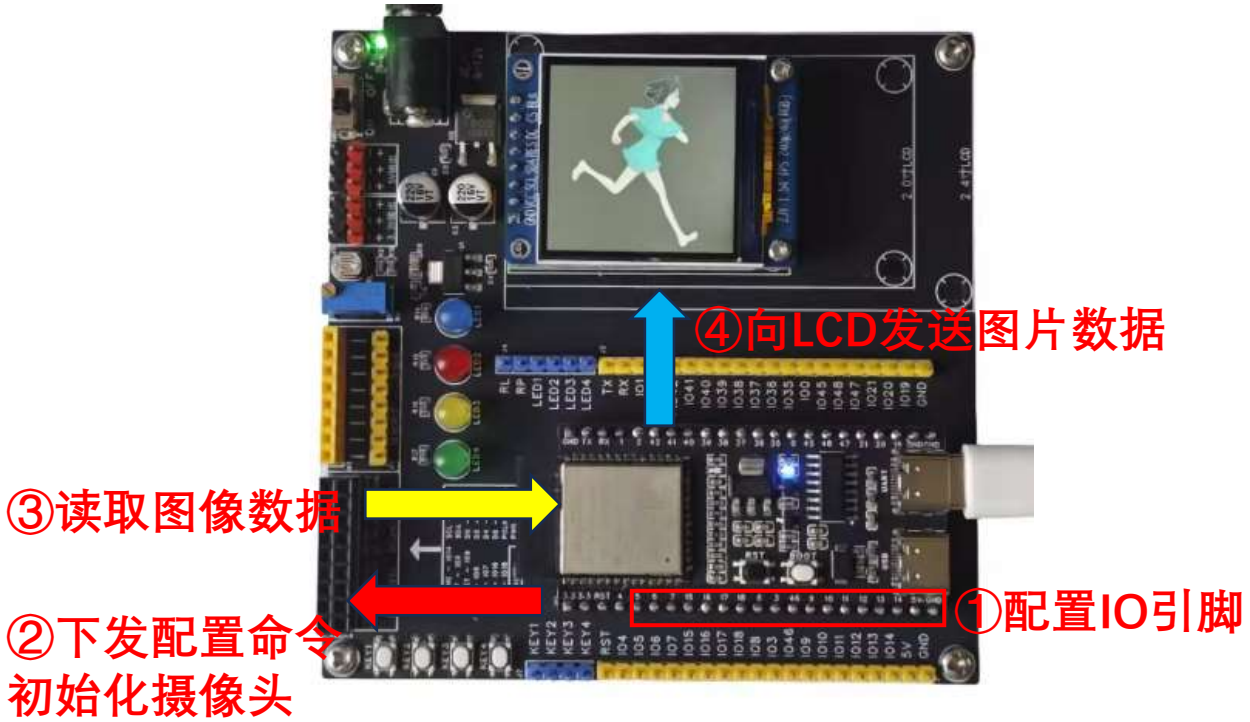




代码编写思路

艾谷科技
Aigo

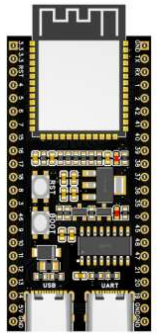
- 调用初始化函数，配置IO引脚，并初始化摄像头
- 调用图像读取函数，读取图像数据
- 调用图片显示函数，在LCD上显示图片



WIFI简介

艾谷科技
Aigo

- WiFi 简介：WIFI是无线局域网（WLAN）的商用俗称。无线局域网（Wireless Local Area Network，WLAN），是基于 IEEE 802.11 系列标准制定的无线联网技术。
- WIFI的工作模式：STA(站点模式)、AP(接入点模式)、AP+STA(混合模式)
- 按照频段分类：
  2.4GHz频段：国际标准频率范围2.400GHz~2.4835GHz
  5GHz频段：国际标准频率范围5.150 GHz ~ 5.825 GHz
- 按照IEEE802.11标准号分类：
  802.11b、802.11g、802.11n、802.11ac、802.11ax、802.11be





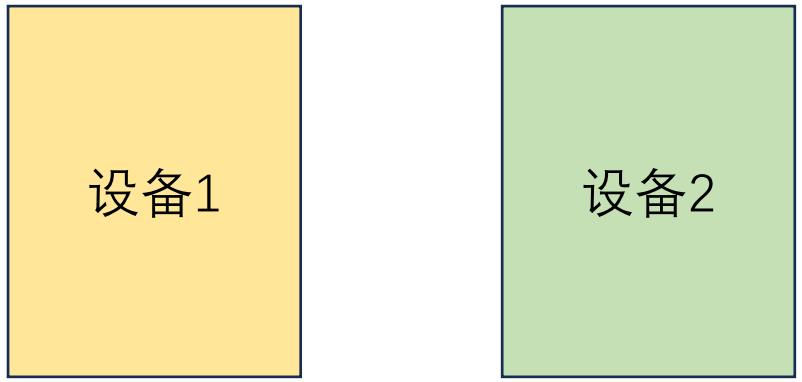


IEEE802.11标准简介

艾谷科技
Aigo

IEEE 802.11是国际电气和电子工程师协会（IEEE）制定的无线局域网技术标准集，是 WiFi 技术的核心底层规范，定义了无线局域网中设备的信号传输、数据交互、组网规则，是全球通用的无线联网技术基础。

所有消费级 WiFi 设备（路由器、手机、嵌入式设备）都必须遵循 802.11 标准才能实现互联互通。



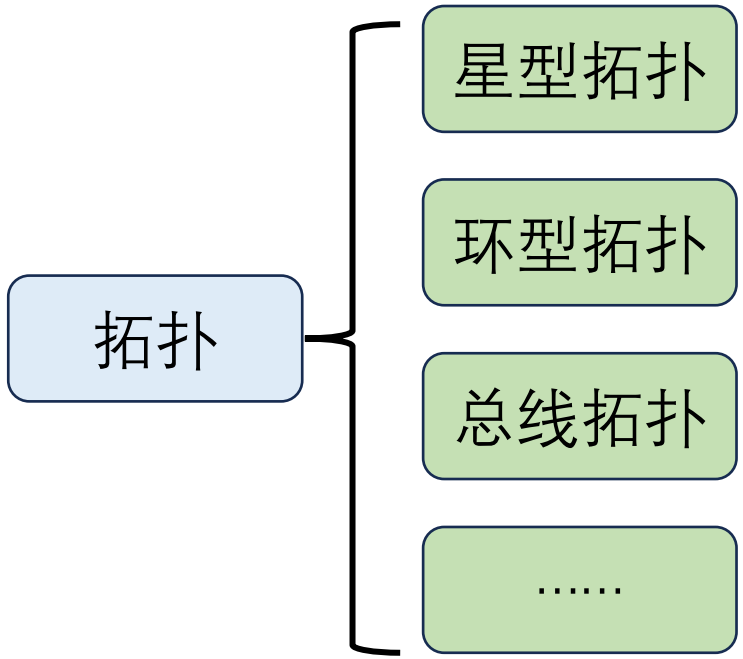
日常俗称	官方标准编号	频段	最高速率
WIFI 1	802.11b	2.4GHz	11Mbps
WIFI 2	802.11g	2.4GHz	54Mbps
WIFI 3 / WIFI 4	802.11n	2.4GHz/5GHz	150Mbps
WIFI 5	802.11ac	5GHz	1.3Gbps
WIFI 6	802.11ax	2.4GHz/5GHz	9.6Gbps
WIFI 7	802.11be	2.4GHz/5GHz	40Gbps

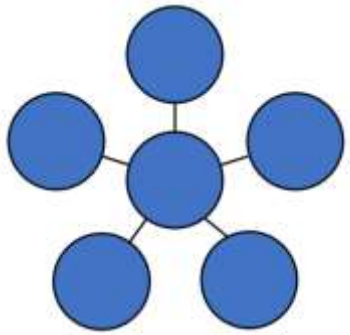
拓扑简介

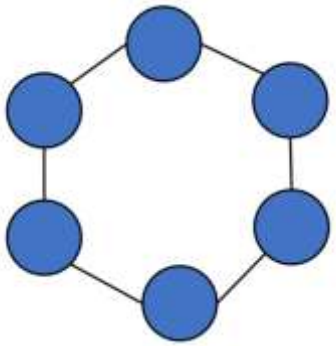
网络拓扑是指网络中设备、节点、链路及它们之间连接关系的几何结构与逻辑结构的总称。

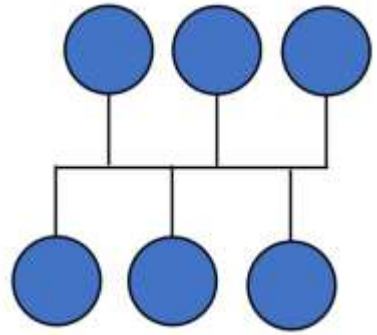
网络拓扑决定了数据在网络中的传输路径、通信效率和可靠性。

在嵌入式领域，星型拓扑是最常用、最主流的拓扑结构。



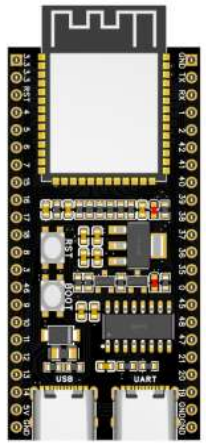






星型网络拓扑

艾谷科技
Aigo







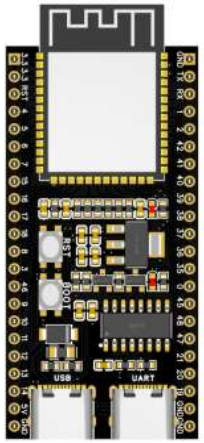


常见的网络名词

艾谷科技
Aigo

无线通信过程中，常见以下名词：客户端、服务端、路由器、IP地址、LWIP协议栈。

IP地址：192.168.2.1



客户端

服务端

数据

LWIP协议栈



路由器

数据

LWIP协议栈

IP地址：192.168.2.2



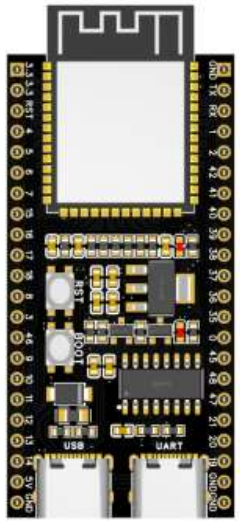
服务端

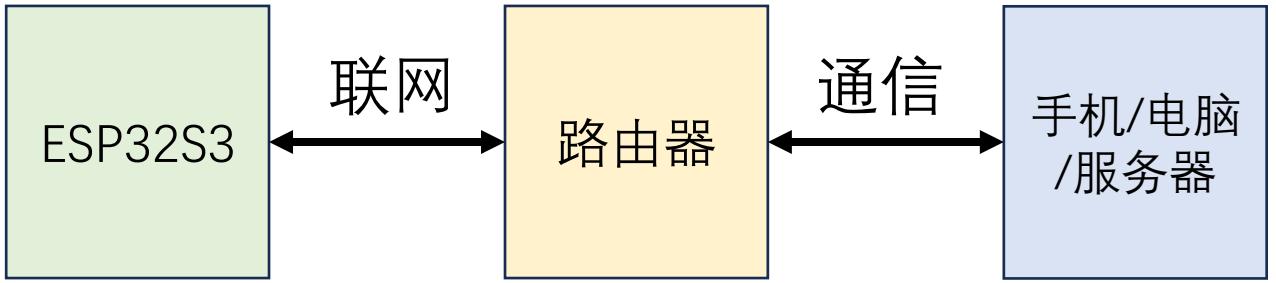
客户端

ESP32S3-WIFI功能简介

艾谷科技
Aigo

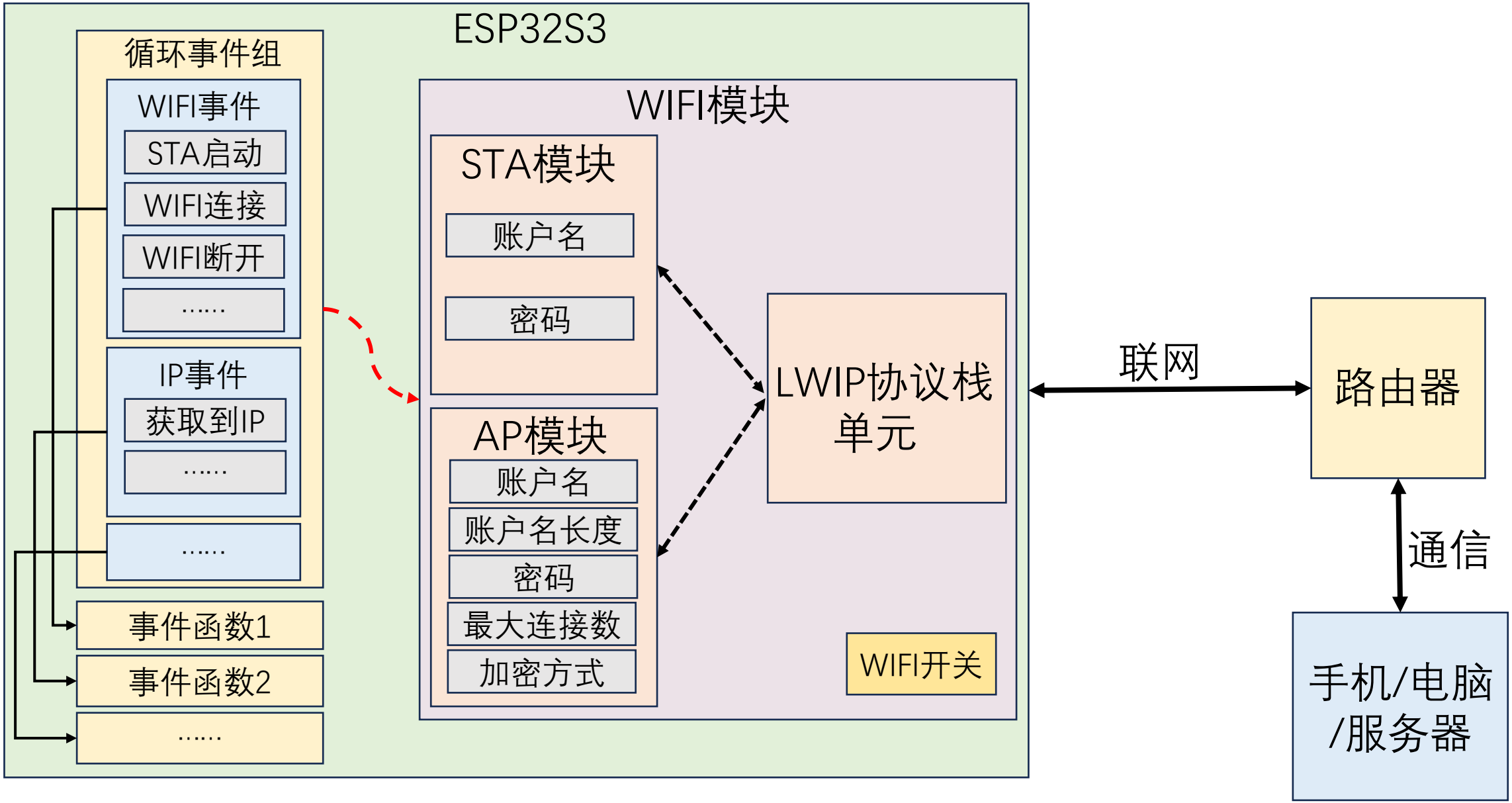
- ESP32S3 内部集成了无线局域网（WLAN）模块;
- 工作频段：仅支持2.4GHz
- 标准：符合802.11 b/g/n 标准
- 工作模式：STA(站点模式)、AP(接入点模式)、STA+AP(混合模式)





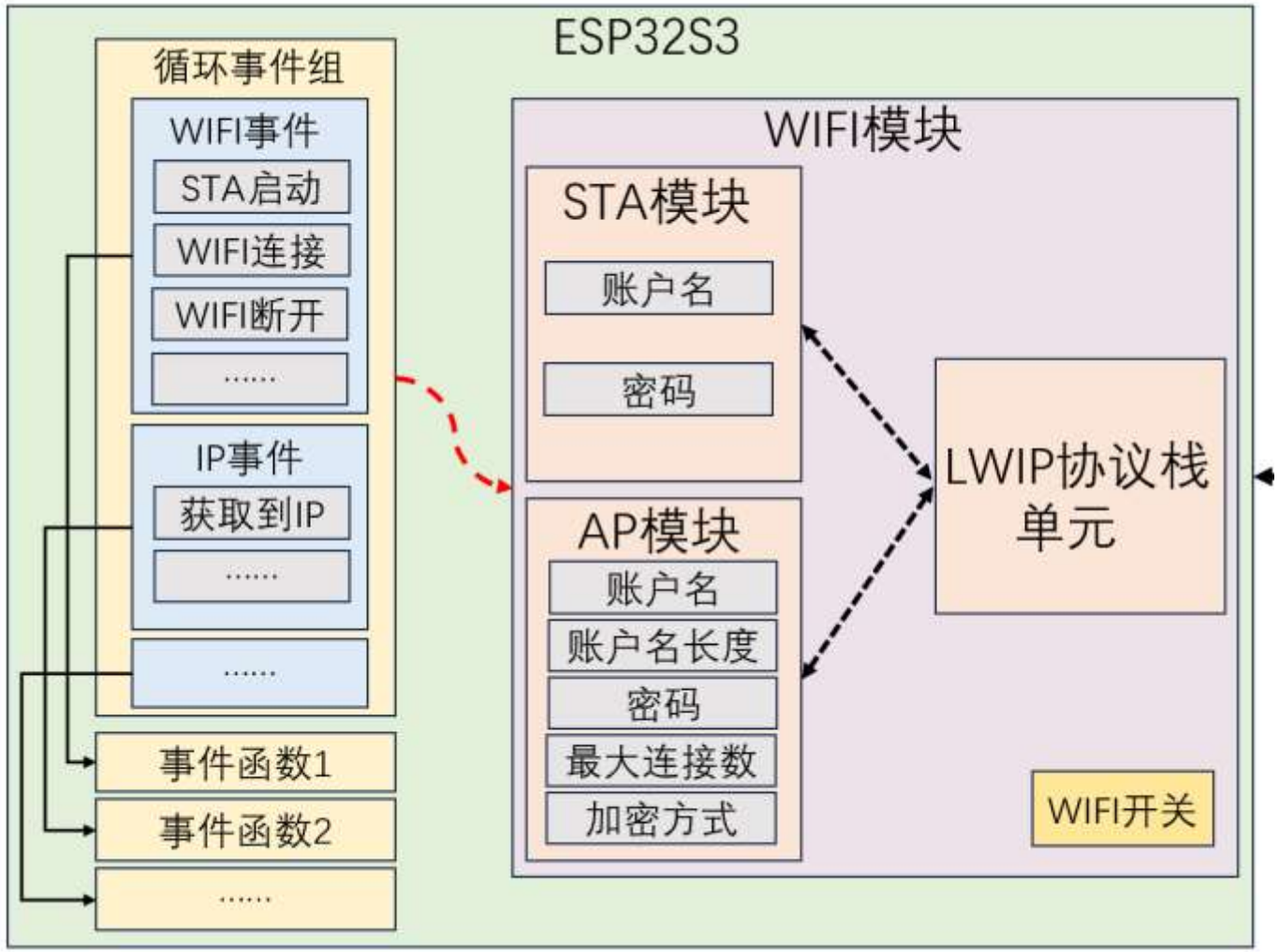
ESP32S3 WIFI工作流程

艾谷科技
Aigo



ESP32S3 WIFI配置流程

艾谷科技
Aigo



- 调用nvs_flash()函数，初始化NVS
- 调用esp_netif_init()函数，初始化LWIP协议栈
- 调用esp_event_loop_create_default()函数，创建循环事件组
- 调用esp_event_handler_register()函数，向循环事件组中注册要监听的事件组以及事件编号，并创建事件函数
- 调用esp_netif_create_default_wifi_sta()函数，将STA模块/AP模块 与LWIP协议栈单元连接
- 调用esp_wifi_init()函数，初始化与WIFI相关的底层硬件
- 调用esp_wifi_set_mode()函数，设置WIFI模块工作模式(STA模式/AP模式)
- 调用esp_wifi_set_config()函数，配置STA/AP模式参数
- 调用esp_wifi_start()函数，启动WIFI模块

- 调用esp_wifi_connect()函数，启动WIFI连接
- 调用esp_wifi_stop()函数，关闭WIFI模块

UTC/时间戳简介

艾谷科技
Aigo

UTC 全称 Coordinated Universal Time（协调世界时），是全球统一的时间基准。UTC代表0时区的时间-英国伦敦格林尼治的当地时间。

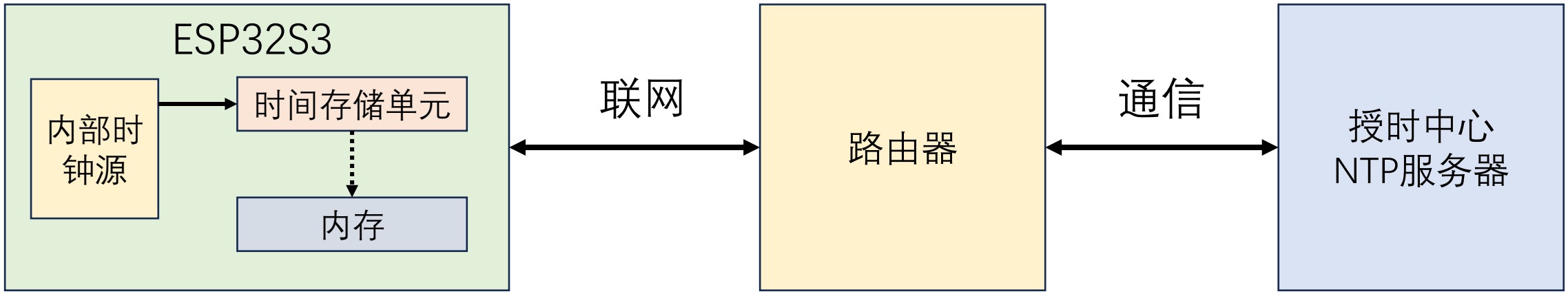
时间戳是从1970年1月1日00：00：00的UTC时间，到当前时刻，一共经过的总秒数。简单来说，时间戳是一个纯数字整数，用一个数字来表示某一个时刻。

城市	时区	标识
英国格林尼治	0时区	UTC
中国北京	东8区	UTC+8
美国纽约	西5区	UTC-5

UTC时间	时间戳
1970.1.1-00:00:00	0
1970.1.1-00:00:20	20
1970.1.1-00:00:50	50
……	……
2026.1.1-12:00:00	1767268800

获取网络时间配置流程

艾谷科技
Aigo



第一步：联网

- 配置ESP32S3为STA模式并连接外网

第二步：时间同步

- 调用esp_sntp_setoperatingmode()函数，设置时间同步模式
- 调用esp_sntp_setservername()函数，设置要访问的NTP服务器
- 调用esp_sntp_init()函数，初始化SNTP
- 调用setenv()函数，设置当前时区
- 调用tzset()函数，使设置时区的操作立刻生效

第三步：读取时间

- 调用time()函数，获取时间存储单元中的时间戳
- 调用localtime()函数，将时间戳转换成日期(年\月\日\时\分\秒)

常用NTP服务器

pool.ntp.org – 全球通用的NTP服务器
cn.pool.ntp.org – 中国区专用的NTP服务器
ntp1.aliyun.com – 阿里云NTP服务器

蓝牙简介

蓝牙（Bluetooth）是一种短距离无线通信技术，工作在2.4 GHz频段，目标是以低功耗、低成本的方式实现设备间的数据交换。

蓝牙版本：从 1999 年发布 1.0 版本至今，蓝牙标准由蓝牙技术联盟（SIG）持续演进，目前已经发展到 5.4 版本。

4.0版本后的蓝牙分支：经典蓝牙和低功耗蓝牙(BLE)

蓝牙分支	设计目标	典型应用
经典蓝牙	高带宽、持续数据流	蓝牙耳机、蓝牙音箱
低功耗蓝牙	功耗低、连接快、小数据量	智能手环、智能家居

ESP32S3N16R8蓝牙功能

艾谷科技
Aigo

- 蓝牙支持：仅支持低功耗蓝牙，不支持经典蓝牙
- 蓝牙版本：最高支持BLE5.0版本
- 使用注意：WIFI和蓝牙共用一个天线，两者需要分时复用



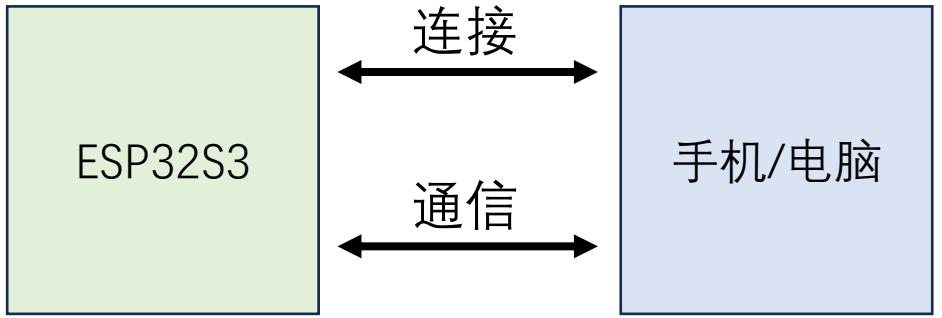
ESP32S3蓝牙使用步骤

艾谷科技
Aigo

WIFI使用步骤：①联网 ②通信



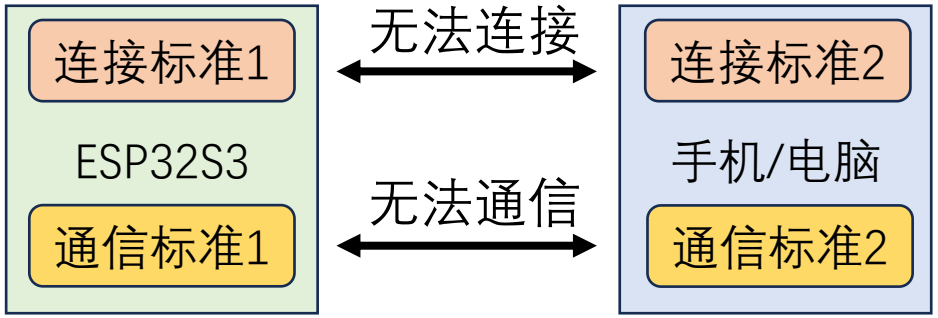
蓝牙使用步骤：①连接 ②通信

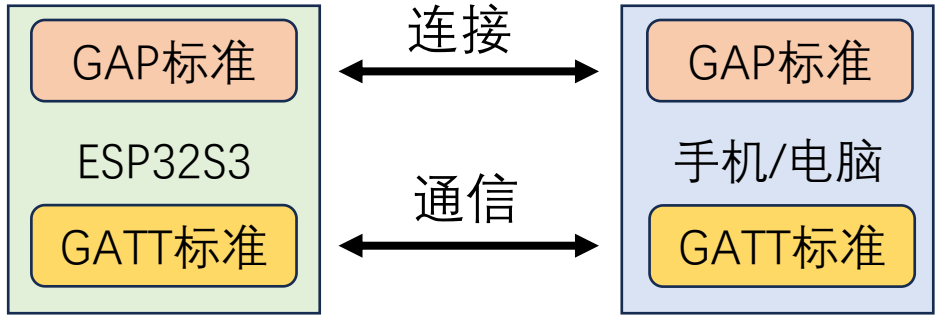


蓝牙标准

艾谷科技
Aigo

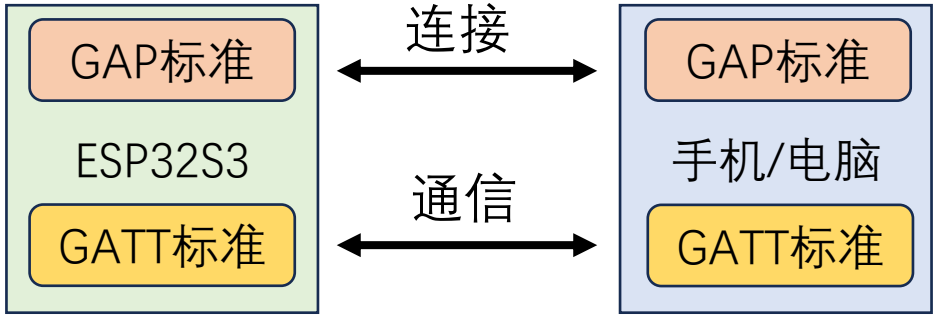
蓝牙设备问题：连接标准不一致、通信标准不一致。
问题处理：蓝牙技术联盟(SIG)制定了目前蓝牙最主流的标准GAP连接标准和GATT通信标准。





GAP和GATT

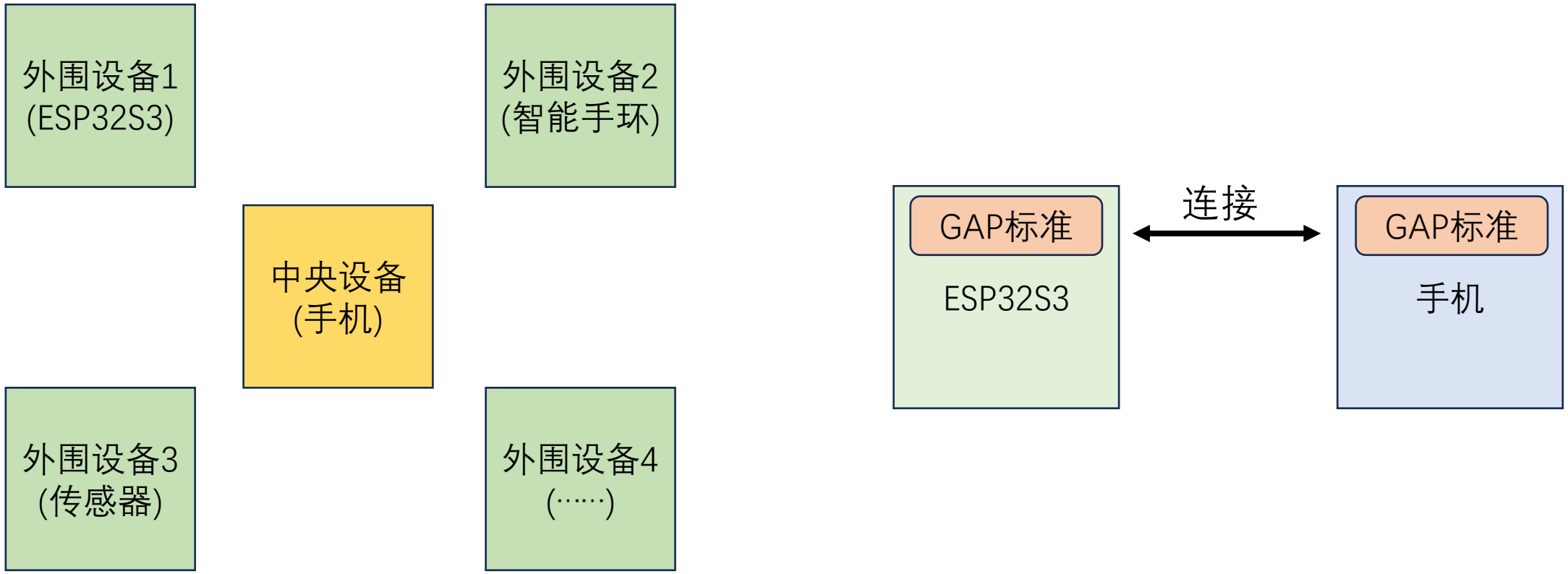
GAP(Generic Access Profile),中文名：通用访问规范。GAP 标准负责蓝牙设备的发现与连接。
GATT(Generic Attribute Profile),中文名：通用属性规范。GATT负责设备连接成功之后的数据交换。



连接阶段 - GAP标准下的蓝牙设备

艾谷科技
Aigo

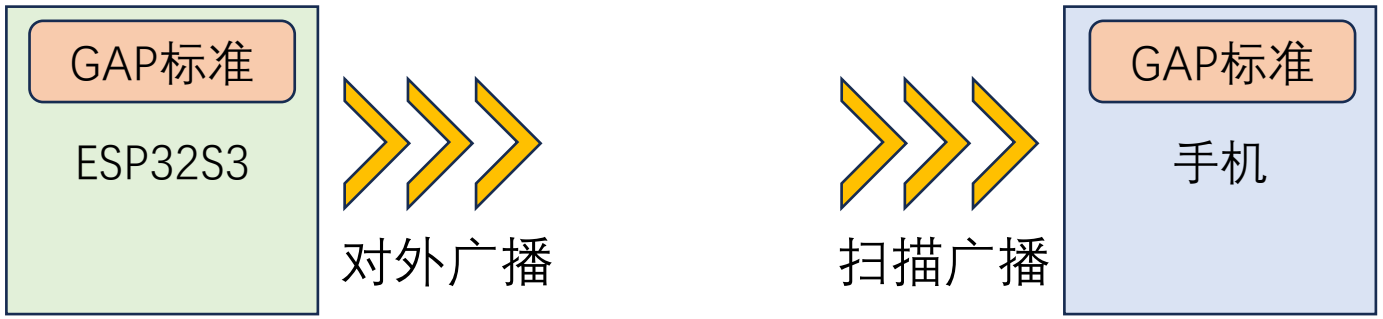
GAP标准定义的两种设备：外围设备、中央设备
外围设备：通常是传感器、智能灯泡这类拥有数据的设备
中央设备：通常是手机、电脑这类可以主动发起连接的设备



连接阶段 -广播与扫描

广播：ESP32S3、传感器、智能灯等外围设备，会对外广播自己的存在。开启广播，证明设备蓝牙准备就绪，等待其他设备连接。

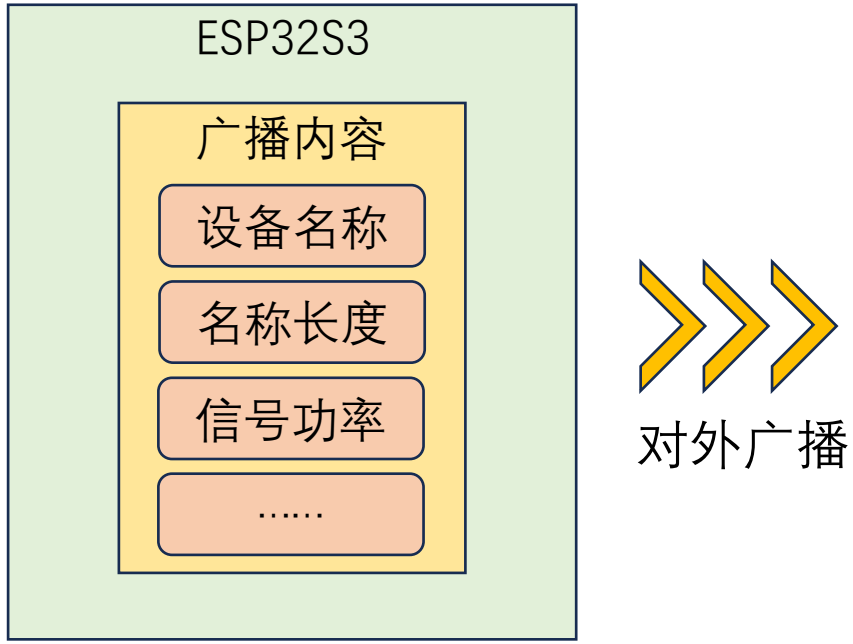
扫描：手机、电脑等中央设备，会主动扫描周围的广播，以寻找目标设备。



连接阶段 –ESP32S3广播内容

为了能够正确建立连接，外围设备的广播信号中需要携带与自身相关的重要信息。

ESP32S3在开启广播之前，需要事先配置广播内容。开启广播后，广播信号将携带这些内容。



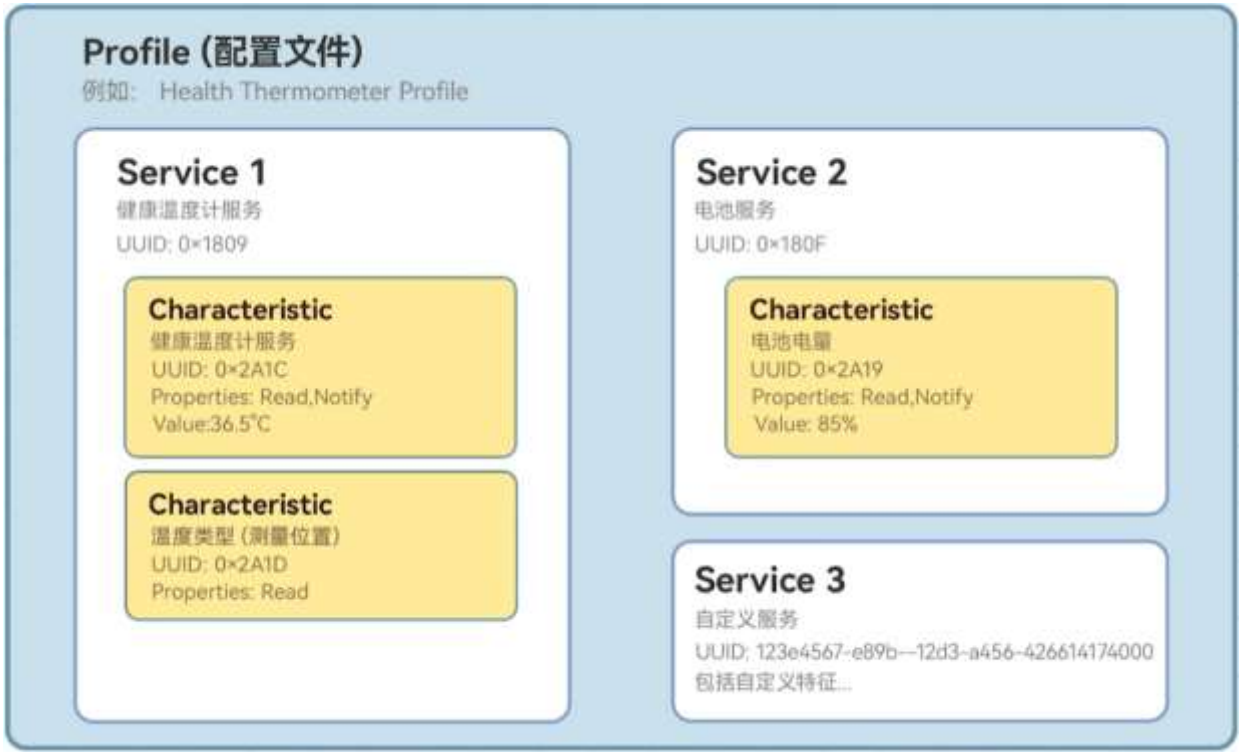
- 使用ble_gap_adv_set_fields()函数配置广播内容
- 使用ble_gap_adv_start()函数开启广播
- 使用ble_gap_adv_stop()函数关闭广播

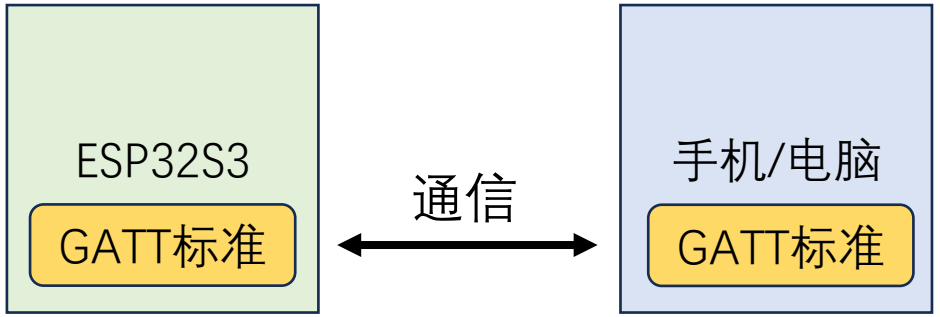
通信阶段 – 特征与服务

艾谷科技
Aigo

外围设备与中央设备通过GAP标准成功进行蓝牙连接之后，进入通信阶段，数据交互基于GATT标准进行。

GATT标准下的配置文件以服务和特征的结构进行描述。





通信阶段 – 特征与服务

艾谷科技
Aigo

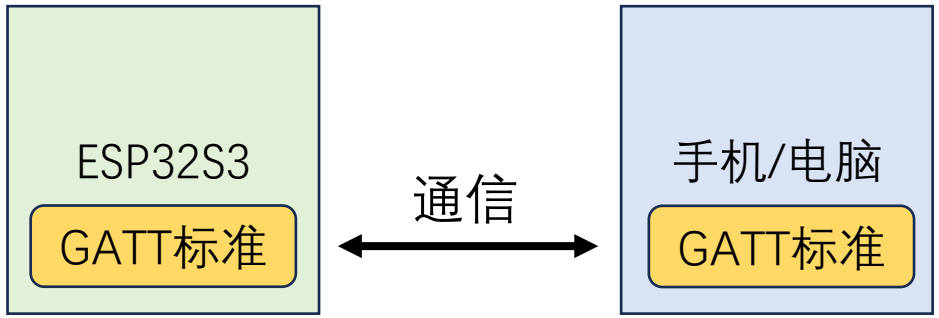
- 通信之前，外围设备向中央设备发送配置文件。
- 服务：UUID+多个特征
- 特征：UUID+属性+值

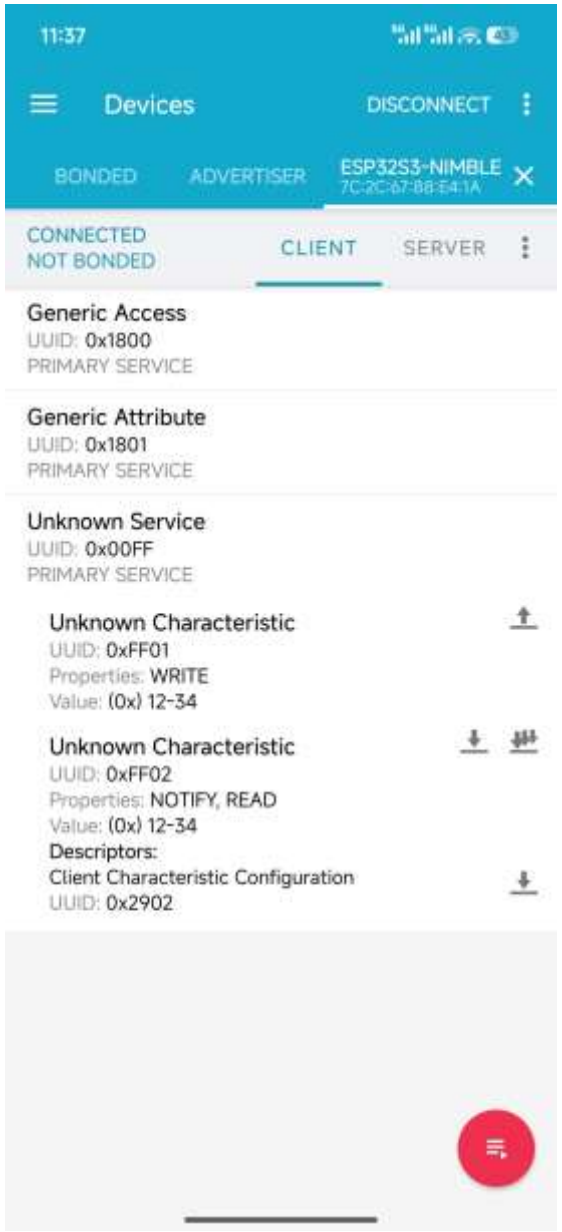
服务	UUID	
	特征	UUID
		值
		属性：Write\Read\Notify
	特征	UUID
		值
		属性：Write\Read\Notify

通信阶段 – ESP32S3服务与特征

艾谷科技
Aigo

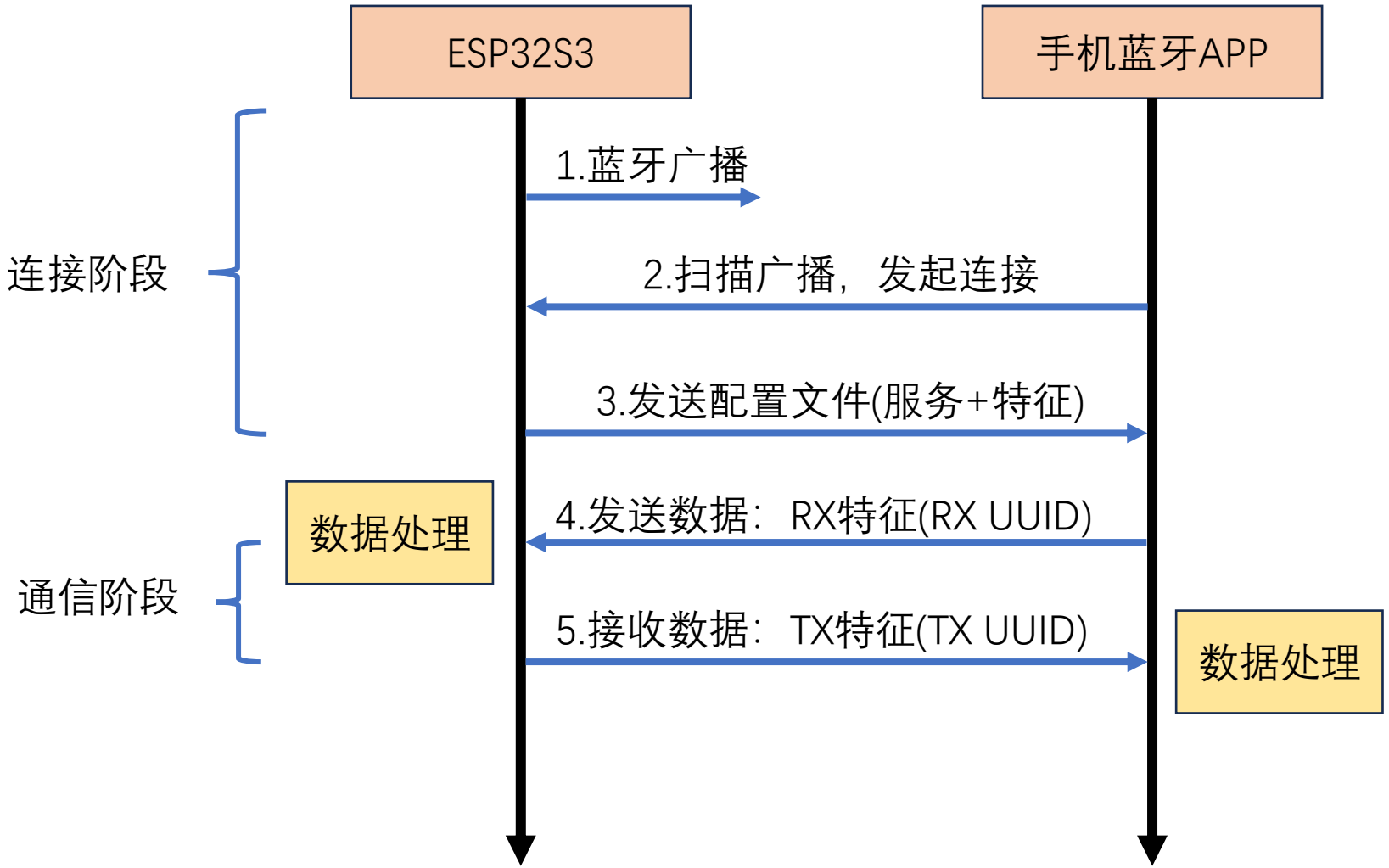
```c
static const struct ble_gatt_svc_def gatt_svcs[] = {
    {
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = BLE_UUID16_DECLARE(0x00FF),
        .characteristics = (struct ble_gatt_chr_def[]) {
            {
                // RX 特征：手机写数据到此
                .uuid = BLE_UUID16_DECLARE(0xFF01),
                .access_cb = gatt_svr_access_cb,
                .flags = BLE_GATT_CHR_F_WRITE,
                .val_handle = &rx_val_handle,
            },
            {
                // TX 特征：ESP32 发送给手机
                .uuid = BLE_UUID16_DECLARE(0xFF02),
                .access_cb = gatt_svr_access_cb,
                .val_handle = &tx_val_handle,
                .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY,
            },
            { 0 }
        },
    },
    { 0 }
};
```





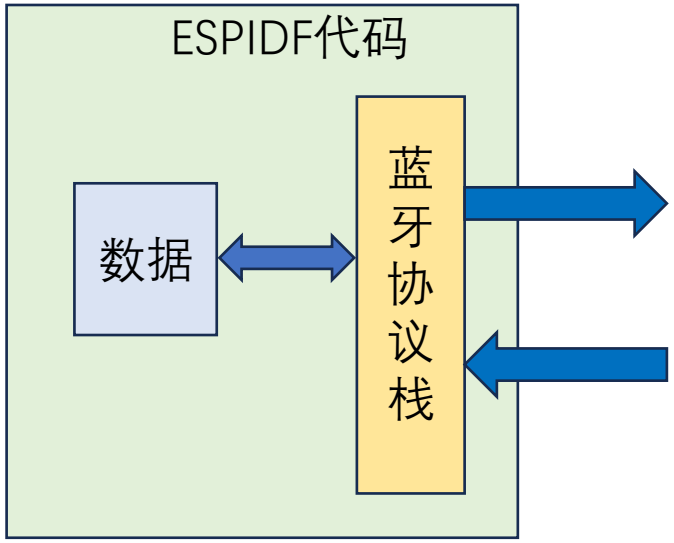
BLE通信流程

艾谷科技
Aigo



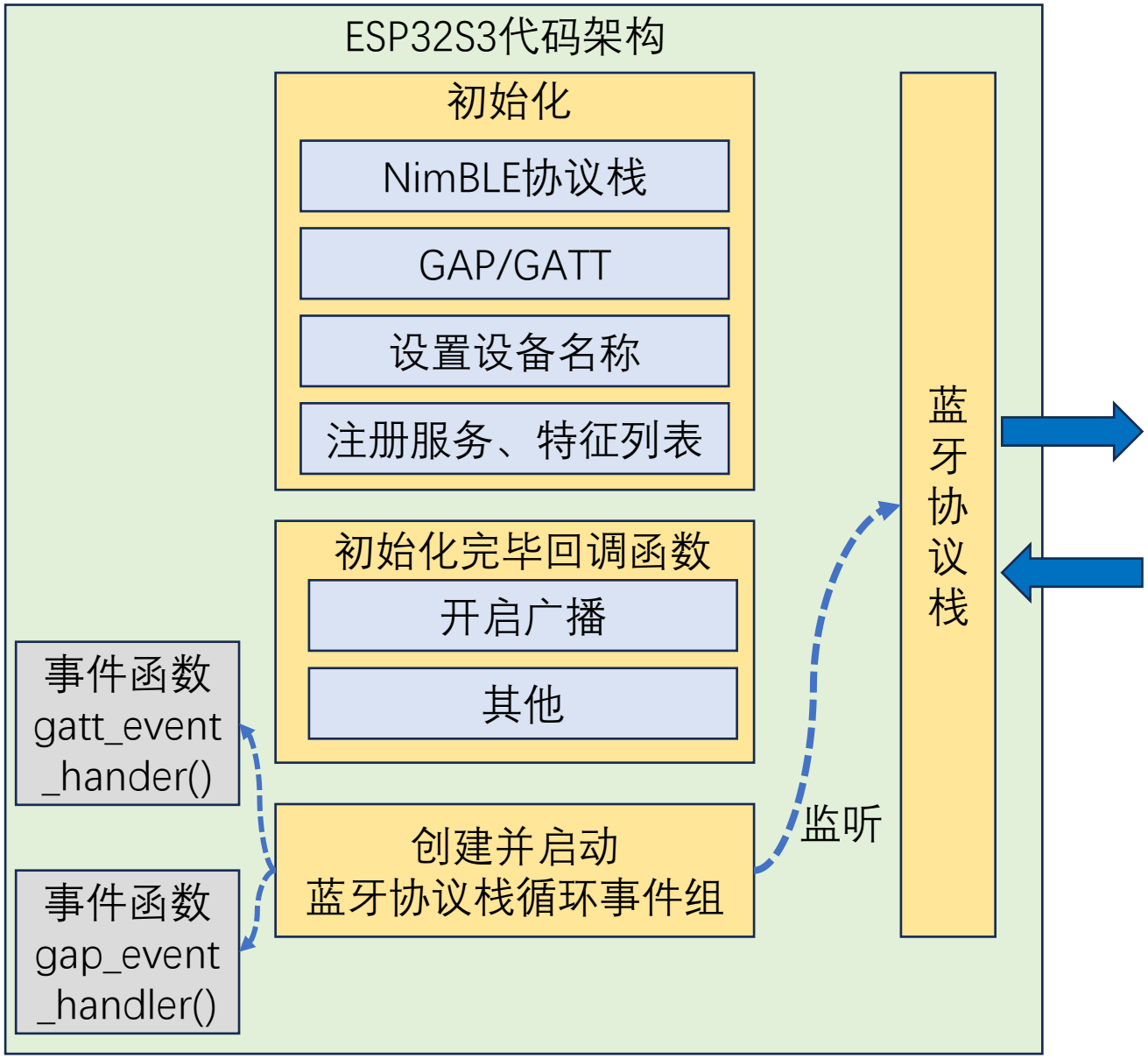
ESPIDF蓝牙协议栈

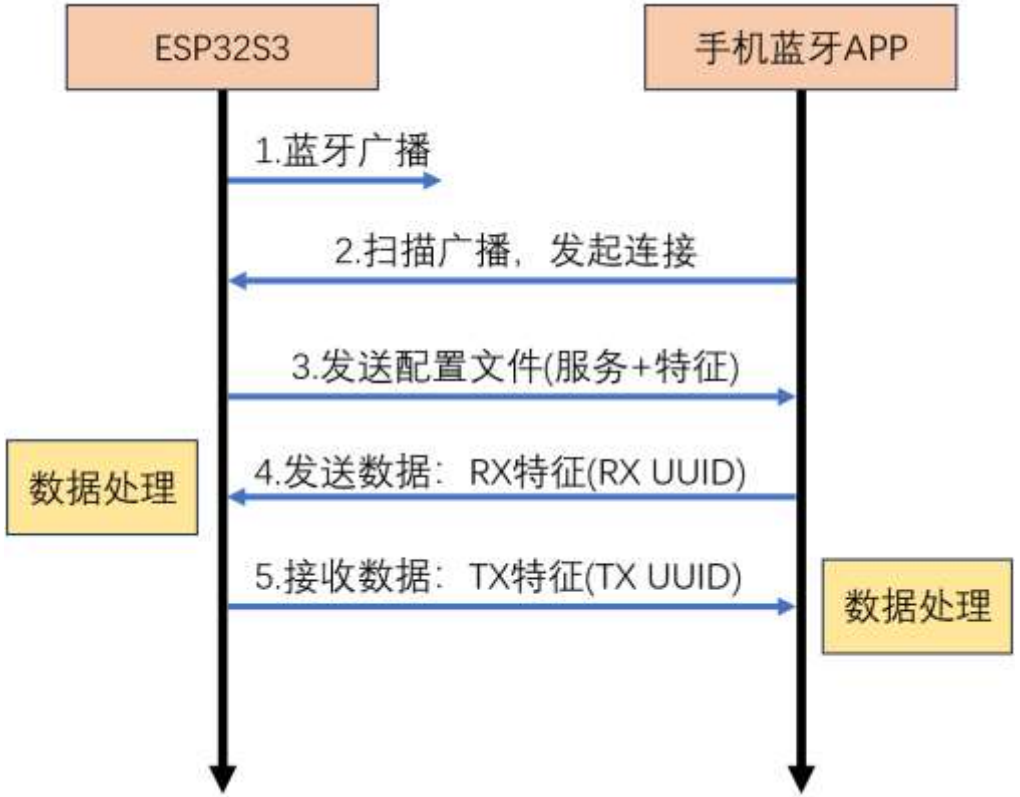
蓝牙协议栈：就是一套按照蓝牙规范实现的分层软件架构，它把无线电信号抽象成应用能调用的 API，让开发者无需关心射频底层细节就能实现蓝牙通信。本质：一套C语言代码。
ESPIDF蓝牙协议栈：Bluedroid、NimBLE



ESP32S3蓝牙工作流程

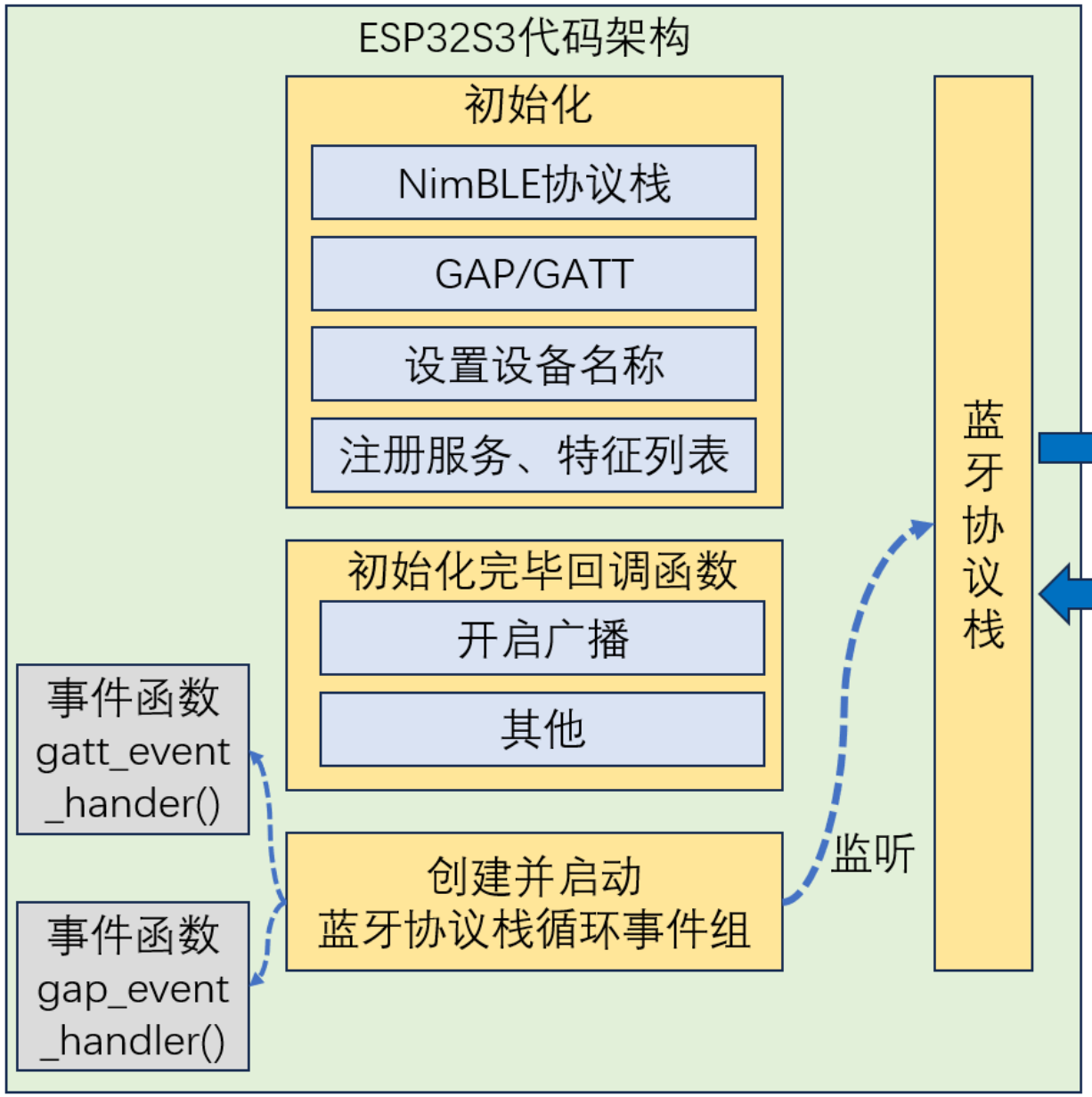
艾谷科技
Aigo





ESP32S3蓝牙配置流程

艾谷科技 Aigo



- 在sdkmenuconfig中开启蓝牙，选择NimBLE蓝牙协议栈
- 调用nvs_flash_init()函数，初始化NVS
- 调用nimble_port_init()函数，初始化NimBLE蓝牙协议栈
- 调用ble_svc_gap_init()、ble_svc_gatt_init()函数，初始化GAP连接标准、GATT通信标准
- 调用ble_svc_gap_device_name_set()函数，设置设备名称
- 调用ble_gatts_count_cfg()、ble_gatts_add_svcs()函数，注册服务特征列表
- 调用ble_hs_cfg.sync_cb = on_sync结构体，注册初始化完毕回调函数on_sync()
- 在on_sync()回调函数中，调用开启广播的函数开启广播
- 调用nimble_port_freertos_init ()函数创建监听任务
- 在监听任务函数中，调用nimble_port_run()函数循环监听蓝牙协议栈产生的事件
- 创建事件函数：gatt_event_hander()、gap_event_handler()